



HiSVP

开发指南

文档版本 01
发布日期 2018-12-10

版权所有 © 深圳市海思半导体有限公司 2018。保留一切权利。

非经本公司书面许可，任何单位和个人不得擅自摘抄、复制本文档内容的部分或全部，并不得以任何形式传播。

商标声明



HISILICON、海思和其他海思商标均为深圳市海思半导体有限公司的商标。

本文档提及的其他所有商标或注册商标，由各自的所有人拥有。

注意

您购买的产品、服务或特性等应受海思公司商业合同和条款的约束，本文档中描述的全部或部分产品、服务或特性可能不在您的购买或使用范围之内。除非合同另有约定，海思公司对本文档内容不做任何明示或默示的声明或保证。

由于产品版本升级或其他原因，本文档内容会不定期进行更新。除非另有约定，本文档仅作为使用指导，本文档中的所有陈述、信息和建议不构成任何明示或暗示的担保。

深圳市海思半导体有限公司

地址：深圳市龙岗区坂田华为基地华为电气生产中心 邮编：518129

网址：<http://www.hisilicon.com>

客户服务电话：+86-755-28788858

客户服务传真：+86-755-28357515

客户服务邮箱：support@hisilicon.com



前言

概述

本文档旨在帮助用户了解 SVP(Smart Vision Platform)的硬件特性、工具链及开发流程，以期达到快速上手和开发出充分利用 SVP 特性的智能方案。



说明

未有特殊说明，Hi3559CV100 与 Hi3559AV100 内容一致。

未有特殊说明，Hi3516DV300 与 Hi3516CV500 内容一致。

产品版本

与本文档相对应的产品版本如下。

产品名称	产品版本
Hi3559A	V100
Hi3559C	V100
Hi3519A	V100
Hi3516D	V300
Hi3516C	V500

读者对象

本文档（本指南）主要适用于以下工程师：

- 技术支持工程师
- 软件开发工程师

符号约定

在本文中可能出现下列标志，它们所代表的含义如下。



符号	说明
 危险	表示有高度潜在危险，如果不能避免，会导致人员死亡或严重伤害。
 警告	表示有中度或低度潜在危险，如果不能避免，可能导致人员轻微或中等伤害。
 注意	表示有潜在风险，如果忽视这些文本，可能导致设备损坏、数据丢失、设备性能降低或不可预知的结果。
 窍门	表示能帮助您解决某个问题或节省您的时间。
 说明	表示是正文的附加信息，是对正文的强调和补充。

修订记录

修订记录累积了每次文档更新的说明。最新版本的文档包含以前所有文档版本的更新内容。

日期	版本	修改描述
2018-12-10	01	新增 5.3.4、5.5.9 和 5.5.10 小节 3.5.2 和 5.13.2 小节涉及修改
2018-11-13	00B14	4.5 小节涉及修改 新增 4.7 和 4.8 小节
2018-10-26	00B13	3.1.6.2 小节，表 3-2 涉及修改 新增 5.9 和 5.10 小节 5.13.2 小节涉及修改
2018-10-15	00B12	1.3 小节，表 1-1 涉及修改 3.1.1，3.6.2、3.6.3 和 4.1.2 小节涉及修改
2018-09-04	00B11	新增第 4 章 修改第 5 章的相关内容 添加 Hi3516CV500 和 Hi3516DV300 内容
2018-08-08	00B10	新增 2.2 小节，2.6.2 小节中的注意涉及修改 3.2.7~3.2.9，3.4.1~3.4.4 小节涉及修改 3.5.1 小节，修改表 3-5 4.1.1、4.1.2、4.2.2 小节涉及修改；新增 4.4.1 和 4.7 小节内容



日期	版本	修改描述
2018-07-30	00B09	添加 Hi3556AV100 的相关内容
2018-07-06	00B08	3.4.2.1、4.1.2 和 4.7.2 小节涉及修改 新增 3.5.1 和 4.7.3 小节
2018-06-20	00B07	4.1.2 小节的步骤 3 和 4.5.3 小节的步骤 2 涉及修改 4.5.4 小节涉及修改 新增 4.6 小节
2018-05-15	00B06	3.4~3.6 小节涉及修改 新增 4.1.2 “Python3.5+caffe 环境配置” 小节 删除原来的 4.1.4 “NNIE 编译器依赖库 OpenCV 安装与配置” 小节 新增 4.5.4~4.6 小节
2018-04-10	00B05	第五次临时版本发布 添加 Hi3519AV100 相关内容
2018-04-04	00B04	第四次临时版本发布 3.5.1 小节，表 3-5 涉及修改 4.2.3、4.2.4、4.5.1 和 4.5.2 小节涉及修改 新增 4.5.3 小节
2018-03-15	00B03	第三次临时版本发布 修改 3.1.1、3.2 和 3.5 小节 新增 3.4 小节和第 4 章
2018-01-30	00B02	第二次临时版本发布 3.3.1.3 小节与 3.3.2.6 小节的 lib64:\$LD_LIBRARY_PATH 改成 lib:\$LD_LIBRARY_PATH 3.5.4、3.8 小节涉及刷新 新增 3.9 小节
2018-01-10	00B01	第一次临时版本发布。



目 录

1 概述.....	1
1.1 SVP 简介	1
1.2 开发框架.....	1
1.3 硬件资源.....	2
1.4 软件开发.....	2
1.5 开发环境.....	3
1.6 相关文档.....	3
2 DSP 开发指南	4
2.1 开发工具介绍.....	4
2.2 DSP 工具链和配置核版本.....	4
2.3 Windows 环境下安装 DSP 工具链和配置核.....	5
2.4 修改 memmap 配置.....	7
2.5 查看 stack 使用.....	7
2.6 Linux 环境下安装 DSP 工具链和配置核	8
2.6.1 安装 DSP 工具链	8
2.6.2 安装 DSP 配置核	15
2.6.3 配置环境变量.....	17
2.7 开发流程.....	17
2.7.1 开发自定义算子.....	18
3 NNIE 开发指南	20
3.1 NNIE 介绍	20
3.1.1 工具链介绍	20
3.1.2 开发流程	21
3.1.3 网络层的分类.....	22
3.1.4 扩展层规则	23
3.1.5 Non-support 层处理方式	41
3.1.6 NNIE 规格	44
3.1.7 NNIE 硬件资源利用率	66
3.2 Prototxt 要求.....	67
3.2.1 deploy.prototxt 输入层格式	67



3.2.2 prototxt layer 描述格式	68
3.2.3 prototxt 激活层描述方式	68
3.2.4 prototxt 针对 Scale/Bias 层书写规范	71
3.2.5 prototxt 针对 RNN 层书写规范	76
3.2.6 prototxt 针对 LSTM 层书写规范	80
3.2.7 prototxt 指定任意中间层上报	84
3.2.8 prototxt 指定任意层配置高精度	85
3.2.9 prototxt 指定某些层由 CPU 执行	86
3.2.10 分段执行网络 prototxt 示例	87
3.3 公开模型下载	93
3.3.1 Googlenet	93
3.3.2 VGG16	93
3.3.3 Resnet-50、resnet-101、resnet-152	94
3.4 Linux 版 NNIE mapper 安装	95
3.4.1 mapper 依赖库 Protobuf 安装与配置	95
3.4.2 mapper 依赖库 OpenCV 安装与配置	97
3.4.3 mapper 依赖库 CUDA 安装与配置	99
3.4.4 mapper 本体安装	100
3.5 Linux 版 nnie_mapper 使用说明	101
3.5.1 nnie_mapper 版本说明	101
3.5.2 配置文件说明	102
3.5.3 nnie_mapper 输出	107
3.6 仿真库使用说明	107
3.6.1 使用限制	108
3.6.2 配置文件设置	108
3.6.3 配置文件说明	109
3.6.4 功能仿真 CUDA 加速配置	110
3.7 NNIE 调试	112
3.7.1 预处理一致性问题	112
3.7.2 多段网络时输入问题	112
3.7.3 精度下降问题	113
4 Runtime 开发指南	114
4.1 Runtime 介绍	114
4.1.1 层次关系	114
4.1.2 工具链介绍	114
4.1.3 开发流程	115
4.1.4 网络层分类	115
4.1.5 Runtime Layer 规格	115
4.2 插件库管理	116



4.2.1 提供 Custom-Layer 代码实现模板.....	116
4.2.2 提供 Custom-Layer 插件库复用功能.....	117
4.2.3 插件库限制	117
4.3 网络拓扑管理.....	117
4.4 级联网络支持.....	119
4.5 优先级配置.....	119
4.6 帧间依赖网络支持.....	120
4.7 全局参数可配置.....	120
4.7.1 线程使用的 CPU 亲和度可配置	120
4.7.2 日志级别可配置.....	121
4.8 模型组可配置.....	122
4.8.1 模型组全局配置项.....	123
4.8.2 input 可配置项	124
4.8.3 模型、连接器可配置项.....	124
4.8.4 modelgroup 文件解析约束	126
4.9 Sample 说明.....	126
4.9.1 图片 Resize.....	126
5 RuyiStudio 工具使用指南	127
5.1 RuyiStudio 安装.....	127
5.1.1 编译链 MinGW-W64 安装	127
5.1.2 Python3.5+caffe 环境配置	131
5.1.3 RuyiStudio 启动方式	135
5.2 RuyiStudio 工具平台公共功能介绍.....	136
5.2.1 启动工具设置 workspace.....	136
5.2.2 透视图功能介绍.....	137
5.2.3 创建工程	140
5.2.4 导入工程	143
5.2.5 工程属性设置.....	147
5.2.6 C/C++代码编辑功能介绍.....	153
5.2.7 C/C++代码编译功能.....	159
5.2.8 C/C++代码调试功能.....	161
5.3 生成 NNIE wk 功能	163
5.3.1 cfg 文件配置模式	163
5.3.2 Prototxt 文件自动标记功能.....	166
5.3.3 标记后的 Prototxt 可视化功能.....	167
5.3.4 网络拓扑图中显示各原生 caffe 支持层的 Shape 信息	170
5.3.5 Prototxt 层属性可视化编辑.....	172
5.3.6 原始 prototxt 编辑功能	177
5.3.7 nnie_mapper 配置项自动生成功能.....	178



5.3.8 生成 NNIE wk 文件	180
5.3.9 Make wk 时产生的文件描述	181
5.4 仿真 NNIE 功能	181
5.4.1 配置文件可视化编辑	181
5.4.2 Sample 代码中一键切换功能仿和指令仿	182
5.4.3 Sample 代码中手动切换功能仿和指令仿切换	184
5.5 数据分析工具	186
5.5.1 网络拓扑图显示功能	186
5.5.2 向量相似性比较功能	188
5.5.3 调试定位信息获取功能	195
5.5.4 目标检测功能	200
5.5.5 输出 caffe 的中间结果	203
5.5.6 还原网络	204
5.5.7 芯片高效模式和 Un-inplace 功能	208
5.5.8 性能仿真结果展示	216
5.5.9 预处理功能	219
5.5.10 拓扑图对比功能	220
5.6 多芯片支持功能	222
5.6.2 创建新工程项目时选择对应的芯片	223
5.6.3 在已有的工程项目中修改对应的芯片	224
5.7 Fine tune 辅助添加量化层功能	225
5.8 生成 Runtime wk 功能	226
5.8.1 cfg 文件配置模式	226
5.8.2 Prototxt 层属性可视化编辑	226
5.8.3 生成 Runtime wk 文件	230
5.9 Runtime 模型管理功能	230
5.9.1 概述	230
5.9.2 选择文件导入模型	231
5.9.3 扫描文件夹导入模型	231
5.9.4 删除导入的模型	231
5.10 模型级联编辑功能	232
5.10.1 概述	232
5.10.2 新建模型级联关系图	232
5.10.3 模型级联关系图编辑	234
5.10.4 编辑元素属性	236
5.10.5 保存级联关系图	238
5.10.6 导出模型组描述文件	238
5.11 客户自定义插件	238
5.11.1 创建工程	238



5.11.2 添加客户自定义插件到客户自定义插件路径下	242
5.11.3 设置客户自定义插件路径	244
5.12 创建 Runtime 工程和切换芯片	245
5.12.1 创建 runtime 工程	245
5.12.2 切换芯片	248
5.12.3 查看项目 Model 和芯片型号	249
5.13 FAQ	252
5.13.1 工具启动时一直卡在启动画面“Loading org.eclipse...”无法打开工具的解决办法	252
5.13.2 打印 caffe 中间输出或还原网络时提示 Layer xxx with type xxx is not supported, please refer to chapter 3.1.4 and FAQ of "HiSVP Development Guide.pdf" to extend caffe!	253
5.13.3 工具在运行仿真库代码时，当仿真库对应的 exe 发生异常崩溃时，所有 printf 信息在控制台上打印不出来的解决办法	263
5.13.4 在创建项目时，出现中文时，导致项目修改文件失败，编译和运行失败的问题	264



插图目录

图 1-1 SVP 开发框架	1
图 2-1 System Overview	5
图 2-2 Find and Install a Configuration Build	6
图 2-3 查看安装好的配置核	7
图 2-4 创建的工程界面	7
图 2-5 查看栈信息操作界面	8
图 2-6 显示栈信息窗口	8
图 2-7 DSP 应用程序开发框图	18
图 3-1 NNIE 软件开发流程	22
图 3-2 在 caffe.proto 文件中扩展 LayerParameter	23
图 3-3 在 caffe.proto 文件中定义新 layer 的 message	24
图 3-4 从 prototxt 中获取参数示例	24
图 3-5 声明自定义的层	25
图 3-6 在实现文件中添加头文件	25
图 3-7 ROI Pooling 层在 prototxt 中的示例	25
图 3-8 CReLU 层实现示例	26
图 3-9 PassThrough 层操作示意图	28
图 3-10 增加 PassThrough 层所需的对 caffe.proto 的修改	29
图 3-11 MXNet 中 RReLU API 说明	31
图 3-12 在 LayerParameter 中添加 MatMulParameter	32
图 3-13 定义 MatMulParameter 参数	32
图 3-14 在 LayerParameter 中添加 PermuteParameter	33
图 3-15 定义 PermuteParameter 参数	33
图 3-16 Faster RCNN 示意图	34
图 3-17 在 LayerParameter 中添加 ROI PoolingParameter	35



图 3-18 定义 ROI PoolingParameter 参数	35
图 3-19 PSROI Pooling 示意图	36
图 3-20 在 LayerParameter 中添加 PSROI PoolingParameter	36
图 3-21 定义 PSROI PoolingParameter 参数	36
图 3-22 Upsample 层示意图	37
图 3-23 在 LayerParameter 中添加 UpsampleParameter	38
图 3-24 定义 UpsampleParameter 参数	38
图 3-25 Normalize 层计算示意图	39
图 3-26 在 LayerParameter 中添加 NormalizeParameter	40
图 3-27 定义 NormalizeParameter 参数	40
图 3-28 Faster RCNN 分段执行示意图	44
图 3-29 NNIE 支持 prototxt 格式说明示意图（左边支持）	68
图 3-30 Faster RCNN 网络自定义分段示意图一	88
图 3-31 Faster RCNN 网络自定义分段示意图二	89
图 3-32 RFCN 网络 mapper 自动分段示意图	90
图 3-33 RFCN 自定义分段示意图	91
图 3-34 SSD 自定义分割方式一	92
图 3-35 SSD 自定义分割方式二	93
图 3-36 Protobuf 下载页面	95
图 3-37 配置文件示意图	108
图 3-38 多段网络示意图	113
图 4-1 Runtime API 交付层次关系图	114
图 4-2 插件库管理步骤图	117
图 4-3 网络拓扑示例图	118
图 4-4 模型级联图	119
图 4-5 帧间依赖网络示意图	120
图 4-6 CPU 配置示意图	121
图 4-7 Modelgroup 可配置项关系图	122
图 4-8 rfcn+alexnet modelgroup 结构图	123
图 5-1 wget 安装可执行文件	127
图 5-2 MinGW-w64 release 版本	128
图 5-3 MinGW-w64 解压后的目录结构	129



图 5-4 Msys 版本.....	129
图 5-5 解压的 msys 放置的位置	130
图 5-6 系统环境 MinGW 环境变量设置	131
图 5-7 libraries_v140_x64_py35_1.1.0.tar.bz2	132
图 5-8 RUYI_PYTHON_PATH	132
图 5-9 Path	133
图 5-10 PYTHONPATH.....	133
图 5-11 解压 caffe.zip 到 caffe 文件夹中	134
图 5-12 拷贝 libraries 中的库文件到指定文件夹中	134
图 5-13 打开 RuyiStudio 工具	135
图 5-14 Ruyi Studio 工具启动界面	136
图 5-15 选择 workspace 的目录.....	136
图 5-16 工具界面	137
图 5-17 选择 Ruyi 透视图.....	137
图 5-18 Ruyi 透视图.....	138
图 5-19 工作区视图	139
图 5-20 视图关闭选项	139
图 5-21 通过 File 菜单创建 Project.....	140
图 5-22 通过鼠标右键创建 Project	140
图 5-23 选择 NNIE Project.....	141
图 5-24 设置 Project 的名字并确认创建工程	142
图 5-25 创建工程后的工程视图.....	143
图 5-26 NNIE PC 端 Sample 工程	143
图 5-27 工程导入方式 1.....	144
图 5-28 工程导入方式 2.....	144
图 5-29 选择外部的项目对话框.....	145
图 5-30 选择工程对话框.....	146
图 5-31 导入 sample 代码后的工程效果	147
图 5-32 工程属性视图	148
图 5-33 采用系统环境变量.....	149
图 5-34 采用自定义环境变量.....	149
图 5-35 设置 linker 为 X64	150



图 5-36 设置 g++的编译参数	151
图 5-37 工程所需的 Include 路径配置	152
图 5-38 工程所需的 Lib 或 Dll 文件配置	153
图 5-39 搜索框	154
图 5-40 Tasks 视图	154
图 5-41 创建任务框	155
图 5-42 在代码中创建任务	155
图 5-43 问题视图	156
图 5-44 属性视图	156
图 5-45 项目管理视图	157
图 5-46 大纲视图	157
图 5-47 引用关系查看	158
图 5-48 断点显示	158
图 5-49 代码辅助功能	159
图 5-50 编译日志	159
图 5-51 编译成功后的文件夹	160
图 5-52 编译报错的日志	160
图 5-53 创建 Application 设置视图	161
图 5-54 重定向标准输出	162
图 5-55 Debug 透视图	162
图 5-56 Mapper Configuration Editor 界面模式打开 cfg 文件	164
图 5-57 Mapper Configuration Editor 视图	165
图 5-58 cfg 文件 Text Editor 视图	166
图 5-59 Prototxt 自动标记功能	167
图 5-60 编辑标记后 Prototxt 文件	167
图 5-61 标记后的 Prototxt 的网络拓扑图	168
图 5-62 标记后的 Prototxt 中 Layer 对应的属性图	169
图 5-63 Inplace 层节点的 Layer Info 视图	170
图 5-64 显示 shape 信息的网络拓扑图	171
图 5-65 更新 shape 信息后的网络拓扑图	172
图 5-66 将当前 Layer 指定为 CPU 运算	173
图 5-67 将当前 Layer 指定为高精度层	174



图 5-68 将当前 Layer 指定为中间上报层	175
图 5-69 当前 Layer 为 Custom 层	176
图 5-70 当前 Layer 为 Proposal 层	177
图 5-71 Prototxt 编辑界面	178
图 5-72 Mapper Configuration Editor 参数界面	179
图 5-73 *.cfg Text Editor 界面	180
图 5-74 Make WK 过程的工具界面	181
图 5-75 仿真配置文件可视化编辑	182
图 5-76 切换功能仿/指令仿的操作	183
图 5-77 控制台打印切换仿真库对应的修改信息	183
图 5-78 修改 g++编译宏	184
图 5-79 编译宏对应的代码位置	185
图 5-80 修改工程依赖的 Libraries	186
图 5-81 网络拓扑图视图	187
图 5-82 向量对比视图	189
图 5-83 选择 dot 文件的向量对比视图	190
图 5-84 双击需要查看的行	191
图 5-85 详细比较结果	192
图 5-86 点击 Show Result 按钮	193
图 5-87 Graph View 显示对比结果	194
图 5-88 保存 csv 格式的报告	195
图 5-89 Dump Network 视图	197
图 5-90 选择保存导出数据的文件夹	197
图 5-91 选择需要 Dump 的开始层	198
图 5-92 选择需要 Dump 的结束层	198
图 5-93 Dump 提示推荐层	199
图 5-94 Dump 导出的信息说明	200
图 5-95 目标检测界面	201
图 5-96 坐标检测文件示例	201
图 5-97 框选结果	202
图 5-98 修改置信度后的框选结果	202
图 5-99 输出 caffe 中间结果	203



图 5-100 在配置的 output Dir 下面得到 caffe 中间结果	204
图 5-101 标记 prototxt.....	205
图 5-102 在 Graph View 上画出对应视图	206
图 5-103 Dump Network 视图.....	207
图 5-104 Dump 出的数据.....	207
图 5-105 还原网络	207
图 5-106 还原出 caffemodel.....	208
图 5-107 芯片高效模式视图.....	209
图 5-108 打开 Prototxt 显示到 Graph View 中.....	210
图 5-109 芯片高效模式执行效果.....	211
图 5-110 Un-inplace 所有 inplace 层的功能.....	212
图 5-111 双击选中需要 un-inplace 的层	213
图 5-112 Un-inplace 单个 inplace 层的功能.....	214
图 5-113 Revert initial State 功能.....	215
图 5-114 Save Prototxt 功能	216
图 5-115 性能仿真结果展示界面.....	217
图 5-116 导入 Dot 文件和性能仿真结果文件夹.....	218
图 5-117 性能仿真结果文件示例.....	218
图 5-118 预处理视图.....	219
图 5-119 导入 Prototxt.....	220
图 5-120 进行预处理另存 Prototxt.....	220
图 5-121 拓扑图对比初始化界面.....	221
图 5-122 拓扑图对比视图.....	221
图 5-123 导入文件	221
图 5-124 网络拓扑图	222
图 5-125 创建 NNIE Project 时设置 SOC Version.....	224
图 5-126 修改已有工程中对应的芯片.....	225
图 5-127 添加量化层视图.....	226
图 5-128 将当前 Layer 指定为高精度运算	227
图 5-129 将当前 Layer 指定为 cpu 运算.....	228
图 5-130 提示是否需要创建 Proposal 类型的插件库.....	229
图 5-131 打开 proposal 层的显示.....	229



图 5-132 运行 Runtime 所需要传入的文件	230
图 5-133 运行 Runtime 的界面	230
图 5-134 Model Zoo 视图	231
图 5-135 模型级联关系图编辑器	232
图 5-136 模型级联关系图向导	233
图 5-137 模型级联关系图基础信息编辑	233
图 5-138 导入模型组描述文件	234
图 5-139 输入数据图元	234
图 5-140 模型图元	235
图 5-141 连接器图元	235
图 5-142 从 Model Zoo 添加模型	235
图 5-143 有效连线	236
图 5-144 无效连线	236
图 5-145 级联关系图属性视图	237
图 5-146 模型组描述文件导出按钮的位置	238
图 5-147 通过鼠标右键创建 Project	239
图 5-148 选择 Custom plugin Project	240
图 5-149 设置 Project 和 layer type 的名字并确认创建工程	241
图 5-150 创建工程后的工程视图	242
图 5-151 生成的*.c 文件	242
图 5-152 通过鼠标右键选择 Add lib to Custom plugin location	243
图 5-153 添加 dll 成功之后的提示	244
图 5-154 打开 preferences 页面	244
图 5-155 打开设置 Custom Plugin 页面	245
图 5-156 通过鼠标右键创建 Project	246
图 5-157 选择 Runtime Project	247
图 5-158 设置 Project 的名字并确认创建工程	248
图 5-159 切换芯片	249
图 5-160 属性页	250
图 5-161 项目 Model 页	251
图 5-162 芯片型号页	252
图 5-163 工具启动卡主无法打开的现象	253



图 5-164 修改 build_win.cmd 文件中的部分配置.....	254
图 5-165 运行 build_win.cmd 文件.....	254
图 5-166 运行打印，下载卡顿.....	255
图 5-167 download_prebuilt_dependencies.py	255
图 5-168 WindowsDownloadPrebuiltDependencies.cmake.....	255
图 5-169 Caffe1.0 基础工程.....	256
图 5-170 检测网的 ssd 网络	257
图 5-171 nnie_ssd_deploy.prototxt 文件的 Normalize 层.....	257
图 5-172 nnie_ssd_deploy.prototxt 文件的 Permute 层	258
图 5-173 下载 caffe-ssd.zip 包	258
图 5-174 增加 NormalizeParameter 和 PermuteParameter 的定义（1）	258
图 5-175 增加 NormalizeParameter 和 PermuteParameter 的定义（2）	259
图 5-176 NormalizeParameter 和 PermuteParameter 层的代码文件路径	259
图 5-177 添加代码 permute_layer.cpp 和 normalize_layer.cpp 到工程文件	260
图 5-178 添加代码 permute_layer.hpp 和 normalize_layer.hpp 到工程文件.....	260
图 5-179 Release 模式编译	261
图 5-180 在代码中增加扩展层.....	261
图 5-181 打印 caffe 中间结果.....	262
图 5-182 成功得到 caffe 的中间输出	262
图 5-183 main 方法中设置缓存 BUFFF 为 NULL.....	263



表目录

表 1-1 不同芯片下的 SVP 硬件资源	2
表 1-2 不同芯片下的 SVP 运行环境	3
表 2-1 不同芯片的 DSP 工具链和配置核	4
表 3-1 SVP NNIE 网络层级联关系表	45
表 3-2 NNIE layer 规格表	51
表 3-3 Scale 层 10 种场景配置	75
表 3-4 Bias 层 6 种使用场景配置	76
表 3-5 芯片和 nnie mapper 版本对应关系表	101
表 3-6 nnie_mapper 配置选项说明	102
表 4-1 模型组全局配置项表	123
表 4-2 input 可配置项	124
表 4-3 bottom/top 数据结构	124
表 4-4 模型、连接器可配置项	125
表 5-1 芯片名称与 mapper 和仿真库的对应关系	222



1 概述

1.1 SVP 简介

SVP(Smart Vision Platform)是海思媒体处理芯片智能视觉异构加速平台。该平台包含了 CPU、DSP、NNIE(Neural Network Inference Engine)等多个硬件处理单元和运行在这些硬件上 SDK 开发环境，以及配套的工具链开发环境。

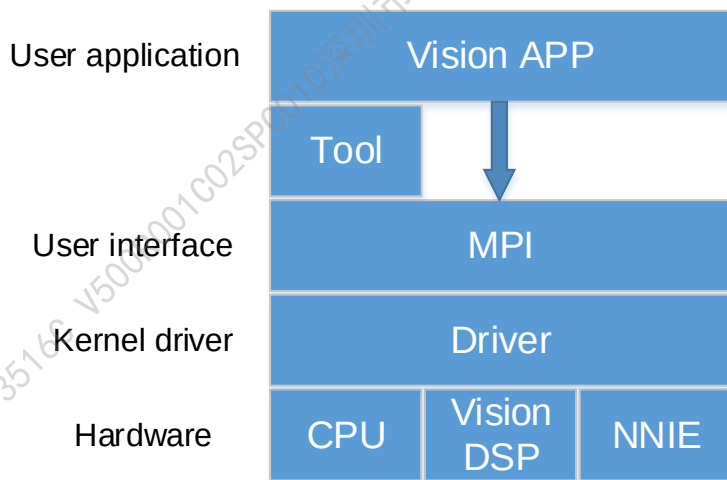
本文档主要介绍 SVP 的硬件特性、配套工具链及开发流程，旨在帮助用户快速入门以及开发出充分利用 SVP 硬件特性的智能应用。软件开发接口介绍请参考《HiSVP API 参考》文档。

1.2 开发框架

SVP 开发框架如图 1-1 所示。目前 SVP 中包含的硬件处理单元有 CPU、vision DSP、NNIE，其中某些硬件可能有多核。

不同的硬件有不同的配套工具链，用户的应用程序需要结合这些工具的使用来开发。

图1-1 SVP 开发框架





1.3 硬件资源

不同的芯片 SVP 会使用不同硬件资源，如表 1-1 所示。

表1-1 不同芯片下的 SVP 硬件资源

芯片	SVP 硬件资源		
	CPU	DSP	NNIE
Hi3559AV100ES	双核 A73+双核 A53	2 个	1 个
Hi3559AV100	双核 A73+双核 A53	4 个	2 个
Hi3519AV100	单核 A53+单核 A53	1 个	1 个
Hi3556AV100	单核 A53+单核 A53	1 个	1 个
Hi3516DV300	单核 A7+单核 A7	0 个	1 个
Hi3516CV500	单核 A7+单核 A7	0 个	1 个

针对 CPU 的具体规格，请参考 ARM 官方文档。

DSP 和 NNIE 的硬件规格，可以参考《Hi35xx Vxxx ultra-HD Mobile Camera SoC 用户指南》。



注意

不同的芯片 SVP 可能会使用不同的硬件资源，即便是使用相同的硬件型号，硬件的配置也不一定相同。

1.4 软件开发

SVP 是海思媒体处理芯片的智能加速平台，因此需要结合海思 MPP 平台一起来进行软件开发，可参考相关文档《HiMPP V4.0 媒体处理软件开发参考》。用户可以根据 SVP 的软硬件特性开发出能最大化利用 SVP 硬件资源的视觉处理应用。



1.5 开发环境

不同的芯片 SVP 会在不同的环境上运行，如表 1-2 所示。

表1-2 不同芯片下的 SVP 运行环境

芯片	操作系统
Hi3559AV100ES	Linux(big.LITTLE)
Hi3559AV100	Linux(big.LITTLE/Multi-Core)
Hi3519AV100	Linux(SMP)
Hi3556AV100	AMP(Linux + Huawei LiteOS)
Hi3516DV300	SMP(Linux)/AMP(Linux + Huawei LiteOS)
Hi3516CV500	SMP(Linux)/AMP(Linux + Huawei LiteOS)

对于 Hi3559AV100ES：SVP 布局在双核 A73 和双核 A53 组成的 big.LITTLE 上，芯片媒体业务布局在单核 A53 上，智能业务需要的图像数据是通过核间通信从单核 A53 上获取。具体多核业务开发可以参考《多核 使用指南》和 SDK 包提供的 sample。

1.6 相关文档

《Hi35xx Vxxx ultra-HD Mobile Camera SoC 用户指南》

《HiSVP API 参考》

《HiMPP V4.0 媒体处理软件开发参考》

《多核 使用指南》



2 DSP 开发指南

2.1 开发工具介绍

Xtensa Xplorer 是 Cadence 为客户提供的针对其 DSP 进行软件开发的一个集成开发环境，其包含了软件开发(software development)、编译(compile)、调试(debugging)、仿真(simulation)、性能分析(profiling)、硬件跟踪(hardware trace)等功能。

Xtensa Xplorer 的安装使用可参考 Cadence 提供的官方文档，以下均简称为 Xplorer。

由于 SVP 在不同芯片上可能使用不同的 DSP；即便是相同型号的 DSP，硬件配置也可能不一样。在有 DSP 的芯片配套开发包中，海思为用户提供了与硬件相同配置的配置核供用户在 Xplorer 下仿真开发。

配置核在 HiSVP_PC_Vx.x.x.x 的 tool 目录下与芯片中 DSP 的型号一致的目录中，如 tool/vp6，其中 “*_linux.tgz” 和 “*_win32.tgz” 分别表示 linux、windows 版本上 Xplorer 的配置核。

2.2 DSP 工具链和配置核版本

表 2-1 是不同芯片上的 DSP 对应的工具链和配置核说明，注意 linux、win32 分别表示 linux、windows 环境下的对应的工具链和配置核。

表2-1 不同芯片的 DSP 工具链和配置核

芯片	工具链	配置核
Hi3559AV100	XtensaTools_RG_2016_4_linux.tgz	VP60_1124_linux.tgz
Hi3559CV100	XtensaTools_RG_2016_4_win32.tgz	VP60_1124_win32.tgz
Hi3519AV100	XtensaTools_RG_2017_7_linux.tgz	VP60_TIE_1012_CHILD_linux.tgz
Hi3556AV100	XtensaTools_RG_2017_7_win32.tgz	VP60_TIE_1012_CHILD_win32.tgz
Hi3516DV300	不支持	不支持
Hi3516CV500	不支持	不支持



注意

- 工具链可以在 Xplorer 中下载，所以不再发布包中提供；
- HiSVP_PC_Vx.x.x.x 的 tool 中对相同配置的 DSP 只提供了 1 个 DSP 配置核，若多于 1 个表示芯片中有不同配置的 DSP。

2.3 Windows 环境下安装 DSP 工具链和配置核

本文假定用户在 windows 环境中已经安装好了 Xplorer7.0.4，并以 Hi3559AV100 VP6 配置核 VP60_1124 为例进行配置核安装，不同芯片不同配置核的在不同 Xplorer 版本的安装类似。

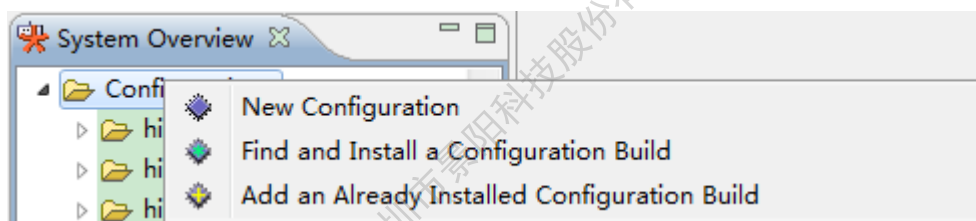


注意

Hi3559AV100 VP6 配置核 VP60_1124 需与 RG2016.4 版本的工具包（见表 2-1 XtensaTools_RG_2016_4_win32.tgz）配套使用，Xplorer7.0.4 默认集成了 RG2016.4 工具包，如用户使用其他版本的 Xplorer，需要另外安装 RG2016.4 工具包。

- 步骤 1. 打开 Xplorer 7.0.4，右键 System Overview 窗口下的 Configurations，如图 2-1 所示，点击 Find and Install a Configuration Build；

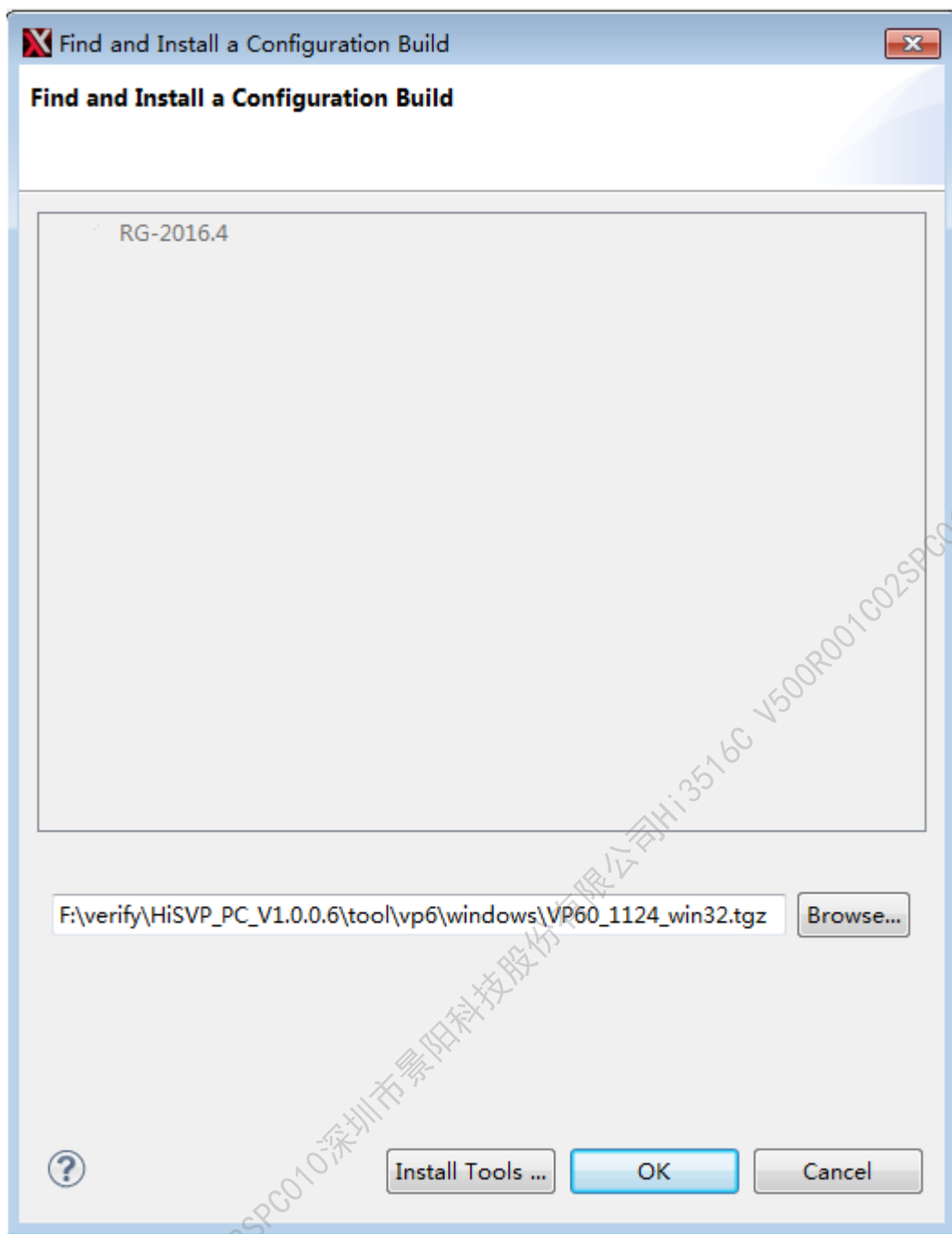
图2-1 System Overview



- 步骤 2. 进入 Find and Install a Configuration Build 界面，点击“Browse”，选择待安装配置核（此处以 VP60_1124_win32 为例），点击“OK”即开始安装；



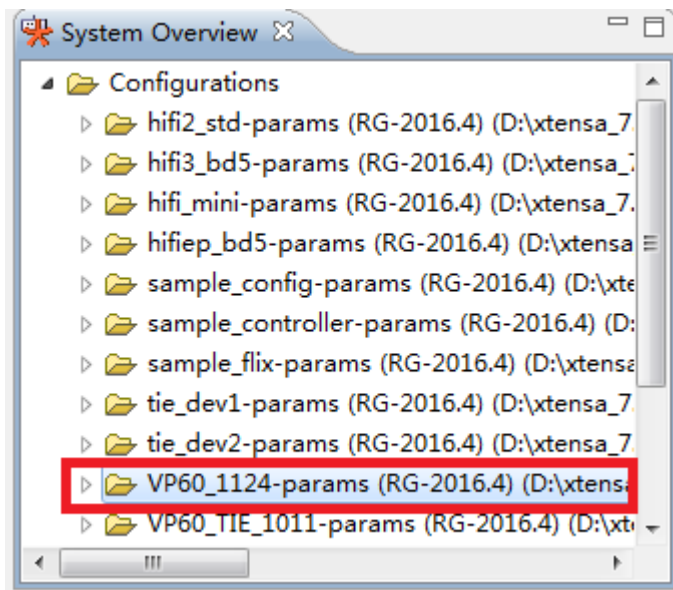
图2-2 Find and Install a Configuration Build



安装好后，在 SystemOverview 界面的 Configurations 下会出现刚刚安装的配置核，如图 2-3 所示，至此，VP60_1124 配置核安装成功。

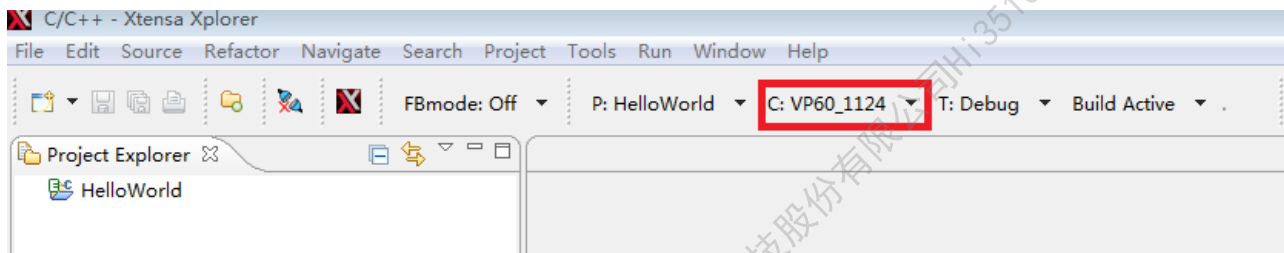


图2-3 查看安装好的配置核



步骤 3. 运行工程（图 2-4 以 HelloWorld 为例）时，按照图 2-4 选择 Configuration 为图 2-4 中的 C:VP60_1124，即表示使用的是海思提供的配置核。

图2-4 创建的工程界面



----结束

2.4 修改 memmap 配置

用户可以根据自己的开发环境来修改 memory map。详细过程参考 Xtensa® Linker Support Packages (LSPs) Reference Manual 的第二章。

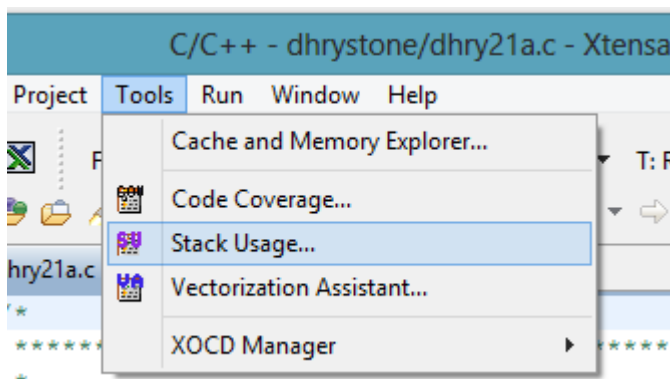
2.5 查看 stack 使用

具体操作步骤如下：

步骤 1. 点击 Xplorer 的 Tools->Statck Usage 选项。

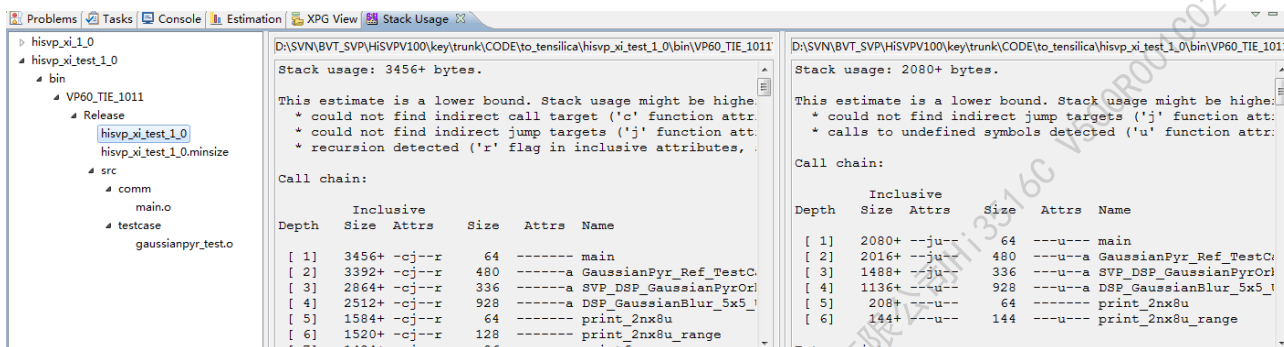


图2-5 查看栈信息操作界面



步骤 2. 此时会出现 Stack Usage 的窗口。选择一个工程，右键点击 Expand All，选择可执行文件右键点击“Show in Left Panel”或“Show in Right Panel”查看该文件的 Stack 使用情况。

图2-6 显示栈信息窗口



----结束

2.6 Linux 环境下安装 DSP 工具链和配置核

在 Linux 环境下安装 Cadence DSP 的工具链和配置核可参考 Cadence 提供的官方文档 dev_tools_install_guide.pdf。用户在利用海思提供的 SDK 包进行板端 DSP 程序开发时，推荐按照以下方式安装 DSP 的工具链和配置核。

2.6.1 安装 DSP 工具链

下面以 Xplorer-7.0.4-linux-installer.bin 版本介绍工具链的安装。步骤如下：

步骤 1. 首先需要以 root 权限在服务器上创建目录/opt/xtensa。

步骤 2. 把 Xplorer-7.0.4-linux-installer.bin 安装包拷贝到/opt/xtensa 目录下，然后执行安装./Xplorer-7.0.4-linux-installer.bin，如下面所示：

```
root@SZX1000040224:/opt/xtensa# ./Xplorer-7.0.4-linux-installer.bin
```



Welcome to the Xtensa Xplorer Setup Wizard.

Please read the following License Agreement. You must accept the terms of this agreement before continuing with the installation.

Press [Enter] to continue :
Cadence Tools Software Use Agreement

Your use of the software you are about to install is subject to one or more licensing agreements.

Portions of this software are subject to the terms of either (1) a technology license agreement between Cadence and the user ("you") - a direct Cadence licensee, (2) an end user license agreement between an existing Cadence licensee and you, or (3) a limited use evaluation agreement between Cadence and you. In addition, each directory contains files that identify any open source licenses or copyrights that apply to each component.

For a copy of the license agreement that applies to your use of this software, please make an inquiry to the organization that provided this copy of the software to you.

Press [Enter] to continue :

Do you accept this license? [y/n]:y

Xtensa Xplorer Installation Directory

Please enter the path to the Xtensa Xplorer root directory. The Xtensa Xplorer 7.0.4 and XtDevTools directories will be installed in this directory.

It is recommended that you use the same Xplorer root directory as your previous installations of Xplorer so that the XtDevTools directory can be shared,



which
will allow this version of Xplorer to use all previous installations of
Xtensa
configurations and tools.

Xtensa Xplorer Root Directory [/opt/xtensa]:

Warning: The installer did not detect an XtDevTools directory in the
location that you specified. If you have previously installed any
version of Xtensa Xplorer, please install this version in the same root
directory so this version can share the same XtDevTools tree.

Do you have an existing installation of Xtensa Xplorer?

Click "Yes" to specify the previous Xplorer root directory
location(usually /usr/xtensa), otherwise click "No" if this is your
first installation of Xplorer.

[Y/n]: n

Select the components you want to install; clear the components you do
not want
to install. Click Next when you are ready to continue.

Xplorer and Xtensa Development Tools : Y (Cannot be edited)

General Sample Configs [Y/n] :y

Xtensa Fusion F1 DSP [Y/n] :y

Xtensa HiFi Family Samples [Y/n] :y

Xtensa TIE base examples builds [Y/n] :y

Xtensa ConnX D2 DSP Engine Reference Cores [Y/n] :y

Is the selection above correct? [Y/n]: y

Xplorer components selected

Selected the following components:

+ Xtensa Xplorer and Development Tools



- + Xtensa Fusion F1 DSP core
- + Xtensa HiFi Family Samples
- + Xtensa ConnX D2 DSP Engine Reference Cores
- + Xtensa TIE base example
- + Xtensa General Samples

Press [Enter] to continue :

Disk Space Report

Installation space report

Required disk space is : 3640 MB

Current disk has free space : 7570 MB

Press [Enter] to continue :

Installation Summary

Xtensa products will be installed as follows

Xtensa Xplorer will be installed to:

/opt/xtensa/Xplorer-7.0.4

Xtensa Tools will be installed to:

/opt/xtensa/XtDevTools/install/tools/RG-2016.4

Xtensa Tools and samples bundles will be stored at:

/opt/xtensa/XtDevTools/downloads/RG-2016.4

Xtensa Xplorer workspace default location:

/opt/xtensa/Xplorer-7.0.4-workspaces

Press [Enter] to continue :

Pre-installation Message



The Xplorer installer runs in 2 phases. The last phase (post-installation) is installing tools and any configurations selected, and may run for several minutes without appearing to make progress. Please be patient.

Press [Enter] to continue :

Setup is now ready to begin installing Xtensa Xplorer on your computer.

Do you want to continue? [Y/n]: y

Please wait while Setup installs Xtensa Xplorer on your computer.

Installing

0% _____ 50% _____ 100%

#####

Post Installation Script Result

Congratulations !! You have finished installing Xplorer-7.0.4

Please review the following message log to make sure of the success of the installation.

=====

```
INSTALLING XtensaTools
===== LOCATE utils plugin =====
WHERE_UTILS_RESULT=/opt/xtensa/Xplorer-
7.0.4/eclipse/plugins/other.xide.external.
utils_7.0.4.2000
INSTALL XTTOOLS COMMAND :: /opt/xtensa/Xplorer-7.0.4/eclipse/jre/bin/java
-cp
/opt/xtensa/Xplorer-
7.0.4/eclipse/plugins/other.xide.external.utils_7.0.4.2000/ut
ils.jar other.xide.external.utils.io.Unpack
/opt/xtensa/XtDevTools/downloads/RG-
2016.4/tools/XtensaTools_RG_2016_4_linux.tgz
/opt/xtensa/XtDevTools/install/tools/
/opt/xtensa/Xplorer-
7.0.4/eclipse/plugins/other.xide.external.utils_7.0.4.2000
INSTALL XtensaTools RESULT :: 0 , SUCCESS
===== LOCATE dynamic plugin files =====
```



```
INSTALL XOS Document Plugin
WHERE_XOS_RESULT=/opt/xtensa/XtDevTools/install/tools/RG-2016.4-
linux/XtensaTools
/doc/xos-1.12.zip
INSTALL XOS COMMAND :: /opt/xtensa/Xplorer-7.0.4/eclipse/jre/bin/java -cp
/opt/xtensa/Xplorer-
7.0.4/eclipse/plugins/other.xide.external.utils_7.0.4.2000/ut
Press [Enter] to continue :
ils.jar other.xide.external.utils.io.Unpack
/opt/xtensa/XtDevTools/install/tools/RG-2016.4-linux/XtensaTools/doc/xos-
1.12.zip
/opt/xtensa/Xplorer-7.0.4/eclipse/plugins/
/opt/xtensa/Xplorer-
7.0.4/eclipse/plugins/other.xide.external.utils_7.0.4.2000
INSTALL XOS HELP PLUGIN RESULT :: 0 , SUCCESS
INSTALL XIPC Document Plugin
WHERE_XIPC_RESULT=/opt/xtensa/XtDevTools/install/tools/RG-2016.4-
linux/XtensaTool
s/doc/xipc.zip
INSTALL XIPC COMMAND :: /opt/xtensa/Xplorer-7.0.4/eclipse/jre/bin/java -
cp
/opt/xtensa/Xplorer-
7.0.4/eclipse/plugins/other.xide.external.utils_7.0.4.2000/ut
ils.jar other.xide.external.utils.io.Unpack
/opt/xtensa/XtDevTools/install/tools/RG-2016.4-
linux/XtensaTools/doc/xipc.zip
/opt/xtensa/Xplorer-7.0.4/eclipse/plugins/
/opt/xtensa/Xplorer-
7.0.4/eclipse/plugins/other.xide.external.utils_7.0.4.2000
INSTALL XIPC HELP PLUGIN RESULT :: 0 , SUCCESS

Checking XtensaRegistry dir...
Check XtensaRegistry RG-2016.4 DIR
DONE:: /opt/xtensa/XtDevTools/XtensaRegistry/RG-2016.4-linux

INSTALLING User selected Xtensa config builds from
XtDevTools/downloads/RG-2016.4/builds IF EXISTS
=====
Press [Enter] to continue :
UNPACK CONFIG RESULT :: 0, SUCCESS
INSTALL CONFIG XRC_D2PM RESULT :: 0, SUCCESS
=====
UNPACK CONFIG RESULT :: 0, SUCCESS
```



```
INSTALL CONFIG XRC_FUSION_AON_ALL_LM RESULT :: 0, SUCCESS
```

```
=====
UNPACK CONFIG RESULT :: 0, SUCCESS
INSTALL CONFIG hifi2_std RESULT :: 0, SUCCESS
```

```
=====
UNPACK CONFIG RESULT :: 0, SUCCESS
INSTALL CONFIG hifi3_bd5 RESULT :: 0, SUCCESS
```

```
=====
UNPACK CONFIG RESULT :: 0, SUCCESS
INSTALL CONFIG hifiep_bd5 RESULT :: 0, SUCCESS
```

```
=====
UNPACK CONFIG RESULT :: 0, SUCCESS
INSTALL CONFIG hifi_mini RESULT :: 0, SUCCESS
```

```
Press [Enter] to continue :
```

```
=====
UNPACK CONFIG RESULT :: 0, SUCCESS
INSTALL CONFIG sample_config RESULT :: 0, SUCCESS
```

```
=====
UNPACK CONFIG RESULT :: 0, SUCCESS
INSTALL CONFIG sample_controller RESULT :: 0, SUCCESS
```

```
=====
UNPACK CONFIG RESULT :: 0, SUCCESS
INSTALL CONFIG sample_flix RESULT :: 0, SUCCESS
```

```
=====
UNPACK CONFIG RESULT :: 0, SUCCESS
INSTALL CONFIG tie_dev1 RESULT :: 0, SUCCESS
```

```
=====
UNPACK CONFIG RESULT :: 0, SUCCESS
INSTALL CONFIG tie_dev2 RESULT :: 0, SUCCESS
```

```
Setting Xplorer configuration and then initializing configuration cache
INITIALIZE Xtensa Xplorer RESULT :: 0, SUCCESS
Press [Enter] to continue :
```

```
-----
Setup has finished installing Xtensa Xplorer on your computer.
```



```
Run Xtensa Xplorer now (Recommended)
(This initializes workspace location defaults) [Y/n]:n
```

----结束

2.6.2 安装 DSP 配置核

下面以 VP60_1124 核为参考说明配置核的安装，其他配置核安装过程类似。

步骤 1. 在 root 权限下，把 VP60_1124_linux.tgz 压缩包拷贝到之前创建的/opt/xtensa 目录，然后解压压缩包，命令 `tar -zxf VP60_1124_linux.tgz`，解压出/opt/xtensa/RG-2016.4-linux/VP60_1124/目录。

步骤 2. 进入/opt/xtensa/RG-2016.4-linux/VP60_1124/目录安装配置核，执行./install。

如下所示：

```
root@SZX1000040224:/opt/xtensa/RG-2016.4-linux/VP60_1124# ./install
```

```
Xtensa Processor Configuration Installation Tool
Copyright (c) 2001-2016 Tensilica Inc.
For Xtensa Tools Version 12.0.4
```

```
Before you can use a new Xtensa processor configuration, you must run
this tool to complete the installation. Two separate packages are
required:
```

```
1) The Xtensa Tools cross-development software toolkit package. These
tools are configuration-independent and are shared by all your Xtensa
processor configurations. You do not need a separate copy for each
configuration.
```

```
2) The configured Xtensa processor files, of which this script is a
part.
```

```
You must have already downloaded both packages and extracted the files
on your system before you can continue.
```

```
Are you ready to proceed? [y] y
Continuing...
```

```
Enter the path to the Xtensa Tools directory:
/opt/xtensa/XtDevTools/install/tools/RG-2016.4-linux/XtensaTools
```

```
The files for this Xtensa processor configuration will now be set up
to work with the installation directories that you have chosen. This
```



process will take a few minutes, and once it has begun the installation directories cannot be changed. If you abort this script after this point, or if you need to change the installation directories for some reason, you will need to start over with the original files that you downloaded for this configuration. (The Xtensa Tools files are not modified in this process so you do not need to reinstall the Xtensa Tools package.) The directories to be used are:

```
Xtensa Tools:          /opt/xtensa/XtDevTools/install/tools/RG-2016.4-  
linux/XtensaTools
```

```
Configured Processor: /opt/xtensa/RG-2016.4-linux/VP60_1124/.
```

```
Do you want to continue? [y] y
```

The files for this processor configuration have now been set to use the directory names you have chosen.

The next installation step is to add this processor configuration to the list of available configurations in a registry of Xtensa cores. The configuration will be registered with the default name, which is the Core ID from the Xtensa Processor Generator. You must ensure that each core in the registry has a unique name. Please refer to the "Xtensa Software Development Toolkit User's Guide" to learn how to register this configuration with a different name.

This script will update only one registry of Xtensa cores, and in most cases, you should use the default Xtensa registry. If you are sharing the Xtensa Tools installation with others, and you do not want this processor configuration to be shared, you can specify an alternate registry. Please refer to the "Xtensa Software Development Toolkit User's Guide" for instructions on adding this configuration to additional Xtensa core registries.

The default registry is:

```
/opt/xtensa/XtDevTools/install/tools/RG-2016.4-linux/XtensaTools/config
```

```
What registry would you like to use? [default]
```

```
/opt/xtensa/XtDevTools/XtensaRegistry/RG-2016.4-linux
```

```
Do you want to make "VP60_1124" the default Xtensa core? [y] y
```

The installation process is now complete.



注意

- 使用海思 SDK 开发时配置核路径最好要安照上面的安装路径指定，否则在开发 DSP 程序时，需要针对实际安装路径去修改 makefile，指定工具链和配置核路径才能编译（可参考 Cadence 提供的《xtensa_xcc_compiler_ug.pdf》文档）
- 如果不同芯片的 DSP 依赖不同 Xtensatools，则需要把要依赖的 Xtensatools 压缩包放到这个路径（/opt/xtensa/XtDevTools/install/tools）进行解压，安装配置核的时候选择这个 Xtensatools 路径。

----结束

2.6.3 配置环境变量

在/etc 目录下打开 profile 文件，添加以下环境变量配置：

- 设置 Cadence license:
`export XTENSAD_LICENSE_FILE=port@serverip`
例如：port 是 28001, serverip 是 192.168.1.100
- 设置配置核参数路径：
`export XTENSA_SYSTEM=/opt/xtensa/XtDevTools/XtensaRegistry/RG-2016.4-linux`
- 设置默认的配置核：
`export XTENSA_CORE=VP60_1124`
- 设置交叉编译工具链路径：
`export PATH="/opt/xtensa/XtDevTools/install/tools/RG-2016.4-linux/XtensaTools/bin:$PATH"`

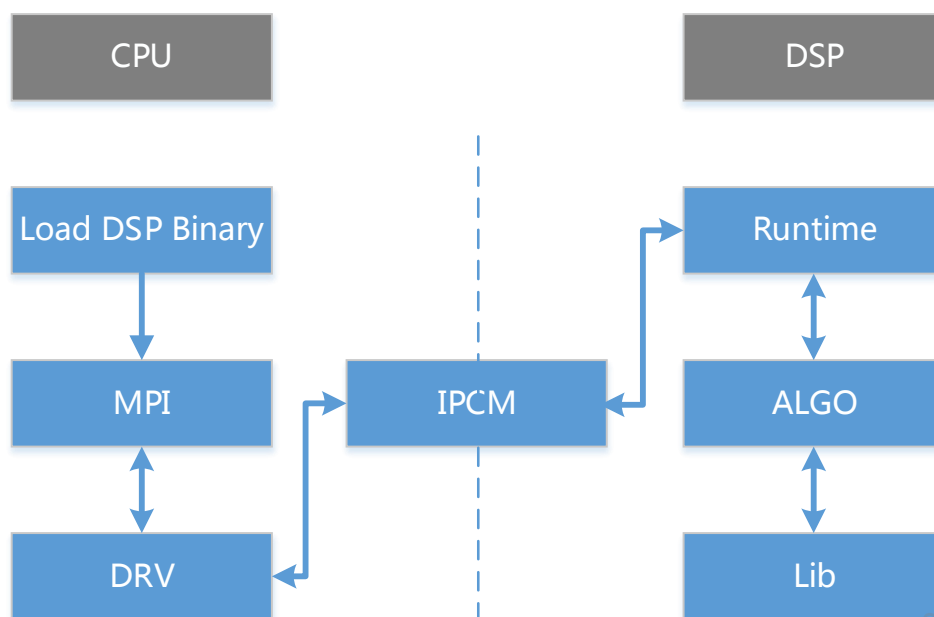
2.7 开发流程

DSP 开发分为 PC 端和板端环境：

- PC 端环境：利用 Xplorer 集成工具进行 DSP 算法开发，利用 Xplorer 集成工具提供的仿真环境可以快速实现对算法的开发和验证，开发完算法代码可以直接移植到板端编译调试。Xplorer 环境参考前面的介绍。
- 板端环境：基于海思提供的 SDK 开发包进行板端程序开发。

DSP 应用程序开发框图如图 2-7 所示。

图2-7 DSP 应用程序开发框图



如图 2-7 所示，DSP 程序的开发和使用分为几部分：

- 在 CPU 端
 - Load DSP Binary：调用 HI_MPI_SVP_DSP_LoadBin 加载 DSP 可执行程序，并且调用 HI_MPI_SVP_DSP_EnableCore 使能 DSP。
 - MPI：DSP MPI 接口。
 - DRV：驱动模块。
 - IPCM：核间通信模块。
- 在 DSP 端
 - Runtime：DSP 上算子的调度管理。
 - ALGO：解析参数，调用 DSP 算子。
 - Lib：DSP 算子库。
 - IPCM：核间通信模块。

2.7.1 开发自定义算子

- 在 CPU 端

步骤 1. 在程序初始化时调用 HI_MPI_SVP_DSP_LoadBin 把 4 个 bin（hi_iram.bin、hi_sram.bin、hi_dram0.bin、hi_dram1.bin）分别加载至对应位置。

步骤 2. 调用 HI_MPI_SVP_DSP_EnableCore 使能 DSP。

步骤 3. 开发基于 HI_MPI_SVP_DSP_RPC/HI_MPI_SVP_DSP_Query 接口封装自定义算子



说明

请参考 Sample(HI_MPI_SVP_DSP_ENCA_Dilate3x3/HI_MPI_SVP_DSP_ENCA_Erode3x3 封装)和《HiSVP API 参考.pdf》接口 HI_MPI_SVP_DSP_RPC/HI_MPI_SVP_DSP_Query 说明。

----结束

- 在 DSP 端

步骤 1. 在发布包 algo/include/int 目录下开发基于帧级或者 Tile 级的算子。



说明

用户可参考 algo/include/int/svp_dsp_frm.h 和 algo/src/svp_dsp_frm.h 开发帧级算子。

步骤 2. 在发布包 algo/src/svp_dsp_algo.c 源文件 SVP_DSP_ALGO_Process 函数添加对自定义算子的调用。

步骤 3. 执行 algo/Makefile 重新对 algo 目录下的代码编译成库。

步骤 4. 执行 runtime/obj/Makefile 把.o、.a 等编译成 elf 文件并转换为 4 个 binary。



说明

4 个 bin 分别为 hi_iram.bin、hi_sram.bin、hi_dram0.bin 和 hi_dram1.bin，并且放在 runtime/obj/bin。

----结束



注意

- 在不同的芯片中 DSP 的个数会不同，而且 IRAM/SRAM/DRAM 的编址也会不一样。具体请参考对应芯片手册。
- 由于逻辑增加了 ReorderBuf，导致 IDMA 有一个使用约束，IDMA 的编程接口 idma_init 函数只能使用 MAX_BLOCK_2/MAX_BLOCK_4/MAX_BLOCK_8，不能使用 MAX_BLOCK_16。
- DSP 的 IDMA 访问的 DDR 地址必须是匿名 MMZ 区域的地址。对于 Hi3559AV100，假设匿名 MMZ 区域首地址是 MMZ_ADDR，则 DSP 的 IDMA 能访问的 DDR 地址范围是 MMZ_ADDR ~ MMZ_ADDR + 3GB - 64MB。



3 NNIE 开发指南

3.1 NNIE 介绍

NNIE 是 Neural Network Inference Engine 的简称，是海思媒体 SoC 中专门针对神经网络特别是深度学习卷积神经网络进行加速处理的硬件单元，支持现有大部分的公开网络，如 Alexnet、VGG16、Googlenet、Resnet18、Resnet50 等分类网络，Faster R-CNN、YOLO、SSD、RFCN 等检测网络，以及 SegNet、FCN 等场景分割网络。

目前 NNIE 配套软件及工具链仅支持以 Caffe 框架，使用其他框架的网络模型需要转化为 Caffe 框架下的模型。

3.1.1 工具链介绍

SVP NNIE 在 HiSVP_PC_Vx.x.x.x.rar 组件包中，提供如下的工具链：

- **nnie_mapper(tool\nnie\linux\mapper 目录)：**简称 **mapper**，该工具将用户开通过开源深度学习框架训练得到的模型转化成在 Hi35xx 芯片上或者在仿真库中可以加载的数据指令文件（文件后缀为 **wk**，后文中提到的 **wk** 文件即由该工具生成）。
- **仿真库(software\x64 目录)：**模拟 NNIE 的硬件执行和软件接口调用，在“相同的输入”下仿真库与硬件得到相同的结果。仿真库可以使用户脱离硬件在 PC 环境下仿真，且便于调试，有助于用户提前快速开发算法原型。仿真库有基于 Visual Studio 和 MinGW 版本，后者集成在 RuyiStudio 中。
- **仿真 Sample 工程(software\sample_simulator 目录)：**，包含仿真 **sample** 源代码供开发者学习参考，支持 Visual Studio 或 RuyiStudio 环境运行。
- **模型包(software\data)：**包含若干 **sample** 中用到的网络的 **caffe** 模型文件及对应的 NNIE mapper 配置文件、**wk** 文件、图像文件等。
- **Windows 版 IDE(tool\nnie\windows 目录)工具 RuyiStudio，**集成 Windows 版的 NNIE mapper 和仿真库，用户可以将仿真 **Sample** 工程导入运行、调试；IDE 还集成了代码编辑、编译、调试、执行、画框、相似度比对等功能，具体参考 5 “[RuyiStudio 工具使用指南](#)” 章节。



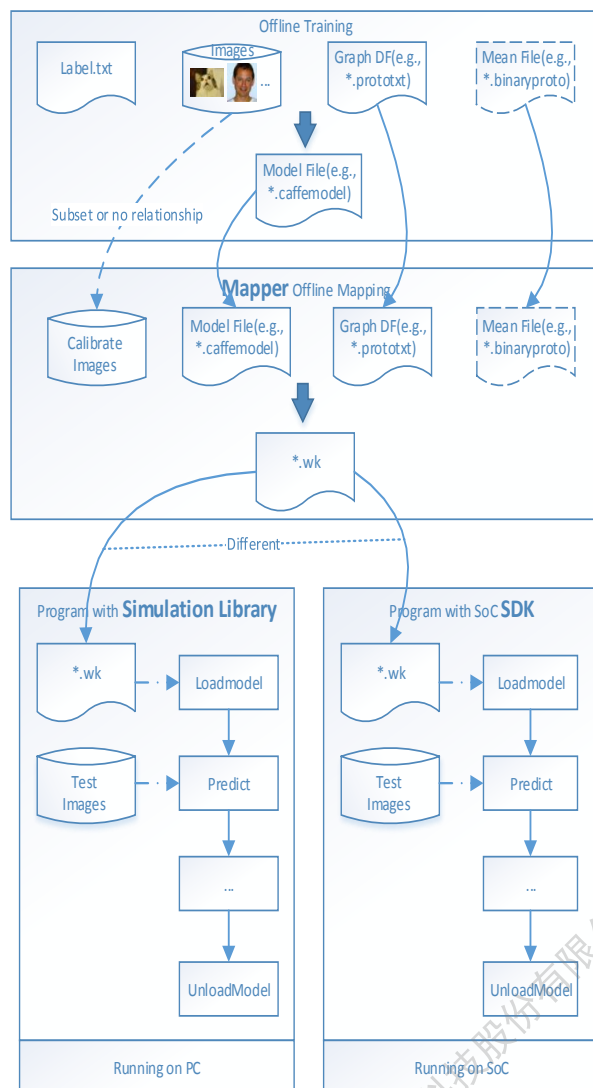
注意

- “相同的输入”是指相同的原始模型，相同的图像数据输入。由 mapper 转化得到的仿真库上的模型和在芯片上的是不相同的。
- NNIE 工具链目前只支持 Caffe 框架，且以 Caffe-1.0 版本为基础。

3.1.2 开发流程

以 Caffe 框架上训练的模型为例，NNIE 的开发流程如图 3-1 所示。在 Caffe 上训练、使用 NNIE 的 mapper 工具转化都是离线的。通过设置不同的模式，mapper 将 *.caffemodel 转化成在仿真器、仿真库或板端上可加载执行的数据指令文件。一般在开发前期，用户可使用仿真器对训练出来的模型进行精度、性能、带宽进行初步评估，符合用户预期后再使用仿真库进行完整功能的仿真，最后将程序移植到板端。

图3-1 NNIE 软件开发流程



3.1.3 网络层的分类

一个网络的层可分为如下的 3 类：

- 标准层：NNIE 支持的 Caffe 标准层，比如 Convolution，Pooling 层等；
- 扩展层：NNIE 支持的公开但非 Caffe 标准层，分为 2 种：
 - 一种是基于 Caffe 框架进行自定义扩展的层，比如 Faster RCNN 中的 ROI Pooling 层、SSD 中 Normalize 层、RFCN 中的 PSROI Pooling 层，SegNet 中的 UpSample 层等；
 - 另外一种来源于其他深度学习框架的自定义层，比如 YOLOv2 中 Passthrough 层等；
- Non-support 层：NNIE 不支持的层，比如 Caffe 中专用于 Training 的层、其他非 Caffe 框架中的一些层或者用户自定义的私有层等。



3.1.4 扩展层规则

Faster RCNN、SSD、RFCN 和 SegNet 等网络都包含了一些原始 Caffe 中没有定义的结构，如 ROI Pooling、Normalize、PSROI Pooling 和 Upsample 等。NNIE 的 mapper 目前仅支持 Caffe 框架，且以 Caffe-1.0 为基础。为了使 mapper 能支持这些网络，需要对原始的 Caffe 进行扩展。



注意

针对使用了扩展层的网络，用户可以二选一操作：

- 按照 3.1.4 节扩展 Caffe 的 caffe.proto 以及扩展层代码实现，并以此基础来进行训练，对应部署的 deploy.prototxt 满足 mapper 要求。
- 使用开源的扩展 caffe 框架来训练（比如 rbgirshick/py-faster-rcnn 等），不需要扩展 caffe.proto 以及扩展层代码实现，但是需检查对应的扩展层规格是满足要求，且最后输入给 mapper 的 deploy.prototxt 必须满足 3.1.4.3 中要求以便 nnie_mapper 识别。

3.1.4.1 扩展 Caffe proto 文件

扩展层可能涉及到一些相关的输入参数，因此需要扩展 Caffe 的 proto 文件，使得 Caffe 在解析 prototxt 文件时能够识别出对应的参数声明。

以 ROI Pooling 层为例，需要修改 Caffe 源代码 src/caffe/proto/caffe.proto 文件：

- 1) 首先需要在 LayerParameter 这个 message 中添加自定义层参数的声明，下图定义了 ROI PoolingParameter 这个参数类型，其名称为 roi_pooling_param，等号右边的数字可以是一个任意的目前 LayerParameter 中没有被使用的字段。

本文档中限定扩展层参数字段从 100000 开始。roi_pooling_param 为声明的该 parameter 对应的名称，可以在源代码中使用 layer_param.roi_pooling_param() 来获取该参数对应的数值。

图3-2 在 caffe.proto 文件中扩展 LayerParameter

```
optional ROI PoolingParameter roi_pooling_param = 100000;  
optional NormalizeParameter norm_param = 100001;  
optional PSROI PoolingParameter psroi_pooling_param = 100002;  
optional UpsampleParameter upsample_param = 100003;
```

- 2) 完成 ROI PoolingParameter 的参数声明后，还需要在 caffe.proto 文件中添加一个新的 message，内部定义该参数具体的字段名称、数据类型、默认数值等。如下图即定义了 ROI Pooling 层有 3 个可以配置的参数，分别为 pooled_h、pooled_w 和 spatial_scale。

图3-3 在 caffe.proto 文件中定义新 layer 的 message

```
// Message that stores parameters used by ROI Pooling Layer
message ROIPoolingParameter {
    // Pad, kernel size, and stride are all given as a single value for equal
    // dimensions in height and width or as Y, X pairs.
    optional uint32 pooled_h = 1 [default = 0]; // The pooled output height
    optional uint32 pooled_w = 2 [default = 0]; // The pooled output width
    // Multiplicative spatial scale factor to translate ROI coords from their
    // input scale to the scale used when pooling
    optional float spatial_scale = 3 [default = 1];
}
```

3.1.4.2 扩展层代码实现

经过上述 caffe.proto 文件的扩展之后，Caffe 就能够识别 prototxt 中层参数相关的设定。但若需要支持扩展层的解析、training、inference，还需要根据扩展层的处理方式，完成对应底层代码实现，如果该层处理要在 CPU 上运行，就要实现对应的 C++ 代码，如果该层处理要在 GPU 上运行，就要实现对应的 CUDA C/C++ 代码。

在实现代码的过程中，层的参数可以使用 Caffe 提供的接口函数，结合 proto 文件中声明的参数名称来获取。以 ROI Pooling 为例：

- 1) Faster RCNN 就在原始的 caffe 中添加了 ROI Pooling 层的 CPU 与 GPU 代码，分别为 roi_pooling_layer.cpp 和 roi_pooling_layer.cu，并将这两个文件放在了 Caffe 源代码的 src/caffe/layers 目录下。在源代码中的参数获取方式如下图，其中 roi_pooling_param 为 proto 文件中定义的 ROIPoolingParameter 的名称，pooled_h、pooled_w 和 spatial_scale 为该参数包含的 3 个具体参数的名称。获取了参数后，就可以在后续的代码实现中使用该参数。

图3-4 从 prototxt 中获取参数示例

```
template <typename Dtype>
void ROIPoolingLayer<Dtype>::LayerSetUp(const vector<Blob<Dtype>*> &bottom,
    const vector<Blob<Dtype>*> &top) {
    ROIPoolingParameter roi_pool_param = this->layer_param_.roi_pooling_param();
    CHECK_GT(roi_pool_param.pooled_h(), 0)
        << "pooled_h must be > 0";
    CHECK_GT(roi_pool_param.pooled_w(), 0)
        << "pooled_w must be > 0";
    pooled_height_ = roi_pool_param.pooled_h();
    pooled_width_ = roi_pool_param.pooled_w();
    spatial_scale_ = roi_pool_param.spatial_scale();
    LOG(INFO) << "Spatial scale: " << spatial_scale_;
}
```

- 2) 在实现底层代码的过程中，主要需要实现该层 Forward 与 Backward 对应的代码，并在文件末尾调用 Caffe 内建的层声明函数，如下图 roi_pooling_layer.cpp 的末尾。如果某些函数没有实现，在代码中加入 NOT_IMPLEMENTED 宏，INstantiate_Class 和 Register_Layer_Class 即为调用 Caffe 内建的层声明函数。

图3-5 声明自定义的层

```
template <typename Dtype>
void ROIPoolingLayer<Dtype>::Backward_cpu(const vector<Blob<Dtype>*>& top,
      const vector<bool>& propagate_down, const vector<Blob<Dtype>*>& bottom) {
    NOT_IMPLEMENTED;
}

#ifdef CPU_ONLY
STUB_GPU(ROIPoolingLayer);
#endif

INSTANTIATE_CLASS(ROIPoolingLayer);
REGISTER_LAYER_CLASS(ROIPooling);

} // namespace caffe
```

- 3) 上述的操作完成了一个名称为 ROIPooling 的 Layer 的声明。当然，代码还有对应的头文件，头文件统一放在 Caffe 源代码的 include/caffe 目录下，ROIPooling 层将头文件整个到了一个名称为 fast_rcnn_layers.hpp 的头文件中。如果有必要也可以将对应的头文件放在 include/caffe/layers 目录下，只要保证其路径的正确性使工程在编译的时候能够找到对应的文件即可。

图3-6 在实现文件中添加头文件

```
#include "caffe/fast_rcnn_layers.hpp"
```

- 4) 完成了上述步骤之后，就可以重新编译整个 Caffe 工程。成功编译后的 Caffe 就可以解析 prototxt 中自定义的层了。如下图为 Faster RCNN 的 prototxt 中包含 ROI Pooling 层的示例，其 layer 的 type 字段为 ROIPooling，并分别定义了 roi_pooling_param 参数中 pooled_w、pooled_h 和 spatial_scale 的具体数值。

图3-7 ROIPooling 层在 prototxt 中的示例

```
layer {
  name: "roi_pool5"
  type: "ROIPooling"
  bottom: "conv5_3"
  bottom: "rois"
  top: "pool5"
  roi_pooling_param {
    pooled_w: 7
    pooled_h: 7
    spatial_scale: 0.0625 # 1/16
  }
}
```


3.1.4.3 扩展层在 prototxt 中的定义

扩展层的实现可以在 Caffe 或其他深度学习框架中实现，扩展层中的第一种是基于 Caffe 框架进行扩展实现的，如 ROI Pooling、Normalize、PSROI Polling 和 Upsample 层等，因此有公开的 prototxt 标准定义。而扩展层中的第二种并不是在 Caffe 框架下实现的，对于这类网络中的自定义层，也需要给出 prototxt 的标准定义，用于在 prototxt 中定义网络中相应的层。

CReLU 层

CReLU 层为 PVANet 中特殊的层结构，其结构如下，在 Caffe 中并没有标准的 CReLU 层作为单独的一层。

注意：在 CReLU 最早提出的论文中《Understanding and Improving Convolutional Neural Networks via Concatenated Rectified Linear Units》(<http://cn.arxiv.org/abs/1603.05201>)，并没有如图 3-8 所述的 Scale/Shift 层，也即没有训练参数。不过，在 NNIE mapper 支持的单独 CReLU 层的实现中，是以 PVANet 网络中的方式来实现，Caffemodel 模型文件中需要包含 Scale 层对应的参数。如果要使用不同与此方式实现的 CReLU 层，可以通过多个 Caffe 标准层组合实现。

图3-8 CReLU 层实现示例

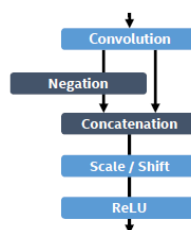


Figure 1: Our C.ReLU building block. **Negation** simply multiplies -1 to the output of Convolution. **Scale / Shift** applies trainable weight and bias to each channel, allowing activations in the negated part to be adaptive.

虽然没有一个独立的 CReLU 层，不过该层所实现的操作可以使用 Caffe 中已有的标准层来进行实现，Negation 使用标准的 Power 层，参数配置为 power 为 1，scale 为 -1.0，shift 为 0；Concatenation 使用标准的 Concat 层将原始的 feature map 和取反后的 feature map 进行拼接；Scale/Shift 使用标准的 Scale 层实现；ReLU 同样使用标准的层实现。

综上，一个 CRelu 层可以使用如下的 prototxt 定义，使用 Caffe 中标准层进行实现。



```
layer {
  name: "conv1_1/neg"
  type: "Power"
  bottom: "conv1_1/conv"
  top: "conv1_1/neg"
  power_param {
    power: 1
    scale: -1.0
    shift: 0
  }
}
layer {
  name: "conv1_1/concat"
  type: "Concat"
  bottom: "conv1_1/conv"
  bottom: "conv1_1/neg"
  top: "conv1_1"
}
```

```
layer {
  name: "conv1_1/scale"
  type: "Scale"
  bottom: "conv1_1"
  top: "conv1_1"
  scale_param {
    bias_term: true
  }
}
layer {
  name: "conv1_1/relu"
  type: "ReLU"
  bottom: "conv1_1"
  top: "conv1_1"
}
```

除了采用多个 Caffe 中标准层组合的方式实现外，NNIE mapper 也支持解析一个独立定义的 CReLU 层，CReLU 层在 prototxt 中进行定义的方式示例如下，layer 的 type 定义为 CReLU。

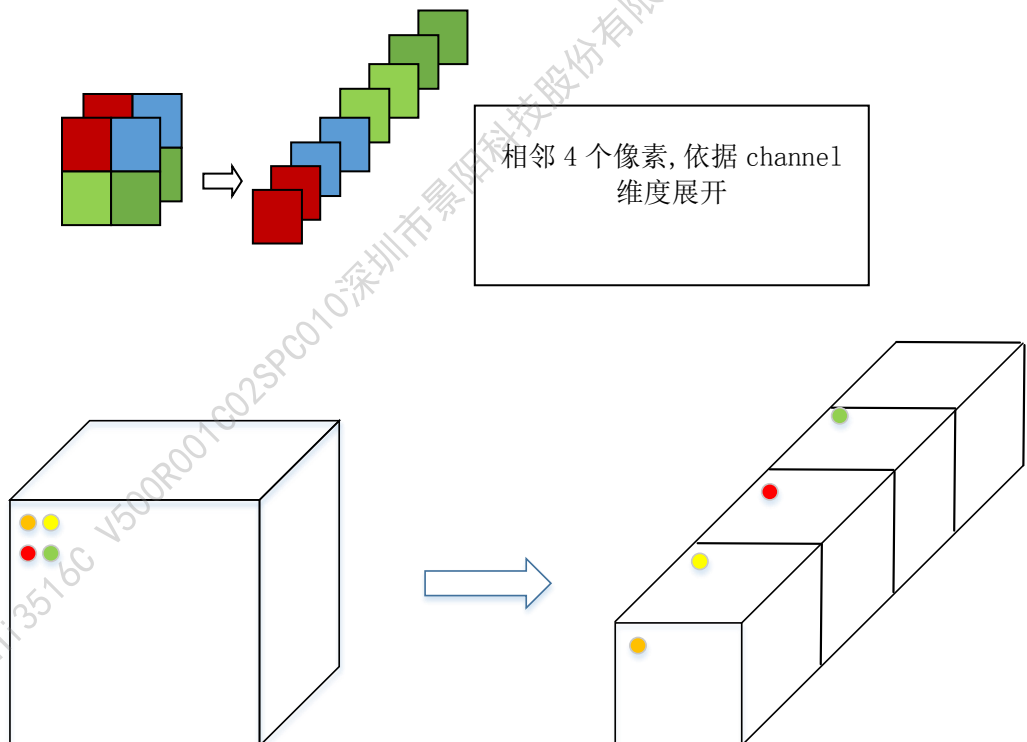
```
layer {  
  name: "crelu_layer"  
  type: "CReLU"  
  bottom: "some_input"  
  top: "some_output"  
  scale_param {  
    bias_term: true  
  }  
}
```

CReLU 的操作为上述限定的操作，由于 Power、Concat 层的参数都是限定的，ReLU 层不包含参数，因此仅仅需要配置 `scale_param`，即 Scale 层对应的参数。`scale_param` 可以复用标准 Scale 层的参数配置，因此不需要对 `caffe.proto` 文件进行扩展，只需要开发 CReLU 层对应的层实现代码即可。

PassThrough 层

PassThrough 层为 Yolo v2 中的一个自定义层，由于 Yolo v2 并不是使用 Caffe 框架实现，因此对于该层没有标准的定义。该层实现的功能为将 feature map 在 spatial 维度上的数据展开到 channel 维度上，原始在 channel 维度上连续的元素在展开后的 feature map 中依然是连续的。如将 $26 \times 26 \times 512$ 的 feature 变成 $13 \times 13 \times 2048$ 的 feature，做法为将相邻的像素展开到 channel 维度，示意图如图 3-9。

图3-9 PassThrough 层操作示意图



该层操作在实现过程中需要给定的参数包括在 spatial 维度上进行展开的窗口大小，以下称为 block，需要定义该 block 窗口的高度 height 和宽度 width，两者均为正整数，分别即为 block_height 和 block_width，且需要保证 block_height 和 block_width 能够被输入 feature map 的 height 和 width 整除。若输入 feature map 的 channel 维度为 num_in，则输出 feature map 的 channel 维度 num_output=num_in*block_height*block_width。如上述的示例中，block_height=2，block_width=2，num_output=2048。

在定义该层的参数时，需要 block_height、block_width 和输出 feature map 的 channel 维度数 num_output。虽然 num_output 可以通过 num_in、block_height、block_width 计算得出，但是在参数定义是还是需要进行设定，用于参数合法性的检查。

PassThrough 层在 prototxt 中进行定义的方式示例如下，layer 的 type 定义为 PassThrough。

```
layer {
  name: "pass_through"
  type: "PassThrough"
  bottom: "some_input"
  top: "some_output"
  pass_through_param {
    num_output: xxx
    block_height: xxx
    block_width: xxx
  }
}
```

为了使 caffe 能够对其进行解析，需要对 proto 文件进行扩展。在 caffe.proto 文件的 LayerParameter 中加入名称为 PassThroughParameter 的定义（下图中的 100004 为一个任意的当前 caffe.proto 的 LayerParameter 中没有被占用的数值）；并在文件中定义一个名称为 PassThroughParameter 的 message 定义。

图3-10 增加 PassThrough 层所需的对 caffe.proto 的修改

```
optional ROIPoolingParameter roi_pooling_param = 100000;
optional NormalizeParameter norm_param = 100001;
optional PSROIPoolingParameter psroi_pooling_param = 100002;
optional UpsampleParameter upsample_param = 100003;
optional PassThroughParameter pass_through_param = 100004;

// Message that stores parameters used by PassThroughLayer
message PassThroughParameter {
  //the number of channels in output feature map
  optional uint32 num_output = 1 [default = 0];
  //size of the spatial window block
  optional uint32 block_height = 2 [default = 0];
  optional uint32 block_width = 3 [default = 0];
}
```



NNIE 对该层的支持规格为在保证参数合法有效的情形下，`block_height` 可以为一个 1 到 255 之间的正整数，`block_width` 可以为一个 1 到 255 之间的正整数，且 `block_height` 和 `block_width` 可以不相等。

DepthwiseConv 层

Depthwise Convolution 层为 Xception 网络中的自定义层，在 `caffe` 框架中同样也是没有进行标准定义的。Depthwise Convolution 实现的操作为针对输入的每一个 `channel`，单独做 $K \times K$ 的卷积，假设输入的 `channel` 是 `M1`，输出结果的 `channel` 依然是 `M1`。因此，可以认为有 `M1` 个 $K \times K \times 1$ 的卷积 `kernel`。

由于该层的计算是基于标准卷积的一种变种，需要定义参数可以复用 `Caffe` 中卷积参数的定义，所以不需要对 `caffe.proto` 文件进行扩展，直接复用 `ConvolutionParameter` 即可。

Depthwise Convolution 层在 `prototxt` 中进行定义的方式示例如下，`layer` 的 `type` 定义为 `DepthwiseConv`。

```
layer {
  name: "depthwise_convolution"
  type: "DepthwiseConv"
  bottom: "some_input"
  top: "some_output"
  convolution_param{
    stride: xxx
    pad: xxx
    kernel_size: xxx
    num_output: xxx
    bias_term: true
  }
}
```

与 `Caffe` 中 `convolution_param` 一样，在定义的时候同样可以使用 `kernel_h`、`kernel_w`、`stride_h`、`stride_w`、`pad_h`、`pad_w` 来定义 `height` 和 `width` 方向上的卷积 `kernel` 参数和 `padding`。

对于 Depthwise Convolution 层的 `bias`，同样是通过 `convolution_param` 中的 `bias_term` 来确定，由于 `Caffe` 的卷积层的 `bias` 的维度是通过读取 `prototxt` 的 `convolution_param` 的 `num_output` 来定义，因此 `DepthwiseConv` 层的定义中，`num_output` 也是 `convolution_param` 中必须要定义的字段，且由于 `DepthwiseConv` 层的计算方式，必须要求 `num_output` 与输入数据的 `channel` 维度大小一致，`bias` 为每个输出 `channel` 对应一个 `bias` 值。



RReLU 层

RReLU 层即 Randomized Leaky ReLU，在 <https://arxiv.org/abs/1505.00853> 中有提到，并在文章中通过 MXNet 框架进行实现，在 caffe 框架中同样也是没有进行标准定义的。RReLU 层类似 Leaky ReLU，只不过在负数区间的斜率与 Leaky ReLU 不同。Leaky ReLU 在负数区间的斜率是配置好固定的，RReLU 在负数区间的斜率在训练中是从一个 $[l,u]$ 区间的均匀分布随机采样确定的，在推理阶段，RReLU 在负数区间的斜率为一个确定的数值 $l+u/2$ 。

如图 3-11 所示为 MXNet 中使用 RReLU 的一个示例，当 `act_type` 设定为 `rrelu` 后，需要设定 `lower_bound` 和 `upper_bound`，即对应以上描述的 l 和 u 。

图3-11 MXNet 中 RReLU API 说明

```
mxnet.symbol.LeakyReLU(data=None, act_type=_Null, slope=_Null, lower_bound=_Null, upper_bound=_Null, name=None, attr=None, out=None,
**kwargs)
```

Applies Leaky rectified linear unit activation element-wise to the input.

Leaky ReLUs attempt to fix the "dying ReLU" problem by allowing a small *slope* when the input is negative and has a slope of one when input is positive.

The following modified ReLU Activation functions are supported:

- *elu*: Exponential Linear Unit. $y = x > 0 ? x : slope * (exp(x)-1)$
- *leaky*: Leaky ReLU. $y = x > 0 ? x : slope * x$
- *prelu*: Parametric ReLU. This is same as *leaky* except that *slope* is learnt during training.
- *rrelu*: Randomized ReLU. same as *leaky* but the *slope* is uniformly and randomly chosen from $[lower_bound, upper_bound]$ for training, while fixed to be $(lower_bound+upper_bound)/2$ for inference.

由于该层的计算是基于 Leaky ReLU 的一种变种，需要定义参数可以复用 Caffe 中 ReLU 参数的定义，所以不需要对 `caffe.proto` 文件进行扩展，直接复用 `ReLUParameter` 即可。

RReLU 层在 `prototxt` 中进行定义的方式示例如下，`layer` 的 `type` 定义为 `RReLU`，`negative_slope` 参数即根据训练时使用的 `upper_bound` 和 `lower_bound` 设定为 $upper_bound+lower_bound/2$ 。

```
layer {
  name: "rrelu_layer"
  type: "RReLU"
  bottom: "some_input"
  top: "some_output"
  relu_param{
    negative_slope: xxx
  }
}
```

MatMul 层

MatMul 即 Matrix Multiplication，该层实现的功能即为两个输入矩阵的乘法操作。该层需要定义两个输入、一个输出。若第一个输入矩阵的维度为 $N \times K$ ，则第二个输入矩阵的行方向维度必须为 K ，列方向维度为 M ，那么该层实现 $N \times K$ 与 $K \times M$ 矩阵的乘法，

输出的维度为 $N \times M$ 。两个输入矩阵的维度信息通过定义一个 `matmul_param` 传入，需要设定三个维度信息，其中 `dim_1` 为第一个输入矩阵行维度、`dim_2` 为第一个输入矩阵列维度/第二个输入矩阵行维度、`dim_3` 为第二个输入矩阵列维度。

定义了该层的输入、输出以及相关的参数后，需要扩展 `caffe.proto` 文件，以使得在 `prototxt` 中定义的 `MatMul` 层和相关参数设定能够被解析。首先在 `caffe.proto` 的 `LayerParameter` 中添加 `MatMulParameter` 的参数声明。

图3-12 在 `LayerParameter` 中添加 `MatMulParameter`

```
optional ROIPoolingParameter roi_pooling_param = 100000;  
optional NormalizeParameter norm_param = 100001;  
optional PSROIPoolingParameter psroi_pooling_param = 100002;  
optional UpsampleParameter upsample_param = 100003;  
optional PassThroughParameter pass_through_param = 100004;  
optional MatMulParameter matmul_param = 100005;
```

之后在 `caffe.proto` 中的新添加一个名称为 `MatMulParameter` 的 `message`，需要定义 `dim_1`、`dim_2`、`dim_3` 三个参数。

图3-13 定义 `MatMulParameter` 参数

```
message MatMulParameter {  
    optional uint32 dim_1 = 1; //row of input matrix one  
    optional uint32 dim_2 = 2; //column of input matrix one and row of input matrix two  
    optional uint32 dim_3 = 3; //column of input matrix two  
}
```

对 `caffe.proto` 文件完成修改后，就可以解析 `prototxt` 中定义的 `MatMul` 层，一个示例的 `MatMul` 层定义如下。

```
layer {  
    name: "matmul_layer"  
    type: "MatMul"  
    bottom: "some_input_one"  
    bottom: "some_input_two"  
    top: "some_output"  
    matmul_param {  
        dim_1: N  
        dim_2: K  
        dim_3: M  
    }  
}
```


Permute 层

Permute 层实现的功能是将一个多维的 Tensor 的维度顺序进行切换，例如将一个 NCHW 顺序的 Tensor 转换为 NHWC 顺序的 Tensor。NNIE 的 Permute 层只支持对一个四维输入 Tensor 进行顺序转换，且转换的顺序只能为 0、2、3、1，即将原始 0、1、2、3 顺序的 Tensor 转换为新的 0、2、3、1 顺序的 Tensor，也就是上述的 NCHW 顺序的 Tensor 转换为 NHWC。

在 SSD 网络中有实现 Permute 层的操作，且 SSD 网络的 Permute 层定义了 PermuteParameter，用于配置输出 Tensor 的维度顺序。所以需要对 caffe.proto 文件进行扩展，添加 PermuteParameter 相应的内容。

图3-14 在 LayerParameter 中添加 PermuteParameter

```
optional ROIPoolingParameter roi_pooling_param = 100000;  
optional NormalizeParameter norm_param = 100001;  
optional PSROIPoolingParameter psroi_pooling_param = 100002;  
optional UpsampleParameter upsample_param = 100003;  
optional PassThroughParameter pass_through_param = 100004;  
optional MatMulParameter matmul_param = 100005;  
optional PermuteParameter permute_param = 100006;
```

图3-15 定义 PermuteParameter 参数

```
message PermuteParameter {  
    // The new orders of the axes of data. Notice it should be with  
    // in the same range as the input data, and it starts from 0.  
    // Do not provide repeated order.  
    repeated uint32 order = 1;  
}
```

具体如图 3-15，Permute 层需要配置的参数为 order，用于配置输出 Tensor 的维度顺序，prototxt 中配置的 order 参数的条目数必须与输入的维度数一致且互不相等，一个示例的 Permute 层定义如下。

```
layer {  
    name: "permute_layer"  
    type: "Permute"  
    bottom: "some_input"  
    top: "some_output"  
    permute_param {  
        order: 0  
        order: 2  
        order: 3  
        order: 1  
    }  
}
```



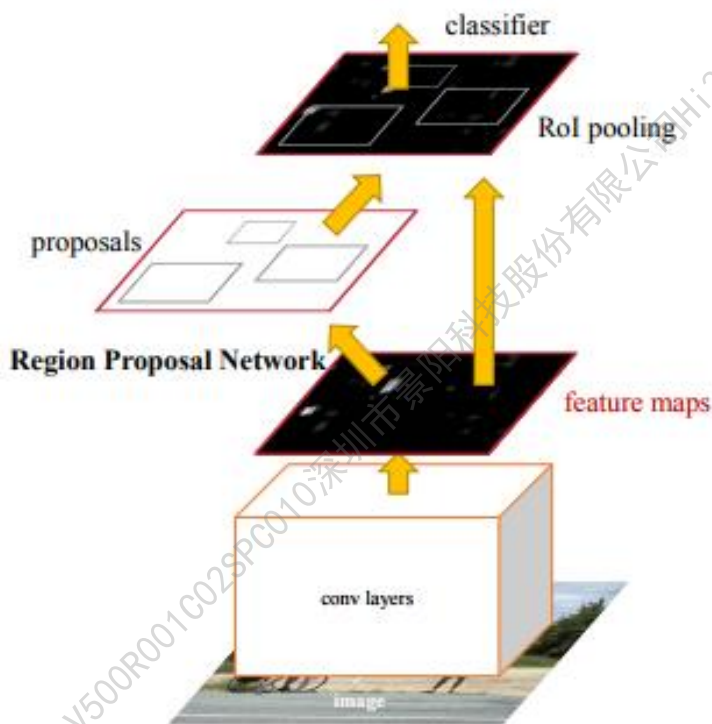

注意

目前仅支持如上示例的 order 配置，对一个四维输入 Tensor 进行 0231 的 Permute 变换。

ROI Pooling 层

ROI Pooling 层为 Faster RCNN 网络中用于将不同图像经过卷积层后得到的 feature map 进行维度统一的操作，输入为一个 feature map 以及需要从中提取的 ROI 框坐标值，输出为一个维度归一化的 feature map。如下图为一个 ROI Pooling 在 Faster RCNN 网络中的应用方式，经过卷积层后，得到输入图片的 feature map，feature map 输入到 RPN 网络输出一系列 ROI 框的坐标数值，然后 ROI Pooling 层根据框的坐标从 feature map 中选取对应的区域，然后对该区域进行一个输出维度统一的 pooling 操作，如 7*7，即使用不同 ROI 框选出的不同尺寸的 feature map 经过 ROI Pooling 后在 spatial 维度上会归一到 7*7，具体做法是将选出的 feature map 在行、列方向上进行 7 等分，然后对划分的 7*7=49 个小区域进行 Maxpooling 操作，每个区域对应输出的一个数值。

图3-16 Faster RCNN 示意图



ROI Pooling 层需要对 caffe.proto 文件进行扩展，定义 ROI PoolingParameter，扩展方式示例如下图，定义参数包括 spatial scale，即输入 feature map 与原始图片的尺寸比

例，用于转换 ROI 的坐标，因为 ROI 坐标是在原图尺寸上的；pooled_h、pooled_w，输出 feature map 在 spatial 维度上 h 和 w 的大小。

图3-17 在 LayerParameter 中添加 ROIPoolingParameter

```
optional ROIPoolingParameter roi_pooling_param = 100000;  
optional NormalizeParameter norm_param = 100001;  
optional PSROIPoolingParameter psroi_pooling_param = 100002;  
optional UpsampleParameter upsample_param = 100003;  
optional PassThroughParameter pass_through_param = 100004;  
optional MatMulParameter matmul_param = 100005;  
optional PermuteParameter permute_param = 100006;
```

图3-18 定义 ROIPoolingParameter 参数

```
// Message that stores parameters used by ROIPoolingLayer  
message ROIPoolingParameter {  
    // Pad, kernel size, and stride are all given as a single value for equal  
    // dimensions in height and width or as Y, X pairs.  
    optional uint32 pooled_h = 1 [default = 0]; // The pooled output height  
    optional uint32 pooled_w = 2 [default = 0]; // The pooled output width  
    // Multiplicative spatial scale factor to translate ROI coords from their  
    // input scale to the scale used when pooling  
    optional float spatial_scale = 3 [default = 1];  
}
```

因此一个 ROIPooling 层在 prototxt 中定义的示例如下。

```
layer {  
  name: "roipooling_layer"  
  type: "ROIPooling"  
  bottom: "some_input_feature_map"  
  bottom: "some_input_rois"  
  top: "some_output"  
  roi_pooling_param {  
    spatial_scale: 0.0625  
    pooled_h: 7  
    pooled_w: 7  
  }  
}
```

PSROI Pooling 层

PSROI Pooling 层的操作与 ROI Pooling 层类似，不同之处在于不同空间维度输出的图片特征来自不同的 feature map channels，且对每个小区域进行的是 Average Pooling，不同于 ROI Pooling 的 Max Pooling，如图 3-19 为一个 PSROI Pooling 层的示意图，

图3-19 PSROI Pooling 示意图

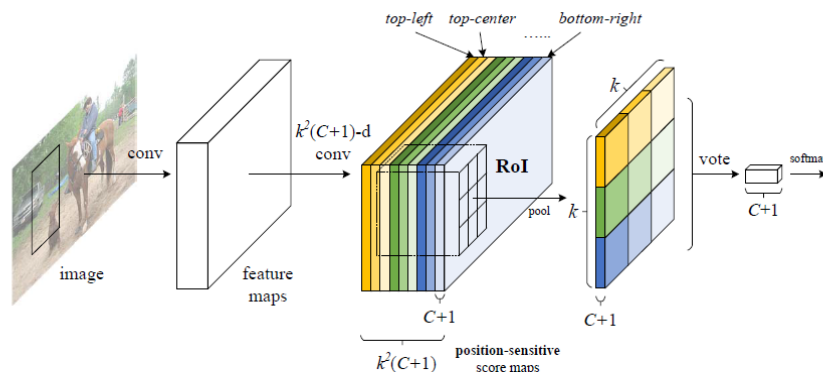


Figure 1: Key idea of **R-FCN** for object detection. In this illustration, there are $k \times k = 3 \times 3$ position-sensitive score maps generated by a fully convolutional network. For each of the $k \times k$ bins in an RoI, pooling is only performed on one of the k^2 maps (marked by different colors).

对于一个输出 $k \times k$ 的结果，不同空间维度的特征取自输入 feature map 中不同的组，即将输入的 feature map 均匀分为 $k \times k$ 组，每组的 channel 数与输出的 channel 一致，得到上述输出。

PSROI Pooling 层需要对 caffe.proto 文件进行扩展，定义 PSROI PoolingParameter，扩展方式示例如图 3-20，定义参数包括 spatial scale，即输入 feature map 与原始图片的尺寸比例；output_dim，输出 feature map 的 channel 数；group_size，输出的 spatial 维度，即上述的 k 。

图3-20 在 LayerParameter 中添加 PSROI PoolingParameter

```
optional ROI PoolingParameter roi_pooling_param = 100000;
optional NormalizeParameter norm_param = 100001;
optional PSROI PoolingParameter psroi_pooling_param = 100002;
optional UpsampleParameter upsample_param = 100003;
optional PassThroughParameter pass_through_param = 100004;
optional MatMulParameter matmul_param = 100005;
optional PermuteParameter permute_param = 100006;
```

图3-21 定义 PSROI PoolingParameter 参数

```
message PSROI PoolingParameter {
    required float spatial_scale = 1;
    required int32 output_dim = 2; // output channel number
    required int32 group_size = 3; // equal to pooled_size
}
```

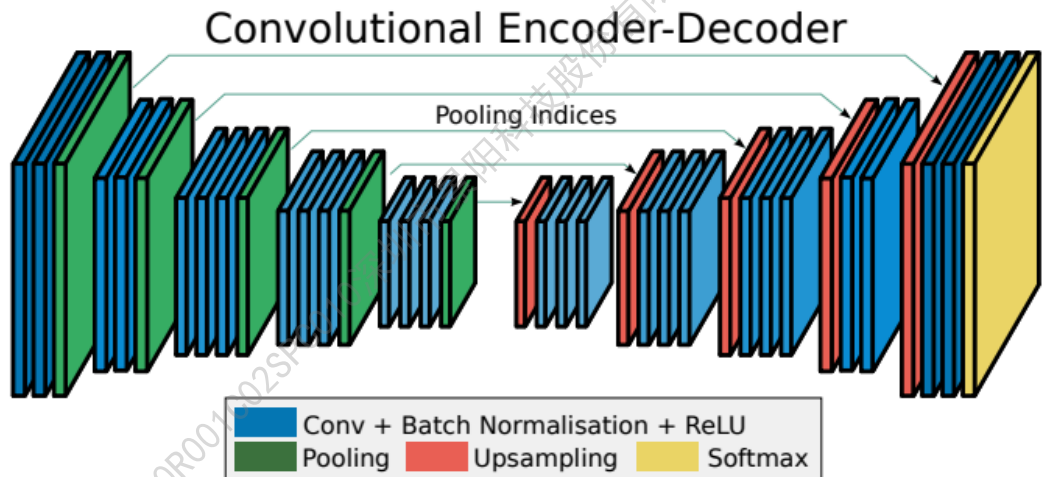
因此一个 PSROIPooling 层在 prototxt 中定义的示例如下。

```
layer {
  name: "psroipooling_layer"
  type: "PSROIPooling"
  bottom: "some_input_feature_map"
  bottom: "some_input_rois"
  top: "some_output"
  psroi_pooling_param {
    spatial_scale: 0.0625
    output_dim: 21
    group_size: 7
  }
}
```

Upsample 层

Upsample 层为 Pooling 层的逆操作，下图为一个示意图，其中每个 Upsample 层均与网络之前一个对应大小输入、输出 Pooling 层一一对应，完成 feature map 在 spatial 维度上的扩充。

图3-22 Upsample 层示意图



Upsample 层需要对 caffe.proto 文件进行扩展，定义 UpsampleParameter，扩展方式示例如下图，定义参数包括 scale，即输出与输入的尺寸比例，如 2；scale_h、scale_w，同 scale，分别为 h、w 方向上的尺寸比例；pad_out_h、pad_out_w，仅在 scale 为 2 时有用，对输出进行额外 padding 在 h、w 方向上的数值；upsample_h、upsample_w，输

出图像尺寸的数值。在 Upsample 的相关代码中，推荐仅仅使用 upsample_h、upsample_w 准确定义 Upsample 层的输出尺寸，其他所有的参数都不推荐继续使用。

图3-23 在 LayerParameter 中添加 UpsampleParameter

```
optional ROIPoolingParameter roi_pooling_param = 100000;  
optional NormalizeParameter norm_param = 100001;  
optional PSROIPoolingParameter psroi_pooling_param = 100002;  
optional UpsampleParameter upsample_param = 100003;  
optional PassThroughParameter pass_through_param = 100004;  
optional MatMulParameter matmul_param = 100005;  
optional PermuteParameter permute_param = 100006;
```

图3-24 定义 UpsampleParameter 参数

```
message UpsampleParameter {  
    // DEPRECATED. No need to specify upsampling scale factors when  
    // exact output shape is given by upsample_h, upsample_w parameters.  
    optional uint32 scale = 1 [default = 2];  
    // DEPRECATED. No need to specify upsampling scale factors when  
    // exact output shape is given by upsample_h, upsample_w parameters.  
    optional uint32 scale_h = 2;  
    // DEPRECATED. No need to specify upsampling scale factors when  
    // exact output shape is given by upsample_h, upsample_w parameters.  
    optional uint32 scale_w = 3;  
    // DEPRECATED. Specify exact output height using upsample_h. This  
    // parameter only works when scale is 2  
    optional bool pad_out_h = 4 [default = false];  
    // DEPRECATED. Specify exact output width using upsample_w. This  
    // parameter only works when scale is 2  
    optional bool pad_out_w = 5 [default = false];  
    optional uint32 upsample_h = 6;  
    optional uint32 upsample_w = 7;  
}
```

因此一个 Upsample 层在 prototxt 中定义的示例如下。

```
layer {  
    name: "upsample_layer"  
    type: "Upsample"  
    bottom: "some_input_feature_map"  
    bottom: "some_input_pool_index"  
    top: "some_output"  
    upsample_param {  
        upsample_h: 224  
        upsample_w: 224  
    }  
}
```

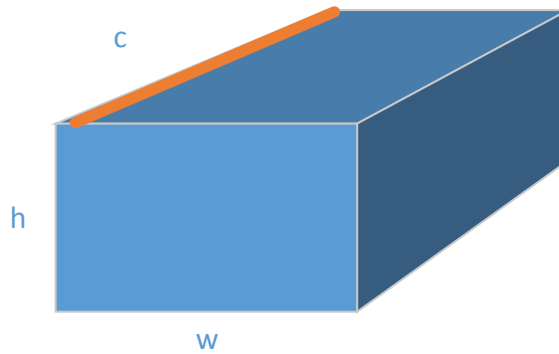
Normallize 层

Normalize 层为 SSD 网络中一个层结构，其进行的操作为对于一个 $c \times h \times w$ 的三维 tensor，输出是同样大小的 tensor，其中间计算为每个元素以 channel 方向的平方和的平方根求 normalize，其具体计算公式为：

$$x'_i = \frac{x_i}{\sqrt{\sum_{i=1}^c x_i^2}}$$

其中分母位置的平方和的累加向量为同一 h 与 w 位置的所有 c 方向的向量，如下图中的橙色区域。

图3-25 Normalize 层计算示意图



经过上述计算归一化后，再对每个 feature map 做 scale，每个 channel 对应一个 scale 值。

Normalize 层需要对 caffe.proto 文件进行扩展，定义 NormalizeParameter，扩展方式示例如下图，定义的参数包括 across_spatial，是否在 channel 或者 spatial 维度上做 normalization，目前仅支持 False 的情形，即只支持在 channel 维度做 normalization；scale_filler，对 feature map 进行 scale 时 scale 参数的初始化方法；channel_shared，scale 在 channel 方向是否共享，默认为 True；eps，防止 normalize 过程中除以 0 的一个很小数值，默认为 $1e-10$ ，且不支持配置；此外在 NNIE 支持的 Normalize 层中，层的参数相比开源实现新添加了 sqrt_a 参数，该参数是用于保证 NNIE 在定点处理过程中数据不会发生溢出。为了防止在上述计算 normalize 过程中上溢出，引入系数 a ，normalize 的计算公式转换为：

$$x'_i = \frac{x_i}{\sqrt{\sum_{i=1}^c x_i^2}} = \frac{\sqrt{a} x_i}{\sqrt{a * \sum_{i=1}^c x_i^2}} = \frac{\sqrt{a} x_i}{\sqrt{\sum_{i=1}^c (\sqrt{a} * x_i) * (\sqrt{a} * x_i)}}$$

其中，系数 a 可以通过离线数据统计得到，记 $sum = \sum_{i=1}^c x_i^2$ ，即保证

$a * |sum|_{\max} \leq 2^{19} + 2^{-12}$ （数据以 20.12 定点方式储存），且保证 $\sqrt{a} \geq 2^{-12}$ （ $\sqrt{a}$ 可

以 20.12 定点方式储存), 即保证系数 $\sqrt{\alpha}$ 不会下溢出。 $\sqrt{\alpha}$ 即为 sqrt_a, 是 normalize 层中需要定义的参数。

sqrt_a 的默认值为 1, 即如果用户没有在 prototxt 中进行配置的话, 该值为 1; 如果配置了 sqrt_a 参数, 则为实际配置的数值。

图3-26 在 LayerParameter 中添加 NormalizeParameter

```
optional ROIPoolingParameter roi_pooling_param = 100000;  
optional NormalizeParameter norm_param = 100001;  
optional PSROIPoolingParameter psroi_pooling_param = 100002;  
optional UpsampleParameter upsample_param = 100003;  
optional PassThroughParameter pass_through_param = 100004;  
optional MatMulParameter matmul_param = 100005;  
optional PermuteParameter permute_param = 100006;
```

图3-27 定义 NormalizeParameter 参数

```
// Message that stores parameters used by NormalizeLayer  
message NormalizeParameter {  
    optional bool across_spatial = 1 [default = true];  
    // Initial value of scale. Default is 1.0 for all  
    optional FillerParameter scale_filler = 2;  
    // Whether or not scale parameters are shared across channels.  
    optional bool channel_shared = 3 [default = true];  
    // Epsilon for not dividing by zero while normalizing variance  
    optional float eps = 4 [default = 1e-10];  
    // square root of a value used in normalize  
    optional float sqrt_a = 5 [default = 1];  
}
```

一个示例的 Normalize 层定义如下。

```
layer {  
    name: "normalize_layer"  
    type: "Normalize"  
    bottom: "some_input"  
    top: "some_output"  
    normalize_param {  
        across_spatial: False  
        channel_shared: True  
        sqrt_a: 0.5  
    }  
}
```



3.1.4.4 扩展层的参考设计

Caffe 官方提供的开发新层的示例，地址
<https://github.com/BVLC/caffe/wiki/Development>。

ROI Pooling，参考设计为 Faster RCNN 网络，开源工程地址
<https://github.com/rbgirshick/py-faster-rcnn>。

Normalize，参考设计为 SSD 网络，开源工程地址
<https://github.com/weiliu89/caffe/tree/ssd>。

PSROI Pooling，参考设计为 RFCN 网络，开源工程地址
<https://github.com/daijifeng001/caffe-rfcn>。

Upsample，参考设计为 SegNet 网络，开源工程地址
<https://github.com/alexgkendall/caffe-segnet>，注意到在使用 Upsample 层时，需要配合网络在 Upsample 层之前的 Maxpooling 层输出的 max 元素的最大值，Averagepooling 由于无法输出对应的 max 元素的 mask，所以无法配合 Upsample 层使用。

CReLU 层，参考设计为 PVANet 网络，开源工程地址 <https://github.com/sanghoon/pva-faster-rcnn>。

上述自定义层为基于 Caffe 框架实现的，下述的自定义层基于其他深度学习框架，用于参考基本实现原理。

PassThrough 层，参考设计为 YOLO v2 网络，开源工程地址为
<https://pjreddie.com/darknet/yolo/>。

Depthwise Convolution 层，参考设计为 Xception 网络，开源工程地址为
<https://github.com/fchollet/keras/blob/master/keras/applications/xception.py>。

RReLU 层，参考论文中的定义 <https://arxiv.org/abs/1505.00853> 以及 MXNet API 中的实现 <http://mxnet.io/api/python/symbol.html>。

Permute 层，参考设计为 SSD 网络，开源工程地址
<https://github.com/weiliu89/caffe/tree/ssd>。

3.1.5 Non-support 层处理方式

当网络中存在 Non-support 层时，需要将网络进行切分，不支持的部分由用户使用 CPU 或者 DSP 等方式实现，统称为非 NNIE 方式。由此整个网络会出现 NNIE->非 NNIE->NNIE... 的分段执行方式。

nnie_mapper 将 NNIE 的 Non-support 层分为两种，“Proposal”层和“Custom”层：

- Proposal 层输出的是矩形信息（Bbox，即 Bounding box）；Proposal 自定义层格式如下，包含 name\type\bottom\top 等关键字段参数，type 必须为“Proposal”，支持任意多个的 bottom，但是 top 只有一个。



```
layer {  
  name: "proposal"  
  type: "Proposal"  
  bottom: "rpn_cls_prob_reshape"  
  bottom: "rpn_bbox_pred"  
  top: "rois"  
}
```

Proposal 层还支持以下参数的解析，但不产生实际的效果，

Proposal	uint32	feat_stride	默认值16
	uint32	base_size	默认值16
	uint32	min_size	默认值16
	float	ratio	NA
	float	scale	NA
	uint32	pre nms topn	默认值6000
	uint32	post nms topn	默认值300
	float	nms_thresh	默认值0.7
	float	remove_thresh	默认值0

- Custom 层输出的是 tensor (feature map、vector)；Custom 自定义层格式如下，仅包含 name\type\bottom\top\ custom_param\shape\dim 关键字段，type 必须为“Custom”，支持任意多个 top\bottom，custom_param 中的 shape 信息是描述 top 输出的形状，如果有多个输出，则按 top 顺序书写多个 shape。用户原有其他的参数字段需删除。



```
layer {
  name: "custom1"
  type: "Custom"
  bottom: "conv1"
  bottom: "conv2"
  bottom: "pooling1"
  top: "custom1_1"
  top: "custom1_2"
  custom_param
  {
    shape {
      dim: 1
      dim: 256
      dim: 64
      dim: 64
    }
    shape {
      dim: 1
      dim: 128
      dim: 128
      dim: 128
    }
  }
}
```

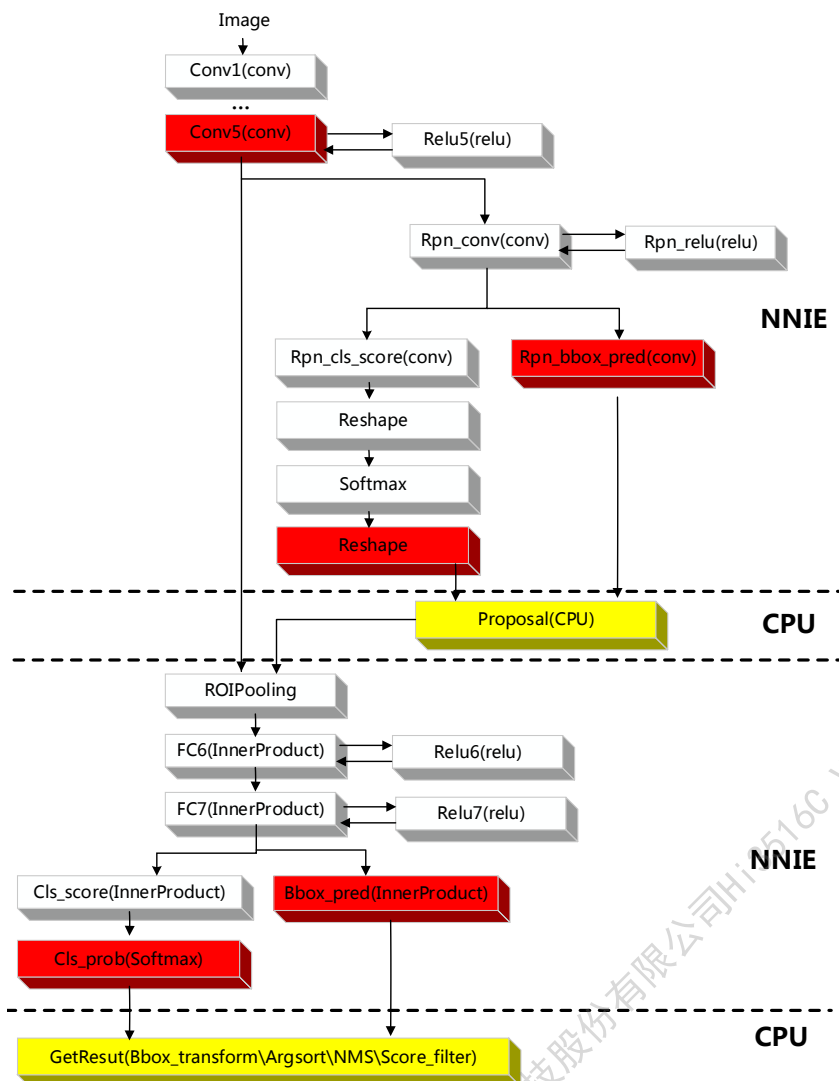
在 `nnie_mapper` 对网络模型进行转化之前，用户需根据上述特性将 Non-support 层修改为“Proposal”层或者“Custom”层，具体修改方式参考“[3.2 Prototxt 要求](#)”。

下图以 Faster RCNN 为例，Faster RCNN 中 RPN 部分的 Proposal 层(Bbox_transform、Argsort、NMS 等操作)输出的是 Bounding Box 信息。图示的 Faster RCNN 分为如下的几段来执行（非 NNIE 图示均由 CPU 来执行）：

- 1) 第 1 段（NNIE 执行），卷积部分，包含 RPN 前面的部分；
- 2) 第 2 段（非 NNIE 执行），Proposal 部分，生成矩形框信息；
- 3) 第 3 段（NNIE 执行），FC 部分，包含 ROI Pooling，得到矩形框调整值和置信度；
- 4) 第 4 段（非 NNIE 执行），结果获取部分，由 FC 部分的矩形框调整值和置信度经过 `bbox_transform`、`argsort`、`NMS`、阈值过滤等步骤获取最终的矩形框和置信度信息，CPU 执行。



图3-28 Faster RCNN 分段执行示意图



3.1.6 NNIE 规格

3.1.6.1 层级联关系

网络层的级联关系如表 3-1 所示；请注意表格中包含有 NNIE 的 Non-support 的“Proposal”层和“Custom”层的级联关系。



表3-1 SVP NNIE 网络层级联关系表

Layer type	Following layer															
	Convolution Deconvolution	Pooling DepthwiseConv	InnerProduct	LRN	BatchNorm	Scale Bias	Eltwise	ReLU PReLU AbsVal TanH Sigmoid BNLL ELU	CReLU	LSTM RNN	Softmax	Exp Log	Reshape Flatten	Split	Slice	Concat
Convolution Deconvolution	●	●	●	●	●	●	●	●	●	×	●	●	●	●	●	●
Pooling Depthwise Conv	●	●	●	●	●	●	●	●	●	×	●	●	●	●	●	●
InnerProduct	×	×	●	×	●	●	●	●	×	×	●	●	●	●	●	●
LRN	●	●	●	●	●	●	●	●	●	×	●	●	●	●	●	●
BatchNorm	●	●	●	●	●	●	●	●	●	×	●	●	●	●	●	●
Scale Bias	●	●	●	●	●	●	●	●	●	×	●	●	●	●	●	●
Eltwise	●	●	●	●	●	●	●	●	●	×	●	●	●	●	●	●



Layer type	Following layer															
	Convolution Deconvolution	Pooling Depth wiseConv	InnerPro duct	LRN	BatchNor m	Scal e Bias	Eltwis e	ReLU PReLU AbsVal TanH Sigmoid BNLL ELU	CReLU	LSTM RNN	Softmax	Exp Log	Reshape Flatten	Split	Slice	Concat
ReLU PReLU AbsVal TanH Sigmoid BNLL ELU	●	●	●	●	●	●	●	●	●	×	●	●	●	●	●	●
CReLU	●	●	●	●	●	●	●	●	●	×	●	●	●	●	●	●
LSTM RNN	×	×	●	×	×	×	●	●	×	×	×	●	●	×	×	●
Softmax	●	●	●	●	●	●	●	●	●	×	●	●	●	●	●	●
Exp Log	●	●	●	●	●	●	●	●	●	×	●	●	●	●	●	●
Reshape Flatten	●	●	●	●	●	●	●	●	●	×	●	●	●	●	●	●
Split	●	●	●	●	●	●	●	●	●	×	●	●	●	●	●	●



Layer type	Following layer															
	Convolution Deconvolution	Pooling Depth wiseConv	InnerPro duct	LRN	BatchNor m	Scal e Bias	Eltwis e	ReLU PReLU AbsVal TanH Sigmoid BNLL ELU	CReLU	LSTM RNN	Softmax	Exp Log	Reshape Flatten	Split	Slice	Concat
Slice	●	●	●	●	●	●	●	●	●	×	●	●	●	●	●	●
Concat	●	●	●	●	●	●	●	●	●	×	●	●	●	●	●	●
SPP	×	×	●	×	●	●	●	●	×	×	●	●	●	●	●	●
Power	●	●	●	●	●	●	●	●	●	×	●	●	●	●	●	●
Threshold	●	●	●	●	●	●	●	●	●	×	●	●	●	●	●	●
MVN	●	●	●	●	●	●	●	●	●	×	●	●	●	●	●	●
Parameter	●	●	●	●	●	●	●	●	●	×	●	●	●	●	●	●
Reduction	●	●	●	●	●	●	●	●	●	×	●	●	●	●	●	●
Proposal	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×
ROIPoolin g	●	●	●	●	●	●	●	●	●	×	●	●	●	●	●	●
PSROIPool ing	●	●	●	●	●	●	●	●	●	×	●	●	●	●	●	●
Normalize	●	●	●	●	●	●	●	●	●	×	●	●	●	●	●	●



Layer type	Following layer															
	Convolution Deconvolutio n	Pooling Depth wiseCo nv	InnerPro duct	LRN	BatchNor m	Scal e Bias	Eltwis e	ReLU PReLU AbsVal TanH Sigmoid BNLL ELU	CReLU	LSTM RNN	Softmax	Exp Log	Reshape Flatten	Split	Slice	Concat
PassThrou gh	●	●	●	●	●	●	●	●	●	×	●	●	●	●	●	●
Upsample	●	●	●	●	●	●	●	●	●	×	●	●	●	●	●	●
Permute	●	●	●	●	●	●	●	●	●	×	●	●	●	●	●	●
MatMul	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×
Custom	●	●	●	●	●	●	●	●	●	×	●	●	●	●	●	●

Layer type	Following layer																
	SP P	Powe r	Thresho ld	MV N	Param eter	Reduct ion	Propos al	ROIPooli ng	PSROIPO oling	Norm alize	PassThr ough	Upsamp le	Permut e	Mat Mul	Custo m	as inp ut	as outp ut
Convoluti on Deconvolu tion	●	●	●	●	●	●	●	●	●	●	●	●	●	×	●	●	●



Layer type	Following layer																
	SP P	Powe r	Thresho ld	MV N	Param eter	Reduct ion	Propos al	ROIPooli ng	PSROIPO oling	Norm alize	PassThr ough	Upsamp le	Permut e	Mat Mul	Custo m	as inp ut	as outp ut
Pooling Depthwise Conv	●	●	●	●	●	●	●	●	●	●	●	●	●	×	●	●	●
InnerProd uct	×	●	●	×	●	×	●	×	×	×	×	×	×	×	●	●	●
LRN	●	●	●	●	●	●	●	●	●	●	●	●	●	×	●	●	●
BatchNor m	●	●	●	●	●	●	●	●	●	●	●	●	●	×	●	●	●
Scale Bias	●	●	●	●	●	●	●	●	●	●	●	●	●	×	●	●	●
Eltwise	●	●	●	●	●	●	●	●	●	●	●	●	●	×	●	●	●
ReLU PReLU AbsVal TanH Sigmoid BNLL ELU	●	●	●	●	●	●	●	●	●	●	●	●	●	×	●	●	●
CReLU	●	●	●	●	●	●	●	●	●	●	●	●	●	×	●	●	●
LSTM RNN	×	●	●	●	●	×	×	×	×	×	×	×	×	×	×	●	●
Softmax	●	●	●	●	●	●	●	●	●	●	●	●	●	×	●	●	●



Layer type	Following layer																
	SP P	Powe r	Thresho ld	MV N	Param eter	Reduct ion	Propos al	ROIPooli ng	PSROIPO oling	Norm alize	PassThr ough	Upsamp le	Permut e	Mat Mul	Custo m	as input	as output
Exp Log	●	●	●	●	●	●	●	●	●	●	●	●	●	×	●	●	●
Reshape Flatten	●	●	●	●	●	●	●	●	●	●	●	●	●	×	●	●	●
Split	●	●	●	●	●	●	●	●	●	●	●	●	●	×	●	●	●
Slice	●	●	●	●	●	●	●	●	●	●	●	●	●	×	●	●	●
Concat	●	●	●	●	●	●	●	●	●	●	●	●	●	×	●	●	●
SPP	×	●	●	×	●	×	●	×	×	×	×	×	×	×	●	●	●
Power	●	●	●	●	●	●	●	●	●	●	●	●	●	×	●	●	●
Threshold	●	●	●	●	●	●	●	●	●	●	●	●	●	×	●	●	●
MVN	●	●	●	●	●	●	●	●	●	●	●	●	●	×	●	●	●
Paramete r	●	●	●	●	●	●	●	●	●	●	●	●	●	×	●	●	●
Reduction	●	●	●	●	●	●	●	●	●	●	●	●	●	×	●	●	●
Proposal	×	×	×	×	×	×	×	●	●	×	×	×	×	×	●	×	●
ROIPooli ng	●	●	●	●	●	●	×	×	×	●	●	●	●	×	●	●	●
PSROIPO oling	●	●	●	●	●	●	×	×	×	●	●	●	●	×	●	●	●



Layer type	Following layer																
	SP P	Powe r	Thresho ld	MV N	Param eter	Reduct ion	Propos al	ROIPooli ng	PSROIPO oling	Norm alize	PassThro ugh	Upsamp le	Permut e	Mat Mul	Custo m	as inp ut	as outp ut
Normalize	●	●	●	●	●	●	●	●	●	●	●	●	●	×	●	●	●
PassThro ugh	●	●	●	●	●	●	●	●	●	●	●	●	●	×	●	●	●
Upsample	●	●	●	●	●	●	●	●	●	●	●	●	●	×	●	●	●
Permute	●	●	●	●	●	●	●	●	●	●	●	●	●	×	●	●	●
MatMul	×	×	×	×	×	×	×	×	×	×	×	×	×	×	×	●	●
Custom	●	●	●	●	●	●	×	●	●	●	●	×	●	×	×	×	×

3.1.6.2 层规格描述

网络层的具体规格如表 3-2 所示；请注意黄底黑字的是扩展层，黄底红字的是 Non-support 层。

表3-2 NNIE layer 规格表

layer	参数类型	参数名	参数含义	描述	规格	
AbsVal	无	无	无	求绝对值	支持	w<=4096
BatchNorm	bool	use_global_stats	false 表示使用滑动平均；true 表示使用全局的，不每次进行计算	-	支持 true	w<=4096
	float	moving_average_fraction	默认为.999，每次迭代滑动平均衰减系数		不支持配置	



layer	参数类型	参数名	参数含义	描述	规格	
	float	eps	默认为 1e-5，用来防止被 0 除		不支持配置	
Bias	int32	axis	默认是 1，不同的值对应的相加维度不同 For example, if bottom[0] is 4D with shape 100x3x40x60, the output top[0] will have the same shape, and bottom[1] may have any of the following shapes (for the given value of axis): (axis == 0 == -4) 100; 100x3; 100x3x40; 100x3x40x60 (axis == 1 == -3) 3; 3x40; 3x40x60 (axis == 2 == -2) 40; 40x60 (axis == 3 == -1) 60	计算两个输入的 elementwise 和	支持两种配置(1,-3)，默认值 1，遵循 caffe 规则	w<=4096
	int32	num_axes	默认为 1，当只有一个 bottom 时有效，指定训练的 bias 参数维度大小		支持配置 1 或者-1，与 axis 配套使用，在张量与 C 维度向量相乘时配置为-1，其余情况为 1，遵循 caffe 规则；	
BNLL	无	无	无	按照一固定公式计算，激活函数	支持	w<=4096
Concat	int32	axis	默认为 1，表示哪个维度 concat，可以为负数	至少两个输入，将它们在某个维度上拼接起来	支持配置，-4~3，不支持 N 维度的 concat	axis 1: w<=UINT_MAX axis 2: w<=16384 axis 3: w<=4096 vecotr: w<=UINT_MAX bottom num<=32
	uint32	concat_dim	DEPRECATED; 跟 axis 含义相同，不支持负值		支持配置 0~3，与 axis 配套使用，遵循 caffe 规则；	



layer	参数类型	参数名	参数含义	描述	规格	
Convolution	uint32	num_output	输出的 channel 大小	对输入做卷积	支持配置，最大 32768;	$w \leq 4096$ $kernel_h < \frac{2048}{w/(16*stride_w)*stride_w}$
	bool	bias_term	默认为 1，表示是否加 bias		支持配置 0 或 1;	
	uint32	pad	默认为 0，表示补边的大小		支持，配置范围 0~255;	
	uint32	kernel_size	kernel 的大小		支持 1~255	
	uint32	stride	默认为 1，stride 的大小		支持 1~255	
	uint32	dilation	默认为 1，dilation 的大小		支持 1~255，配置后 $(kernel-1)*dilation+1 < 256$;	
	uint32	pad_h	默认为 0，表示 height 方向的 padding		支持，配置范围 0~255;	
	uint32	pad_w	默认为 0，表示 width 方向的 padding		支持，配置范围 0~255;	
	uint32	kernel_h	kernel 的 height 大小		支持，配置范围 1~255;	
	uint32	kernel_w	kernel 的 width 大小		支持，配置范围 1~255;	
	uint32	stride_h	stride 的 height 大小		支持 1~255	
	uint32	stride_w	stride 的 width 大小		支持 1~255	
	uint32	group	默认为 1，group 的大小		支持最大 2048	
	int32	axis	默认为 1，axis 值之前的维度认为是独立输入，axis 之后的维度认为是空间维度，即在做卷积时会求和		不支持配置	
Deconvoluti	uint32	num_output	输出的 channel 大小	卷积的反过程，将卷	支持配置，最大 32768;	$w \leq 4096$



layer	参数类型	参数名	参数含义	描述	规格	
on	bool	bias_term	默认为 1，表示是否加 bias	积的前向传播和后向传播置换	支持配置 0 或 1；	kenel_h <= (32768/w-1)*stride_h + 1
	uint32	pad	默认为 0，表示补边的大小		支持，配置范围 0~255；	
	uint32	kernel_size	kernel 的大小		支持 1~255	
	uint32	stride	默认为 1，stride 的大小		支持 1~255	
	uint32	dilation	默认为 1，dilation 的大小		支持 1~255，配置后(kernel-1)*dilation+1<256；	
	uint32	pad_h	默认为 0，表示 height 方向的 padding		支持，配置范围 0~255；	
	uint32	pad_w	默认为 0，表示 width 方向的 padding		支持，配置范围 0~255；	
	uint32	kernel_h	kernel 的 height 大小		支持，配置范围 1~255；	
	uint32	kernel_w	kernel 的 width 大小		支持，配置范围 1~255；	
	uint32	stride_h	stride 的 height 大小		支持 1~255	
	uint32	stride_w	stride 的 width 大小		支持 1~255	
	uint32	group	默认为 1，group 的大小		支持最大 2048	
	int32	axis	默认为 1，axis 值之前的维度认为是独立输入，axis 之后的维度认为是空间维度，即在做卷积时会求和		不支持配置	
Eltwise	EltwiseOp	operation	默认为 SUM，element-wise 的操作，加、乘、求最大	进行 elementwise 的计算操作，包括相加、	支持,相加、相乘、求最大；	w<=4096 bottom num<=32



layer	参数类型	参数名	参数含义	描述	规格	
	float	coeff	blob_wise 系数	相乘、求最大	支持，用线性 20.12 表示	
	bool	stable_prod_grad	对于乘操作，大于两输入的情况，选择是否使用更慢但更稳定的方法求梯度		不支持配置	
ELU	float	alpha	默认为 1，公式中的 alpha 系数	一种固定公式的激活函数	支持，用线性 20.12 表示	w<=4096
Exp	float	base	默认为-1.0	计算 exp	支持，-1 或是正数	w<=4096
	float	scale	默认为 1.0		支持，用线性 20.12 表示	
	float	shift	默认为 0.0		支持，用线性 20.12 表示	
Flatten	int32	axis	flatten 开始的维度	将输入的 blob 转成向量	axis 取值范围[-4,3]，默认值为 1，-4 与 0 含义相同。	张量：输出 w<=65536 向量： w<=UINT_MAX
	int32	end_axis	flatten 结束的维度		end_axis 取值范围[-4,3]，默认值为-1，表示最后一条轴。End_axis 应大于 axis。	
InnerProduct	uint32	num_output	输出的维度大小	全连接层，计算内积	支持，最大支持 32768；	8 bits: w<=4096 16 bits: w<=4096
	bool	bias_term	是否加 bias		支持配置 0 或 1；	
	int32	axis	进行内积计算的维度		不支持配置 当 LSTM 后面接 FC 时，支持且仅支持 axis=2	
	bool	transpose	是否转置权值矩阵		支持配置 0 或 1；	
Input	BlobShape	shape	定义 blob 的大小，可以每个分别定义，可以统一为同样大小，	-	支持最大 input 节点 32 个	w<=4096



layer	参数类型	参数名	参数含义	描述	规格	
			可以不定义			
Log	float	base	默认为-1.0	计算 log	支持-1 和任意正数	w<=4096
	float	scale	默认为 1.0		支持任意值	
	float	shift	默认为 0.0		支持任意值	
LRN	uint32	local_size	默认为 5	一种 normalization 的方法	支持 3、5、7	local size 3: w<=4096 local size 5/7: w<=4096
	float	alpha	默认为 1.0		支持, 用线性 20.12 表示	
	float	beta	默认为 0.75		支持, 用 1.5.7 表示	
	NormRegion	norm_region	默认为 ACROSS_CHANNELS, 可选 WITHIN_CHANNEL		只支持 ACROSS_CHANNEL	
	float	k	默认为 1.0		支持配置, 用线性 20.12 表示;	
LSTM	uint32	num_output	默认为 0, 输出的维度大小	LSTM 网络	支持, 最大支持 5456	num_output<=5456
	bool	debug_info	默认为 false, 是否输出 debug_info		不支持配置	
	bool	expose_hidden	默认为 false, 表示是否增加额外输入输出层, 对于 LSTM 是同时增加 2 层		支持配置 0 或 1;	
MVN	bool	normalize_variance	默认为 true, 设为 false 时只减均值	$x' = (x - \text{mean}(x)) / \text{std}(x)$	支持配置 true 或 false	w<=4096
	bool	across_channels	默认为 false, 设为 true 时就是 CHW 视为一个向量		支持配置 true 或 false	
	float	eps	默认为 1e-9, 用来防止除 0		不支持配置	



layer	参数类型	参数名	参数含义	描述	规格	
Parameter	BlobShape	shape	定义 blob 的大小	caffe 中的数据结构都是 blobs，很难区分 data 和 parameter。开发者提供 parameter 这个层来给用户指定参数的来源。比如，在 FC 中 $C=AB$ ，如果 B 是 bottom，那么 A 就一定是参数，且参数是由训练得到的，而如果用户想要做一个 FC 操作且 AB 均是从前面节点得来，这时可以插入一个 parameter layer，指定 FC 的 bottom 是 B，参数来源是 parameter layer，而 parameter layer 来源为 A，即实现这一功能。 该层的增加提高了 caffe 应用的灵活性	支持解析，但是 caffe 当前版本里没有实现该层的代码；	NA
Pooling	PoolMethod	pool	默认为 MAX，pooling 的类型，另一种是 AVE	对输入做 pooling	支持配置 0 或 1；	$w \leq 4096$ $kernel_h \leq 2048 / (w / (16 * stride_w) * stride_w)$
	uint32	pad	默认为 0，表示补边的大小		支持，配置范围 0~255；	
	uint32	pad_h	默认为 0，表示 height 方向的 padding		支持，配置范围 0~255；	



layer	参数类型	参数名	参数含义	描述	规格	
	uint32	pad_w	默认为 0，表示 width 方向的 padding		支持，配置范围 0~255；	
	uint32	kernel_size	kernel 的大小		支持，配置范围 1~255；	
	uint32	kernel_h	kernel 的 height 大小		支持，配置范围 1~255；	
	uint32	kernel_w	kernel 的 width 大小		支持，配置范围 1~255；	
	uint32	stride	默认为 1，stride 的大小		支持，配置范围 1~255；	
	uint32	stride_h	stride 的 height 大小		支持，配置范围 1~255；	
	uint32	stride_w	stride 的 width 大小		支持，配置范围 1~255；	
	bool	global_pooling	默认为 false，是否全平面做 pooling		支持配置 0 或 1；	
Power	float	power	默认为 1.0	$f(x)=(\text{shift}+\text{scale}*x)^{\text{power}}$	支持部分配置；-3、-2、-1、-1/2、1/2、1、2、3；	w<=4096
	float	scale	默认为 1.0		支持配置，用线性 20.12 表示；	
	float	shift	默认为 0.0		支持配置，用线性 20.12 表示；	
PReLU	bool	channel_shared	默认为 false，表示负向斜率是否 channel 共享	带参数的 ReLU 激活函数	支持配置，用线性 20.12 表示；	w<=4096
Reduction	Reduction Op	operation	默认为 SUM，可选 ASUM、SUMSQ、MEAN	将输入的 blob reduction 成一个 scalar	支持 SUM、ASUM、SUMSQ、MEAN	w<=4096
	int32	axis	默认为 0，表示从第几个维度开始后面的维度全部 reduction		支持[2,3]	



layer	参数类型	参数名	参数含义	描述	规格	
	float	coeff	默认为 1.0, 输出的系数		支持 20.12 范围内的配置	
ReLU	float	negative_slope	默认为 0, 负值的斜率	ReLU 激活函数	支持配置, 用线性 20.12 表示;	w<=4096
Reshape	BlobShape	shape	指示输出的维度大小, 0 表示跟 bottom 一致, -1 表示该维度由输入的 blob 及输出的其他维度决定	维度变换	支持[1,4]个 shape_dim 配置	output w<=4096
	int32	axis	默认为 0, 表示 shape 中第一数值与输出的第几个起始维度对应		支持[-4,3]范围内可配, 不支持 N 维度, 默认值 0, 遵循 caffe 规则	
	int32	num_axes	默认为-1, 表示从 axis 开始做几个维度的 reshape		支持[-1,3]范围内可配, 默认值-1 表示对 axis 起始的所有轴进行变换	
RNN	uint32	num_output	默认为 0, 输出的维度大小	RNN 网络	支持, 最大支持 16384	num_output<=16384
	bool	debug_info	默认为 false, 是否输出 debug_info		不支持配置	
	bool	expose_hidden	默认为 false, 表示是否增加额外输入输出层, 对于 RNN 是增加 1 层		支持配置 0 或 1;	
Scale	int32	axis	默认为 1, 输入的 bottom[0]做 scale 的起始维度, axis 不同则 bottom[1]的 shape 不同	计算两个输入的 elementwise 乘, 支持两个输入维度大小不一致	支持两种配置(1,-3), 默认值 1, 遵循 caffe 规则	w<=4096
	int32	num_axes	默认是 1, 当只有一个 bottom 的时候有效, 表示做 scale 的维度, -1 表示 axis 开始全部, 0 表示是参数只有一个 scalar		支持配置 1 或者-1, 与 axis 配套使用, 在张量与 C 维度向量相乘时配置为-1, 其余情况为 1, 遵循 caffe 规则;	



layer	参数类型	参数名	参数含义	描述	规格	
	bool	bias_term	默认为 false，表示是否加 bias		支持配置 0 或 1；	
Sigmoid	NA	NA	NA	sigmoid 激活函数	支持	w<=4096
Slice	int32	axis	默认为 1，表示切块的维度	沿某个维度切割输入的 blob，输出多个切割的 blob	支持[-4,3]范围内可配，不支持 N 维度，默认值 1，遵循 caffe 规则	top num<=32
	uint32	slice_point	表示输出的各个 blob 的切割点，不定义的则均分		支持，切割点数小于 32；	
	uint32	slice_dim	默认为 1，跟 axis 含义相同		支持配置 1~4，与 axis 配套使用，遵循 caffe 规则；	
Softmax	int32	axis	默认为 1，计算 softmax 的维度	计算 softmax 函数	支持配置，支持 c、h、w 维度；	axis 1: vector c<=65536 tensor c<=4096 axis 3: w<=4096
Split	NA	NA	NA	复制输入的 blob 到多个节点	支持	top num<=32
SPP	uint32	pyramid_height	spp 的高，即做 2^n 次 pooling	对不同的输入做金字塔的 pooling，输出一样大小	支持，2^n 次，n<7；pooling kernel 小于等于 255	等同 pool+reshape+concat 的规格
	PoolMethod	pool	默认为 MAX，pooling 的类型，另一种是 AVE		支持配置 0 或 1；	
TanH	NA	NA	NA	tanh 激活函数	支持	w<=4096
Threshold	float	threshold	默认为 0，输出比较的结果，1 或 0	输入与一个阈值相比较，大于则输出 1，小于等于则输出 0	支持配置，用线性 20.12 表示；	w<=4096



layer	参数类型	参数名	参数含义	描述	规格	
Normalize	bool	across_spatial	是否在 channel 方向做 L2 norm	L2 norm	支持:false: 在 channel 维度做 normalize; 不支持:true: 在 HW 维度做 normalize; (SSD 采用在 channel 维度做)	无限制
	FillerParameter	scale_filler	scale 参数初始化方式		支持, 在 prototxt 中定义 scale 参数, scale*L2-norm,如果没有这个参数就从 caffemodel 去读取	
	bool	channel_shared	scale 参数在 channel 间是否共享		不支持 true: 只用 scale_filler[0]或者 caffemodel 中参数[0]; 支持 false: 全用;	
	float	eps	默认为 1e-10, 用来防止被 0 除		不支持配置	
	float	sqrt_a	数据缩放比例, 防止定点处理溢出		支持, sqrt_a>0, 默认值为 1, 20.12 表示	
PSROIPooling	float	spatial_scale	进行 ROI Pooling 特征输入的尺寸相比原始输入的比例	类似 ROI Pooling, 不同位置的输出来自不同 channel 的输入	支持, 从原图坐标生成 feature map 坐标, 下到指令	$w \leq 4096$ $2048 / (w * (output_dim / 16)) / (group_size * group_size / 16) > 2$ $w * h \leq 32768 / (output_dim / 16)$
	int32	output_dim	输出 channel 数量		支持, 输出 ROI 的 channel 维度大小	
	int32	group_size	输出的 size, h 和 w 维度相同		支持, 输出 ROI 的 H 和 W 大小, H=W	
ROI Pooling	uint32	pooled_h	输出的 height	从 feature map 中根据	支持	$w < 4096$



layer	参数类型	参数名	参数含义	描述	规格	
	uint32	pooled_w	输出的 width	bounding box 的坐标 crop 出对应部分后再根据输出 size 的配置进行 max pooling	支持	2048/w/(c/16/16) >= 2 w*h <= 32768
	float	spatial_scale	进行 ROI Pooling 特征输入的尺寸相比原始输入的比例		支持，从原图坐标生成 feature map 坐标，下到指令	
Upsample	uint32	scale	DEPRECATED; 尺寸放大的比例，w 和 h 方向上一致，默认为 2	Pooling 操作的逆操作	支持，尺寸放大的比例，w 和 h 方向上一致，默认为 2，但不能配置，只能 2，要求与前面 max pooling 一致；	pooling mask: w<=4096 upsampling: w<=4096
	uint32	scale_h	DEPRECATED; h 方向尺寸放大的比例		不支持	
	uint32	scale_w	DEPRECATED; w 方向尺寸放大的比例		不支持	
	bool	pad_out_h	DEPRECATED; 是否需要在输出 H 维度-1;		不支持	
	bool	pad_out_w	DEPRECATED; 是否需要在输出 W 维度-1;		不支持	
	uint32	upsample_h	输出在 h 方向上的尺寸大小		支持，输出 H 大小，要求与实际算出来一致，否则告警；与上面的 scale 参数不会同时用，同时出现采用本条参数；	
	uint32	upsample_w	输出在 w 方向上的尺寸大小		支持，输出 W 大小，要求与实际算出来一致，否则告警；与上面的 scale 参数不会同时用，同时出现采用本	



layer	参数类型	参数名	参数含义	描述	规格	
					条参数;	
Permute	uint32	order	转置的顺序,	支持 faster rcnn 中定义的 permute 旋转,	只支持 order=0,order=2,order=3,order=1 模式	w<=4096
MatMul	uint32	dim_1	第一个矩阵的行方向维度	矩阵乘法的操作,第二个矩阵必须是转置的矩阵 (两个矩阵的 width 相同)	支持	8 bits: dim_2<=10208 16 bits: dim_2<=5104
	uint32	dim_2	第一个矩阵的列方向维度/第二个矩阵的列方向维度		支持	
	uint32	dim_3	第二个矩阵的行方向维度		支持	
RReLU	float	negative_slope	负值的斜率	随机化的 leaky ReLU, 在训练的时候 negative_slope 为给定均匀分布区间内的随机值, inference 的时候, negative_slope 固定为随机值的期望	支持	w<=4096
DepthwiseConv	uint32	num_output	输出的 channel 大小	对输入做 depthwise 的卷积	支持配置, 必须配置为与 input channel 值一样的大小;	w<=4096 kenel_h <= 2048/(w/(16*stride_w)*stride_w)
	bool	bias_term	默认为 1, 表示是否加 bias		支持配置 0 或 1;	
	uint32	pad	默认为 0, 表示补边的大小		支持, 配置范围 0~255;	
	uint32	kernel_size	kernel 的大小		支持 1~255	
	uint32	stride	默认为 1, stride 的大小		支持 1~255	
	uint32	dilation	默认为 1, dilation 的大小		不支持配置	



layer	参数类型	参数名	参数含义	描述	规格	
	uint32	pad_h	默认为 0，表示 height 方向的 padding		支持，配置范围 0~255;	
	uint32	pad_w	默认为 0，表示 width 方向的 padding		支持，配置范围 0~255;	
	uint32	kernel_h	kernel 的 height 大小		支持，配置范围 1~255;	
	uint32	kernel_w	kernel 的 width 大小		支持，配置范围 1~255;	
	uint32	stride_h	stride 的 height 大小		支持 1~255	
	uint32	stride_w	stride 的 width 大小		支持 1~255	
	uint32	group	默认为 1，group 的大小		不支持配置	
	int32	axis	默认为 1，axis 值之前的维度认为是独立输入，axis 之后的维度认为是空间维度，即在做卷积时会求和		不支持配置	
PassThrough	uint32	num_output	第一个矩阵的行方向维度	Yolo v2 网络中的 pass through 层	支持，其数值必须等于输入 channel 值 *block_height*block_width	w<=4096 block_height <= 2048/(w/(16*block_width)*block_width * (c/16))
	uint32	block_height	第一个矩阵的列方向维度/第二个矩阵的行方向维度		支持,配置范围 1~255, 且必须能够被输入 H 方向大小整除	
	uint32	block_width	第二个矩阵的列方向维度		支持,配置范围 1~255, 且必须能够被输入 W 方向大小整除	
CReLU	int32	axis	默认为 1，输入的 bottom[0]做	PVA 网络中的 CReLU	只支持配置为 1	同



layer	参数类型	参数名	参数含义	描述	规格	
			scale 的起始维度	层		power+concat+scale+relu 规格限制
	int32	num_axes	默认是 1，当只有一个 bottom 的时候有效，表示做 scale 的维度		只支持配置为 1	
	bool	bias_term	默认为 false，表示是否加 bias		只支持配置为 true	
Custom	repeated BlobShape	shape	输出的 blob 维度	用户自定义层，输出 feature map	支持[1,32]个输出配置	无限制
Proposal	uint32	feat_stride	默认值 16	-	只支持解析，不产生实际效果	-
	uint32	base_size	默认值 16	-		
	uint32	min_size	默认值 16	-		
	float	ratio	-	-		
	float	scale	-	-		
	uint32	pre_nms_topn	默认值 6000	-		
	uint32	post_nms_topn	默认值 300	-		
	float	nms_thresh	默认值 0.7	-		
	float	remove_thresh	默认值 0	-		



3.1.6.3 其他特性

- 支持任意中间层上报：除了网络的输出层，用户若还关心网络的某个中间层输出时可指定该层上报输出，中间层输出不影响后续隐藏层继续运算；
- 支持任意层高精度（16bit）计算模式：某些网络层对精度要求高时可以指定这些层为高精度模式，但是注意高精度模式的计算能力会比低精度要弱；
- 支持任意切分：用户有自定义私有层或者 NNIE 不支持的层时，需要对网络进行切分，同时也支持用户对支持的层进行切分，但是建议用户尽量减少分段段数，减少硬件执行过程中的软件交互开销，以期获得更高的性能；
- 支持 binary/ternary/sparse 参数的高压缩比定制压缩，用户提供的参数只有三种值且正负对称时，mapper 会自动进入 Ternary 模式；用户提供的参数只有两种值且包含 0 时，mapper 会自动进入 Binary 模式；用户提供的参数稀疏比在 80% 以上时自动进入 sparse 模式；这三种模式相对 normal 模式能提供更高的压缩比，以达到更高的带宽提升；
- 硬件预处理支持 YUV 转换成 RGB：从海思媒体 SOC 的其他模块中，比如 VI、VPSS、VGS（参考《HiMPP V4.0 媒体处理软件开发参考》）等模块中获取的均是 YUV 数据，而用户训练网络模型使用的一般是 RGB 数据。NNIE 硬件支持 YUV 输入转到 RGB，因此用户在实际使用场景中可以把 YUV 转为 RGB 交给 NNIE 硬件实现；
- 支持高效模式完成 in-place 激活运算：类似 Conv+ReLU 计算，使用 3.6 章节中的 in-place 描述方案，硬件执行效率更高；
- 支持多输入、多输出：支持多输入、多输出（包含任意上报的输出）的网络配置；
- 支持网络结构的自动优化：例如同轴 Concat 级联时将被优化成一层以提升处理效率；
- 支持自动识别片上 Cache 以提升处理性能：数据在隐藏层内传递时，如果数据足够小，可以完整的存入片内 RAM，mapper 将实现片上数据传递以提升处理性能；

3.1.7 NNIE 硬件资源利用率

在现有硬件规格下，用户可以参考以下准则来提升硬件利用率：

- 总体原则：
 - 一般情况下，相同计算量场景，尽量增加单层的计算量，减少网络层数，以减少层之间切换造成的利用率损失，提高端到端的利用率。
 - 尽量使用卷积、反卷积、Pooling、FC 层，提高硬件资源利用率，减少 LRN、MVN、Normalize、Softmax 层的使用。
 - 建议使用 RELU、Sigmod、Tanh、PRELU、RRELU 激活函数并使用 in-place 的配置方式（与前一层共享 blob），以便硬件高效完成计算；
- 特别地针对卷积、反卷积、Pooling、DepthwiseConv 层运算：
 - 输出特征数据尺寸在 16*16 时，读参数的 DDR 带宽与卷积计算基本匹配；特征数据尺寸更小时，计算单元受限于 DDR 带宽会无法全部利用。



- 硬件进行卷积计算时，数据按 16 组并行计算，输入的 C 维度是 16 倍数，输出的维度 C 是 4 的倍数，输出的 W 维度是 16 的倍数时，可以充分利用 MAC 的计算性能。

3.2 Prototxt 要求

NNIE mapper 对 prototxt 的输入层格式、layer 格式、激活层、Scale、Bias 层、RNN、LSTM 层及特殊的中间层上报、高精度配置、指定支持层有 CPU 执行等有特定的规范约束。

3.2.1 deploy.prototxt 输入层格式

deploy.prototxt 输入层支持如下两种格式，n 维度的 dim 值建议写 1，mapper 会根据参考图片路径中的图片张数自动生成 n 值：

格式一：

```
input: "data"
input_shape{
  dim:1
  dim:3
  dim:224
  dim:224
}
```

格式二：

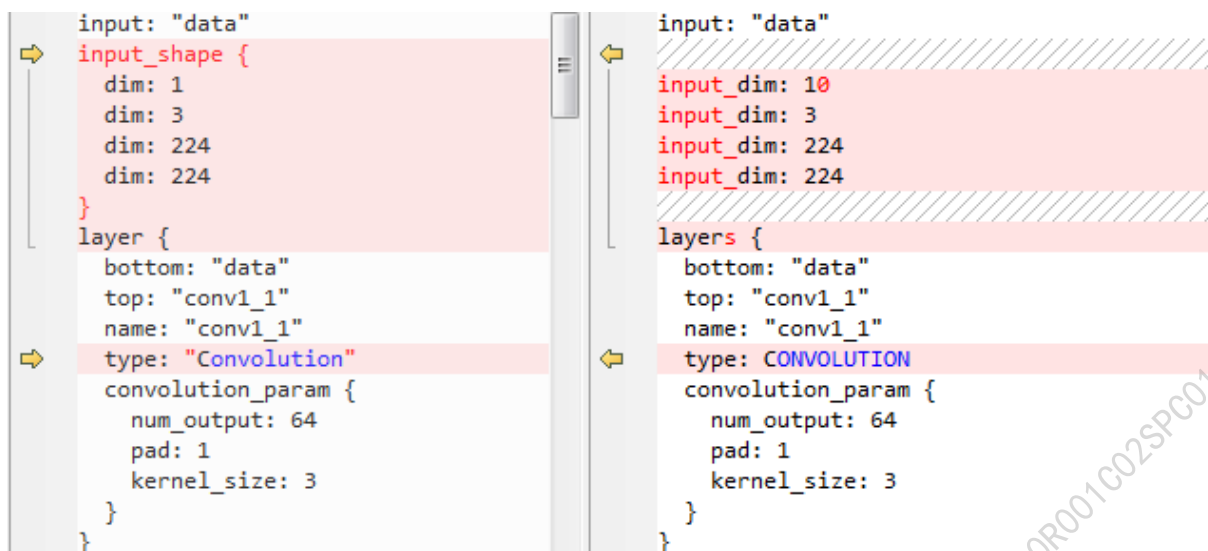
```
layer {
  name: "data"
  type: "Input"
  top: "data"
  input_param {
    shape: {
      dim: 1
      dim: 3
      dim: 227
      dim: 227
    }
  }
}
```



3.2.2 prototxt layer 描述格式

prototxt 格式上只支持图 3-29 左边的格式；层以 layer 开头，type 以带双引号字符串表示。

图3-29 NNIE 支持 prototxt 格式说明示意图（左边支持）



3.2.3 prototxt 激活层描述方式

- 1) batchnorm, scale, bias, relu, sigmoid, tanh、prelu、absval 激活层 bottom 与 top 名称相同时，即 in-place 方式，该层激活在 NNIE 中执行时会与被优化处理，性能更高。



```
layer {
  bottom: "res5c_branch2a"
  top: "res5c_branch2b"
  name: "res5c_branch2b"
  type: "Convolution"
  convolution_param {
    num_output: 512
    kernel_size: 3
    pad: 1
    stride: 1
    bias_term: false
  }
}

layer {
  bottom: "res5c_branch2b"
  top: "res5c_branch2b"
  name: "bn5c_branch2b"
  type: "BatchNorm"
  batch_norm_param {
    use_global_stats: true
  }
}

layer {
  bottom: "res5c_branch2b"
  top: "res5c_branch2b"
  name: "scale5c_branch2b"
  type: "Scale"
  scale_param {
    bias_term: true
  }
}
```

- 2) mapper 也支持非 in-place 配置激活函数，即 bottom 和 top 名称不同；此时 NNIE 上运行该激活层与前一层独立，但支持的激活类型更丰富。格式如下所示：



```
layer {
  bottom: "res5c_branch2a"
  top: "res5c_branch2b"
  name: "res5c_branch2b"
  type: "Convolution"
  convolution_param {
    num_output: 512
    kernel_size: 3
    pad: 1
    stride: 1
    bias_term: false
  }
}
layer {
  bottom: "res5c_branch2b"
  top: "bn5c_branch2b"
  name: "bn5c_branch2b"
  type: "BatchNorm"
  batch_norm_param {
    use_global_stats: true
  }
}
layer {
  bottom: "bn5c_branch2b"
  top: "scale5c_branch2b"
  name: "scale5c_branch2b"
  type: "Scale"
  scale_param {
    bias_term: true
  }
}
```

- 3) batchnorm 层必须有 batch_norm_param 参数，并且 use_global_stats 必须为 true，格式如下所示：



```
layer {  
  bottom: "res5c_branch2b"  
  top: "res5c_branch2b"  
  name: "bn5c_branch2b"  
  type: "BatchNorm"  
  batch_norm_param {  
    use_global_stats: true  
  }  
}
```

3.2.4 prototxt 针对 Scale/Bias 层书写规范

对于 Scale/Bias 层，NNIE 当前仅支持如下三种使用场景：

1. $c \times h \times w$ 的张量数据输入，每一个点对应一个 scale 值或 bias 值，即 scale、bias 参数也是 $c \times h \times w$ 的张量；
2. $c \times h \times w$ 的张量数据输入，每一个 channel 对应一个 scale 值或 bias 值，或者同时对应一个 scale 和一个 bias 值，即 scale、bias 参数是一个 c 维的向量；
3. c 维向量数据输入，每一个点对应一个 scale 值或 bias 值，或者同时对应一个 scale 和一个 bias 值，即 scale、bias 参数也是一个 c 维的向量。

Scale/Bias 层的参数来源分两种场景：

1. 在线场景：Scale/Bias 参数是网络某一层的输出，即在线计算得到；
2. 离线场景：Scale/Bias 参数从 caffemodel 中解析，即离线训练得到；

针对 Scale/Bias 层，prototxt 书写需符合如下规范要求：

- 在线场景要求 bottom 数目为 2，且顺序固定，bottom[0]为当前层的 Feature map 输入，bottom[1]为当前层的参数输入；



```
layer {  
    name: "scale1"  
    type: "Scale"  
    bottom: "mvn1"  
    bottom: "data2"  
    top: "scale1"  
    scale_param {  
        axis: 1  
        num_axes: 3  
        bias_term: false  
    }  
}
```

```
layer {  
    name: "bias1"  
    type: "Bias"  
    bottom: "mvn1"  
    bottom: "data2"  
    top: "bias1"  
    bias_param {  
        axis: 1  
        num_axes: 3  
    }  
}
```

- 离线场景 bottom 数目为 1，当 Scale/Bias 的输入为向量时，num_axes 支持配置为 1 或-1，两者意义相同；



```
layer {  
    name: "scale1"  
    type: "Scale"  
    bottom: "fc1"  
    top: "scale1"  
    scale_param {  
        axis: 1  
        num_axes: 1  
        bias_term: false  
    }  
}
```

```
layer {  
    name: "bias1"  
    type: "Bias"  
    bottom: "fc1"  
    top: "bias1"  
    bias_param {  
        axis: 1  
        num_axes: 1  
    }  
}
```

- 不论在线还是离线场景，Scale 层数据输入和参数输入都是 $c \times h \times w$ 的张量时，只支持 `bias_term` 为 `false`;



```
layer {
  name: "data2"
  type: "Input"
  top: "data2"
  input_param {
    shape {
      dim: 3
      dim: 27
      dim: 27
    }
  }
}
layer {
  name: "mvn1"
  type: "MVN"
  bottom: "data1"
  top: "mvn1"
  mvn_param {
    normalize_variance: false
    across_channels: true
  }
}
layer {
  name: "scale1"
  type: "Scale"
  bottom: "mvn1"
  bottom: "data2"
  top: "scale1"
  scale_param {
    axis: 1
    num_axes: 3
    bias_term: false
  }
}
```

- 不论在线还是离线场景，Scale/Bias 层的 axis 参数只能配置 1 或-3，两者意义相同；



```
layer {  
    name: "scale1"  
    type: "Scale"  
    bottom: "mvn1"  
    bottom: "data2"  
    top: "scale1"  
    scale_param {  
        axis: 1  
        num_axes: 3  
        bias_term: false  
    }  
}
```

```
layer {  
    name: "bias1"  
    type: "Bias"  
    bottom: "mvn1"  
    bottom: "data2"  
    top: "bias1"  
    bias_param {  
        axis: 1  
        num_axes: 3  
    }  
}
```

NNIE 当前支持的 3 种使用场景可以细分为如下 16 种具体场景：

表3-3 Scale 层 10 种场景配置

序号	数据输入	参数输入	axis	num_axes	bias_term
1	c*h*w 张量	在线输入, c*h*w 张量	1 或-3	-	false
2	c 维向量	在线输入, c 维向量	1 或-3	-	true
3	c 维向量	在线输入, c 维向量	1 或-3	-	false
4	c*h*w 张量	在线输入, c 维向量	1 或-3	-	true
5	c*h*w 张量	在线输入, c 维向量	1 或-3	-	false
6	c*h*w 张量	离线输入, c*h*w 张量	1 或-3	-1	false
7	c 维向量	离线输入, c 维向量	1 或-3	1 或-1	true



序号	数据输入	参数输入	axis	num_axes	bias_term
8	c 维向量	离线输入, c 维张量	1 或-3	1 或-1	false
9	c*h*w 张量	离线输入, c 维向量	1 或-3	1	true
10	c*h*w 张量	离线输入, c 维向量	1 或-3	1	false

表3-4 Bias 层 6 种使用场景配置

序号	数据输入	参数输入	axis	num_axes
1	c*h*w 张量	在线输入, c*h*w 张量	1 或-3	-
2	c 维向量	在线输入, c 维向量	1 或-3	-
3	c*h*w 张量	在线输入, c 维向量	1 或-3	-
4	c*h*w 张量	离线输入, c*h*w 张量	1 或-3	-1
5	c 维向量	离线输入, c 维向量	1 或-3	1 或-1
6	c*h*w 张量	离线输入, c 维张量	1 或-3	1

3.2.5 prototxt 针对 RNN 层书写规范

RNN 的基本计算公式如下 (与 Caffe 一致):

输入: x_t, h_{t-1}

输出: o_t, h_t

隐藏层: $h_t = \tanh(w_{xh}x_t + w_{hh}h_{t-1} + b_h)$

输出层: $o_t = \tanh(w_{ho}h_t + b_o)$

若有额外固定输入向量 x_{static} , 计算公式变为:

输入: x_t, h_{t-1}

输出: o_t, h_t

隐藏层: $h_t = \tanh(w_{xh}x_t + w_{hh}h_{t-1} + w_{sh}x_{static} + b_h)$

输出层: $o_t = \tanh(w_{ho}h_t + b_o)$

基于以上计算公式, RNN 的 prototxt 有以下 4 种场景:



3.2.5.1 通常场景 (expose_hidden=false, 1 输入, 1 输出)

Input: $x_t(1 \sim T)$

Output: $o_t(1 \sim T)$

Prototxt 示例如下, 有 1 个 bottom 和 1 个 top, 分别对应 $x_t(1 \sim T)$ 和 $o_t(1 \sim T)$:

- 若 recurrent_param 的 expose_hidden 没配, 默认值为 false;
- 不支持 recurrent_param 的 debug_info, 配置不会生效。

```
layer {
  name: "rnn1"
  type: "RNN"
  bottom: "data0_xt"
  top: "rnn1_ht"
  recurrent_param {
    num_output: 1000
    weight_filler {
      type: "uniform"
      min: -0.08
      max: 0.08
    }
    bias_filler {
      type: "constant"
      value: 0
    }
  }
}
```

3.2.5.2 隐藏参数显式输入输出 (expose_hidden=true, 2 输入, 2 输出)

Input: $x_t(1 \sim T), h_0$

Output: $o_t(1 \sim T), h_T$

Prototxt 示例如下, 有 2 个 bottom 和 2 个 top, 其中 bottom[0]对应 $x_t(1 \sim T)$, bottom[1]对应 h_0 , top[0]对应 $o_t(1 \sim T)$, top[1]对应 h_T ;

- h_0 的向量维度须与 recurrent_param 的 num_output 一致;
- 只有第一个 bottom 与第一个 top 有时间的维度。



```
layer {
  name: "rnn1"
  type: "RNN"
  bottom: "data0_xt"
  bottom: "data1_h0"
  top: "rnn1_ht "
  top: "rnn1_hT"
  recurrent_param {
    num_output: 1000
    expose_hidden=1
    weight_filler {
      type: "uniform"
      min: -0.08
      max: 0.08
    }
    bias_filler {
      type: "constant"
      value: 0
    }
  }
}
```

3.2.5.3 具有额外的固定输入（expose_hidden=false, 2 输入, 1 输出）

Input: $x_t(1 \sim T), x_{static}$

Output: $o_t(1 \sim T)$

Prototxt 示例如下, 有 2 个 bottom 和 1 个 top, 其中 bottom[0]对应 $x_t(1 \sim T)$, bottom[1] 对应 x_{static} , top 对应 $o_t(1 \sim T)$;

- $x_t(1 \sim T)$ 的向量维度, 必须与 x_{static} 的向量维度一致;
- 如果 recurrent_param 的 expose_hidden 没配, 默认值为 false;
- 不支持 recurrent_param 的 debug_info, 配置不会生效。



```
layer {
  name: "rnn1"
  type: "RNN"
  bottom: "data0_xt"
  bottom: "data0_xstatic"
  top: "rnn1_ht "
  recurrent_param {
    num_output: 1000
    weight_filler {
      type: "uniform"
      min: -0.08
      max: 0.08
    }
    bias_filler {
      type: "constant"
      value: 0
    }
  }
}
```

3.2.5.4 既有固定输入，又有显式的隐藏参数（expose_hidden=true，3 输入，2 输出）

Input: $x_t(1 \sim T), x_{static}, h_0$

Output: $o_t(1 \sim T), h_T$

Prototxt 示例如下，有 3 个 bottom 和 2 个 top，其中 bottom[0]对应 $x_t(1 \sim T)$ ，bottom[1]对应 x_{static} ，bottom[2]对应 h_0 ，top[0]对应 $o_t(1 \sim T)$ ，top[1]对应 h_T ；

- h_0 的向量维度必须与 recurrent_param 的 num_output 一致；
- $x_t(1 \sim T)$ 的向量维度，必须与 x_{static} 的向量维度一致；
- 只有第一个 bottom 与第一个 top 有时间的纬度。



```

layer {
  name: "rnn1"
  type: "RNN"
  bottom: "data0_xt"
  bottom: "data1_xstatic"
  bottom: "data1_h0"
  top: "rnn1_ht"
  top: "rnn1_hT"
  recurrent_param {
    num_output: 1000
    expose_hidden=1
    weight_filler {
      type: "uniform"
      min: -0.08
      max: 0.08
    }
    bias_filler {
      type: "constant"
      value: 0
    }
  }
}

```

3.2.6 prototxt 针对 LSTM 层书写规范

RNN 的基本计算公式如下 (与 Caffe 一致):

输入门: $i_t = \text{sigmoid}(w_{xi}x_t + w_{hi}h_{t-1} + b_i)$

遗忘门 $f_t = \text{sigmoid}(w_{xf}x_t + w_{hf}h_{t-1} + b_f)$

输入值: $g_t = \tanh(w_{xg}x_t + w_{hg}h_{t-1} + b_g)$

当前cell: $c_t = i_t * g_t + f_t * c_{t-1}$

输出门: $o_t = \text{sigmoid}(w_{xo}x_t + w_{ho}h_{t-1} + b_o)$

若有额外固定输入向量 x_{static} , 计算公式变为:

输入门: $i_t = \text{sigmoid}(w_{xi}x_t + w_{hi}h_{t-1} + w_{si}x_{static} + b_i)$

遗忘门 $f_t = \text{sigmoid}(w_{xf}x_t + w_{hf}h_{t-1} + w_{sf}x_{static} + b_f)$

输入值: $g_t = \tanh(w_{xg}x_t + w_{hg}h_{t-1} + w_{sg}x_{static} + b_g)$

当前cell: $c_t = i_t * g_t + f_t * c_{t-1}$

输出门: $o_t = \text{sigmoid}(w_{xo}x_t + w_{ho}h_{t-1} + w_{so}x_{static} + b_o)$



基于以上计算公式，LSTM 的 prototxt 有以下 4 种场景：

3.2.6.1 通常场景（expose_hidden=false, 1 输入，1 输出）

Input: $x_t(1 \sim T)$

Output: $h_t(1 \sim T)$

Prototxt 示例如下，有 1 个 bottom 和 1 个 top，分别对应 $x_t(1 \sim T)$ 和 $h_t(1 \sim T)$ ：

- 若 recurrent_param 的 expose_hidden 没配，默认值为 false；
- 不支持 recurrent_param 的 debug_info，配置不会生效。

```
layer {
  name: "lstm1"
  type: "LSTM"
  bottom: "data0_xt"
  top: "lstm1_ht"
  recurrent_param {
    num_output: 1000
    weight_filler {
      type: "uniform"
      min: -0.08
      max: 0.08
    }
    bias_filler {
      type: "constant"
      value: 0
    }
  }
}
```

3.2.6.2 隐藏参数显式输入输出（expose_hidden=true, 3 输入，3 输出）

Input: $x_t(1 \sim T), h_0, c_0$

Output: $h_t(1 \sim T), h_T, c_T$

Prototxt 示例如下，有 3 个 bottom 和 3 个 top，其中 bottom[0] 对应 $x_t(1 \sim T)$ ，bottom[1] 对应 h_0 ，bottom[2] 对应 c_0 ，top[0] 对应 $o_t(1 \sim T)$ ，top[1] 对应 h_T ，top[2] 对应 c_T ：

- h_0 ， c_0 的向量维度须与 recurrent_param 的 num_output 一致；
- 只有第一个 bottom 与第一个 top 有时间的维度；
- 不支持 recurrent_param 的 debug_info，配置不会生效。



```
layer {
  name: "lstm1"
  type: "LSTM"
  bottom: "data0_xt"
  bottom: "data1_h0"
  bottom: "data2_c0"
  top: "lstm1_ht"
  top: "lstm1_hT"
  top: "lstm1_cT"
  recurrent_param {
    num_output: 1000
    expose_hidden = 1
    weight_filler {
      type: "uniform"
      min: -0.08
      max: 0.08
    }
    bias_filler {
      type: "constant"
      value: 0
    }
  }
}
```

3.2.6.3 具有额外的固定输入（expose_hidden=false，2 输入，1 输出）

Input: $x_t(1 \sim T), x_{static}$

Output: $h_t(1 \sim T)$

Prototxt 示例如下，有 2 个 bottom 和 1 个 top，其中 bottom[0] 对应 $x_t(1 \sim T)$ ，bottom[1] 对应 x_{static} ，top 对应 $h_t(1 \sim T)$ ；

- $x_t(1 \sim T)$ 的向量维度，必须与 x_{static} 的向量维度一致；
- 只有第一个 bottom 与第一个 top 有时间的纬度；
- 如果 recurrent_param 的 expose_hidden 没配，默认值为 false；
- 不支持 recurrent_param 的 debug_info，配置不会生效；



```
layer {
  name: "lstm1"
  type: "LSTM"
  bottom: "data0_xt"
  bottom: "data1_xstatic"
  top: "lstm1_ht"
  recurrent_param {
    num_output: 1000
    weight_filler {
      type: "uniform"
      min: -0.08
      max: 0.08
    }
    bias_filler {
      type: "constant"
      value: 0
    }
  }
}
```

3.2.6.4 既有固定输入，又有显式的隐藏参数（`expose_hidden=true`，4 输入，3 输出）

Input: $x_t(1 \sim T), x_{static}, h_0, c_0$

Output: $o_t(1 \sim T), h_T, c_T$

Prototxt 示例如下，有 4 个 bottom 和 3 个 top，其中 bottom[0]对应 $x_t(1 \sim T)$ ，bottom[1]对应 x_{static} ，bottom[2]对应 h_0 ，bottom[3]对应 c_0 ，top[0]对应 $o_t(1 \sim T)$ ，top[1]对应 h_T ，top[2]对应 c_T ；

- h_0, c_0 的向量维度必须与 recurrent_param 的 num_output 一致；
- $x_t(1 \sim T)$ 的向量维度，必须与 x_{static} 的向量维度一致；
- 只有第一个 bottom 与第一个 top 有时间的纬度；
- 不支持 recurrent_param 的 debug_info，配置不会生效。



```
layer {  
  name: "lstm1"  
  type: "LSTM"  
  bottom: "data0_xt"  
  bottom: "data1_xstatic"  
  bottom: "data2_h0"  
  bottom: "data3_c0"  
  top: "lstm1_ht"  
  top: "lstm1_hT"  
  top: "lstm1_cT"  
  recurrent_param {  
    num_output: 1000  
    expose_hidden = 1  
    weight_filler {  
      type: "uniform"  
      min: -0.08  
      max: 0.08  
    }  
    bias_filler {  
      type: "constant"  
      value: 0  
    }  
  }  
}
```

3.2.7 prototxt 指定任意中间层上报

中间层是指不在网络段结尾处的层。

用户需要中间层结果输出时，需对应层的“top”域中添加“_report”标识符进行标注。

如果某一中间层有多个 top 都需要输出，用户可以为每一个 top 添加上报标注。



```
layer {
  name: "conv5 "
  type: "Convolution"
  bottom: "conv4"
  top: "conv5_report"
  convolution_param {
    num_output: 256
    kernel_size: 3
    pad: 1
    stride: 1
  }
}
```

mapper 解析_report 规则如下:

- top 后续无节点, 自然上报, _report 不增加上报点;
- top 对应的后续节点有多个 bottom, 且其中一个 bottom 是 cpu 层, 则该 top 上报;
- top 对应的后续节点是 cpu 层 (其中, cpu 层指 proposal、custom、_cpu 层);
- top 有后续节点, _report 增加上报点;
- custom 有 top 加_report, 报错;
- proposal 有 top 加_report, 不报错, 也不增加上报点;
- _cpu 有 top 加_report, 不报错, 也不增加上报点;
- data 层加_report, 不会报错, 也不会增加上报点;
- inplace 激活, _report 应加在 conv 层上, 原因是多个激活共享了 conv 的 blob, 因此这些层只输出一个 blob, 加在激活层上不会报错, 也不会增加上报点, 其他 inplace 层加_report 的情况, 按此规则同样处理;
- conv 加激活, 如果用户想在 conv 层上报, 必须把两个节点拆开 (激活写成 non-inplace 方式, 即激活的 top、bottom 不同名);

3.2.8 prototxt 指定任意层配置高精度

用户指定自定义计算精度 (compile_mode = 2) 时, 在对应层的层名后加上高精度 “_hp” (16 比特) 标记, 可实现指定任意层为高精度输入, 格式如下所示。



```
layer {
  name: "conv5_hp"
  type: "Convolution"
  bottom: "conv4"
  top: "conv5"
  convolution_param {
    num_output: 256
    kernel_size: 3
    pad: 1
    stride: 1
  }
}
```

只有在 `compile_mode = 2` 的情况下，才可以由用户通过“_hp”标记指定某些层为高精度处理，其他情况下（`compile_mode` 配置为 0 或 1），在层名后加高精度“_hp”标记，编译器会报错。

3.2.9 prototxt 指定某些层由 CPU 执行

对于 mapper 支持层，可以通过在 `name` 字段增加 `_cpu` 标记来指定该层切换为 `cpu` 执行（包含 CPU、DSP 等非 NNIE 执行的，均使用 `_cpu` 标志），格式如下所示。

```
layer {
  bottom: "rpn_cls_score"
  top: "rpn_cls_score_reshape"
  name: "rpn_cls_score_reshape_cpu"
  type: "Reshape"
  reshape_param {
    shape {
      dim: 0
      dim: 2
      dim: -1
      dim: 0
    }
  }
}
```



注意如下的限制：

- _cpu 不能添加在 data 层；
- data、custom、proposal 层后面不能接_cpu 层；
- _cpu 不能添加在 inplace 的激活层；
- _cpu 层后面 inplace 连接的激活层也是_cpu 层；
- _cpu 层后面不能接 upsample 层；

3.2.10 分段执行网络 prototxt 示例

针对 FasterRCNN、RFCN、SSD 等存在 Non-support 层的网络，NNIE 采用软硬件协同的方法来完成这类网络的运算。下面给出经典网络的分段以及一个构造网络的示例。

3.2.10.1 Faster RCNN 网络

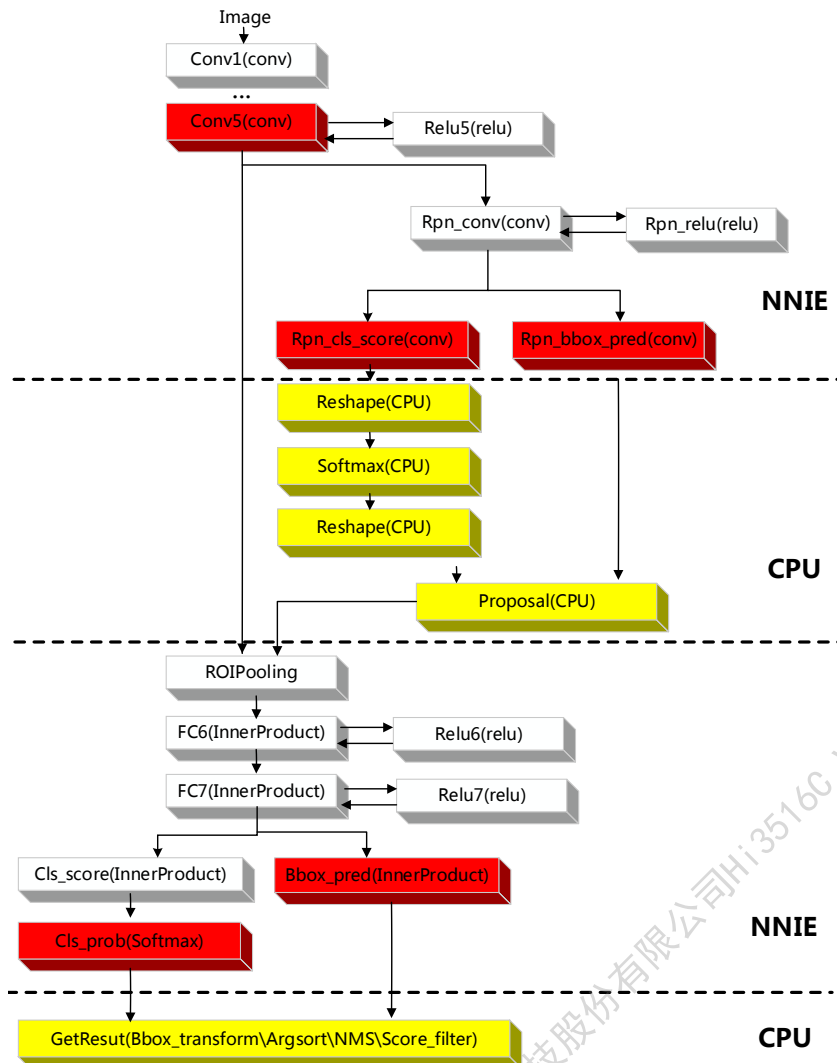
参考“[3.1.5 Non-support 层处理方式](#)”描述，Faster RCNN 分为“NNIE→非 NNIE→NNIE→非 NNIE”4 段来执行：

- 1) 自动切分方式：用户需要将 Proposal 层按约定的格式书写，以辅助 mapper 自动完成网络切割，切分后的网络见[图 3-28](#)；
- 2) 自定义切分方式一：如果用户因某些原因（比如从整体性能考虑）希望手动切割网络，将某些 NNIE 支持的层转由 CPU 处理（或者 DSP 等非 NNIE 硬件），可通过在 layer name 字段上增加_cpu 标记来完成，格式如下：

```
layer {  
  bottom: "rpn_cls_score"  
  top: "rpn_cls_score_reshape"  
  name: "rpn_cls_score_reshape_cpu"  
  type: "Reshape"  
  reshape_param { shape { dim: 0 dim: 2 dim: -1 dim: 0 } }  
}
```

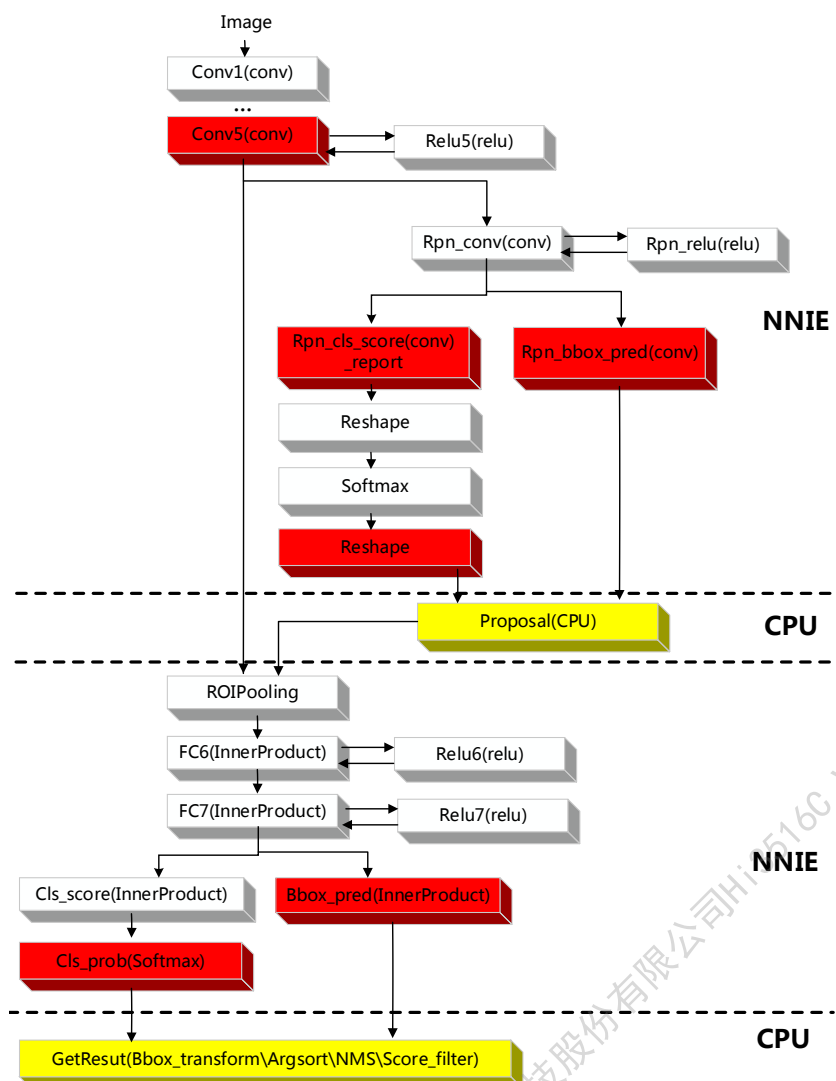
将[图 3-28](#) 的 Faster RCNN Rpn_cls_score 层后面的 Reshape->Softmax->Reshape 跟 Proposal 层切分在一起且由 CPU 执行的网络，如[图 3-30](#)：

图3-30 Faster RCNN 网络自定义分段示意图一



- 3) 自定义切分方式二：如果用户因某些原因需要 Rpn_cls_sore 的输出，而且跟 Proposal 相连接的 Reshape->Softmax->Reshape 希望 NNIE 执行，那么只需按照 3.2.7 prototxt 指定任意中间层上报中描述，指定 Rpn_cls_sore 中间层输出，如图 3-31：

图3-31 Faster RCNN 网络自定义分段示意图二



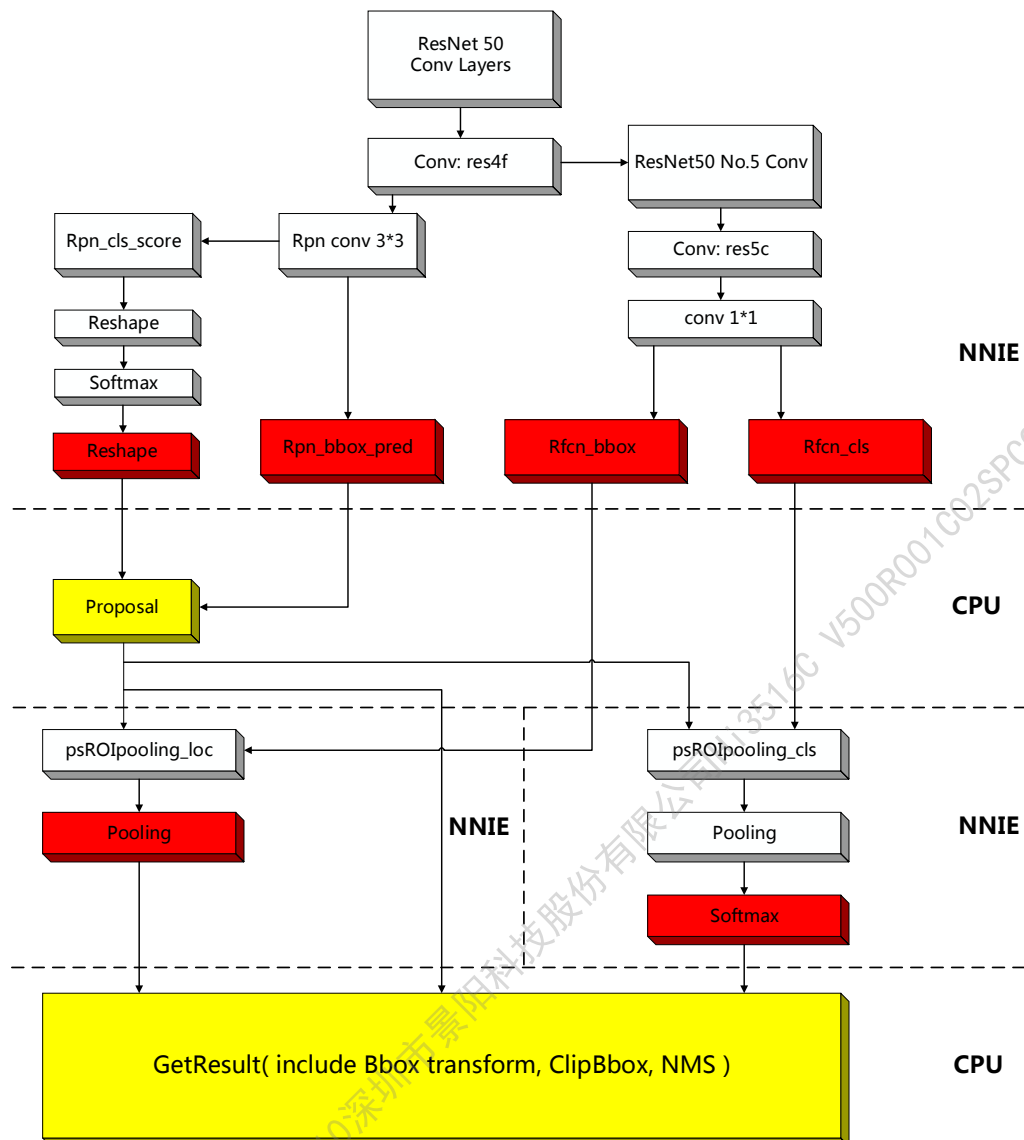
3.2.10.2 RFCN 网络

RFCN 跟 Faster RCNN 类似，RPN 部分的 Proposal 层(Bbox_transform、Argsort、NMS 等操作)输出的是 Bounding Box 信息，由此网络被分为 5 段 NNIE：

- 1) 第 1 段（NNIE 执行），卷积部分，包含 RPN 前面的部分；
- 2) 第 2 段（非 NNIE 执行），Proposal 部分，生成矩形框信息；
- 3) 第 3 段（NNIE 执行），左边的 PsRoiPooling 部分以及后续的 Pooling 等，得到矩形框调整值；
- 4) 第 4 段（NNIE 执行），右边的 PsRoiPooling 部分以及后续的 Pooling 等，得到矩形框置信度；

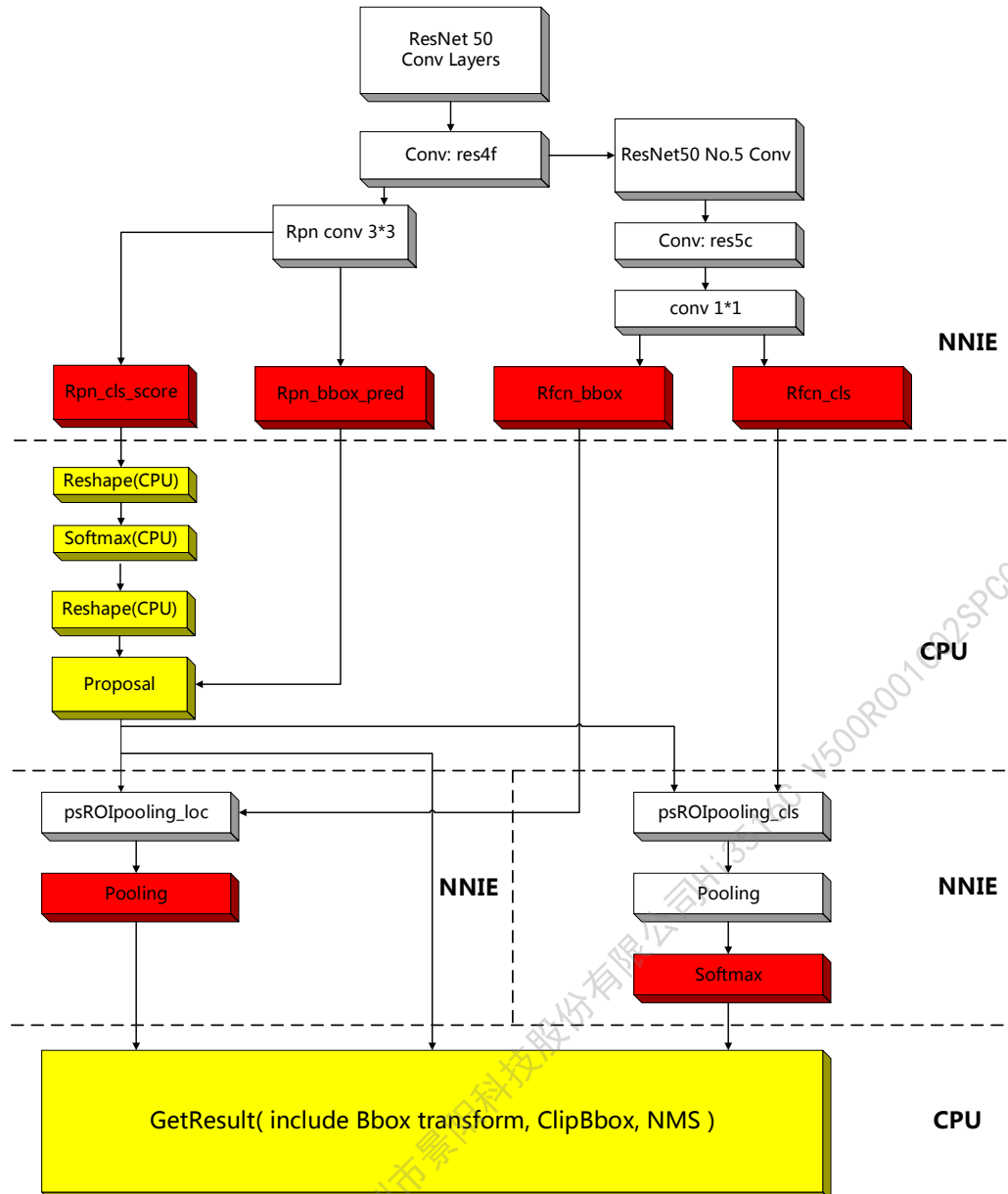
- 5) 第 5 段（非 NNIE 执行），结果获取部分，由 3、4 段得到的矩形框调整值和置信度经过 bbox_transform、argsort、NMS、ClipBox 等步骤获取最终的矩形框和置信度信息。

图3-32 RFCN 网络 mapper 自动分段示意图



类似 Faster RCNN，考虑将 Reshape->Softmax->Reshape->Proposal 一起又 CPU 执行，则变为如图 3-33：

图3-33 RFCN 自定义分段示意图

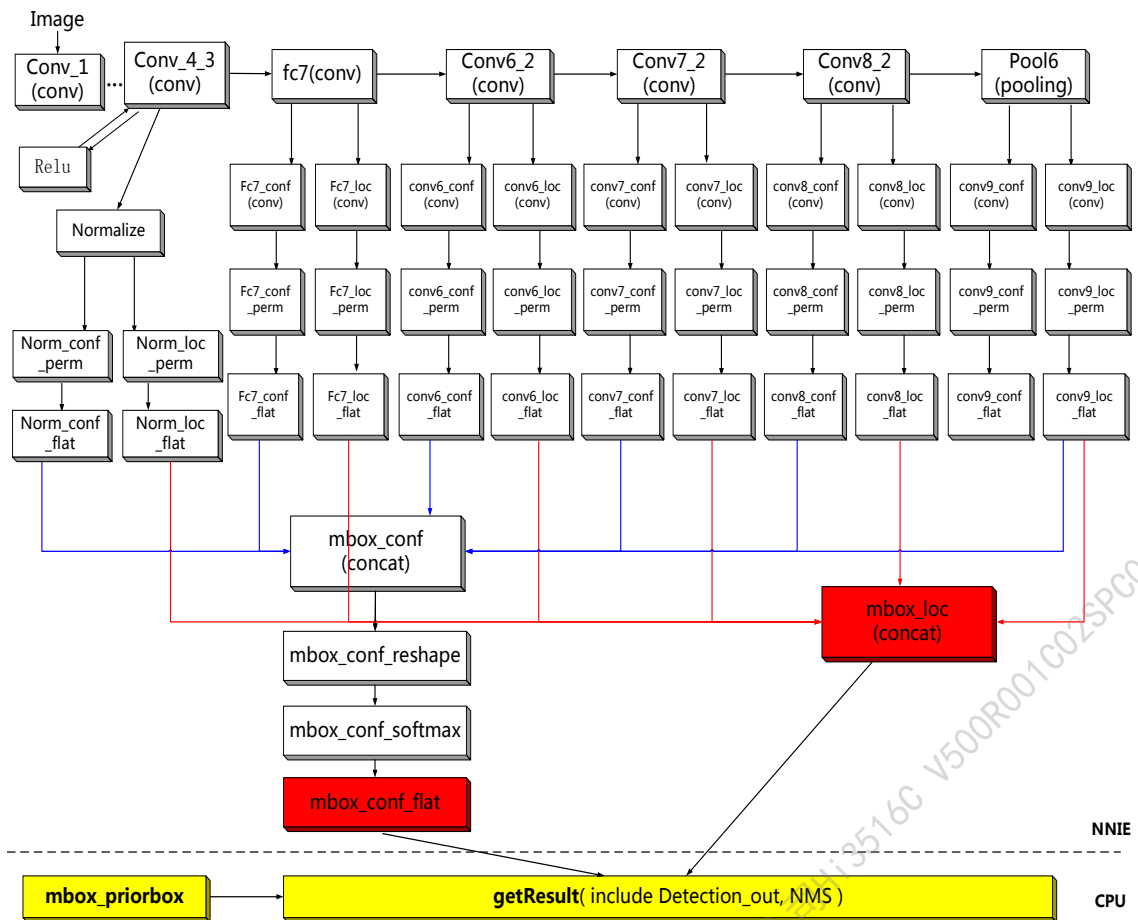


3.2.10.3 SSD 网络

SSD 分为 2 段，前面的部分由 NNIE 执行，后面的一段由 CPU 执行。在 SSD 的 prototxt 中，除跟 prior_box 相关的层不支持，其他的层均支持；建议用户采取删除 prototxt 中跟 prior_box 相关的层的方法来修改 prototxt。

- 1) 自定义切分方式一：NNIE 执行尽可能多的层，删除跟 prior_box 相关的层即可；

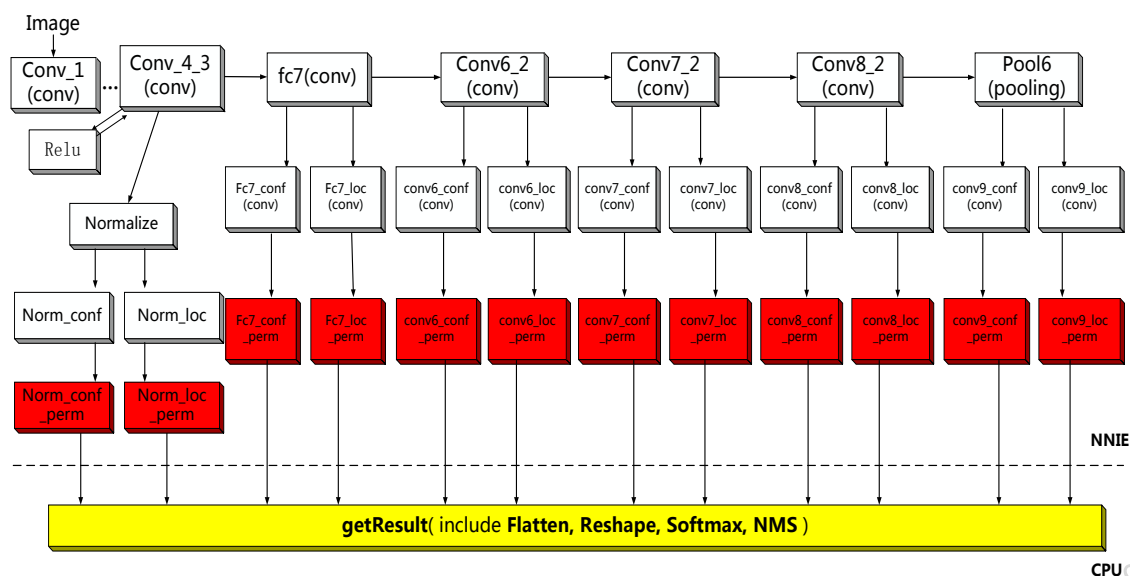
图3-34 SSD 自定义分割方式一



2) 自定义切分方式二：NNIE 执行到 permute 层，prototxt 删除其后的层；



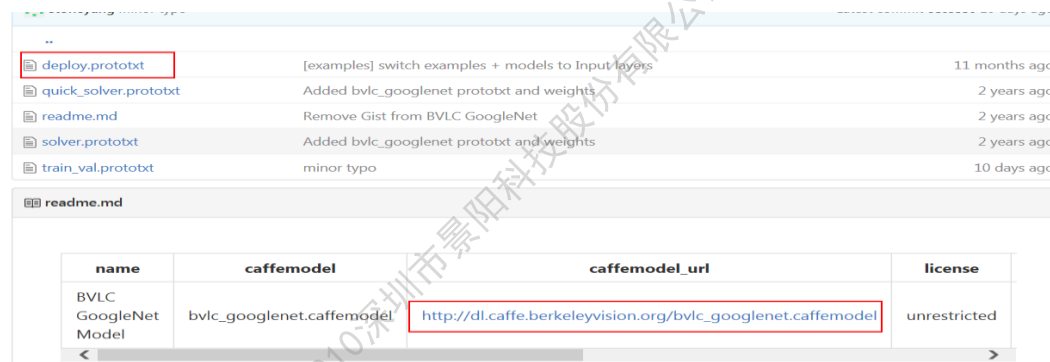
图3-35 SSD 自定义分割方式二



3.3 公开模型下载

3.3.1 Googlenet

下载链接: https://github.com/BVLC/caffe/tree/master/models/bvlc_googlenet



3.3.2 VGG16

prototxt 下载链接(注: prototxt 格式需按 3.2 Prototxt 要求修改):
http://www.robots.ox.ac.uk/~vgg/research/very_deep/



Models

We release our two best-performing models, with 16 and 19 weight layers (denoted as configurations *D* and *E* use the models.

The models are compatible with the [Caffe](#) toolbox. They are available in the Caffe format from the [Caffe Zoo](#)

- 16-layer model: [information page](#) in the Caffe Zoo, [weights \(528 MB\)](#), [layer configuration](#)
- 19-layer model: [information page](#) in the Caffe Zoo, [weights \(548 MB\)](#), [layer configuration](#)

The models can also be used with the [MatConvNet](#) toolbox. They are available in the MatConvNet format from

caffemodel 下载链接: https://deeptdetect.com/applications/list_models/

	Caffe	Tensorflow	Source
AlexNet	Y	N	BVLC
SqueezeNet	Y	N	DeepScale
Inception v1 / GoogleNet	Y	Y	BVLC / Google
Inception v2	N	Y	Google
Inception v3	N	Y	Google
ResNet 50	Y	Y	MSR
ResNet 101	Y	Y	MSR
ResNet 152	Y	Y	MSR
Inception-ResNet-v2	N	Y	Google
VGG-16	Y	Y	Oxford
VGG-19	Y	Y	Oxford

3.3.3 Resnet-50、resnet-101、resnet-152

prototxt 下载链接: <https://github.com/KaimingHe/deep-residual-networks/tree/master/prototxt>

Branch: master	deep-residual-networks / prototxt /	Create new file	Find file	History
Kaiming He prototxt		Latest commit 872cc54 on Feb 3 2016		
..				
ResNet-101-deploy.prototxt	prototxt	a year ago		
ResNet-152-deploy.prototxt	prototxt	a year ago		
ResNet-50-deploy.prototxt	prototxt	a year ago		

Caffemodel 下载链接: https://deeptdetect.com/applications/list_models/



Models are provided for image and text classification

Generic image models

These models are pre-trained on the 1000 classes of the ImageNet ILSVRC Challenge. They can be used as is or finetuned to more targeted applications.

	Caffe	Tensorflow	Source	Top-1 Accuracy (ImageNet)
AlexNet	Y	N	BVLC	57.1%
SqueezeNet	Y	N	DeepScale	59.5%
Inception v1 / GoogleNet	Y	Y	BVLC / Google	67.9%
Inception v2	N	Y	Google	72.2%
Inception v3	N	Y	Google	76.9%
ResNet 50	Y	Y	MSR	75.3%
ResNet 101	Y	Y	MSR	76.4%
ResNet 152	Y	Y	MSR	77%
Inception-ResNet-v2	N	Y	Google	79.79%
VGG-16	Y	Y	Oxford	70.5%
VGG-19	Y	Y	Oxford	71.3%

3.4 Linux 版 NNIE mapper 安装

linux 版本的 NNIE mapper 编译环境：ubuntu 14.04， gcc version 4.8.5

下文以 ubuntu14.04 为例安装 linux 版本的 NNIE mapper。

3.4.1 mapper 依赖库 Protobuf 安装与配置

3.4.1.1 安装包下载

请至 <https://github.com/google/protobuf/releases/tag/v3.5.1>，进入下载页面点击 protobuf-all-3.5.1.tar.gz：

图3-36 Protobuf 下载页面

Protocol Buffers v3.5.1

liujisi released this on 21 Dec 2017 · 25 commits to 3.5.x since this release

Assets

[protobuf-all-3.5.1.tar.gz](#)

[protobuf-all-3.5.1.zip](#)

[protobuf-cpp-3.5.1.tar.gz](#)



3.4.1.2 编译与安装

- 步骤 1. 将 protobuf-all-3.5.1.tar.gz 移至目标目录（假定为/home/test），终端输入 tar xvfz protobuf-all-3.5.1.tar.gz 进行解压。
- 步骤 2. 进入解压后目录，执行 mkdir build_tmp & cd build_tmp 创建并进入 build 目录，终端输入 cmake -Dprotobuf_BUILD_TESTS=OFF -Dprotobuf_BUILD_SHARED_LIBS=ON -DCMAKE_INSTALL_PREFIX=/home/test/protobuflib/protobuf-3.5.1 ../cmake，指定安装目录为 home/test/protobuflib/protobuf-3.5.1。
- 步骤 3. 继续在当前 build_tmp 目录进行编译，终端输入 make 命令完成编译。
- 步骤 4. 在终端输入 **make install** 完成 protobuf 的安装，最终安装目录即步骤 2 所指定目录。

----结束



注意

如果发生 Permission denied 导致无法新建、复制文件(夹)的情况，需要给目标文件夹 开通权限。

3.4.1.3 环境变量设置

把 Protobuf 的头文件和 lib 文件加入相应环境变量中，注意替换“/home/test”为用户自己的目标目录。

- 步骤 1. 在终端中输入 vi ~/.bashrc

- 步骤 2. 进入 vi 编辑 bashrc 文件，在文件末尾添加：

```
export PATH=/home/test/protobuflib/protobuf-3.5.1/bin:$PATH
export LD_LIBRARY_PATH=/home/test/protobuflib/protobuf-
3.5.1/lib:$LD_LIBRARY_PATH
export PKG_CONFIG_PATH=/home/test/protobuflib/protobuf-
3.5.1/lib/pkgconfig
```

- 步骤 3. 退出 vi，在终端输入 source ~/.bashrc

----结束



说明

用户也可以在当前终端中以命令形式依次输入步骤 2 追加内容，该方法只会对当前终端生效。

3.4.1.4 测试 Protobuf 是否成功安装

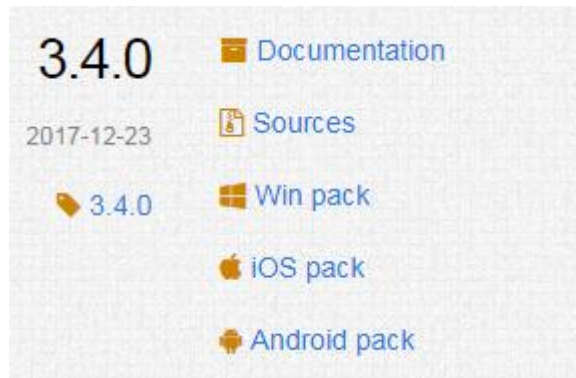
在终端输入 **protoc --version**，如果执行通过，并显示版本号，表明环境正确。



3.4.2 mapper 依赖库 OpenCV 安装与配置

3.4.2.1 OpenCV 安装包下载

请到 OpenCV 的官方网站: <http://opencv.org/releases.html> 下载 Opencv3.4.0。



3.4.2.2 OpenCV 依赖库安装

根据需要安装 OpenCV 依赖库，其中 cmake 是必需的。

步骤 1. 安装 cmake，输入：

```
sudo apt-get install cmake
```

步骤 2. 安装 GTK+ 2.0 或更高版本（可选），输入：

```
sudo apt-get install libgtk2.0-dev
```

步骤 3. 安装 libav 开发包：libavcodec-dev, libavformat-dev, libswscale-dev，输入：

```
sudo apt-get install libavcodec-dev libavformat-dev libswscale-dev
```

----结束

3.4.2.3 OpenCV 安装包解压

步骤 1. 进入 install 目录，将下载好的 opencv-3.4.0.zip 复制到 install。并在终端输入 unzip opencv_3.4.0.zip，得到 opencv-3.4.0 文件夹。

步骤 2. 在 opencv-3.4.0 内，建一个 build 文件夹，用来存放编译文件。

----结束

3.4.2.4 OpenCV 的安装前配置—使用 cmake 方式

步骤 1. 进入 build 目录，cd build

步骤 2. 在终端输入 cmake -D CMAKE_BUILD_TYPE=RELEASE -D CMAKE_INSTALL_PREFIX=/home/test/opencvlib/opencv3.4.0 -D WITH_GPHOTO2=OFF -D WITH_GTK=OFF -D WITH_CUDA=OFF -D WITH_FFMPEG=OFF -D WITH_LAPACK=OFF ..



说明

WITH_GPHOTO2=OFF 和 WITH_GTK=OFF WITH_CUDA=OFF -D WITH_FFMPEG=OFF -D WITH_LAPACK=OFF 用于关闭编译依赖项，建议关闭编译依赖项，降低编译复杂度。

----结束

3.4.2.5 OpenCV 的编译与安装

步骤 1. 将工作目录移动到 opencv 的 build 目录(本文是/install/opencv-3.4.0/build)，输入 `make`

该过程将运行较长时间，可以使用“`make -j ?`”代替 “`make`”命令进行并行编译，此处的“?”为要并行使用的 CPU 核心数（job 数），可跟电脑的具体 CPU 核数修改该数字。

步骤 2. Make 完毕后，在终端中输入：`make install` 完成 OpenCV 的安装。

----结束

3.4.2.6 OpenCV 环境变量设置

步骤 1. 在终端中输入 `vi ~/.bashrc` 进入 `vi` 编辑 profile 文件

步骤 2. 在文件末尾添加：

```
export PATH=/home/test/opencvlib/opencv3.4.0/bin:$PATH
export
LD_LIBRARY_PATH=/home/test/opencvlib/opencv3.4.0/lib:$LD_LIBRARY_PATH
export PKG_CONFIG_PATH=/home/test/opencvlib/opencv3.4.0/lib/pkgconfig
```

步骤 3. 退出 vi，在终端输入 `source ~/.bashrc`

----结束



说明

如果按照本文所述，OpenCV 会安装在/home/test/opencvlib/opencv3.4.0 目录中，如果变更了安装目录，环境变量设置时应该同步变更。

3.4.2.7 测试 OpenCV 是否安装成功

步骤 1. 将工作目录转到 OpenCV 解压目录下的/samples/cpp/处。

步骤 2. 然后尝试编译其中人脸识别的示例程序，输入 `gcc `pkg-config --cflags opencv` -o facedetect facedetect.cpp `pkg-config --libs opencv` -lstdc++`

如果顺利编译，说明 OpenCV 安装成功，并且会在该目录下生成 facedetect 的可执行程序。



说明

如果不是复制该命令的话，请注意 符号`不是单引号，而是反引号(大键盘数字 1 左边)。

步骤 3. 尝试运行该程序，输入 `./facedetect ../data/lena.jpg`，该程序会通过电脑的摄像头捕捉图像，并用椭圆表示检测到的人脸。如果成功运行，说明 OpenCV 安装成功。

----结束



注意

如果编译通过，但运行的时候提示 error while loading shared libraries: libopencv_objdetect.so.3.4 错误。请执行 `sudo ldconfig` 命令，再用 `sudo ldconfig -v | grep -i opencv` 查看动态链接库是否已刷新。可以手动更新一下索引库，输入 `sudo updatedb`。再运行该程序，若还出现问题，请重启系统。

3.4.3 mapper 依赖库 CUDA 安装与配置

若使用 CPU 版本的 mapper（参考 3.5.1 “nnie_mapper 版本说明”），可以跳过该节。

3.4.3.1 CUDA 安装包下载

请到 CUDA 的官方网站：<https://developer.nvidia.com/cuda-80-download-archive> 下载 CUDA 8.0。

Category	Options
Operating System	Windows, Linux, Mac OSX
Architecture	x86_64, ppc64le
Distribution	Fedora, OpenSUSE, RHEL, CentOS, SLES, Ubuntu
Version	16.04, 14.04
Installer Type	runfile (local), deb (local), deb (network), cluster (local)



说明

安全起见请在 shell 输入 `lspci | grep -i nvidia` 确认是否支持 CUDA。

----结束

3.4.3.2 CUDA 安装

请参考官网安装指南 <https://docs.nvidia.com/cuda/cuda-installation-guide-linux/index.html#ubuntu-installation>



3.4.4 mapper 本体安装

步骤 1. 在目标目录中新建 bin 目录，将 HiSVP_PC_Vx.y.z.w \tool\nnie\linux 下的 mapper 拷贝到该目录，下文以 Hi3559AV100 nnie_mapper_11 为例，gpu 加速及其他芯片版本类似。

```
@svpl62:~/bin$ ls
nnie_mapper_11
```

步骤 2. 在终端中输入 `vi ~/.bashrc` 进入 `vi` 编辑 profile 文件

步骤 3. 在文件末尾添加（注意替换为用户自己的目标路径）：

```
export PATH=/home/bin/:$PATH
```

步骤 4. 退出 vi，在终端输入 `source ~/.bashrc`

步骤 5. 以 sample 对应的 data 目录为例，进入 alexnet 目录

```
/sample/data/classification/alexnet$ ls
alexnet_func.cfg  alexnet_inst.cfg  alexnet_no_group_func.cfg  alexnet_no_group_inst.cfg  inst  model
```

目录中的 *.cfg 文件说明参考 3.5.1 “nnie_mapper 版本说明”；

model 目录中 prototxt 要求详见 3.2 “Prototxt 要求” 章节，caffemodel 严格遵循 caffe 规则，mean.txt 或者 *.binaryproto 为预处理减均值文件。

步骤 6. 返回 data 上一级目录，命令行输入

`nnie_mapper_11 ./data/classification/alexnet/alexnet_no_group_func.cfg` 即可运行 `nnie_mapper_11`，如下打印表示 `nnie_mapper` 正常执行：

```
sample$ nnie_mapper_11 ./data/classification/alexnet/alexnet_no_group_func.cfg
Mapper Version 1.1.2.0_B010 (NNIE_1.1) 1807281514145495
begin net parsing....
end net parsing
begin prev optimizing....
end prev optimizing....
begin net quantalizing....
end quantalizing
begin POST optimizing....
end POST optimizing
begin NNIE[0] mem allocation....
end NNIE[0] memory allocating
begin NNIE[0] instruction generating....
end NNIE[0] instruction generating
begin lbs binary code generating....
end lbs binary code generating
```

步骤 7. 在 cfg 指定生成一个后缀为 wk 的文件，即为仿真器上的可执行文件；



```
alexnet_no_group_func.cfg
1 [prototxt_file] ./data/classification/alexnet/model/bvlc_alexnet_no_group_deploy.prototxt
2 [caffemodel_file] ./data/classification/alexnet/model/bvlc_alexnet_no_group.caffemodel
3 [batch_num] 256
4 [net_type] 0
5 [sparse_rate] 0
6 [compile_mode] 0
7 [is_simulation] 1
8 [log_level] 2
9 [instruction_name] ./data/classification/alexnet/inst/alexnet_no_group_func
10 [RGB_order] BGR
11 [data_scale] 0.0039062
12 [internal_stride] 16
13 [image_list] ./data/classification/imagenet/image_ref_list.txt
14 [image_type] 1
15 [mean_file] ./data/classification/alexnet/model/imagenet_mean.binaryproto
16 [norm_type] 1

/sample/data/classification/alexnet/inst$ ls
alexnet_func.wk  alexnet_inst.wk  alexnet_no_group_func.wk  alexnet_no_group_inst.wk
```

----结束

3.5 Linux 版 nnie_mapper 使用说明

3.5.1 nnie_mapper 版本说明

表3-5 芯片和 nnie mapper 版本对应关系表

芯片	NNIE 版本	Mapper 版本
Hi3559AV100	nnie1.1	nnie_mapper_11 nnie_mapper_gpu_11
Hi3559CV100	nnie1.1	nnie_mapper_11 nnie_mapper_gpu_11
Hi3519AV100	nnie1.2	nnie_mapper_12 nnie_mapper_gpu_12
Hi3556AV100	不支持	不支持
Hi3516DV300	nnie1.2	nnie_mapper_12 nnie_mapper_gpu_12
Hi3516CV500	nnie1.2	nnie_mapper_12 nnie_mapper_gpu_12

nnie_mapper_xy 是纯 CPU 的 mapper 版本，nnie_mapper_gpu_xy 是 GPU 加速的 mapper 版本。



注意

- NNIE 板端运行、仿真，必须按上表的芯片型号和 mapper 版本匹配，否则可能出现意想不到的错误。
- GPU 版本的 nnie_mapper 都需要安装 CUDA8.0。
- GPU 版本 mapper 和 CPU 版本的 mapper 编译出来的 wk 会有不同的地方，原因是 cuda 版本和 CPU 版本浮点的表示引起，caffe 的 CPU 和 GPU 版本也会有些许差异。

3.5.2 配置文件说明

nnie_mapper 配置选项说明如表 3-6 所示。

表3-6 nnie_mapper 配置选项说明

配置选项	取值范围	描述
prototxt_file	-	网络描述文件，详细要求见 3.2 “Prototxt 要求”，描述外支持情况与 caffe 相同。
net_type	{0, 1, 2}	网络的类型。 0: CNN（不包含 LSTM/RNN/ROI Pooling/PSROI Pooling 的任意网络）； 1: ROI/PSROI（包含 ROI Pooling 和 PSROI Pooling 的网络）； 2: Recurrent（包含 LSTM、RNN 的网络）；
caffemodel_file	-	网络模型数据文件。
image_list	-	NNIE mapper 用于数据量化的参考图像 list 文件或 feature map 文件。该配置跟 image_type 相关。 NNIE mapper 量化时需要的图片是典型场景图片，建议从网络模型的测试场景随机选择 20~50 张作为参考图片进行量化，选择的图像要尽量覆盖模型的各个场景（图像要包含分类或检测的目标，如分类网的目标是苹果、梨、桃子，则参考图像至少要包含苹果、梨、桃子。比如检测人、车的模型，参考图像中必须由人、车，不能仅使用人或者无人无车的图像进行量化）。图片影响量化系数，选择典型场景的图片计算出来的量化系数对典型场景的量化误差越小。所以请不要选择偏僻场景、过度曝光、纯黑、纯白的图片，请选择识别率高，色彩均匀的典型场景图片。 网络中如果存在多个输入层，则需要配置多个 image_list 项，顺序、个数与 prototxt 完全对应。 如果网络的数据输入是灰度或者 RGB 图像输入，即 image_type 配置不为 0，image_list 配置为所有参考图



配置选项	取值范围	描述
		片的 list，内容示意如下图图示，图片的格式支持以下几种： ".bmp", ".dib", ".jpeg", ".jpg", ".jpe", ".jp2", ".png", ".webp", ".pbm", ".pgm", ".ppm", ".sr", ".ras", ".tiff", ".tif", ".BMP", ".DIB", ".JPEG", ".JPG", ".JPE", ".JP2", ".PNG", ".WEBP", ".PBM", ".PGM", ".PPM", ".SR", ".RAS", ".TIFF", ".TIF" <pre>image_ref_list.txt 1 ../data/classification/imagenet/ref/ILSVRC2012_val_00000001.JPEG 2 ../data/classification/imagenet/ref/ILSVRC2012_val_00000002.JPEG 3 ../data/classification/imagenet/ref/ILSVRC2012_val_00000003.JPEG 4 ../data/classification/imagenet/ref/ILSVRC2012_val_00000004.JPEG</pre> 如果网络的输入是 feature map 或者 FC 向量输入，即 image_type 配置为 0，将 c*h*w 个点（即一个完整的张量）以浮点文本的形式输出在一行内，点与点之间以空格或逗号分隔。如果是多帧输入，则每一行输出一个完整的张量。图示如下： <pre>single_custom_multi_top_2_custom_1.txt 1 0.852727574554 2.22428260776 7.28222365077 3.63192138208 4.44889580721 9.30500569598 7.51946751475 8.61053507313 5.11505773185 0.120343927741 1.55175787525 2.31107244679 7.32313007852 1.67330342699 9.80961173192 6.58306217981 8.58678546437 0.375591352241 2.96989362764 5.70649920558 5.16233427138 9.16785497049 3.61658342832 5.07637698021 5.11637303464 5.80114673548 9.99059787443 4.91098978583 8.48376954078 6.55287913647 7.33294496285 4.70459812637 6.85133955872 0.136886336092 1.42374282923 8.30464445037 9.1317536825 8.29587184813 0.294779163483 9.83021344124 3.69606456616 1.06406590556 7.67306035696 0.897114288304 4.88711021009 6.59023263591 7.48311238822 2.725971777 3.46687892092 2.99688353124 1.57899233521 2.83895671381 5.2338373429 2.66663094513 7.56908910552 6.84177125319 5.16009163347 6.81419291752 3.82894977639 8.12135448168 6.48772744135 7.2850829301 8.49627904474 1.11722630204</pre> Recurrent 输入时，格式等同于 feature map 输入，每行一个向量，一句话写成连续的多行，多句量化时需要将每一句的帧数都补齐为最大帧数。
image_type	{0,1,3,5}	表示网络实际执行时输入给网络的数据类型，该配置跟 image_list 相关。 0：表示网络数据输入为 SVP_BLOB_TYPE_S32（参考《HiSVP API 参考》）或者向量的类型（VEC_S32 和 SEQ_S32）；此时要求 image_list 配置为 feature map 文件； 1：表示网络数据输入为 SVP_BLOB_TYPE_U8（普通的灰度图和 RGB 图）类型；此时要求 image_list 配置是 RGB 图或者灰度图片的 list 文件； 3：网络数据输入为 SVP_BLOB_TYPE_YUV420SP 类型； 5：网络数据输入为 SVP_BLOB_TYPE_YUV422SP 类型； 当配置为 3 或者 5 时，image_list 配置为 RGB 图片的 list 文件。
norm_type	{0, 1, 2, 3,}	表示对网络数据输入的预处理方法。注意 image_type



配置选项	取值范围	描述
	4, 5}	配置为 0 时, <code>norm_type</code> 只能配置为 0; <code>image_type</code> 配置为 3 或者 5 时, 网络输入数据为 YUV 图像, 但是 NNIE 硬件会根据 <code>RGB_order</code> 配置项自动转为 RGB 或者 BGR 图像, 此时 <code>norm_type</code> 配置方法跟 <code>image_type</code> 为 1 时一致。 0: 不做任何预处理; 1: mean file, 减图像均值; 2: channel mean_value, 减通道均值; 3: data_scale, 对图像像素值乘以 data_scale; 4: mean file with data_scale, 减图像均值后再乘以 data_scale; 5: channel mean_value with data_scale, 减通道均值后再乘以 data_scale。
data_scale	(1/4096, FLT_MAX) default: 0.00390625	数据预处理缩放比例, 配置为浮点数, 配合 <code>norm_type</code> 使用 本参数可省略, 默认为 $0.00390625 = 1/256$。FLT_MAX 等于 $3.402823466e+38$。
mean_file	-	<code>norm_type</code> 为 1、4 时, 表示均值文件 xxx.binaryproto; <code>norm_type</code> 为 2、5 时, 表示通道均值文件; <code>norm_type</code> 为 0、3 时, 用户也需要配置 <code>mean_file</code> 项, 但具体内容可以是一个无效路径, 比如 null; 通道均值文件 mean.txt 中每一行的浮点数表示对应的通道均值, 如单通道只有一个值。
batch_num	[0, 256] default: 256	0/1: single (单张) 模式; >1: batch (多张) 模式。 采用 single 模式 mapper 一个任务只能处理一张图片, 内部存储全部为一张图片分配, 减少数据调度次数。 采用 batch 模式, 在计算 FC 时 batch_num 张图片同时计算, 计算资源利用率高。
sparse_rate	[0, 1] default: 0	NNIE 引擎采用了参数压缩技术以减少带宽占用, 为了提高压缩率, 可对 FC 参数进稀疏处理。 用户通过 <code>sparse_rate</code> 数值指定多少比例的 FC 参数稀疏为 0, 例如配 0.5, 则 FC 参数有 50% 将被稀疏为 0, 由于数据变的稀疏, 压缩模块会获得更好的压缩率。稀疏值越高, 计算 FC 时所需参数带宽越低, 但精度会有所下降。
compile_mode	{0, 1, 2}	0: Low-bandwidth(低带宽模式, 默认): 通过量化算法使参数与数据位宽最少, 使系统所需带宽达到最



配置选项	取值范围	描述
	default: 0	小，但会有精度损失； 1: High-precision(高精度模式): 结果精度最好，但是性能会下降； 2: User-specify(用户配置模式): 需要用户在 prototxt 中标明所有使用高精度计算的层，标注规则请见 prototxt_file 说明；
compress_mode	{0, 1} default: 0	配置压缩模式。 0: Normal 模式（包含 Normal、Ternary、Binary、Sparse 四种压缩模式的自动切换）； 1: Bypass 模式，关闭压缩功能。 要求： 可不填，默认为 Normal 模式；用户提供的参数只有三种值且正负对称时，nnie_mapper 会自动进入 Ternary 模式；用户提供的参数只有两种值且包含 0 时，nnie_mapper 会自动进入 Binary 模式；
max_roi_frame_cnt	[1, 5000] default: 300	包含 ROI/PSROI 网络的 RPN 阶段输出的候选框最大数目。 默认值：300。
roi_coordinate_file	-	Mapper 在网络模型转化过程中输入给 ROI Pooling 或 PSROI Pooling 层的配置参数，用于指定 ROI 框的坐标信息，每一行五个值，分别代表 batch_index(int)、left_x (float)、top_y (float)、right_x (float)、bottom_y (float)，不同的框以换行符分隔。 框坐标是在 caffe 中使用输入给 mapper 的 image_list 的相同图片运行到 RPN 层的输出结果，如 Faster RCNN 网络中 Proposal 层的 top 为 rois，在 caffe forward 结束后，通过 np.savetxt('rois.txt', net.blobs['rois'].data[...], fmt="%.6f") 保存框坐标为文件。需要保证两者图片输入顺序相同，同时要保证 caffe 运行时输入给网络的分辨率跟配置给 mapper 的 prototxt 中的分辨率相同。 For example: 0 734.01 147.02 806.03 294.04 0 723.05 157.06 818.07 306.08 1 749.09 170.10 817.11 310.12 1 678.13 220.14 855.15 374.16 如果一个网络中有多个 ROI Pooling 或 PSROI Pooling 层，则需要配置多行坐标文件，个数与 ROI Pooling 或 PSROI Pooling 层个数对应，配置的顺序也需要与 prototxt 内对应层顺序相同；
is_simulation	{0, 1}	网络模型转化类型。



配置选项	取值范围	描述
	default: 0	0: Chip, 芯片模式, 网络模型转化成在芯片上加载的 wk 文件, 指令仿真也使用此模式; 1: Simulation, 仿真模式, 网络模型转化成在 PC 端仿真上加载的 wk 文件, 功能仿真使用此模式;
instructions_name	string length < 120 default: inst	nnie_mapper 生成的知识库文件名称。 默认生成如下格式的知识库名: inst.wk; 用户也可以自行修改生成的知识库名字。
internal_stride	{16, 32} default: 16	用户根据 DDR 颗粒对应的最佳读写效率配置中间结果的对齐方式。 要求: DDR3 对应 16, DDR4 对应 32, 可不填, 默认为 16;
is_check_prototxt	{0, 1} default: 0	检查网络描述文件标志。 0: mapper 模式, 对 prototxt、caffemodel 等进行转化。 1: 网络过滤器模式, 对 prototxt 文件是否符合支持规格进行检查。
log_level	{0, 1, 2, 3} default: 0	设置是否开启日志文件, 以及配置打印的等级, 本参数可省略, 当省略时, 为不打印日志文件。 0: 打印 main 函数流程, cfg 文件等信息; 1: 打印 nnie_mapper 解析到的文件信息, 包含 image_list、prototxt、内存分配过程; 2: 打印中间表示信息; 3: 打印详细信息, 有大量文件输出, 转化耗时较长, 请谨慎使用;
recurrent_tmax	[1, 1024] default: 1024	Recurrent 网络 (包含 LSTM/RNN 层) 每一句话的最大帧数, 支持 [1, 1024] 范围内的配置, 减小配置值可以减小临时缓存大小。
RGB_order	{RGB, BGR} default: BGR	image_type 设置为 0 时, 该参数无效; image_type 设置为 1 时, 表示输入给网络的 RGB 图像的 RGB 三通道的顺序; image_type 设置为 3、5 时, 表示 YUV 图像数据转成 RGB Planar 或者 BGR Planar 图像输入给网络。 本参数可省略。

【注意】



Custom 层需要为 nnie_mapper 配置输入层配置项，以下例子展示包含一个 data 层和一个双输出 Custom 层的配置文件，

```
1 [prototxt_file] ./sigle_custom_multi_top_2.prototxt
2 [caffemodel_file] ./sigle_custom_multi_top_2.caffemodel
3 [net_type] 0
4 [batch_num] 1
5 [norm_type] 3
6 [mean_file] ignore ①
7 [image_type] 1
8 [image_list] ./images
9 [norm_type] 3
10 [mean_file] ignore ②
11 [image_type] 0
12 [image_list] ./sigle_custom_multi_top_2_custom_1_1
13 [norm_type] 3
14 [mean_file] ignore ③
15 [image_type] 0
16 [image_list] ./sigle_custom_multi_top_2_custom_1_2
17
```

即当前用例有一个 data 层，为其配置①号框内的配置项，Custom 有两个 top，为其配置②③两个框内的配置项，image_list 为一个文件名，文件内容是 Custom 对应 top 输出的 caffe 结果，格式见 image_list 配置项说明，文本格式的浮点数，以空格或逗号分隔，一个 tensor(c*h*w)一行；

3.5.3 nnie_mapper 输出

nnie_mapper 默认输出如下文件：

- *.wk: mapper 运行成功时生成的知识库文件；用户可通过 instructions_name 配置选项更改保存路径和文件名。
- cnn_net_tree.dot: mapper 运行成功时生成的网络结构 dot 描述；保存在工程所在目录中，使用 graphviz 可显示为 dot 视图。
- mapper_error.log: mapper 运行出错时生成的错误日志。
- mapper_debug.log: mapper 运行日志。

3.6 仿真库使用说明

NNIE PC 端仿真库分为功能仿真(nniefc*.lib)、指令仿真(nnieit*.lib)和性能仿真：

- 功能仿真仅从功能一致性的角度去模拟硬件，有 CUDA 加速功能，速度较快；
- 指令仿真从指令一致性的角度去模拟硬件，速度较慢；
- 性能仿真输出各层的带宽、cycle 数仿真结果；性能仿真默认配置下跟指令仿一并执行。

功能仿真、指令仿真、硬件三者的结果保持完全一致。

3.6.1 使用限制

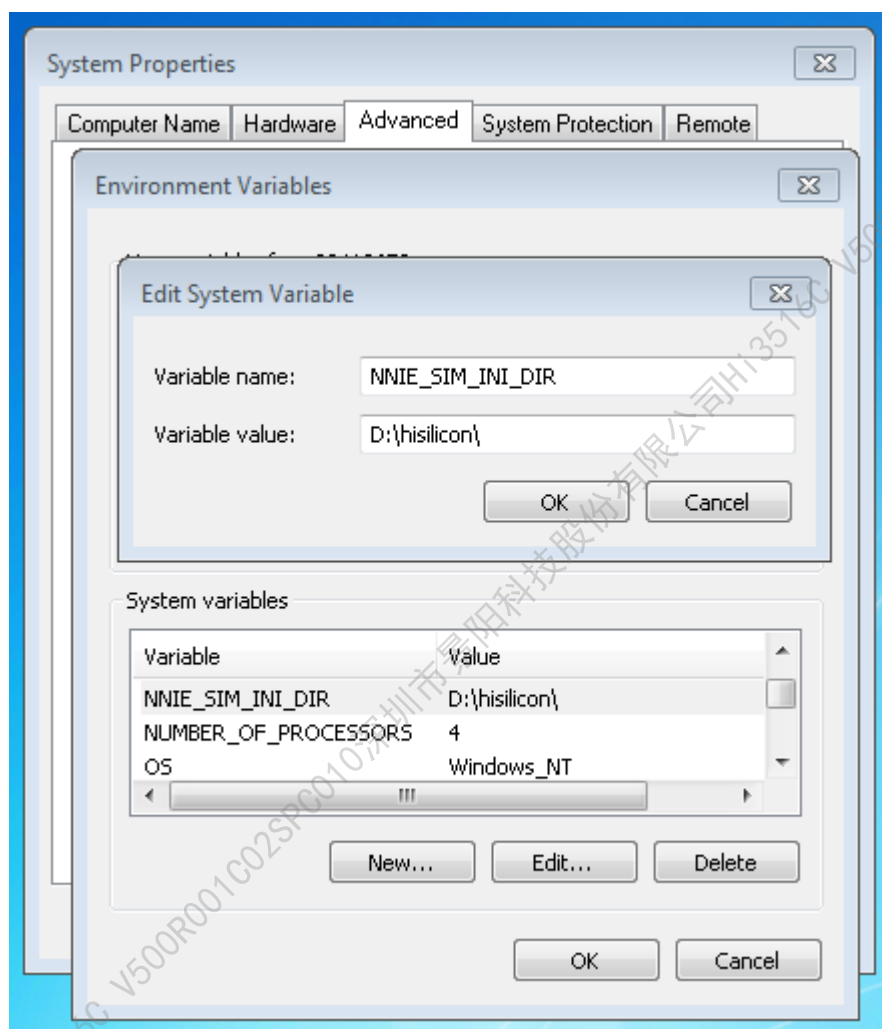
当前版本的仿真库仅支持单线程执行，不支持多线程仿真。

3.6.2 配置文件设置

仿真库配置文件 `nnie_sim.ini` 可以控制仿真库的中间层结果输出、CUDA 加速配置、性能仿真和指令仿真开关等选项。

- 仿真库首先尝试从可执行程序同级文件夹载入配置文件 `nnie_sim.ini`;
- 用户也可通过配置系统环境变量 `NNIE_SIM_INI_DIR` 指定配置文件 `nnie_sim.ini` 所在文件夹路径；如图 3-37 表示 ini 配置文件在 `D:\hisilicon\` 目录中。
- 此外，仿真库还会尝试从以下固定系统路径载入配置文件 `nnie_sim.ini`

图3-37 配置文件示意图



Windows 环境： `C:/hisilicon/nnie_sim.ini`



用户若需要修改仿真库配置项，可以根据上述不同方式，将 HiSVP_PC_Vx.x.x.x/software/hisilicon 中的配置文件 `nnie_sim.ini` 模板拷贝到可执行程序执行同级文件夹、环境变量 `NNIE_SIM_INI_DIR` 指定路径或固定系统路径（windows 的 `C:/hisilicon/`），并修改对应配置项。

配置文件 `nnie_sim.ini` 加载优先级：可执行程序执行同级文件夹 > 环境变量 `NNIE_SIM_INI_DIR` 指定路径 > 固定系统路径

若上述三种方式仿真库读取配置文件 `nnie_sim.ini` 均失败，则使用默认配置。

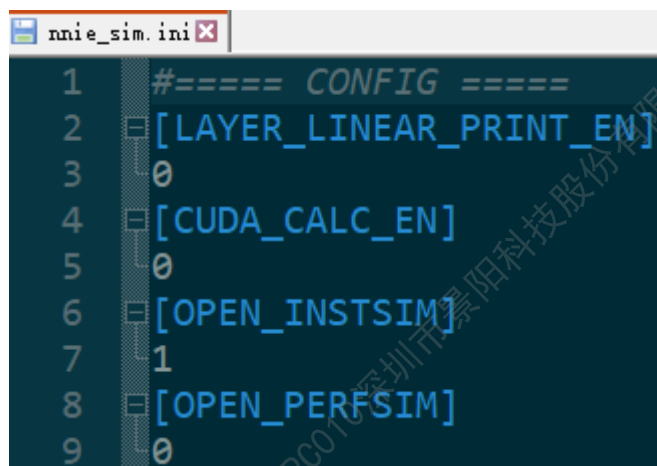


注意

- 在仿真器 sample 的 Visual Studio 解决方案 (HiSVP_PC_Vx.x.x.x/software/sample_simulator/simulator_sample_vc14.sln) 中执行 sample 项目程序时(以项目 `sample_func_vc14` 为例)，程序执行同级文件夹为项目名文件夹(如 `HiSVP_PC_Vx.x.x.x/software/sample_simulator/sample_func_vc14/`)，欲使用程序执行同级文件夹的方法载入配置文件，需把配置文件 `nnie_sim.ini` 拷贝到该目录下。
- 在 RuyiStudio 中运行仿真 sample 程序的配置方法见 5.4.1 “配置文件可视化编辑”。

3.6.3 配置文件说明

中括号[]内指定配置选项，第二行以 0/1 控制配置选项开关，1 表示使能。



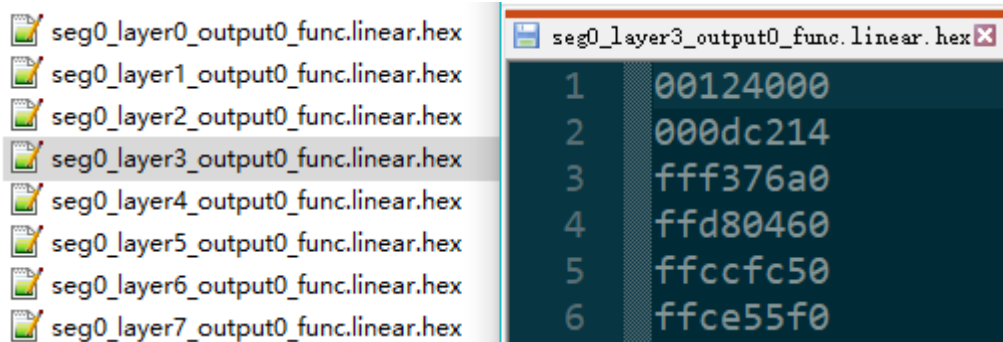
[LAYER_LINEAR_PRINT_EN] 控制仿真中间层结果保存使能。该项对功能仿真库和指令仿真库均生效。如

功能仿真：

`sample_simulator\sample_func_vc14\func_layer_output_linear\seg0_layer0_output0_func.linear.hex`

指令仿真：

`sample_simulator\sample_inst_vc14\inst_layer_output_linear\seg0_layer0_output0_inst.linear.hex`



中间层结果以 32 位定点数 20.12 格式保存。

[CUDA_CALC_EN] 控制 CUDA 计算使能。该项仅对功能仿真库生效。

仿真 CUDA 配置方法详见 3.6.4 “功能仿真 CUDA 加速配置”。

[OPEN_INSTSIM] 控制指令仿真使能。该项仅对指令仿真库生效。

[OPEN_PERFSIM] 控制性能仿真使能。该项仅对指令仿真库生效。



注意

- 配置文件中以#表示注释，行首、行尾勿添加空格、Tab 等内容。
- 在跑多段网络时，如果将[OPEN_PERFSIM]置 1，则必须将[OPEN_INSTSIM]置 1。

3.6.4 功能仿真 CUDA 加速配置

3.6.4.1 环境要求

- Windows 7 / 10
- CUDA 版本 >= 8.0
- 建议显存 >= 6G

3.6.4.2 架构支持

SM_30, SM_35, SM_37

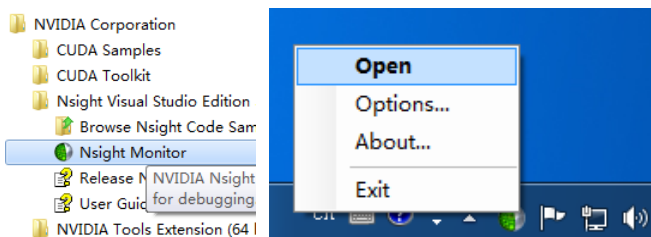
SM_50, SM_52, SM_53

SM_60, SM_61, SM_62

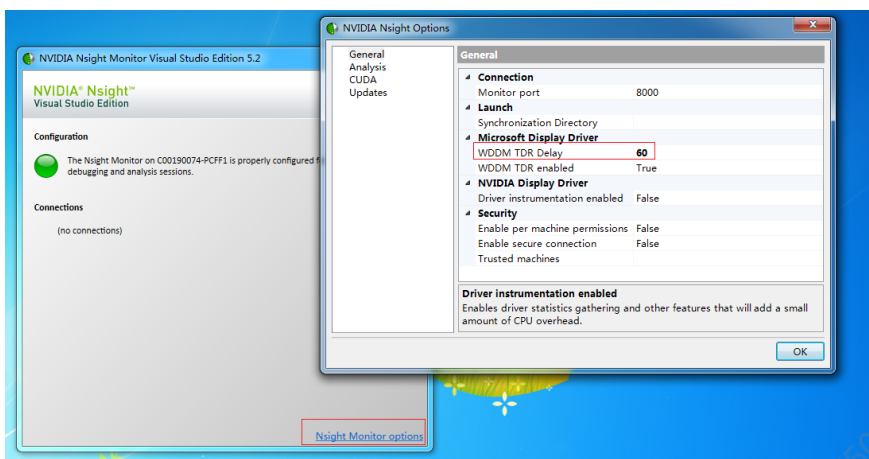
3.6.4.3 配置步骤

步骤 1. 确认已安装 NVIDIA CUDA，要求 CUDA 版本至少 8.0；

步骤 2. 以管理员权限在 开始菜单->NVIDIA Corporation->Nsight Visual Studio Edition 打开 Nsight Monitor。若点击无反应则可能 Nsight Monitor 已运行，可在桌面右下角右键点击 Nsight Monitor 图标，选择 Open 打开。

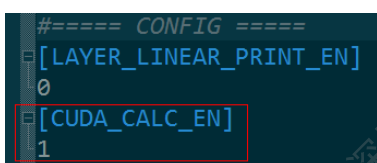


步骤 3. 点击 Nsight Monitor 窗口右下角的 Nsight Monitor options，打开配置窗口。



步骤 4. 修改 Nsight Monitor options 中 General 标签页的 WDDM TDR Delay，默认为 2，调整至适当值如 60，点击确认后重启计算机。此项表示 GPU 占用图形界面无响应时的重启时间，当 CUDA 连续计算超过该设定值时，会重启显示驱动，终止计算，恢复图形界面响应。

步骤 5. 在 NNIE 仿真器配置文件 C:\hisilicon\nnie_sim.ini 中，修改 CUDA 使能配置项 [CUDA_CALC_EN]。修改为“1”使能仿真器 CUDA 加速。



步骤 6. 运行 NNIE 功能仿真器程序时，提示如下图红框所示的设备信息，即 NNIE 仿真器 CUDA 加速调用成功。



```
Fri Apr 27 17:05:15 2018 I create layer(0): preprocess
Fri Apr 27 17:05:15 2018 I create layer(1): convolution
Fri Apr 27 17:05:15 2018 I create layer(2): poolingave
Fri Apr 27 17:05:15 2018 I create layer(3): convolution
Fri Apr 27 17:05:15 2018 I create layer(4): poolingave
Fri Apr 27 17:05:15 2018 I create layer(5): innerproduct
Fri Apr 27 17:05:15 2018 I create layer(6): innerproduct
Fri Apr 27 17:05:15 2018 I create layer(7): softmax
Fri Apr 27 17:05:15 2018 I start [seg(0) layer(0)]: preprocess
Fri Apr 27 17:05:15 2018 I end [seg(0) layer(0)]: preprocess
Fri Apr 27 17:05:15 2018 I start [seg(0) layer(1)]: convolution
*****
CUDA Device(0) Properties
Identify: Quadro M5000
Clock Rate: 1038000 kHz
Max Grid Size: 2147483647 * 65535 * 65535
Max Threads Dim: 1024 * 1024 * 64
Max Threads per Block: 1024
Number of Multiprocessors: 16
32bit Registers Available per Block: 65536
Warp Size: 32 threads
*****
cudaSetDevice(0)
Fri Apr 27 17:05:15 2018 I end [seg(0) layer(1)]: convolution
Fri Apr 27 17:05:15 2018 I start [seg(0) layer(2)]: poolingave
Fri Apr 27 17:05:15 2018 I end [seg(0) layer(2)]: poolingave
Fri Apr 27 17:05:15 2018 I start [seg(0) layer(3)]: convolution
```

----结束

3.7 NNIE 调试

NNIE mapper 运行无错误提示且在上板或者仿真库中能正常运行，但是结果不对或精度不正常时，可以参考下述步骤进行问题定位。

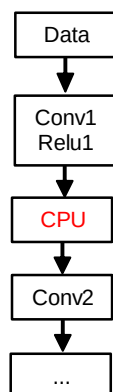
3.7.1 预处理一致性问题

nnie_mapper 支持的预处理方式有 6 种，参考 3.5.1 节中表 3-6 的 norm_type 说明。首先要保证 NNIE mapper 的预处理方式跟训练时候的一致。

3.7.2 多段网络时输入问题

网络有 Proposal 或 Custom 层时，网络被切分为多段。以下图示为例，NNIE mapper 要求 conv2 层需要一个输入，该输入是该网络在 Caffe 中使用输入给 mapper 的参考图像在 caffe 中运行到 CPU 层的结果。输入给 mapper 的 Custom 层输出格式参考表 3-6 的 image_list 说明，Proposal 层输出格式参考表 3-6 的 roi_coordinate_file 说明。

图3-38 多段网络示意图



3.7.3 精度下降问题

将 `nnie_mapper` 配置选项 `log_level` 设置为 3，`nnie_mapper` 会输出各层量化推断结果，每层结果保存在文件名为“`layername_output$Id_$C_$H_$W_quant.linear.hex`”，记为结果 A；

上板打印网络对应各层结果保存名为 `seg$Id_layer$Id_output$Id_board.linear.hex`，记为结果 B₀；

指令仿打印结果保存名为 `seg$Id_layer$Id_output$Id_inst.linear.hex`，记为结果 B₁；

功能仿真打印结果保存名为 `seg$Id_layer$Id_output$Id_func.linear.hex`，记为结果 B₂；

使用 NNIE 工具包中 `tool/nnie1.1/python_tool` 中的脚本 `CNN_convert_bin_and_print_featuremap.py` 打印出在 Caffe 上运行的每层结果保存名为 `layername_output$Id_$C_$H_$W_caffe.linear.float`，记为结果 C；

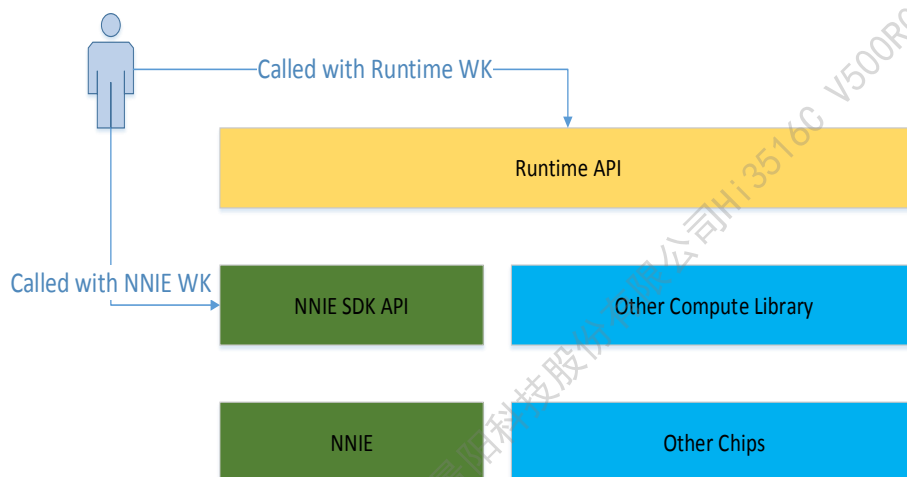
使用 UI 的比较工具分析结果 A\B₀\B₁\B₂\C，出现相似度异常的层则表示该层出现了问题，在使用 UI 工具或者手动把相关层的数据给到海思进行问题复现和分析定位。

4 Runtime 开发指南

4.1 Runtime 介绍

4.1.1 层次关系

图4-1 Runtime API 交付层次关系图



4.1.2 工具链介绍

SVP Runtime 在 HiSVP_PC_Vx.x.x.x.rar 组件包中，提供如下的工具链：

- Runtime Sample 工程（software\sample_runtime 目录），包含完整的 Runtime Sample 源代码供开发者学习参考，支持 Virtual studio 或 RuyiStudio 环境运行。
- 模型包(software\data)：包含若干 sample 中用到的网络的 caffe 模型文件及对应的 Runtime mapper 配置文件、wk 文件、图像文件等。
- Windows 版 IDE(tool\nnie1.1\windows 目录)工具 Ruyi Studio，集成 Windows 版的 Runtime mapper 和仿真库，用户可以将 Runtime Sample 工程导入；IDE 还集成了客户自定义插件、runtime dot 图等功能，具体参考 5 “[RuyiStudio 工具使用指南](#)” 章节。



- 当前 runtime_mapper 仅支持在 RuyiStudio 中使用，不支持脱离 RuyiStudio 单独命令行调用。

4.1.3 开发流程

以 Caffe 框架上训练的模型为例，NNIE 的开发流程如图 3-1 所示。在 Caffe 上训练、使用 NNIE 的 mapper 工具转化都是离线的。通过设置不同的模式，mapper 将 *.caffemodel 转化成在仿真器、仿真库或板端上可加载执行的数据指令文件。一般在开发前期，用户可使用仿真器对训练出来的模型进行精度、性能、带宽进行初步评估，符合用户预期后再使用仿真库进行完整功能的仿真，最后将程序移植到板端。

4.1.4 网络层分类

对于 Runtime 而言，一个网络的层可分为如下的 3 类：

- NNIE-Layer: NNIE 支持层，可见 3.1.3 网络层的分类小节，其中标准层、扩展层都属于 NNIE 支持的层。
- Runtime-Layer: 由 Runtime 默认提供的运行在非 NNIE 芯片上的层，如 Faster RCNN/RFCN 网络中的 Proposal 层等。
- Custom-Layer: NNIE 和 Runtime 均不支持的层，需要由用户自行编码实现。

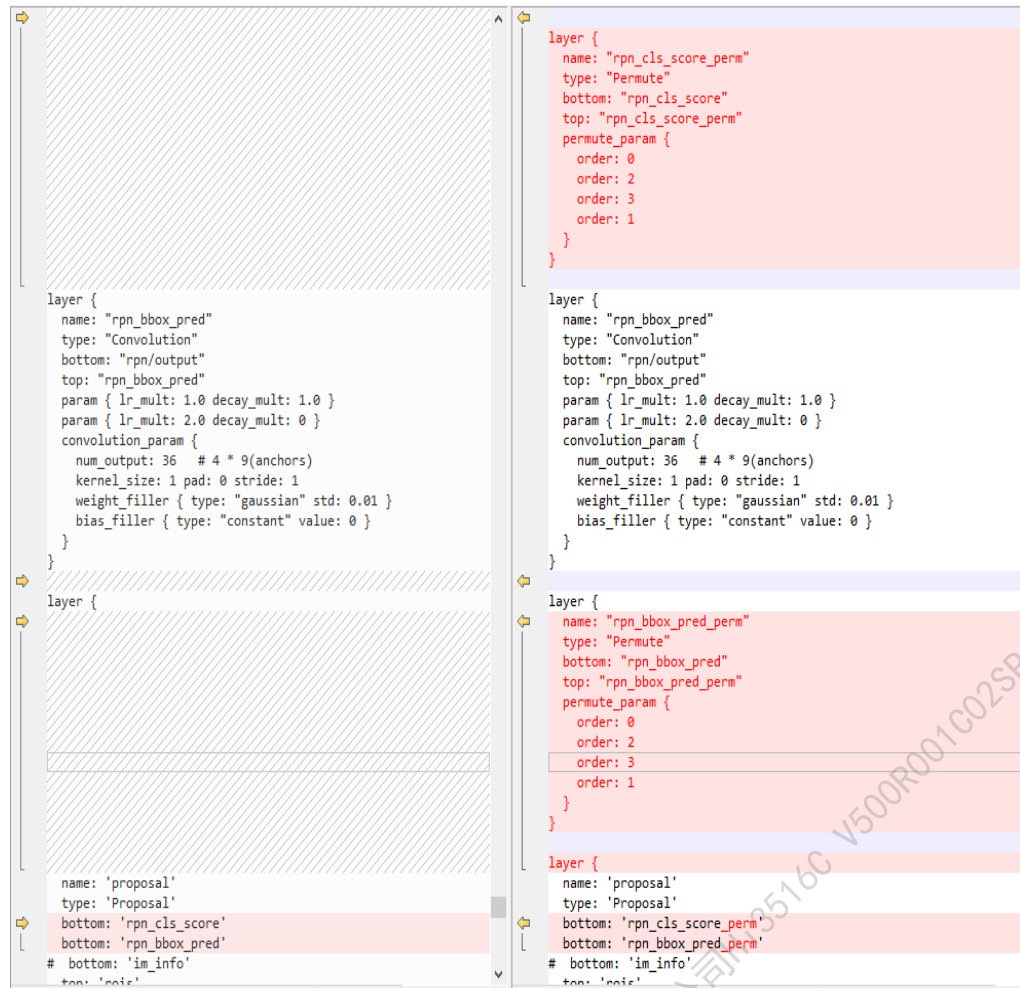
4.1.5 Runtime Layer 规格

Runtime-Layer 默认支持层：

Faster RCNN/RFCN 网络模型内使用的 Proposal 层，CPU 实现，代码开源。

Layer Type	Chip Type	Support Net	Restrict
Proposal (with Permute) (without Permute)	CPU	Faster RCNN RFCN	1. 支持图 3-30 FRCNN 网络 (不支持图 3-31) 2. 支持图 3-33 RFCN 网络 (不 支持图 3-32)

With Permute 和 Without Permute 区别为 Permute 由 CPU/NNIE 实现，对应的 prototxt 差异为（左边 with，右边 without）：



4.2 插件库管理

Runtime-Layer 和 Custom-Layer 均以动态库的方式呈现，在网络运行时实时加载。

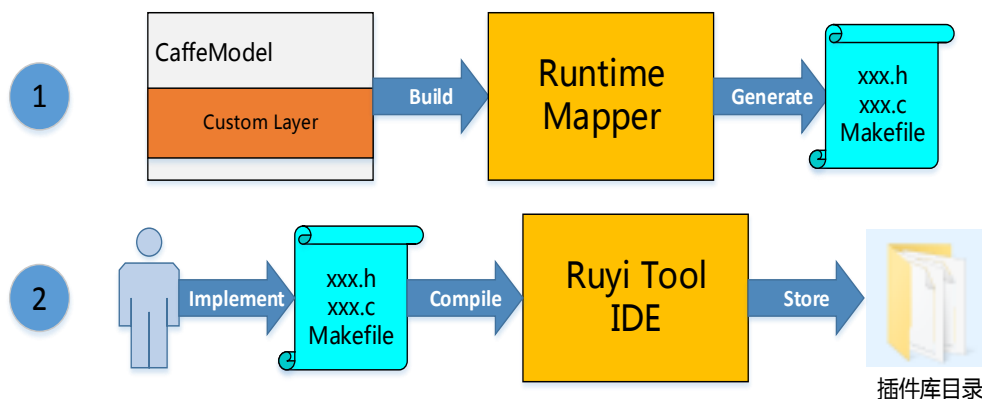
在 Ruyi 工具生成 WK 流程中的标记界面即需要进行 Runtime-Layer 和 Custom-Layer 动态库的注册。

4.2.1 提供 Custom-Layer 代码实现模板

Ruyi 工具在识别到客户网络模型中存在 Custom Layer 时，会自动生成函数实现模板，方便客户进行算法实现；

客户实现具体算法后，通过 Ruyi 工具编译成插件库，并存放至插件库目录中。

图4-2 插件库管理步骤图



4.2.2 提供 Custom-Layer 插件库复用功能

Runtime 支持相同 Layer Type 的 Custom-Layer 动态插件库复用：

- 提供指定插件库路径功能，客户可选择本机 PC 中含有之前已实现 Custom Layer 插件库的目录作为后续复用库查询目录；或者客户可以将之前已实现插件库放至默认插件库查询目录中。
- Ruyi 工具识别到 Prototxt 中含有 Custom-Layer 时，通过 Custom Layer Type 自动查询插件库目录下是否含有相同 Type 的库：
 - 存在：自动关联其中某一个库，且客户依旧可以选择其他库或新建实现。
 - 不存在：提示客户新建实现。

4.2.3 插件库限制

当前暂不支持带有训练参数的层作为 Runtime-Layer 和 Custom-Layer。

如果 Custom-Layer 带有训练参数，需要客户自编码获取训练参数值做处理。

4.3 网络拓扑管理

Runtime 支持用户使用 prototxt 方式灵活组网。

例如 Rfcn→Connector→alexnet 的网络拓扑结构 prototxt 可以表示为：

图4-3 网络拓扑示例图

```

Name: "rfcn_alexnet"
priority: 1

input {
  name: "data" //global input data
}

model {
  name: "rfcn" //rfcn model name
  bottom: { name: "data" } //input data of rfcn
  top: { name: "proposal" } //report 0: proposal
  top: { name: "cls_prob_reshape" } //report 1: cls_prob_reshape
  top: { name: "bbox_pred_reshape" } //report 2: bbox_pred_reshape
  cops {
    name: "proposal" //custom op name
  }
}

connector {
  name: "rfcn_conn_alexnet" //connector name
  bottom: { name: "data" } //input 0: data
  bottom: { name: "rfcn.proposal" } //input 1: proposal of model rfcn
  bottom: { name: "rfcn.cls_prob_reshape" } //input 2: cls_prob_reshape of model rfcn
  bottom: { name: "rfcn.bbox_pred_reshape" } //input 3: bbox_pred_reshape of model rfcn
  top: {
    name: "rect_image" //report: rect_image
    type: U8 //data type of rect_image
    shape: {dim:64 dim:227 dim:227 dim:3} //dimension information
  }
}

model {
  name: "alexnet" //alexnet mode name
  bottom: { name: "rfcn_conn_alexnet.rect_image" } //input: rect_image of rfcn_conn_alexnet
  top: { name: "prob" } //report: prob
}

```

从图 4-3 看出，model，connector 通过 bottom，top 的连接关系构成一张完整的网络拓扑结构。

当前版本对网络拓扑结构的 prototxt 主要约束如下：

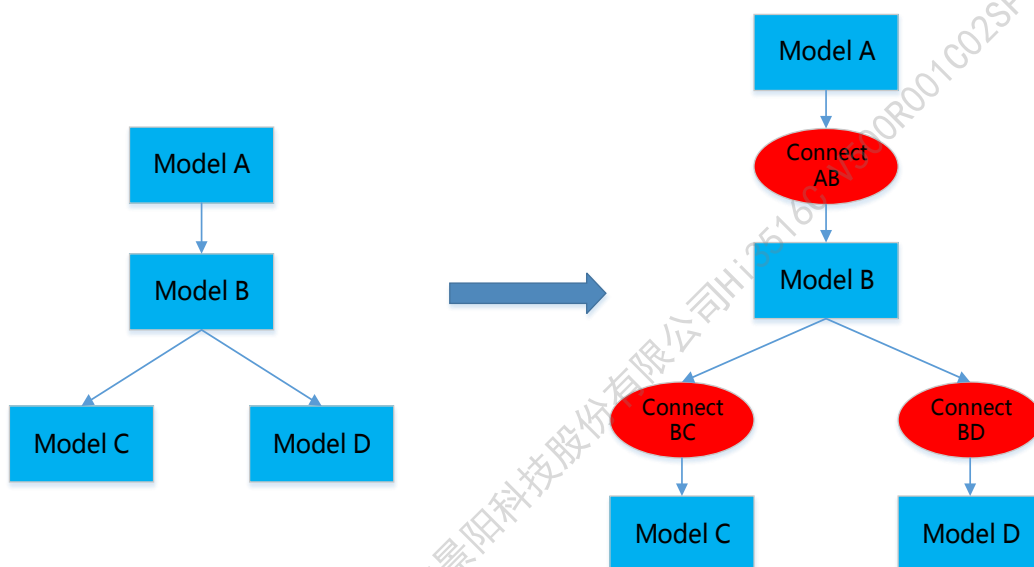
- Model、Connector、Input 的名称需要全局唯一。
- bottom 为 Input 输入，其要求如下：
 - 不要求和离线 wk 文件中 input 名称一致。
 - 输入的 bottom 须顺序对应排列，model 中的 bottom 需要和 wk 中的输入顺序一致，connector 需要和函数注册的输入参数顺序一致。
 - 上接 input 层时，其名称直接为 input name，例如：bottom: { name: "data" }
 - 上接非 input 层时，其名称为上接 Model/Connector 的名称加上需要使用的 report 节点名称，例如：bottom: { name: "rfcn.proposal" }
- top 为上报节点，其要求如下：
 - 不同模型的 top 名称可以相同。
 - 必须和离线 wk 文件中的 output 名称一致。
 - 输出的 top 须顺序对应排列，model 中的 top 需要和 wk 中的输出顺序一致，connector 需要和函数注册中的输出参数顺序一致。
- 当前支持 model、connector、priority、name、bottom、top、cops、type、shape、frame_depend、max_tmpbuf_size_mb(当前只支持 4G 以下)等配置选项，具体请参考 sample 代码。

4.4 级联网络支持

Runtime 支持级联网络 inference，相同组名标识的网络模型为一个模型组。

- 模型组内模型间通过【连接器】进行连接。
- 【连接器】可以和【连接器】相连接。
- 【连接器】支持多输入多输出。
- 【连接器】可以作为模型组输入。
- 【连接器】为用户自实现代码，以注册函数方式和 Model 一起在调用 LoadModelGroup 函数时作为参数传递下来。
- 可通过动态设置【连接器 AB】输出中的 N 维度为 0，来控制所有下接模型和连接器不再继续运行，N 维度为 HI_RUNTIME_BLOB_S 结构体中的 u32Num 变量。
- 所有的输入输出 BLOB 内存均由客户进行分配，模型内的辅助内存用户不需要管理。

图4-4 模型级联图



级联 API 调用，具体可见《HiSVP API 参考》第三章 Runtime 章节。

4.5 优先级配置

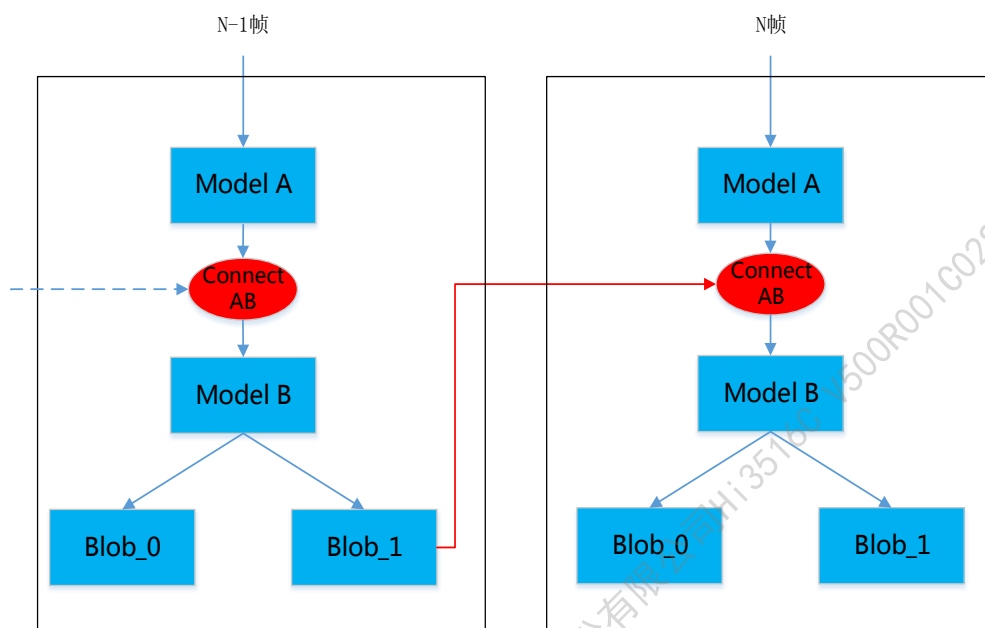
Runtime AI DDK 支持模型组优先级，按照模型内分段颗粒度，优先分配硬件资源给优先级更高的分段进行运算；优先级相同分段按照 First Come First Serve 的方式进行处理。

4.6 帧间依赖网络支持

Runtime AI DDK 支持帧间依赖网络 inference，当前帧的执行需要以前面帧的执行结果为其输入。

- N 帧的输入以 N-1 帧的输出作为其输入，形成一种帧与帧之间相互依赖的网络结构
- 典型的跟踪网网络结构

图4-5 帧间依赖网络示意图



4.7 全局参数可配置

Runtime 初始化时，用户可以根据使用环境配置 Runtime 相关全局参数，用户可以配置如下信息：

- runtime 监听线程使用的 cpu 亲和度
- 客户自定义插件以及 connector 对象使用使用的 cpu 亲和度
- 日志级别

4.7.1 线程使用的 CPU 亲和度可配置

runtime 监听线程、插件库以及连接器对象(CPU 上运行)都运行在 CPU 线程中，用户可以通过配置 CPU 线程达到性能的最优化。

runtime 支持客户配置 CPU 线程属性，可以通过配置的项如下：

- CPU 的个数
- CPU 线程在哪个 CPU 核上运行

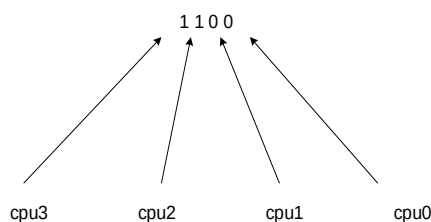


说明

对于非运行在 CPU 上的插件库和连接器，此配置无作用。

- CPU 线程配置说明

图4-6 CPU 配置示意图



多个 CPU 组成一个二进制 xxxxx 结构，CPU0 对应最右边的一位，CPU1 对应前一位，依此类推；1 代表线程运行此 CPU 核上，0 代表线程不运行在此 CPU 核上。

CPU 线程也可以指定使用的 CPU 核的范围，比如 6，即 0110，代表此 CPU 线程可以运行在 CPU1 或者 CPU2 上。

- CPU 线程配置举例

```
HI_SVPRT_RUNTIME_Init("cpu_task_affinity:4 cpu_task_affinity:8", NULL);
```

- 两个 `cpu_task_affinity` 代表启动两个 CPU 线程
- `cpu_task_affinity` 设置为 4、8
- 4 对应 0100，代表使用 CPU2
- 8 对应 1000，代表使用 CPU3

4.7.2 日志级别可配置

有五种级别：

- VERBOSE
- DEBUG
- INFO
- WARNING
- ERR

日志级别优先级高低：ERR>WARNING>INFO>DEBUG>VERBOSE

不配置日志级别的情况下，默认级别为 INFO。



系统运行时，会打印出设置的级别及以上更高级别的所有打印。

例如：

```
HI_SVPRT_RUNTIME_Init("log_level: WARNING cpu_task_affinity:8", NULL);
```

log_level 为 WARNING，代表设置的级别为 WARNING，则系统运行时，会打印 WARNING 和 ERR 的日志信息。

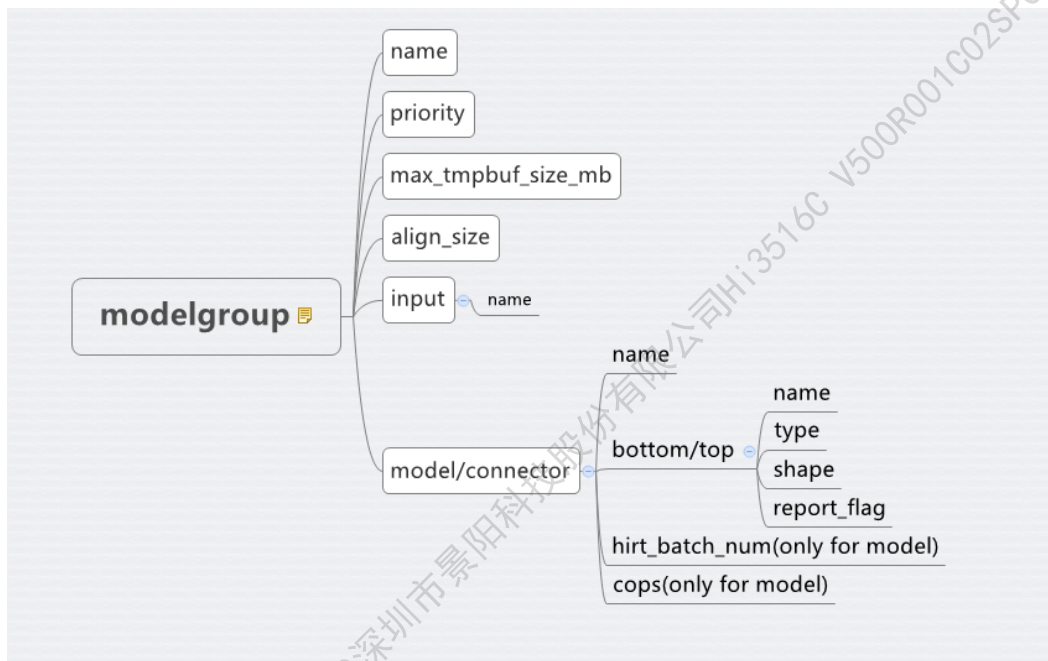
4.8 模型组可配置

Runtime 支持传入的模型组可配置，用户可以根据实际需求进行配置。

模型组配置包括：模型组全局配置项、input 配置项、model/connector 配置项

其配置项关系如下图 4-7 所示：

图4-7 Modelgroup 可配置项关系图



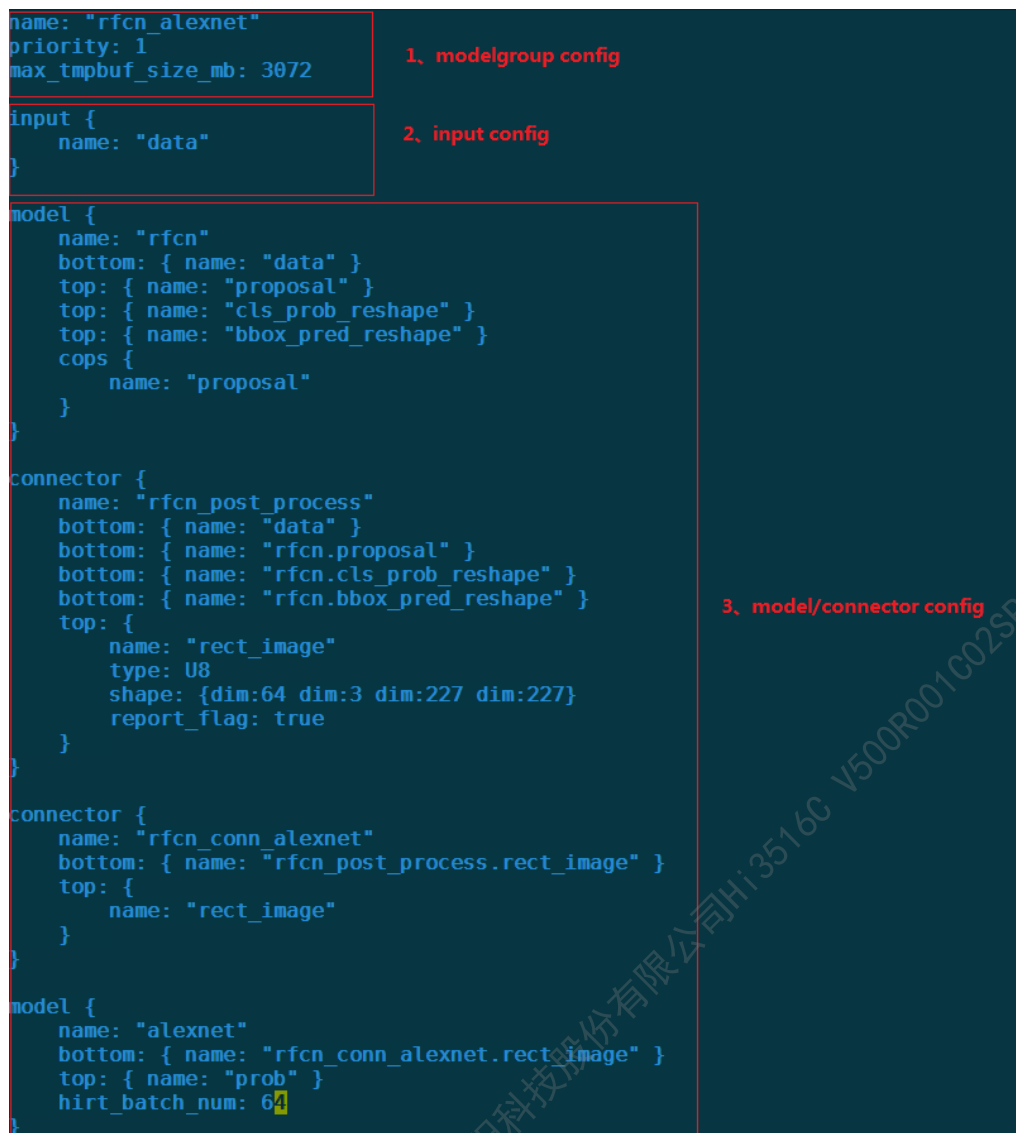
图中，各节点关系如下：

- 模型组全局配置项：name、priority、max_tmpbuf_size_mb、align_size
- input 配置项：input
- model 配置项：name、bottom、top、hirt_batch_num、cops
- connector 配置项：name、bottom、top
- bottom/top：name、type、shape、report

以 rfcn+alexnet 举例，其 modelgroup 如图 4-8 所示：



图4-8 rfcn+alexnet modelgroup 结构图



图中标注 1 的部分为模型组全局配置项，标注 2 的部分为 input 配置，标注 3 的部分为 model/config 配置。

4.8.1 模型组全局配置项

以下配置只针对模型组有效，其配置表如表 4-1 所示：

表4-1 模型组全局配置项表

配置项名称	简要描述	类型	取值范围	默认值	必填
name	模型组名称	字符串	最大长度 64	无	是
priority	模型组优先级	整型	[0,4]	无	是



配置项名称	简要描述	类型	取值范围	默认值	必填
max_tmpbuf_size_mb	模型组配置的最大 tmpbuff 内存	整型	(0,4096)	1024	否
align_size	模型组的对齐方式	整型	16、32	16	否

- name: 模型组名称，其名称对于不同的模型组是唯一的。
- priority: 模型组优先级，优先级高低顺序为：0 > 1 > 2 > 3 > 4，0 是最高优先级，4 是最低优先级
- max_tmpbuf_size_mb: 模型组配置最大 tmpbuff 内存，单位 MB。
- align_size: 模型组的对齐方式，支持 16，32 字节对齐；

当模型组的 align_size 和 wk 中的 align_size 冲突时，系统会报错。

4.8.2 input 可配置项

input 只包含 name 字段，其描述如表 4-2 所示：

表4-2 input 可配置项

配置项名称	简要描述	类型	取值范围	默认值	必填
name	用户数据输入节点	字符串	最大长度 64	无	是

对同一个模型组文件，各个 input 的名称必须是唯一的。

4.8.3 模型、连接器可配置项

模型、连接器可配置项包括：name、top、bottom 信息。其中，name 字段和 4.8.2 “input 可配置项”章节 input 的 name 字段的约束一致。

4.8.3.1 top、bottom 信息包含的数据结构

其数据结构如表 4-3 所示：

表4-3 bottom/top 数据结构

配置项名称	简要描述	值信息	必填
name	top/bottom 名称	字符串，最大长度 129	是
type	数据类型	S32、U8、YVU420SP、YVU422SP、VEC_S32、SEQ_S32	否
shape	shape 信息	包含 n、c、h、w 四个维度	否
report_flag	report 标记	true、false	否



- **name:** bottom/top 名称，最大长度：129，
 - 如果上接 input 层，则直接填入 input 节点的名称
 - 如果上接 model，则填入 model name + "." + top name(model)的格式
 - 如果上接 connector，则填入 connector name + "." + top name(connector)的格式
 - 名称不唯一。
- **type:** 指定 top, bottom 的数据类型
 - bottom: 不需要填入 type 信息
 - top: 如果下接 connector，未下接 model，则必须填入 type 信息，如图中的 connector 否则，可以不填 type 信息
- **shape:** 包含四个维度信息，包含 n、c、h、w 四个维度，各维度为正整型，例如：
 - shape: {dim:64 dim:3 dim:227 dim:227}，shape 的各个 dim 值的大小>=1
 - bottom: 不需填入 shape 信息
 - top: 如果下接 connector，未下接 model，则必须填入 shape 信息，否则，可以不填 shape 信息
- **report_flag:** 标志此 top 是否为 report 层。
 - true: 表示 report 层，需要客户分配相关内存；
 - false: 内存由 runtime 内部分配；
 以下情况需要设置 report_flag 为 true:
 - 用户需要关注的 Blob
 - Blob 在 modelgroup 中为叶子节点(无下接节点)
 - 帧间依赖的连接点

4.8.3.2 模型、连接器可配置项的信息

其配置项如表 4-4 所示：

表4-4 模型、连接器可配置项

配置项名称	简要描述	取值范围	必填
name	model/connector 名称	字符串 最大长度 64	是
bottom	依赖的数据输入信息	请参考 bottom 结构	否
top	输出输出信息	请参考 top 结构	否

- **Name:** 名称，最大长度：64，对于模型组内的模型、连接器，其名称必须为唯一
- **bottom/top:** 请参考 4.8.3.1 “top、bottom 信息包含的数据结构”章节。

4.8.3.3 模型独有配置

- **hirt_batch_num:** 正整型，模型配置的最大 batch 数目。



$0 \leq \text{hirt_batch_num} \leq 256$

- 不设置 `hirt_batch_num` 或者 `hirt_batch_num = 0`, runtime 以 wk 内部的 batch number 进行 `tmpbuff` 分配。
- 设置了 `hirt_batch_num`, runtime 会取 `hirt_batch_num` 和 wk 中的 batch number 两种最小值进行 `tmpbuff` 分配。
- 用户需要保证 `hirt_batch_num` 大于实际输入 batch 数。

- `cops`: 标志模型内部的 cop 信息, 例如 proposal 如下:

```
model {  
    name: "rfcn"  
    bottom: { name: "data" }  
    top: { name: "proposal" }  
    top: { name: "cls_prob_reshape" }  
    top: { name: "bbox_pred_reshape" }  
    cops {  
        name: "proposal"  
    }  
}
```

标记 proposal 为 custom op

这个例子中, proposal 即为 custom op, 又为上报层。

4.8.4 modelgroup 文件解析约束

- 配置以关键字匹配, 自动忽略额外的字符, 例如:
`shape:{dim:2 xxxdim:3 dim:4 dim:5}`, 自动忽略 `xxx` 三个字符, 得到的 shape 信息为 `n=2、c=3、h=4、w=5`。
- 所有数字配置只支持 10 进制, 不支持其他进制类型。
- 当关键字只能有一个时, 以第一个匹配的为准, 例如:
`model {name: "aaa" name: "bbb"}`, 则匹配到的 model name 为 "aaa"

4.9 Sample 说明

4.9.1 图片 Resize

检测网输出的目标通常需要 `resize` 成分类网要求的分辨率后再输入到分类网进行处理, 可以在 runtime 的 connector 中实现图片的 `Resize`。Sample 中 `resizeROI` 接口提供了两种 `Resize` 的实现: 板端使用 IVE; 仿真使用 OpenCV。目前相同的输入使用 IVE 和 OpenCV 得到的结果会有细微差异, 这是由硬件的定点运算引起的。因此 sample 中经过 `resize` 后板端和仿真的结果无法保证完全一致。

如果需要保证 `resize` 结果一致来对比板端和仿真结果, 建议的处理方式是: 将仿真得到的检测网输出通过板端 `resizeROI` 的 IVE 接口获取 `resize` 后的图片保存下来, 再输入到仿真的分类网。



5 RuyiStudio 工具使用指南

RuyiStudio 集成 windows 版的 NNIE mapper 和仿真库，具有生成 NNIE wk 功能、仿真 NNIE 功能，同时具有代码编辑、编译、调试、执行功能、网络拓扑显示、目标检测画框、向量相似度对比、调试定位信息获取等功能。

RuyiStudio 集成 windows 版的 NNIE mapper 基于 Visual Studio 2015 64bit 版本编译，所以其依赖库也需要使用 Visual Studio 2015 64 bit 进行编译。

RuyiStudio 集成仿真库基于 MinGW-W64 7.3.0 编译，所以其依赖的编译链环境也需要是 MinGW-W64。

5.1 RuyiStudio 安装

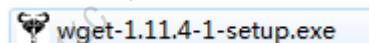
在 RuyiStudio 工具同目录下会有一个 ruyi_env_setup 的文件夹，可以进入该文件夹用脚本进行 mingw, python 及 caffe 的一键环境配置。本章节提供一键脚本配置方法和手动配制方法。

5.1.1 编译链 MinGW-W64 安装

5.1.1.1 脚本一键安装

- 步骤 1. 安装 wget，从 <http://gnuwin32.sourceforge.net/packages/wget.htm> 网页下载 wget 安装包（选择 Complete package, except sources），下载得到 wget-1.11.4-1-setup.exe（如图 5-1 所示），双击安装，将 wget.exe 所在目录添加到系统环境变量 PATH 中，用于从网页下载相关包。建议在系统环境变量中，先创建一个变量名为 WGET_PATH 的环境变量，变量值即为 wget.exe 所在路径，也就是 GnuWin32\bin 路径，然后再将 %WGET_PATH% 添加到系统环境变量 PATH 中。

图5-1 wget 安装可执行文件



- 步骤 2. 点击 ruyi_env_setup 文件夹下的 setup_mingw.bat，就可以自动下载 mingw 和 msys 的安装包，解压到指定位置并配置环境变量。



----结束

5.1.1.2 手动安装

步骤 1. 进入官方下载网址：<https://sourceforge.net/projects/mingw-w64/files/mingw-w64/>，选择 7.3.0 的 x86_64-posix-seh 版本下载（此版本支持 c++11 特性）。



注意

如果安装配置了其他的 mingw 版本则 sample 工程下仿真库依赖的 opencv 库功能可能会异常，因其 opencv 库是在指定版本上编译出来的。

图5-2 MinGW-w64 release 版本

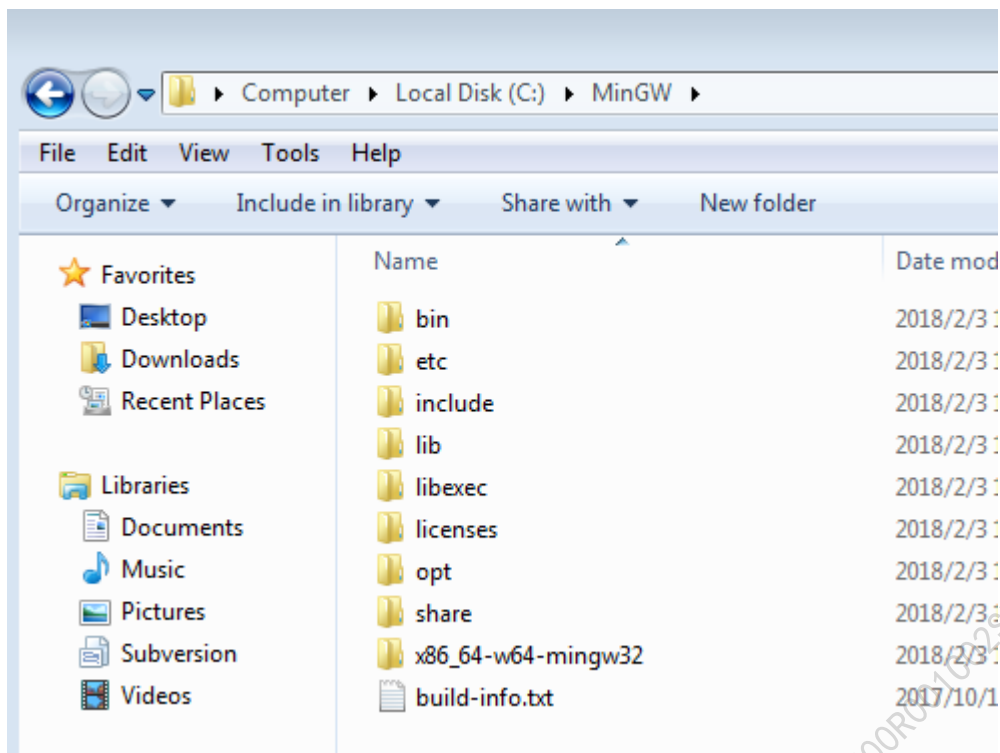
MinGW-W64 GCC-7.3.0

- x86_64-posix-sjlj
- **x86_64-posix-seh**
- x86_64-win32-sjlj
- x86_64-win32-seh
- i686-posix-sjlj
- i686-posix-dwarf
- i686-win32-sjlj
- i686-win32-dwarf

下载后解压到安装目录，注意后缀是.7z 的压缩格式，推荐 C 盘根目录。解压后的目录如图 5-3 所示，后面步骤的路径均以下图路径进行说明。

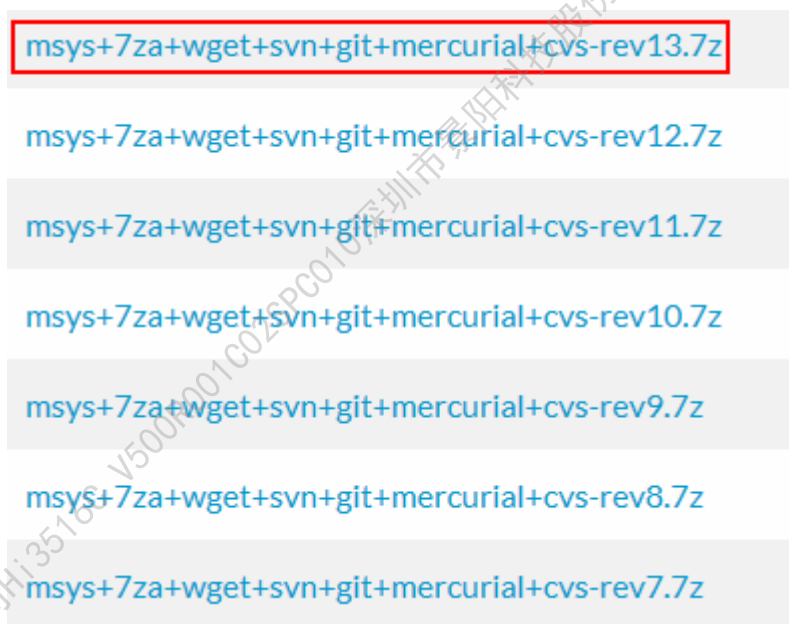


图5-3 MinGW-w64 解压后的目录结构



步骤 2. 从 <https://sourceforge.net/projects/mingwbuilds/files/external-binary-packages/> 选择 `msys+7za+wget+svn+git+mercurial+cvs-rev13.7z` 进行下载，msys 中会提供用于清理 Ruyi 工程缓存时用到的 `rm.exe`。

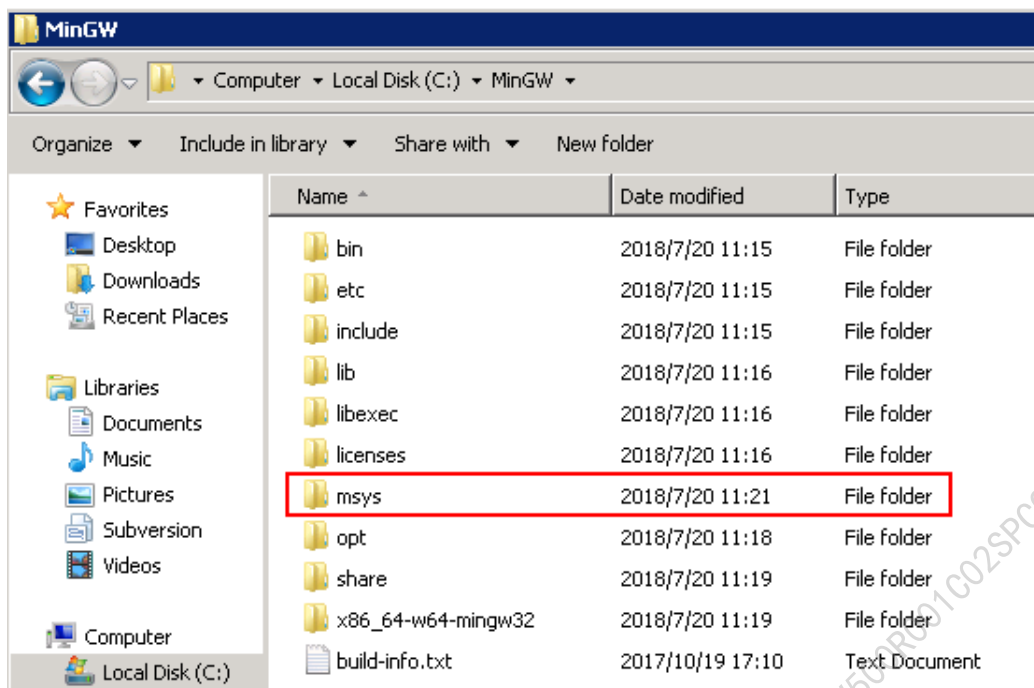
图5-4 Msys 版本





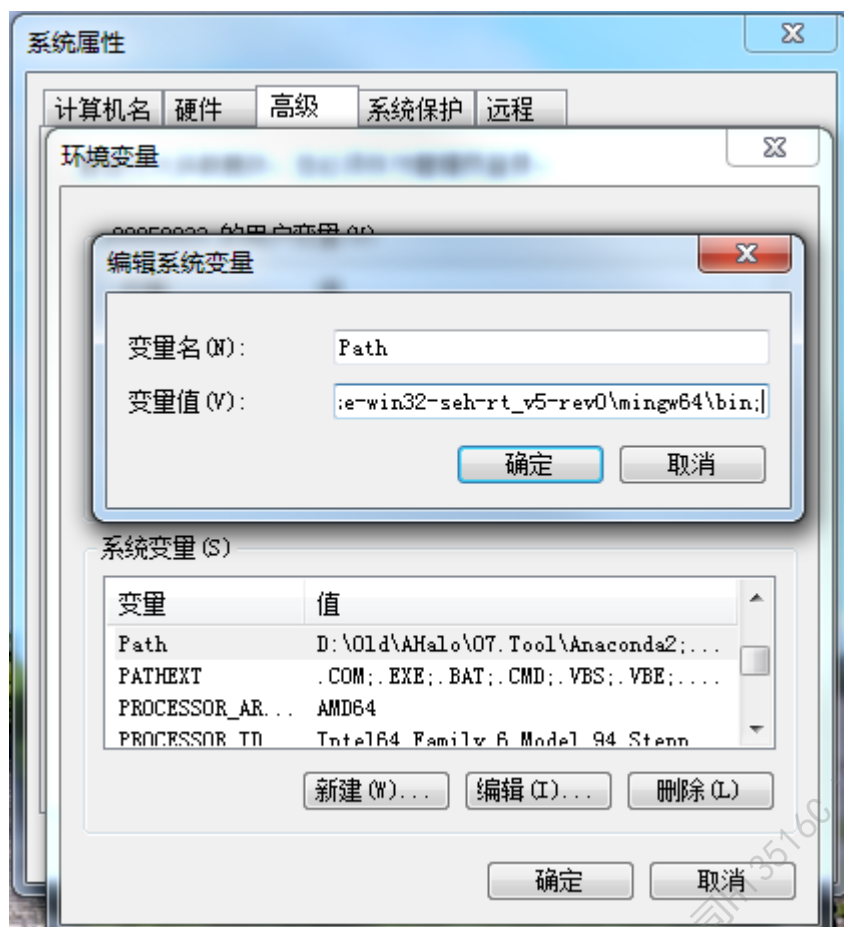
步骤 3. 将下载得到的 `msys+7za+wget+svn+git+mercurial+cvs-rev13.7z` 解压到 MinGW 文件夹下

图5-5 解压的 msys 放置的位置



步骤 4. 设置环境变量，计算机上点击右键->属性->高级系统设置->环境变量，在系统变量中添加以下变量（若已有同名变量则添加变量值用分号隔开）：变量名 `Path`，变量值 MinGW 的 bin 路径和 msys 的 bin 路径 `C:\MinGW\bin; C:\MinGW\msys\bin`，如图 5-6 所示。

图5-6 系统环境 MinGW 环境变量设置



步骤 5. 在 C:\MinGW\bin 目录下，将 x86_64-w64-mingw32-gcc.exe 再拷贝一份并重命名为 mingw32-gcc.exe，否则 RuyiStudio 工具不能自动识别到 MinGW 工具链。

步骤 6. 重新启动计算机。

----结束

5.1.2 Python3.5+caffe 环境配置

5.1.2.1 脚本一键安装

步骤 1. 同 5.1.1.1 步骤 1

步骤 2. 从 <https://github.com/willyd/caffe-builder/releases> 下载 libraries_v140_x64_py35_1.1.0.tar.bz2 并放置在 ruyi_env_setup 文件夹下。



图5-7 libraries_v140_x64_py35_1.1.0.tar.bz2

Version 1.1.0

willyd released this on Mar 4 2017 · 4 commits to master since this release

Assets

[libraries_v120_x64_py27_1.1.0.tar.bz2](#)

[libraries_v140_x64_py27_1.1.0.tar.bz2](#)

[libraries_v140_x64_py35_1.1.0.tar.bz2](#)

[Source code \(zip\)](#)

[Source code \(tar.gz\)](#)

步骤 3. 点击 setup_python.bat 进行安装，setup_python.bat 脚本会调用 python_bat 文件夹下的 setup_download_python.bat、setup_extract_python.bat 和 setPath_python.bat，下载 python，caffe 的依赖包，并配置环境变量。如果环境变量配置失败（如环境变量超过 1024 个字符），或者在 win10 系统下配置系统环境变量，可参考 5.1.2.2 的描述，进行手动配置。

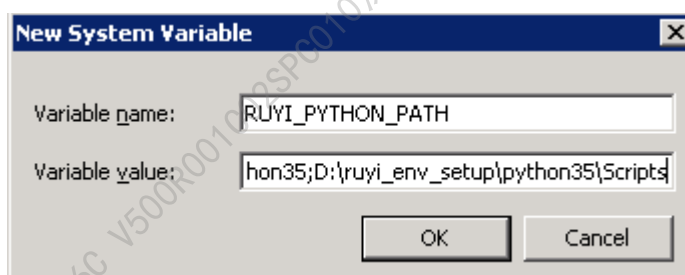
步骤 4. 重新启动计算机。

----结束

5.1.2.2 手动安装

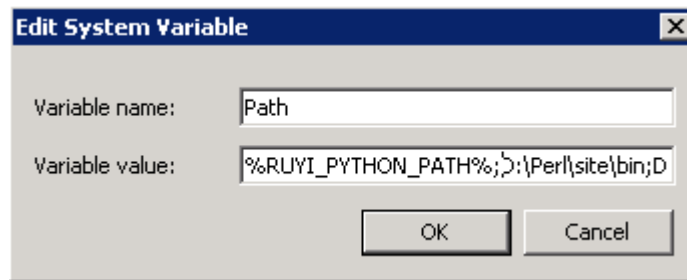
步骤 1. 在系统环境变量中增加变量 RUYI_PYTHON_PATH，如果当前 bat 脚本放置在 D:\rui_env_setup 目录下，则变量值为 D:\rui_env_setup\python35; D:\rui_env_setup\python35\Scripts; D:\rui_env_setup\python35\Library\bin，如图 5-8 所示。

图5-8 RUYI_PYTHON_PATH



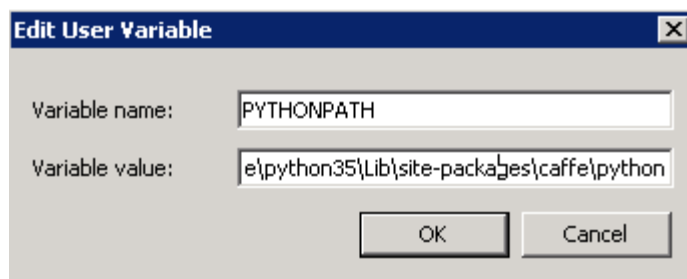
步骤 2. 添加%RUYI_PYTHON_PATH%到系统环境变量 path 的开头。

图5-9 Path



步骤 3. 手动添加用户环境变量 PYTHONPATH, D:\ruiy_env_setup\python35\Lib\site-packages\caffe\python, 用于工具识别到 caffe。

图5-10 PYTHONPATH



步骤 4. 参照 requirement.txt 中给出的版本号下载相关的包。除过 caffe.zip 和 opencv_python-3.4.0.12-cp35-cp35m-win_amd64.whl (详见步骤 5 和步骤 6), 其余包都是直接放到脚本创建的 python35 文件夹下进行解压。

步骤 5. caffe.zip 需要放到 python35\Lib\site-packages 下并解压, 如图 5-11 所示, 然后解压 libraries_v140_x64_py35_1.1.0.tar.bz2, 将 libraries\bin 目录下的 caffe_hdf5.dll, caffe_hdf5_hl.dll, caffe_zlib1.dll, glog.dll, libgcc_s_seh-1.dll, libgfortran-3.dll, libopenblas.dll, libquadmath-0.dll, VCRUNTIME140.dll, libraries\x64\vc14\bin 目录下的 opencv_core310.dll, opencv_imgcodecs310.dll, opencv_imgproc310.dll 以及 libraries\lib 下的 boost_chrono-vc140-mt-1_61.dll, boost_filesystem-vc140-mt-1_61.dll, boost_python-vc140-mt-1_61.dll, boost_system-vc140-mt-1_61.dll, boost_thread-vc140-mt-1_61.dll, gflags.dll 拷贝到 python35\Lib\site-packages\caffe\python\caffe 文件夹下, 如图 5-12 所示。



图5-11 解压 caffe.zip 到 caffe 文件夹中

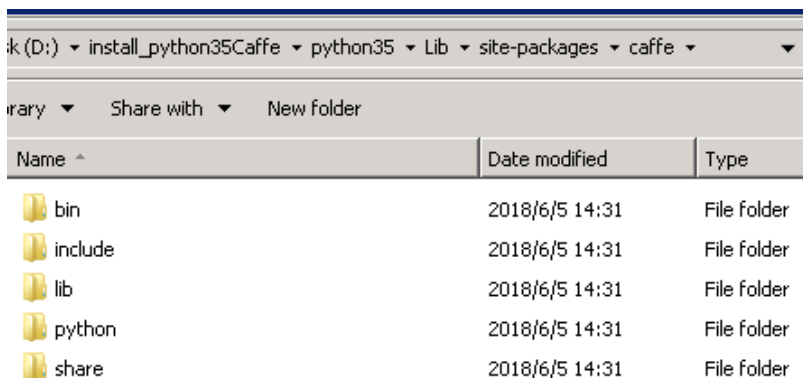
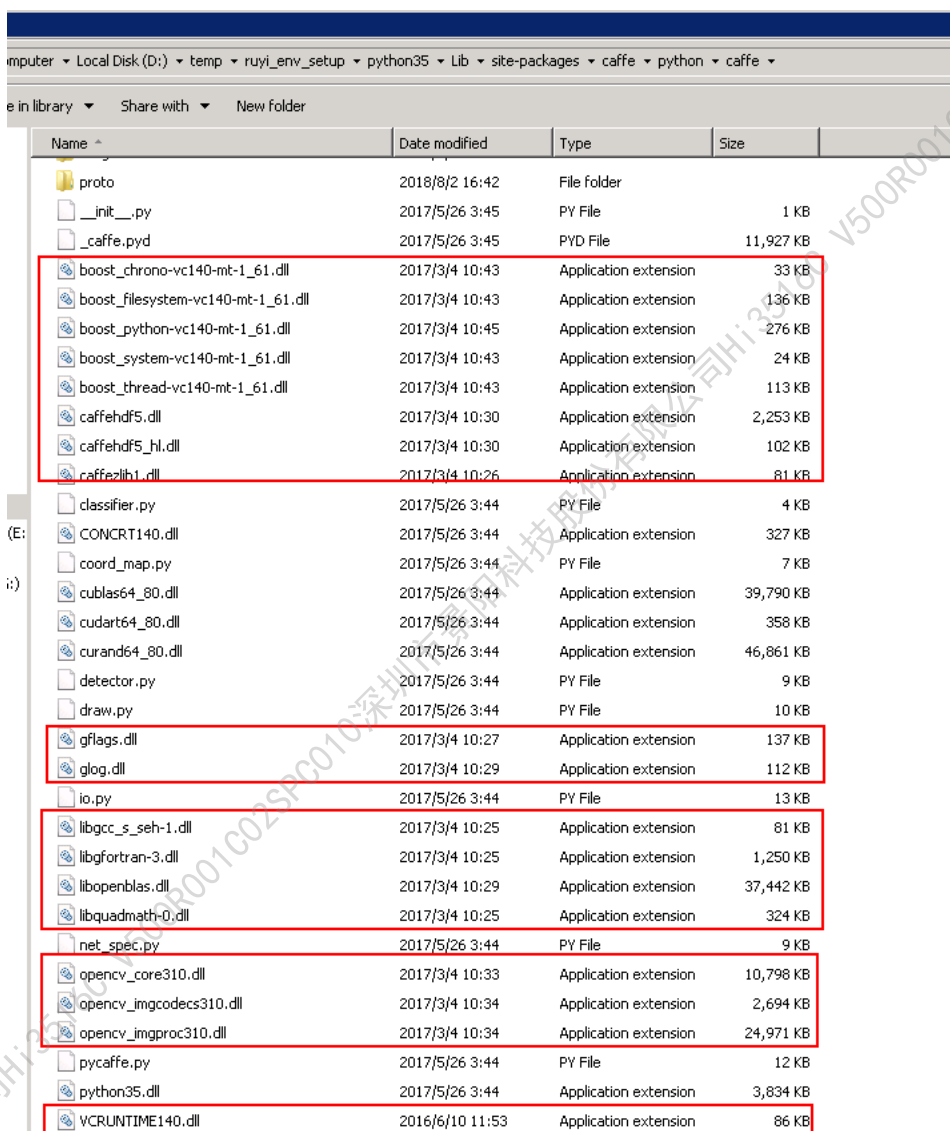


图5-12 拷贝 libraries 中的库文件到指定文件夹中





步骤 6. opencv_python-3.4.0.12-cp35-cp35m-win_amd64.whl 在 python35\Lib\site-packages, 需要从 cmd 进到 python35\Lib\site-packages 目录, 用 pip install opencv_python-3.4.0.12-cp35-cp35m-win_amd64.whl 指令进行安装。

步骤 7. 如果涉及到监测网中 roi_pooling 实现的 proposal 层的 shape 信息或者 python 层的实现是 rpn 时, 需要引入 rpn 相关的 python 实现库, 相关操作才能正常执行, 可以直接运行 setup_roi_caffe.bat 脚本进行安装, 也可以参考 requiemnts 提供的云盘路径, 从外部下载获取所有内容。

setup_python.bat, setup_mingw.bat, setup_roi_caffe.bat, 7z.exe 和 7z.dll 需要放在同一目录下。

步骤 8. 重新启动计算机。

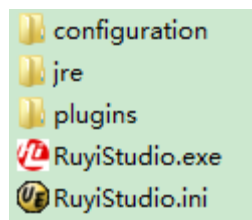
----结束

5.1.3 RuyiStudio 启动方式

步骤 1. 从 SDK 包中取到 RuyiStudio-X.X.X 工具, 拷贝到 PC 上 (PC 要求安装 Win7 64 位操作系统或 Win10 64 位操作系统) 的某个本地硬盘, 工具中已经集成 JRE, 无需单独安装。

步骤 2. 解压 RuyiStudio-X.X.X.zip, 双击工具目录下的 RuyiStudio.exe, 打开 RuyiStudio 工具, 如图 5-13 所示。

图5-13 打开 RuyiStudio 工具



工具启动界面如图 5-14 所示。



图5-14 Ruyi Studio 工具启动界面



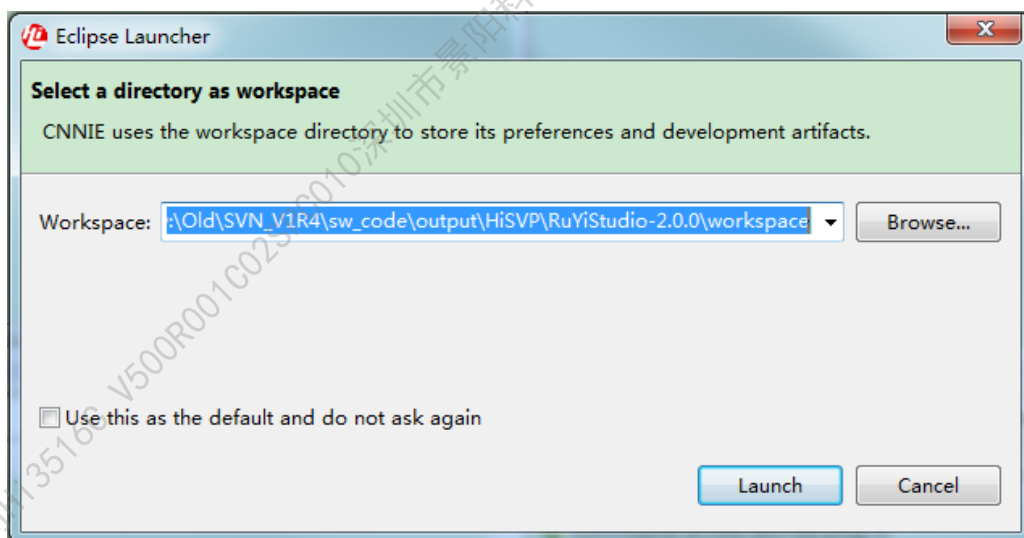
----结束

5.2 RuyiStudio 工具平台公共功能介绍

5.2.1 启动工具设置 workspace

解压 RuyiStudio-X.X.X.zip 后，双击 RuyiStudio.exe，打开工具，选择工作空间，即为选择之后工具中创建的项目的工作空间，如图 5-15 所示。

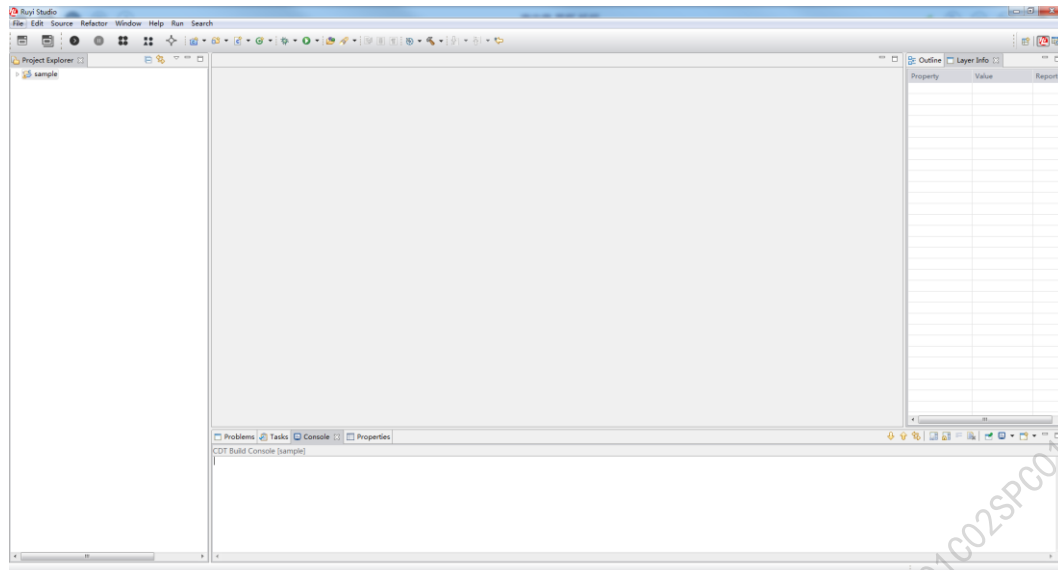
图5-15 选择 workspace 的目录





打开工具后，显示的视图如图 5-16，主要由工程视图，主界面视图和控制台视图组成。

图5-16 工具界面



5.2.2 透视图功能介绍

5.2.2.1 Ruyi 透视图

Ruyi 透视图是 RuyiStudio 工具默认的透视图，在工具中右上角，如图 5-17，默认选中 Ruyi 透视图。

图5-17 选择 Ruyi 透视图



Ruyi 透视图是 NNIT Make wk 专用的透视图，包括 RuyiStudio 自定义的功能：Make

WK 功能



，终止 Make WK 功能



，Graph View 功能



功能



，Vector Comparison 统计向量比较功能



中间结果



，Restore Network 还原网络



，Simulator Performance 性能仿真结果

展示



，Add Quant Layer 添加量化层

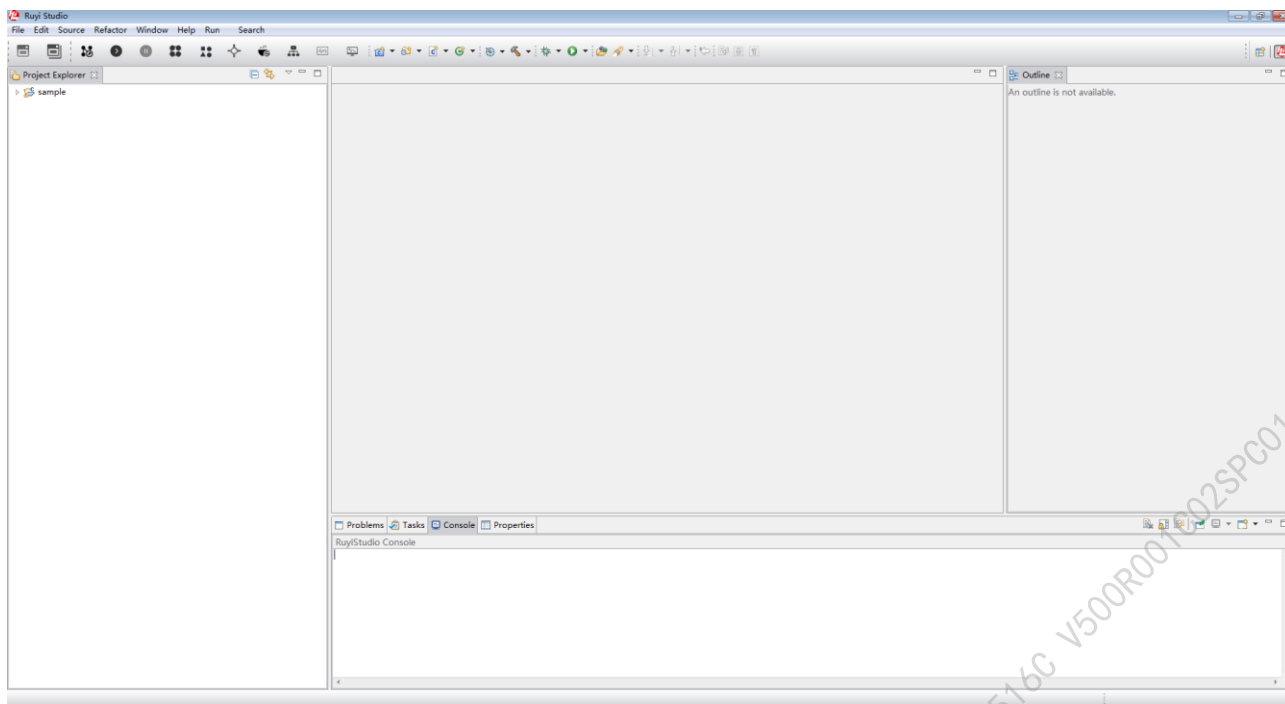


。除此之外，Ruyi 透视图还包含了 C/C++



工程需要使用的 Build 功能, Debug 调试功能, Run 运行功能。Ruyi 透视图默认视图如图 5-18。

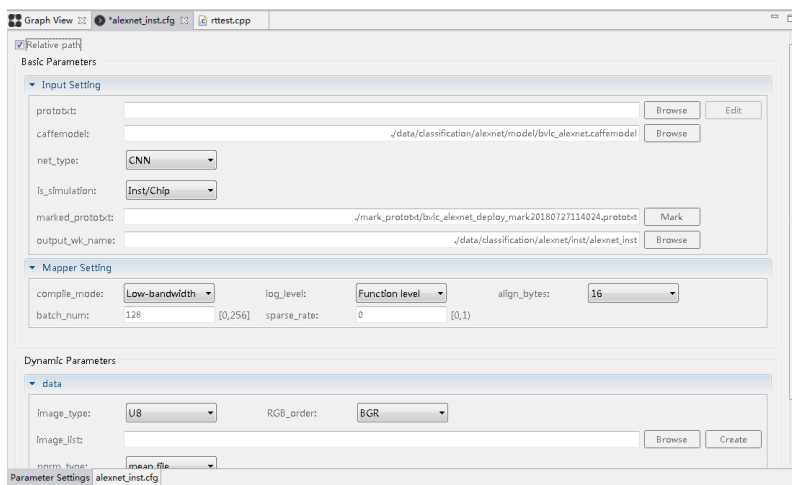
图5-18 Ruyi 透视图



整个透视图包含 4 个视图区域：

- 工具左侧为 Project Explorer 视图，用于显示用户创建或导入的各种工程；
- 下侧为各种信息视图，用于显示工具的问题，后台任务列表，软终端与对象属性；
- 右侧含有 Outline 视图，在进行 C++ 代码编辑时，显示当前 C++ 文件或头文件的结构及组成部分；还有 LayerInfo 视图，在标记 prototxt 文件后，可以显示网络的层属性。
- 中间为工作区，打开的代码文件、cfg 文件，DOT 图视图等其他工具功能视图均出现在此处。

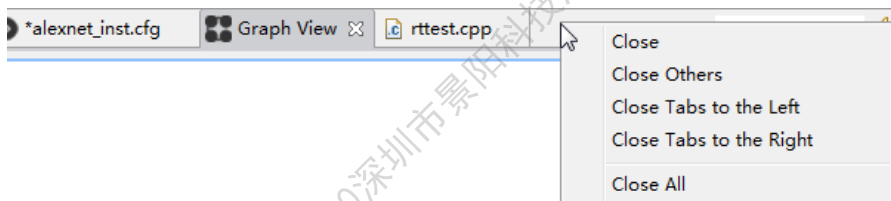
图5-19 工作区视图



任一视图区域及其中的视图均可执行以下操作：

- 点击最小化按钮 ，可以将当前视图区域最小化到最近的边栏；
- 点击最大化按钮 ，可以将当前视图区域最大化，几乎占满整个窗口。其他的视图区域将全部最小化到最近的边栏；
- 点击视图标签上的关闭按钮 （对于非选中状态的视图标签，需要鼠标悬停后按钮才会出现）可以关闭对应的视图；
- 拖动视图标签可以将选中的视图移动到别的视图区域；
- 在视图区域标签栏的空白部分点击右键，会弹出关闭标签页的选项菜单：

图5-20 视图关闭选项



- Close: 关闭当前处于选中状态的视图（区域内有视图时才会出现）；
- Close Others: 关闭当前未处于选中状态的视图（区域内的视图多于 1 个时选项才出现）；
- Close Tabs to the Left: 关于选中视图左侧的所有视图（选中视图不是最左边的一个视图时选项才出现）；
- Close Tabs to the Right: 关闭选中视图右侧的所有视图（选中视图不是最右边的一个视图时选项才出现）；
- Close All: 关闭所有视图（区域内的视图多于 1 个时选项才出现）；



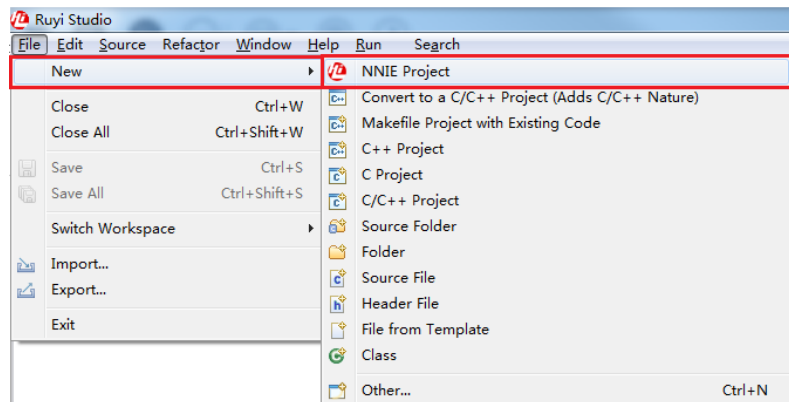
5.2.3 创建工程

5.2.3.1 创建过程

步骤 1. 工程创建，可以采用以下任一方式进行：

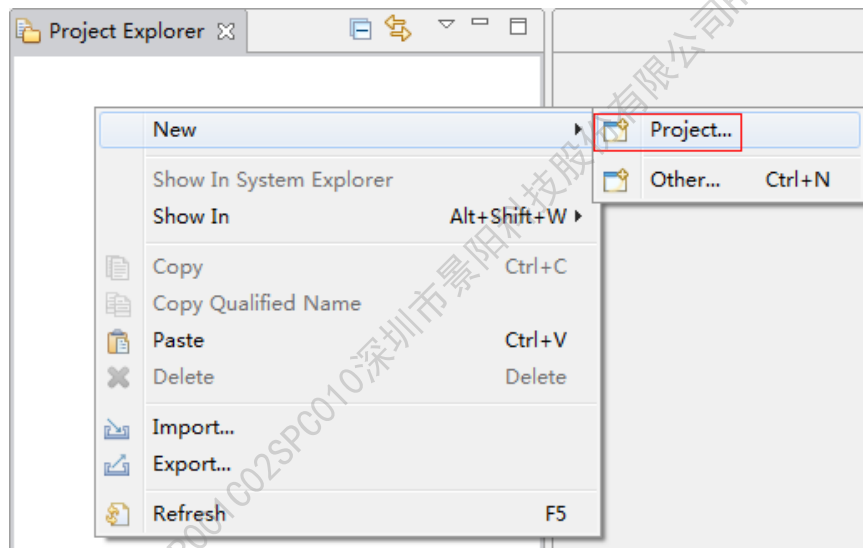
依次点击 File→NNIE Project，如图 5-21 所示。

图5-21 通过 File 菜单创建 Project



通过 Project Explorer 视图中鼠标右键创建 Project，如图 5-22 所示。

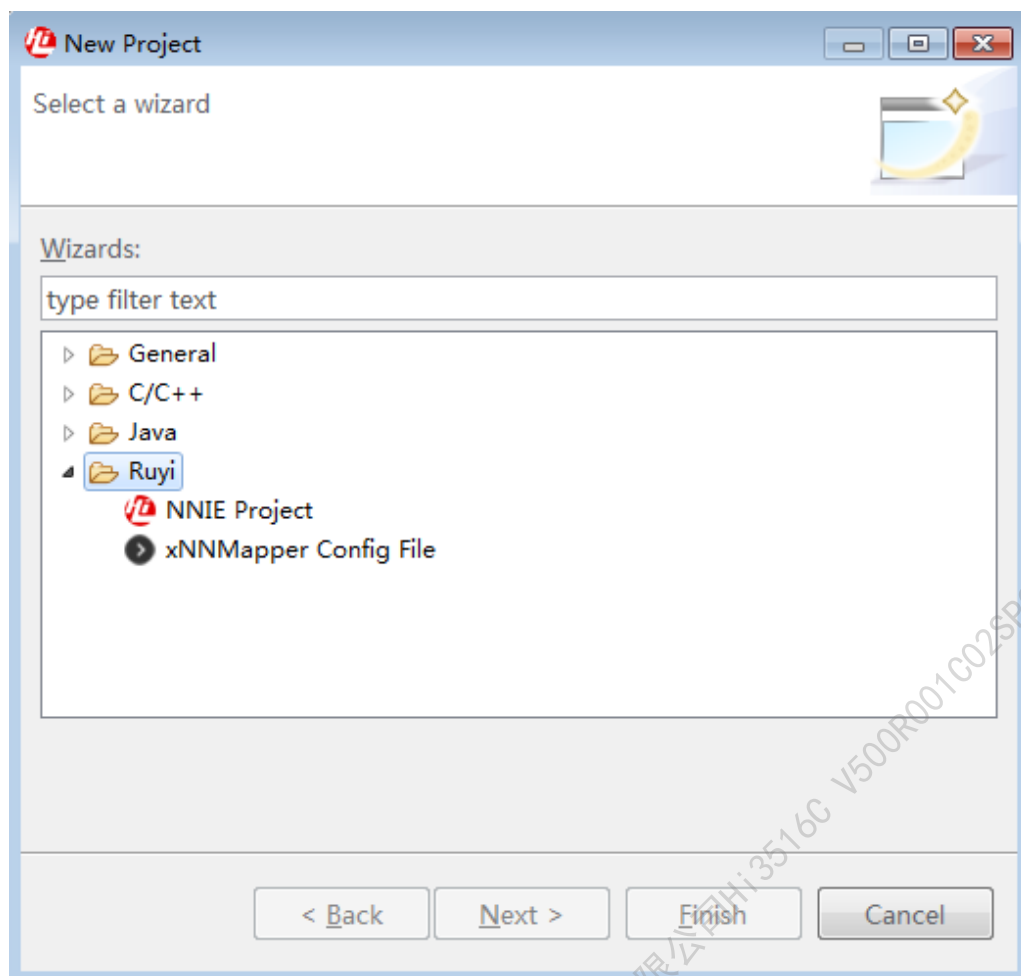
图5-22 通过鼠标右键创建 Project



步骤 2. 选择 NNIE project，点击 Next，如图 5-23 所示。



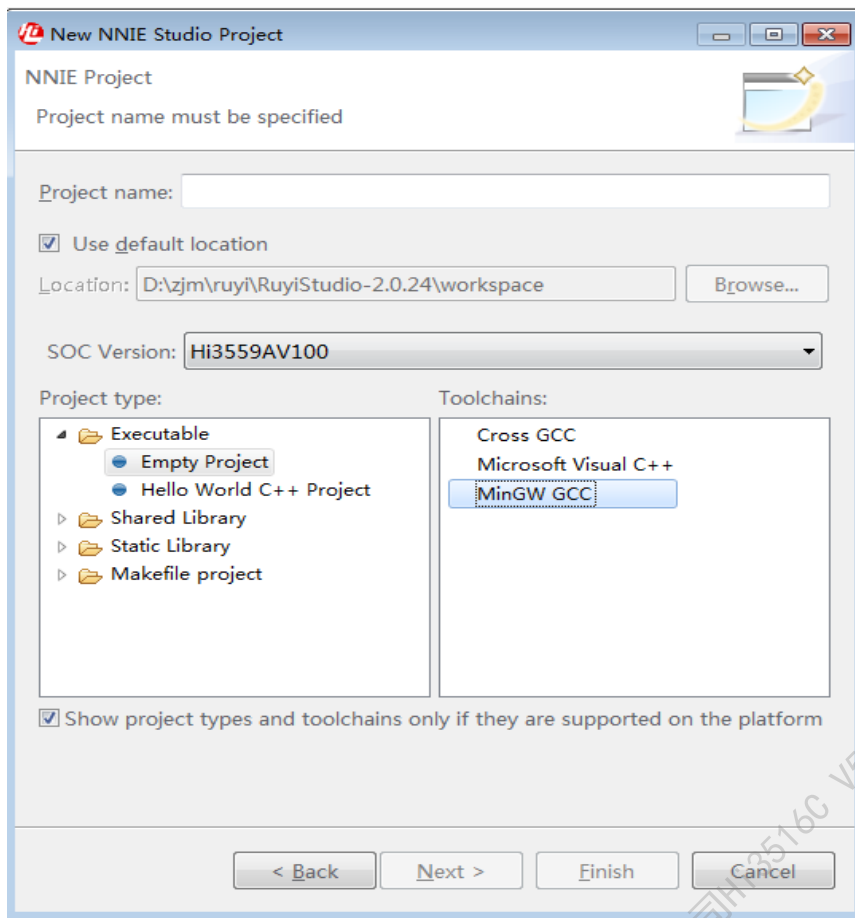
图5-23 选择 NNIE Project



- 步骤 3. 设置 Project 名称，选择工程所属芯片，默认选择 Hi3559AV100；点击 Finish 完成工程创建；注意只有当 MinGW 安装好之后才会在 Toolchains 里显示 MinGW GCC，如图 5-24 所示。



图5-24 设置 Project 的名字并确认创建工程



----结束



注意

- 若用户的工作空间路径或工程名包含非 ASCII 字符，工程将无法创建。
- 若用户的工程路径（工作空间路径+工程名整合后的路径）字符数大于操作系统限制（Windows 系统下为 256），则工程将无法创建。

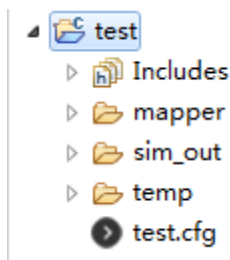
5.2.3.2 工程文件说明

创建后的工程如图如 图 5-25 所示。包含两个空文件夹和一个配置文件：

- *.cfg: 用于配置 NNIE mapper 运行参数的 cfg 文件；
- mapper 文件夹用于存放 NNIE mapper 后输出的文件；
- temp 文件夹用于存放 mapper 前准备过程中的输出文件；
- sim_out 保存调用仿真库运行时根据 hisilicon/nnie_sim.ini 配置的仿真输出文件。



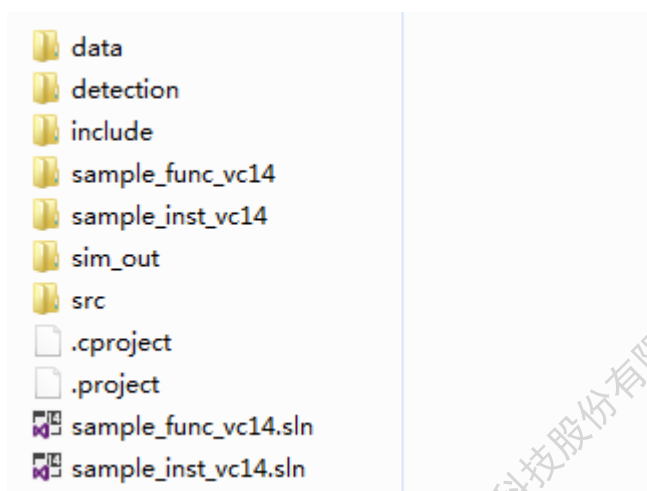
图5-25 创建工程后的工程视图



5.2.4 导入工程

HiSVP_PC_Vx.x.x.x\software\sample 是 NNIE PC 端仿真库的 sample 工程，其中包含.cproject 和.project 文件，表示该工程是可以被导入到 RuyiStudio 的 NNIE 工程，如图 5-26 所示。

图5-26 NNIE PC 端 Sample 工程



步骤 1. 鼠标左键单击“File”->“Import”，如图 5-27 所示；或者鼠标右键点击 Project Explore，在弹出菜单中选择“Import”，如图 5-28 所示。



图5-27 工程导入方式 1

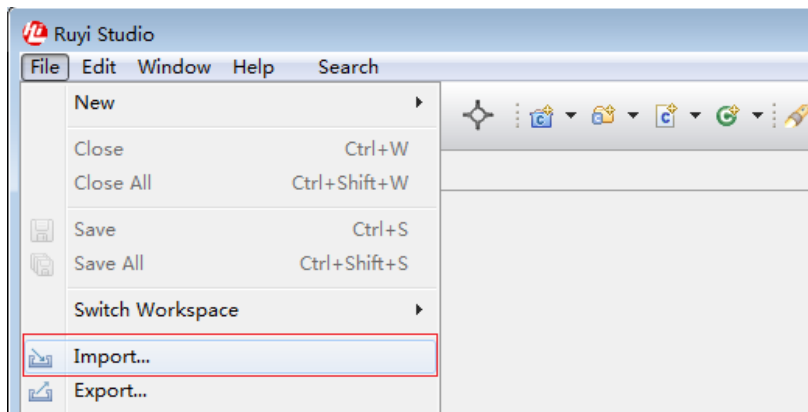
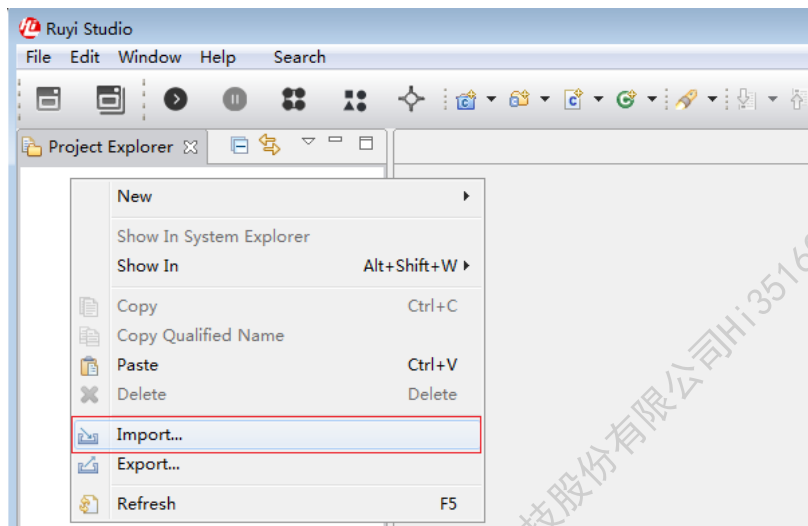


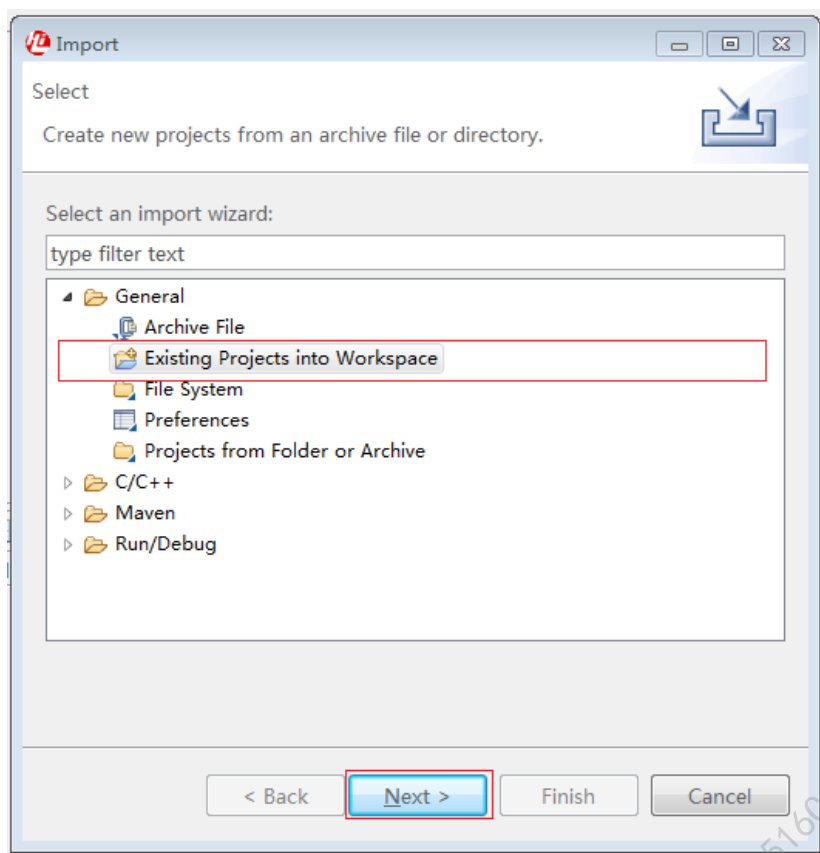
图5-28 工程导入方式 2



步骤 2. 在弹出的导入对话框中，选择“Existing Projects into Workspace”，如图 5-29 所示。



图5-29 选择外部的项目对话框



步骤 3. 在弹出的对话框中，如图 5-30 所示，选择工程所在路径，并选择该路径下要导入的 sample 工程，点击“Finish”按钮，导入成功后工程将显示在“Project explore”视图中，如图 5-31 所示。



图5-30 选择工程对话框

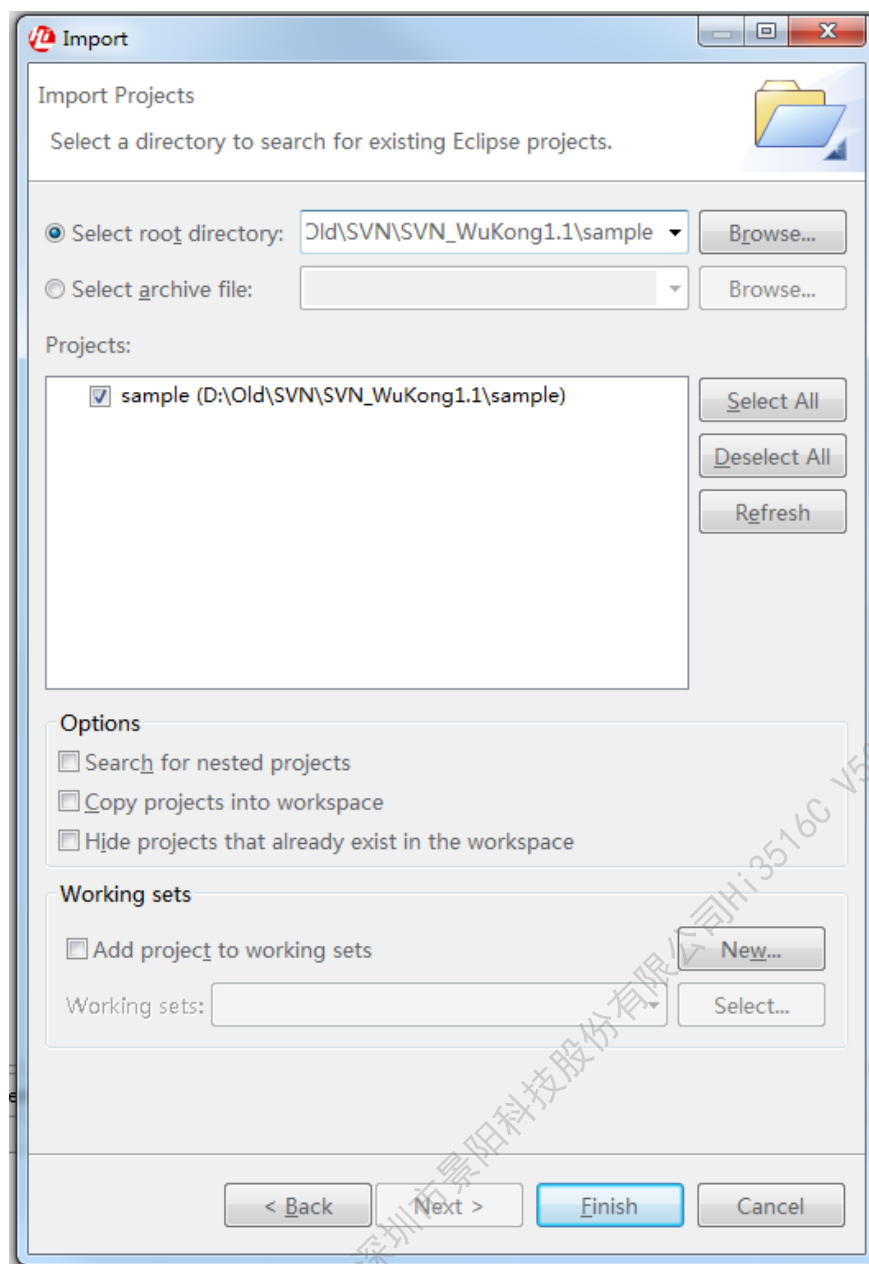
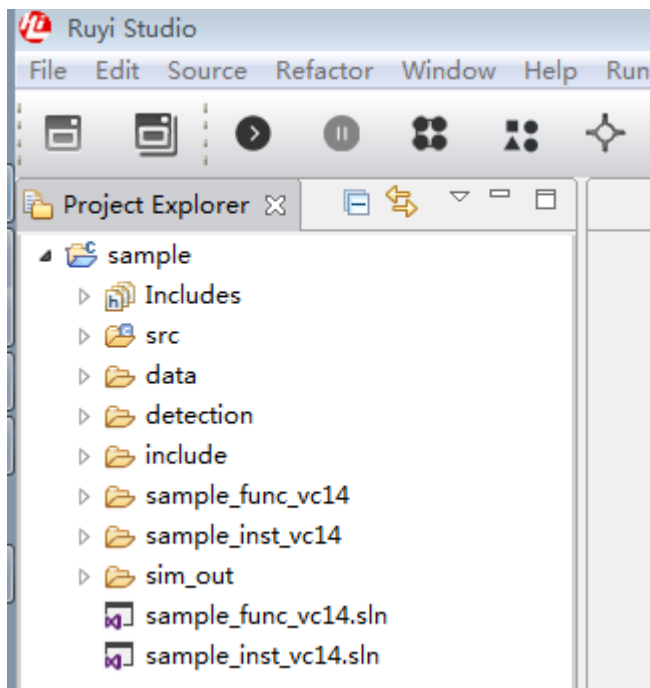


图5-31 导入 sample 代码后的工程效果



----结束

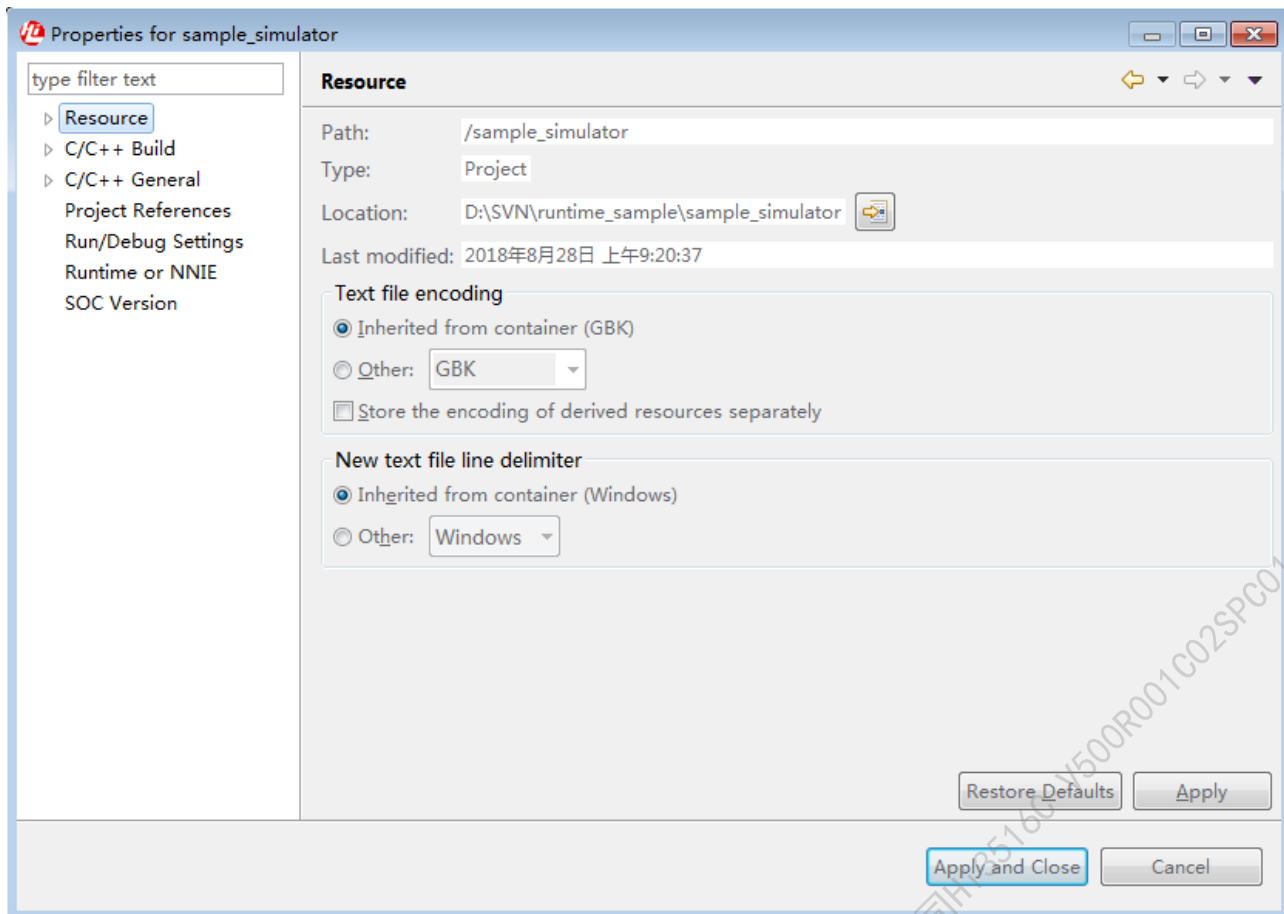
**注意**

- 请勿导入包含非 ASCII 字符的工程目录。使用这样的目录可能带来 sample 无法编译，调试时无法查看源代码等一系列问题。
- 若导入后工程内存在绝对路径字符数超出系统限制（Windows 系统下为 256）的文件，则这些文件可能无法在工具中被读取或用于编译。

5.2.5 工程属性设置

选中工程右键选择 Properties，可以对工程属性进行配置，如[图 5-32](#)所示。

图5-32 工程属性视图



5.2.5.1 C/C++ Build 中 Environment 配置

在工程属性的 C/C++ Build-> Environment 属性页中，可以进行项目自定义生成环境配置，该属性页可以读取系统环境变量，设置变量名，用于 C/C++ 工程的依赖或编译。

如果当前 C/C++ 工程是使用 MinGW 编译链，按 5.1.1 “[编译链 MinGW-W64 安装](#)”小节安装后不再需要进行设置。

如果当前 C/C++ 工程是使用 VS 编译链，编译需要使用到 VS 的 cl.exe 工具，C++ 库函数及 Include 头文件，固需要设置 LIB, INCLUDE, PATH 环境变量。

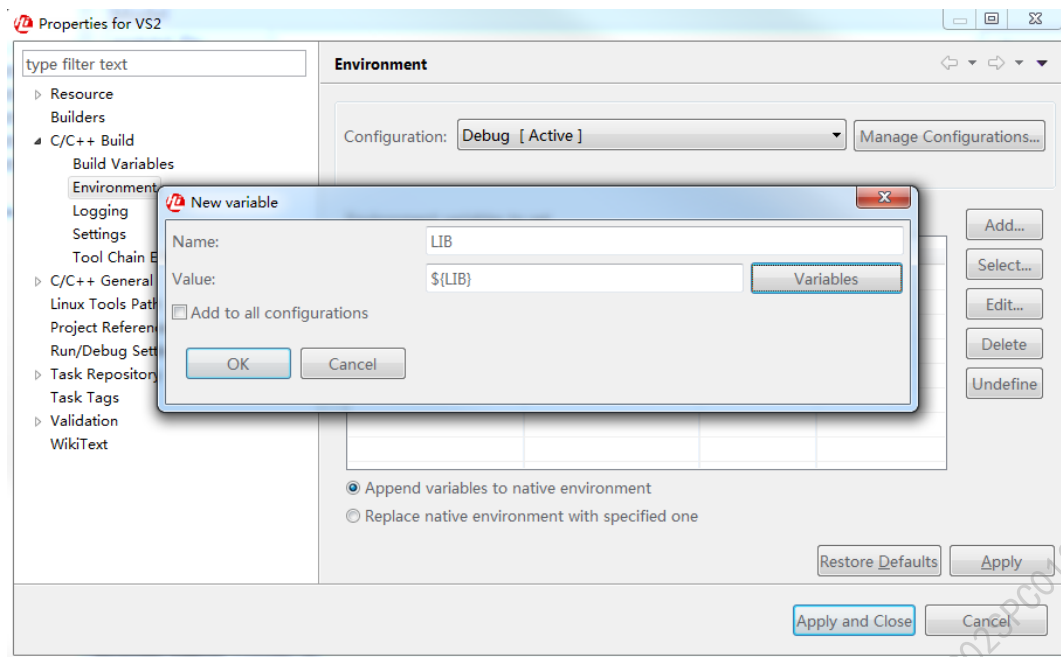
- LIB 主要指定 VS 相关的 LIB 路径
- INCLUDE 主要制定 VS 相关库的 INCLUDE 头文件路径
- PATH 环境变量主要目的是指定 MinGW 中的 cl.exe 的 bin 路径

设置的方式有以下两种：

- 采用系统环境变量，点击 Add，分别新增 LIB, INCLUDE, PATH，在 Variables 中搜索对应的系统环境变量，然后选中，如图 5-33 所示。

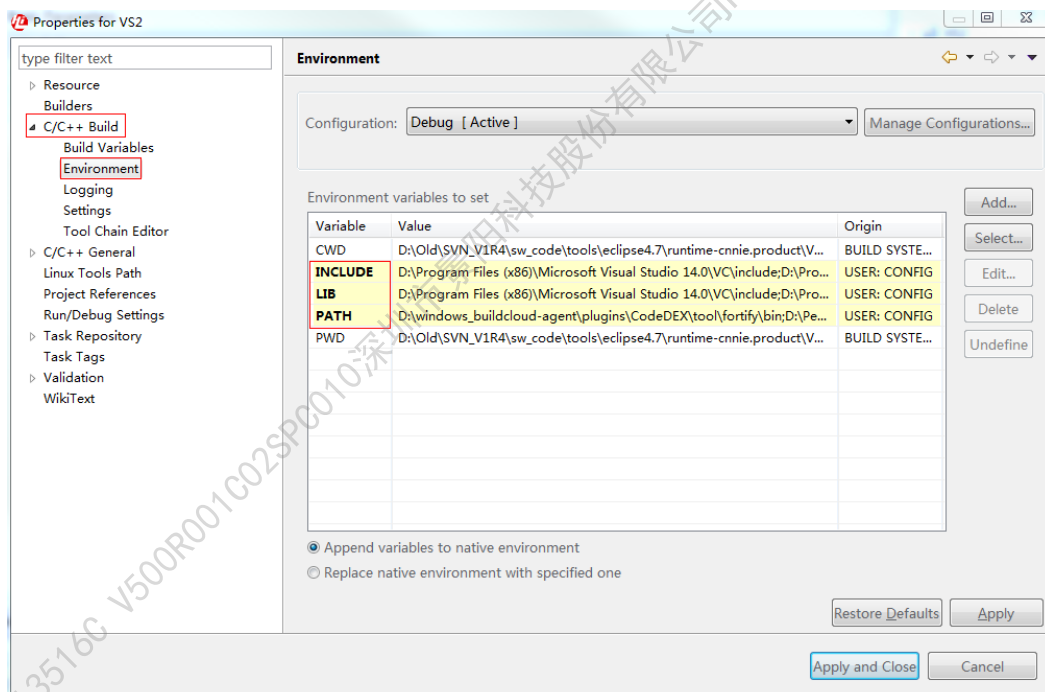


图5-33 采用系统环境变量



- 自定义环境变量，点击 Add，分别新增 LIB，INCLUDE，PATH，在 Value 中输入对应的环境变量路径，如图 5-34 所示。

图5-34 采用自定义环境变量





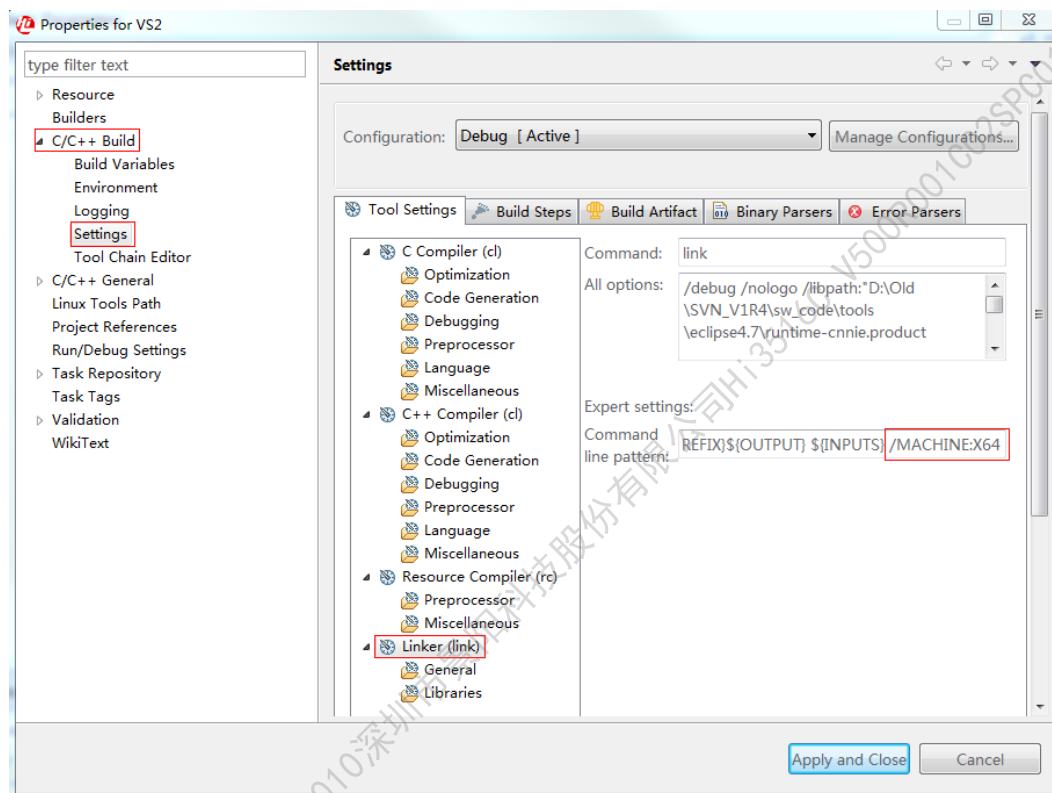
注意

PATH 默认为 PC 环境变量中的 PATH 路径，需要注意如果 PATH 中有之前配置过其他的 MinGW 的 bin 路径，则新增配置的 MinGW-W64 7.3.0 的 bin 路径可能会不生效，用户可以在图 5-34 的 PATH 中确认是否有多处 MinGW 路径，如果有则需要把此处 PATH 中的 MinGW 路径保证指定唯一 MinGW-W64 7.3.0 的 bin 路径。

5.2.5.2 C/C++ Build 中 Setting 配置

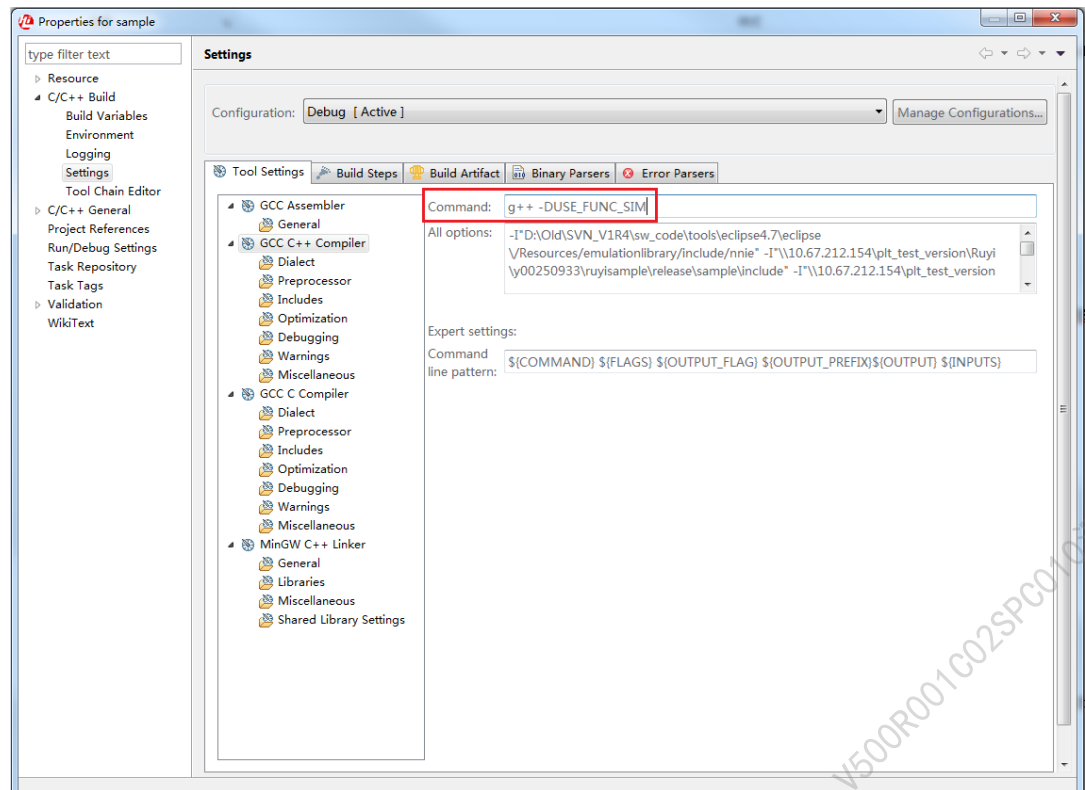
使用 "Settings" 属性页中的 Tool Settings，选项卡自定义在生成配置中使用的工具和工具选项。可以在此处对编译发送的命令进行配置，用于 C/C++ 工程的依赖或编译，如图 5-35 将 linker 设置为 X64 用于链接 64 位的 lib 或 dll。

图5-35 设置 linker 为 X64



当编译前需要指定 g++ 的调试参数时，需要在 Setting 处的 GCC C++ Compiler 页签中 g++ 命令后添加，如图 5-36 所示。

图5-36 设置 g++ 的编译参数

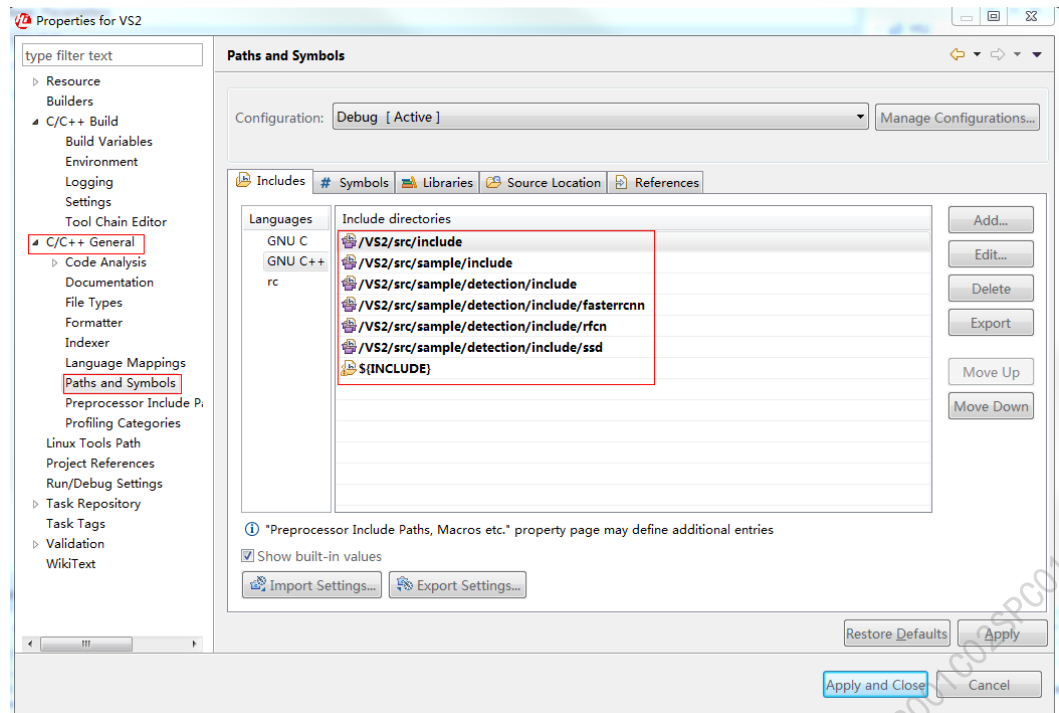


5.2.5.3 C/C++ General 中 Paths and Symbols 配置

在工程属性的 C/C++ General->Paths and Symbols 中的 Includes 和 Libraries 选项卡中，这里配置了 NNIE 工程编译相关头文件和路径和依赖的库：

- **Include** 配置中点击 Add 后有三种引入类型可以选择：
 - 选择 **Workspace**，可以将 workspace 中的 include 文件路径都引入到此处；
 - 选择 **Variables**，可以选择系统环境变量中的变量，如 **INCLUDE** 引入到此处；
 - 选择 **File System** 则可以将任意路径的文件目录引入到此处，如图 5-37 所示。

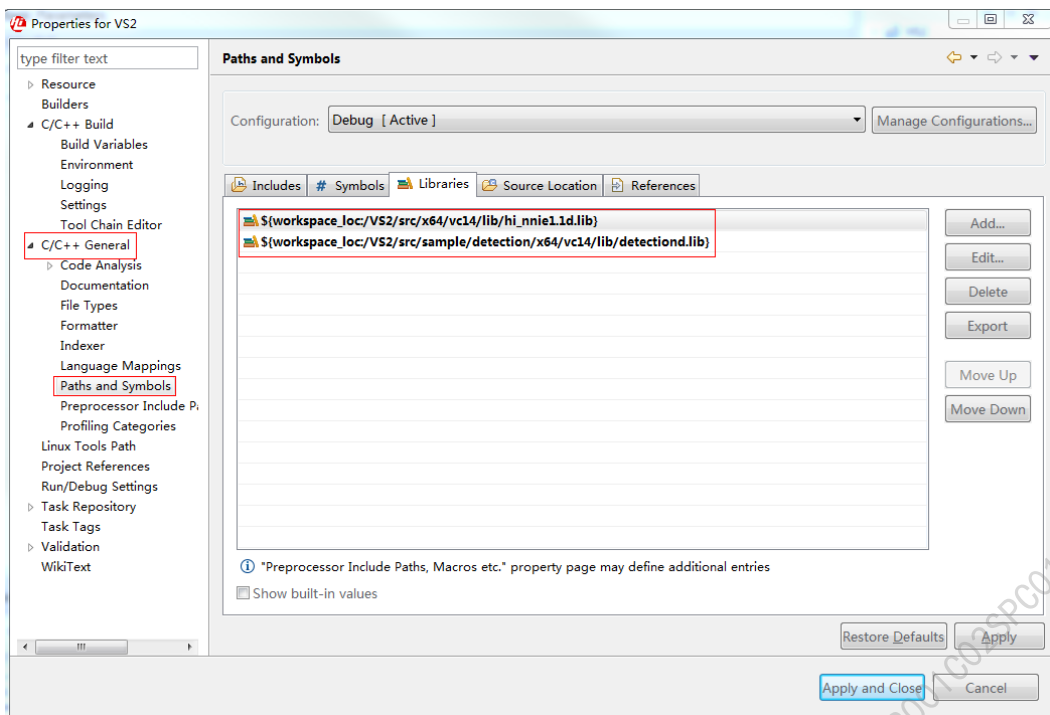
图5-37 工程所需的 Include 路径配置



- Libraries 配置中点击 Add 后同样有三种引入类型可以选择：
 - 选择 Workspace，可以将 workspace 中的 lib 或 dll 文件路径都引入到此处；
 - 选择 Variables，可以选择系统环境变量中的变量，如 LIB 引入到此处；
 - 选择 File System 则可以将任意路径的文件目录引入到此处，如图 5-38 所示。



图5-38 工程所需的 Lib 或 DLL 文件配置



说明

C/C++ Properties 属性页配置较多，此处介绍的主要和 NNIE 工程编译相关，如需了解其他配置请参考《C/C++ Development User Guide》文档。网址如下：
http://help.eclipse.org/neon/index.jsp?topic=%2Forg.eclipse.cdt.doc.user%2Fconcepts%2Fcdt_o_home.htm

5.2.6 C/C++代码编辑功能介绍

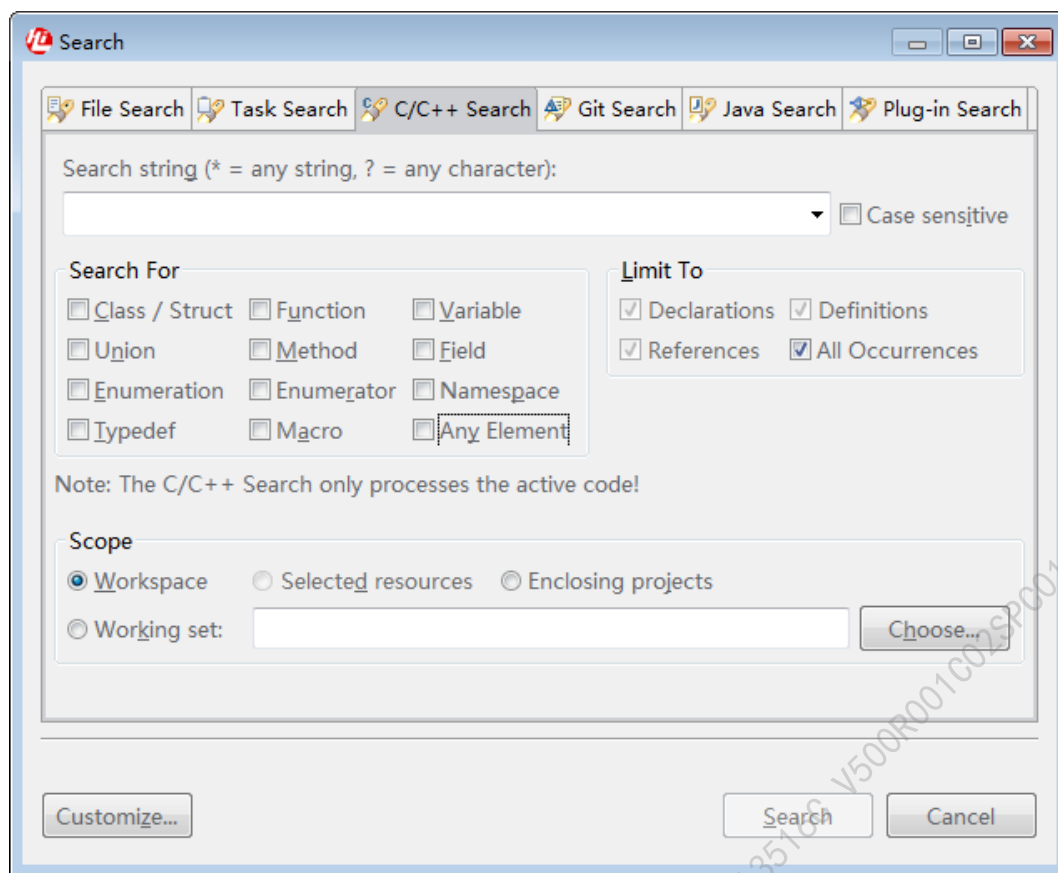
在创建 NNIE 工程后，可以添加在工程中添加 C/C++代码。

5.2.6.1 搜索功能

在创建的 C++项目中，打开一个 C 文件，在该编辑器界面按住 Ctrl+H 键或者在菜单栏点击 Search→Search，会弹出一个搜索框，如图 5-39 所示。



图5-39 搜索框

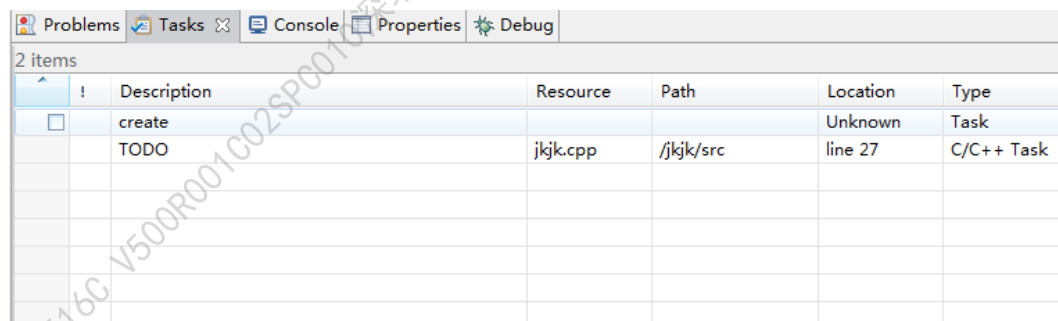


在输入框输入需要查询的类、函数、变量、枚举的关键字，选择需要查询的类、函数、变量、枚举，点击 Search 按钮，就能定位到需要查询的位置。

5.2.6.2 任务功能

步骤 1. 在创建的工程中，有一个 Tasks 视图，如图 5-40 所示。

图5-40 Tasks 视图

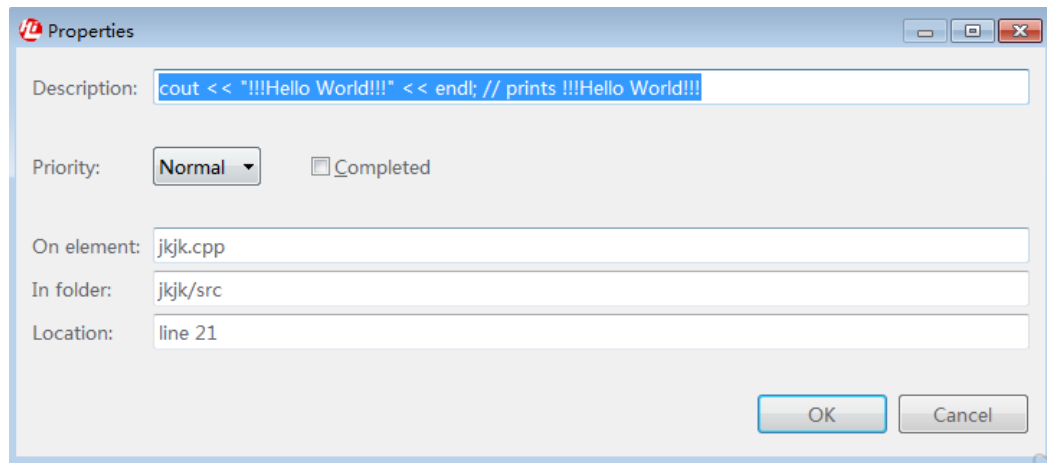


步骤 2. 在 Tasks 视图中可以查看所有的任务，点击其中一个任务，可以定位到任务的位置。



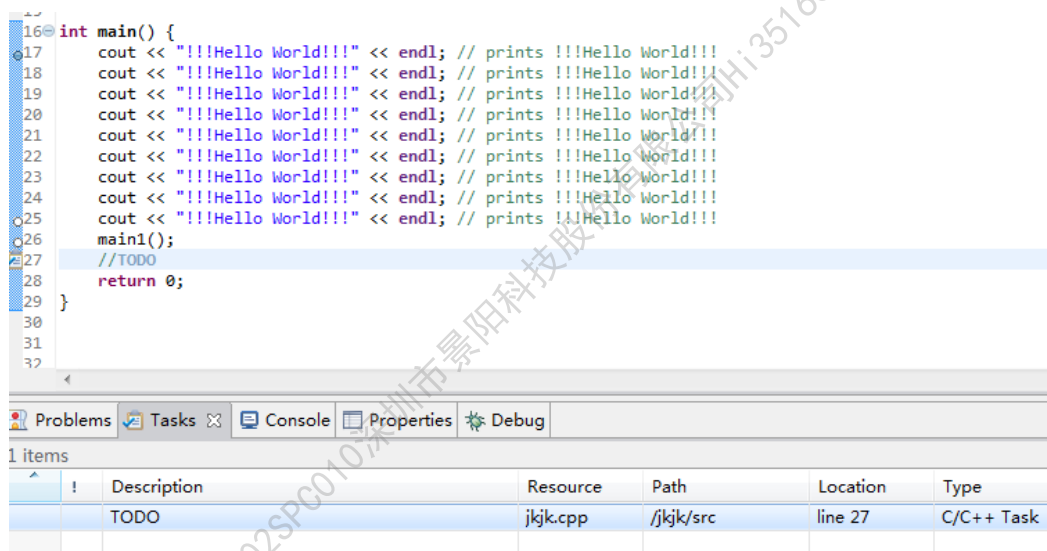
步骤 3. 在编辑器的左边框，选择需要加断点的行，点击鼠标右键，选择 Add Task，会弹出一个对话框如图 5-41 所示。

图5-41 创建任务框



步骤 4. 在代码中添加注释如“//TODO”格式的注释，会在 Tasks 视图添加一条任务。如图 5-42 所示。

图5-42 在代码中创建任务



----结束

5.2.6.3 定位问题功能

在创建的 C++ 工程中，有一个 Problem 视图，如图 5-43 所示。



图5-43 问题视图

Problems	Tasks	Console	Properties	Debug
1 error, 0 warnings, 0 others				
Description	Resource	Path	Location	Type
Errors (1 item)				
Invalid arguments '	jkjk.cpp	/jkjk/src	line 25	Semantic Err...

在视图中会显示所有工程中出现的问题，在视图中会显示详细的报错信息，报错的文件，路径以及位置。点击该问题，能在快速在代码中定位到当前错误的位置。

5.2.6.4 查看文件属性功能

在创建的 C++ 工程中，有一个 Properties 视图。在工程中，选中一个文件，Properties 视图会显示该文件的信息，包括名字、路径、权限、文件修改记录和文件大小。如图 5-44。

图5-44 属性视图

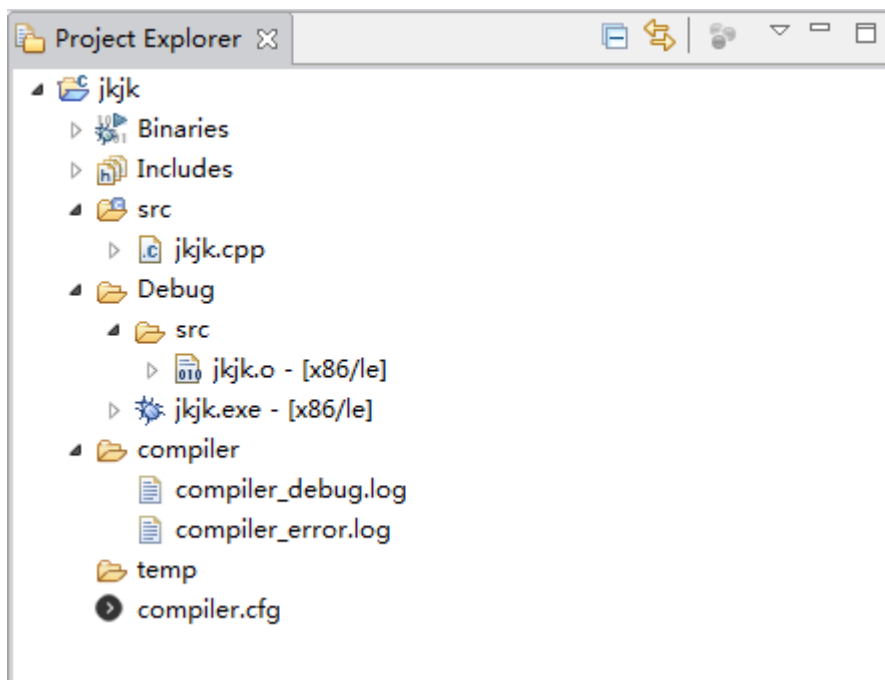
Problems	Tasks	Console	Properties
/jkjk/src/jkjk.cpp			
Resource	Property	Value	
	Info		
	derived	false	
	editable	true	
	last modified	2017年12月8日 上午11:19:41	
	linked	false	
	location	D:\C_SVN\sw_code\src\runtime-cnnie.product\jkjk\src\jkjk.cpp	
	name	jkjk.cpp	
	path	/jkjk/src/jkjk.cpp	
	size	1,086 bytes	

5.2.6.5 项目资源管理功能

在工具的界面中，有一个 Project Explorer 视图，如图 5-45 所示。



图5-45 项目管理视图

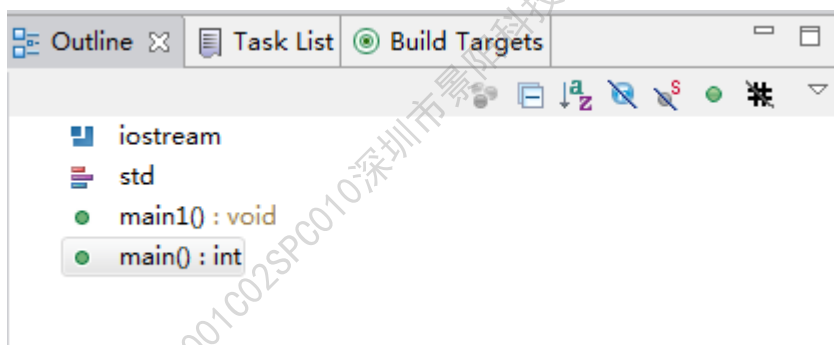


在该视图中，可以查看当前工作空间所有创建的工程，同时也可以对创建的工程进行资源理，可以对工程的文件进行创建、修改、删除和查看。

5.2.6.6 大纲视图功能

在工具的界面中，有一个 Outline 视图，在创建的工程中，打开一个 C 文件，在 Outline 视图中，可以显示该文件的所有的函数、头文件以及变量，如图 5-46 所示。

图5-46 大纲视图



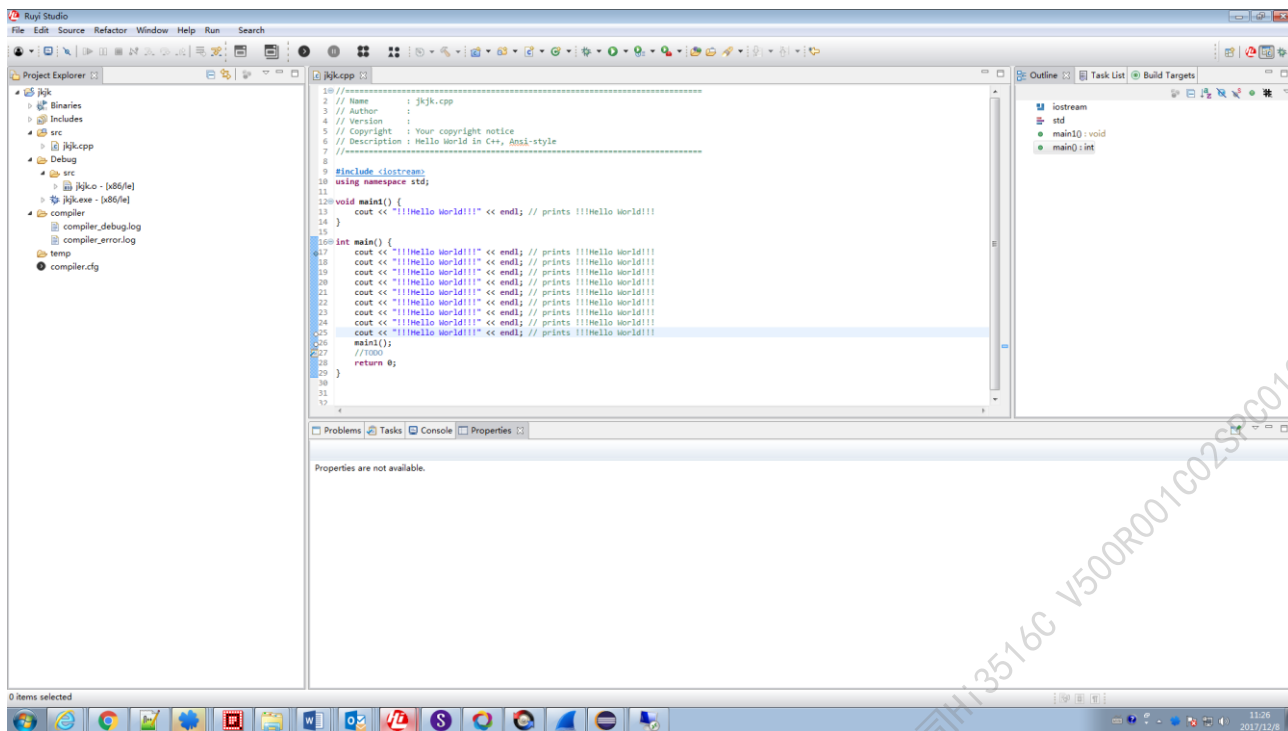
在该视图可以选择需要查看的函数、头文件和变量，双击选中的属性，可快速定位到对应的代码中。



5.2.6.7 引用关系查看功能

打开工程中创建的一个 C 文件，选择当前文件中的函数或者头文件，按住 Ctrl 键，选中的函数或者头文件会出现下划线。如图 5-47 所示。

图5-47 引用关系查看

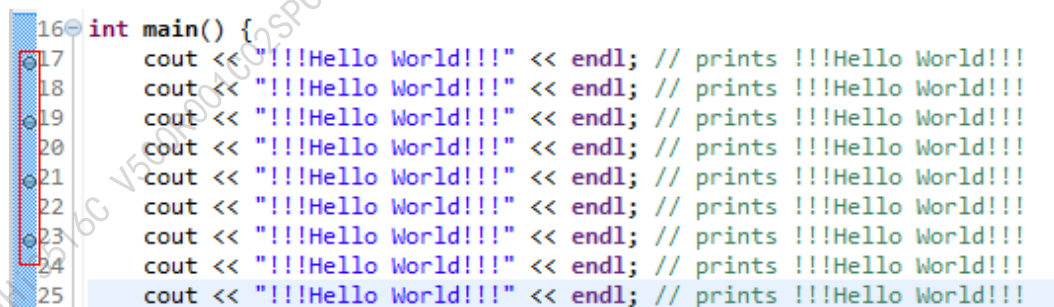


此时按住 Ctrl 键不放，点击鼠标左键，会跳转到选择函数或者头文件的定义处。

5.2.6.8 集成调试功能

打开工程中创建的一个 C 文件，在打开文件的编辑处，在编辑器的左边框，选择需要加断点的行，点击鼠标右键，选择 Toggle BreakPoint，会在该处出现一个圆点，如图 5-48 所示。

图5-48 断点显示



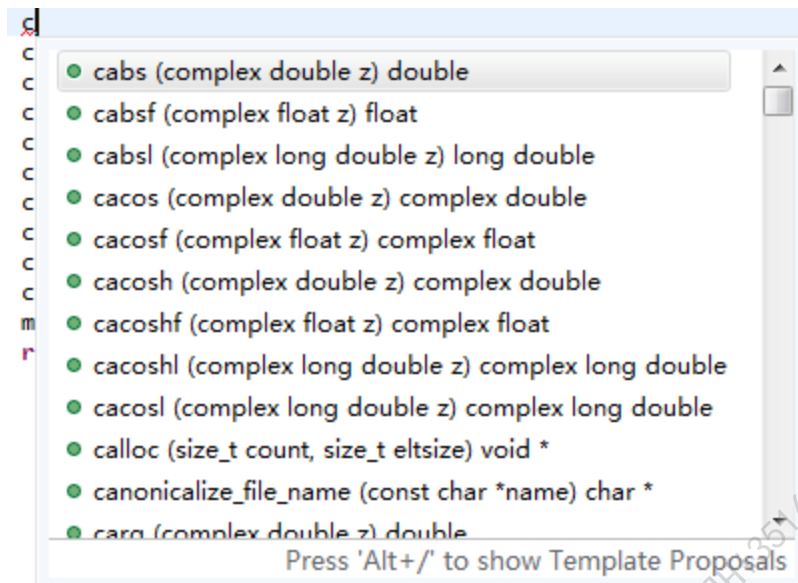


该圆点功能为调试 Debug 的断点，当使用 Debug 调试的时候，程序走到这里就会暂停。

5.2.6.9 代码内容辅助功能

打开工程中创建的一个 C 文件，再打开文件的编辑处，对文件进行编译，在编辑的过程中没有输入完整的关键字，此时可以按住 Alt+/ 键，会弹出该当前输入关键字有关的函数、变量的对话框，如图 5-49 所示。

图5-49 代码辅助功能



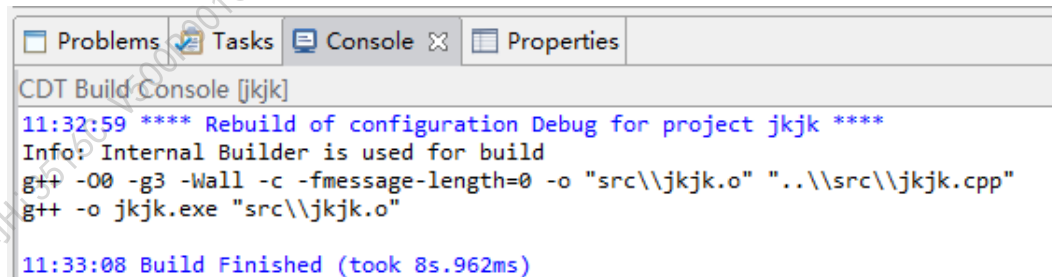
在弹出的框中选择所需的函数或者变量。选择的对象会输入到当前的编辑行。

5.2.7 C/C++代码编译功能

编译 NNIE 的 C/C++工程时需要提前配置工程的 Include，Lib 及 Path 路径，请参考 5.2.5 “工程属性设置” 章节。配置完成后，方可执行如下编译操作。

在项目资源中选择一个 C++项目，点击  按钮，点击完按钮之后会在 Console 视图打印当前项目的编译日志如图 5-50。

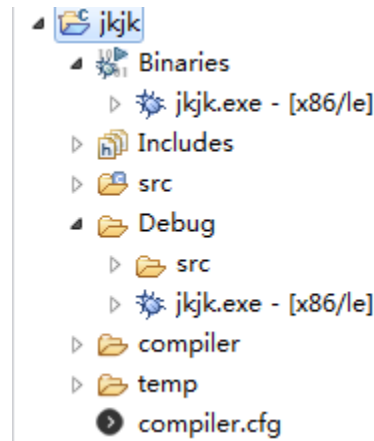
图5-50 编译日志





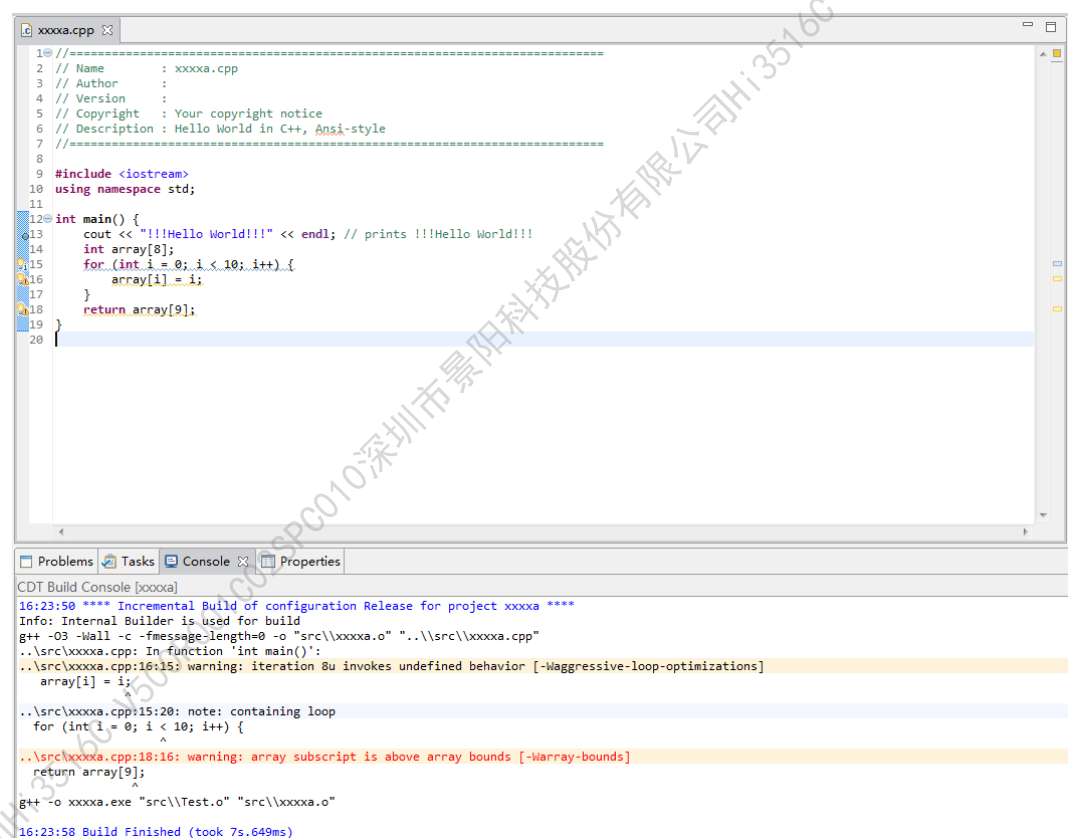
编译成功后会在当前的项目中生成一个 Release 或者 Debug 文件夹，文件夹包含一个 src 文件夹和对应的工程文件名的 exe 文件。如图 5-51。

图5-51 编译成功后的文件夹



在对选中的文件进行编译时，当出现错误时，Console 会打印编译报错的日志。如图 5-52。

图5-52 编译报错的日志





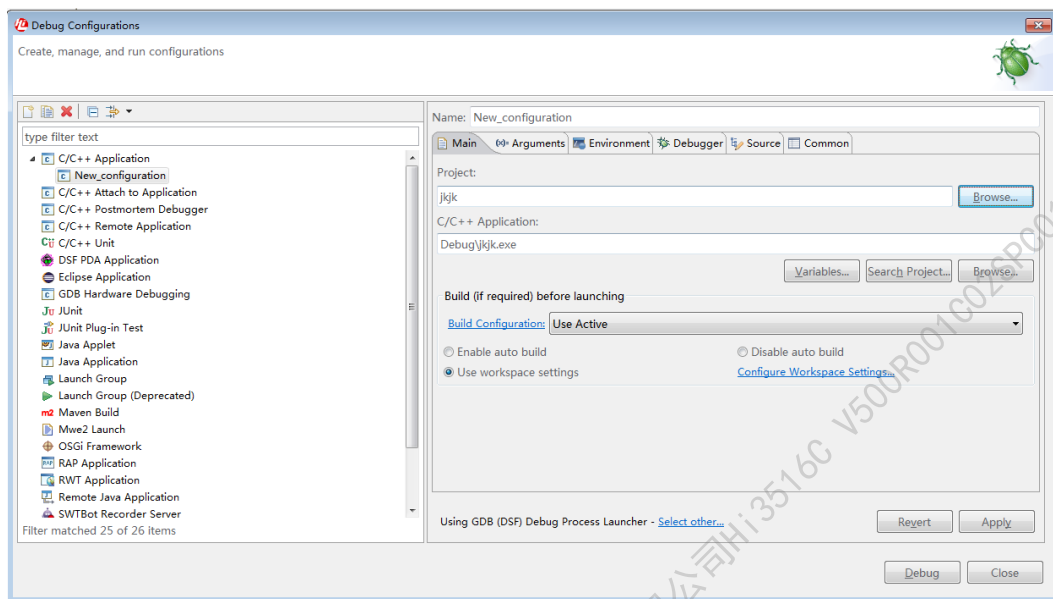
在打印的报错日志中，点击对应的报错日志，可定位到编译出错的代码中。

5.2.8 C/C++代码调试功能

5.2.8.1 创建 C/C++ Application

步骤 1. 代码编译通过后，点击 Debug 按钮 ，点击小三角，选择 Debug Configure，弹出对话框，如图 5-53。

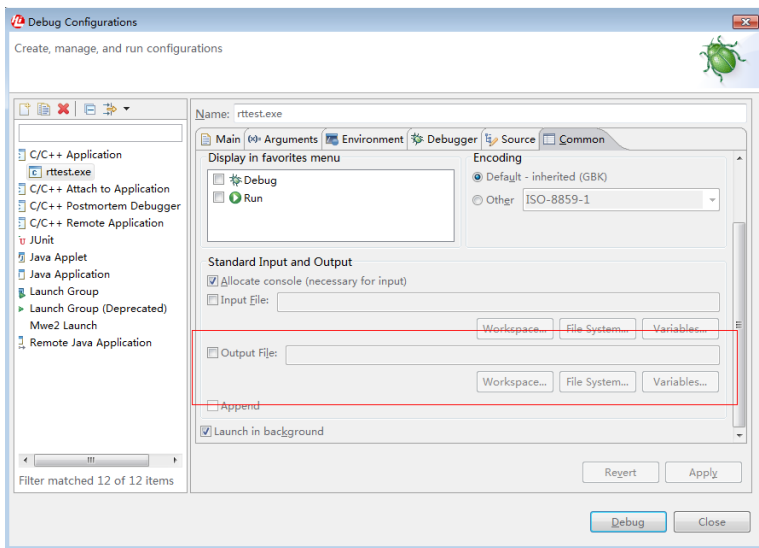
图5-53 创建 Application 设置视图



步骤 2. 由于工具界面 Console 视图能容纳的日志行数有限，因此一般情况下需要重定向标准输出。步骤 1 的 Application 设置视图中选择 Common 页签，在 Output File 中指定一个文件作为日志输出文件即可。

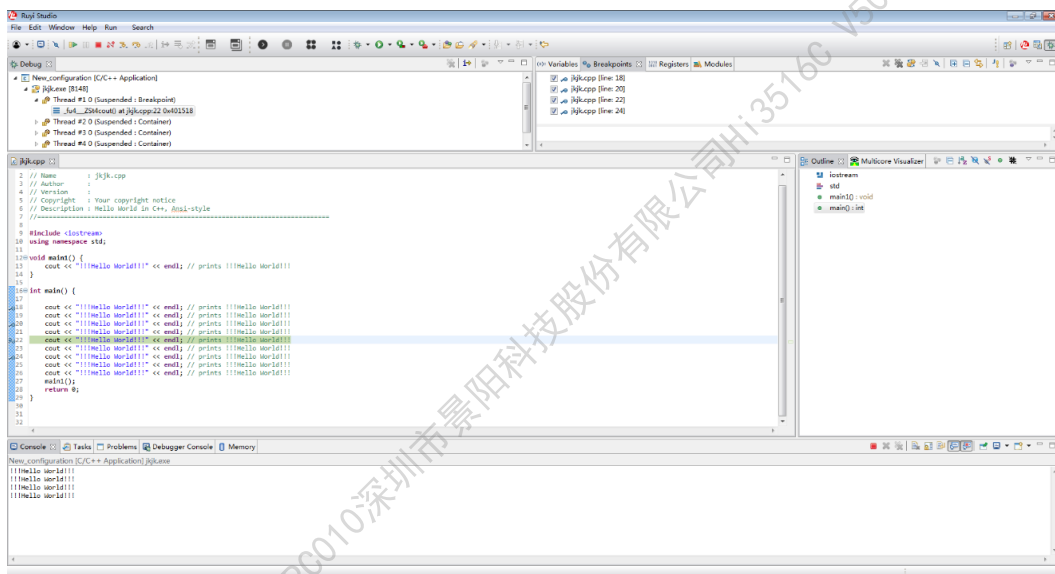


图5-54 重定向标准输出



步骤 3. 设置完成后点击 Debug 按钮，可以选择跳转到 debug 透视图，如图 5-55 所示。

图5-55 Debug 透视图



5.2.8.2 Debugger 视图按钮功能

- 调试实例----调试名称和状态。
- 线程实例----线程数量和状态。
- 堆栈框架和实例----堆栈框架数量，函数，文件名，文件行数
- 移除所有中止的发布----在调试视图中清除所有的中止进度。
- 跳过断点----在调试的过程中跳过所有的断点，不在打了断点的地方停止。



-  恢复----运行程序。
-  暂停----暂停执行当前选中的调试目标线程。
-  终止----结束选中的调试会话或者进度。影响的行为依赖于在调试视图中选中的条目的类型。
-  步进执行----执行当前语句，但会跟踪到子程序的内部。
-  单步执行----执行当前语句，但是跳过子程序的调用。
-  指令步进模式----启用指令步进模式，当按下此按钮，单步和步进操作是按汇编指令一条条执行，否则，是按源码的语言形式一条条执行。

5.2.8.3 C/C++工程相关调试说明

C/C++工程具体调试功能，请参考《C/C++ Development User Guide》文档，链接如下：

http://help.eclipse.org/neon/index.jsp?topic=%2Forg.eclipse.cdt.doc.user%2Fconcepts%2Fcdt_o_home.htm

5.3 生成 NNIE wk 功能

新建工程或者导入已有工程后，会出现后缀为“.cfg”的 NNIE mapper 配置文件。

5.3.1 cfg 文件配置模式

5.3.1.1 界面配置模式

默认双击左侧 Project Explorer 视图中 xxx.cfg 文件，cfg 文件将以 Mapper Configuration Editor 的模式打开界面模式，或者选中 cfg 文件右键选择 Open with-> Mapper Configuration Editor，如图 5-56，打开后的界面如图 5-57。



图5-56 Mapper Configuration Editor 界面模式打开 cfg 文件

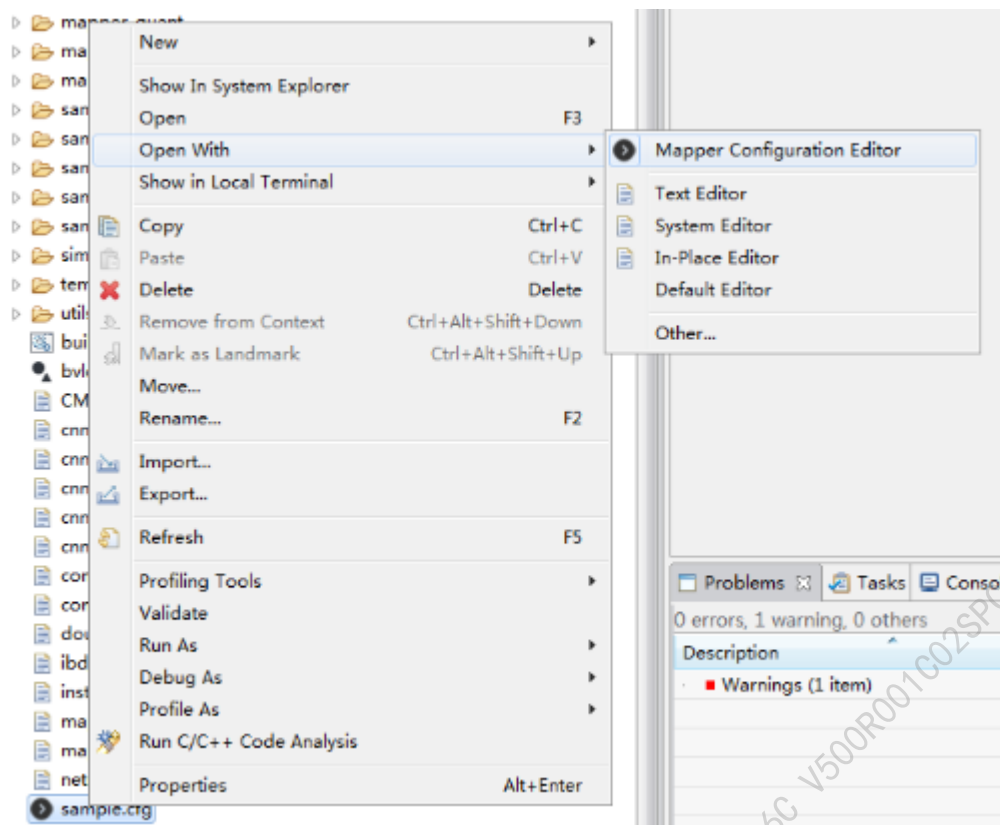
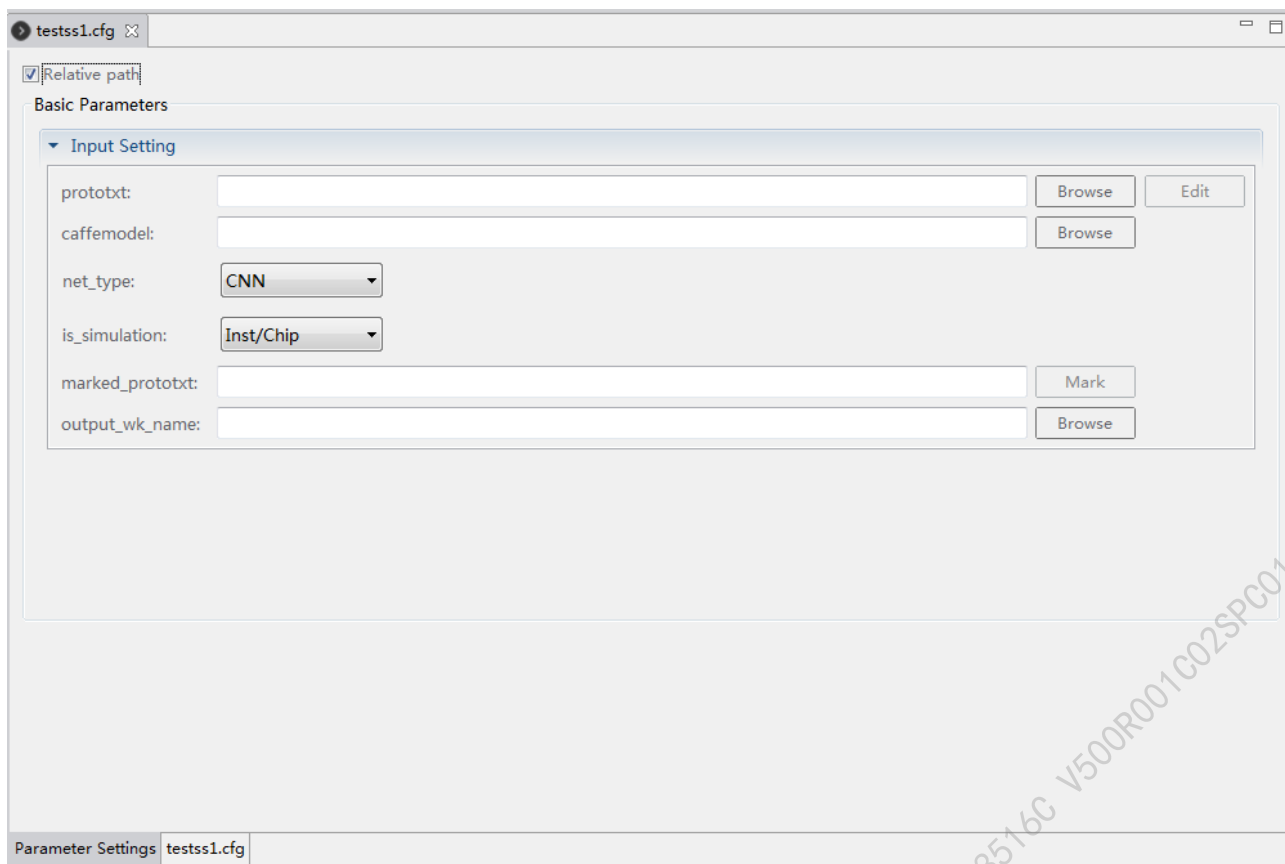




图5-57 Mapper Configuration Editor 视图

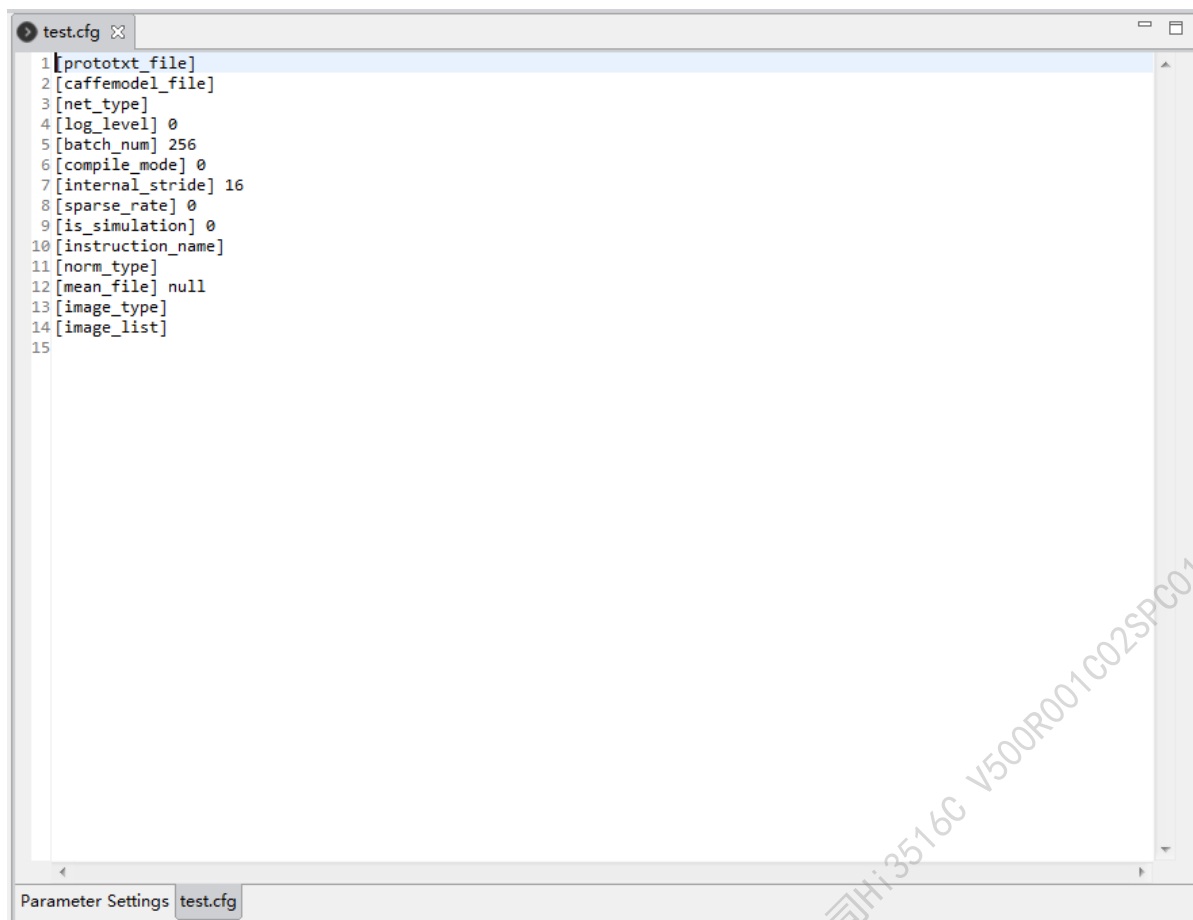


5.3.1.2 文本编辑模式

选中 cfg 文件右键选择 Open with->Text Editor, 或者点击图 5-58 左下角的 test.cfg 按钮可以切换到 cfg 文件的文本编辑视图;

cfg 文本文件可以在 linux 版本的 nnie_mapper 中编译。

图5-58 cfg 文件 Text Editor 视图



5.3.2 Prototxt 文件自动标记功能

在 cfg 文件界面视图中，如图 5-35，点击 Browse 按钮导入 prototxt，导入 prototxt 后，需要选择 net_type/caffemodel 才会进行预处理，RuyiStudio 自动将 NNIE 不支持的层修改为 Custom 或 Proposal 层，该功能为预处理的第一个步骤。

因用户导入的 Prototxt 文件中可能有 nnie_mapper 不支持的层类型，RuyiStudio 自动识别 prototxt 层并根据 NNIE mapper Non-support 层处理方式将不支持的层修改为 Custom 或者 Proposal 层，且生成一个标记后的 prototxt 文件，路径存放在 Project 工程 mark_prototox 文件目录下，命名规则为原 Prototxt 文件名_mark_时间戳.prototxt 文件，如图 5-59 所示。

RuyiStudio 将作为 ROIpooling 或者 PSROIpooling 层输入的 Non-support 层则修改为“Proposal”层，其他则作为“Custom”层。

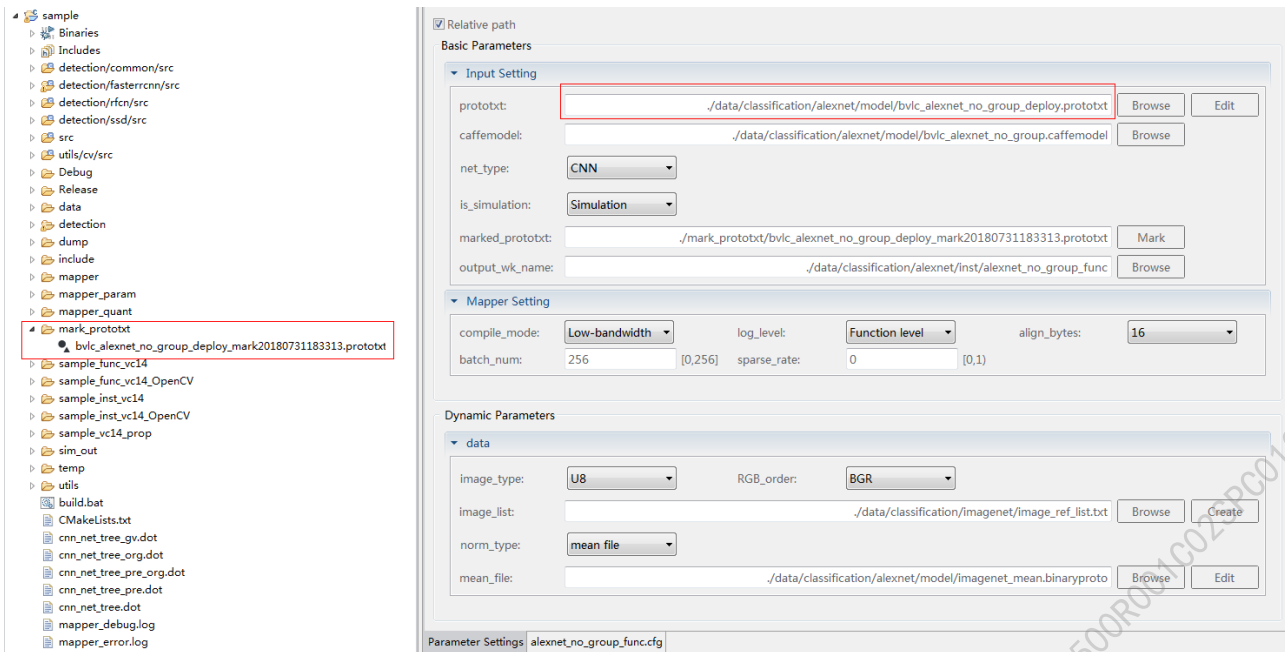
RuyiStudio 支持解析 prototxt 的层类型如下：

"Convolution", "Deconvolution", "Pooling", "DepthwiseConv", "InnerProduct", "LRN", "BatchNorm", "Scale", "Bias", "Eltwise", "ReLU", "PReLU", "AbsVal", "TanH", "Sigmoid", "BNLL", "ELU", "CReLU", "LSTM", "RNN", "Softmax", "Exp", "Log", "Reshape", "Flatten", "Split", "Slice", "Concat", "SPP", "Power", "Threshold", "MVN", "Parameter",



"Reduction", "Proposal", "ROIpooling", "PSROIpooling", "Normalize", "PassThrough", "Upsample", "Permute", "MatMul", "Custom", "Input", "Dropout".

图5-59 Prototxt 自动标记功能



5.3.3 标记后的 Prototxt 可视化功能

点击如图 5-60 中 Mark 按钮进入工具自动标记后的 Prototxt 的网络拓扑图，如图 5-61。

图5-60 编辑标记后 Prototxt 文件

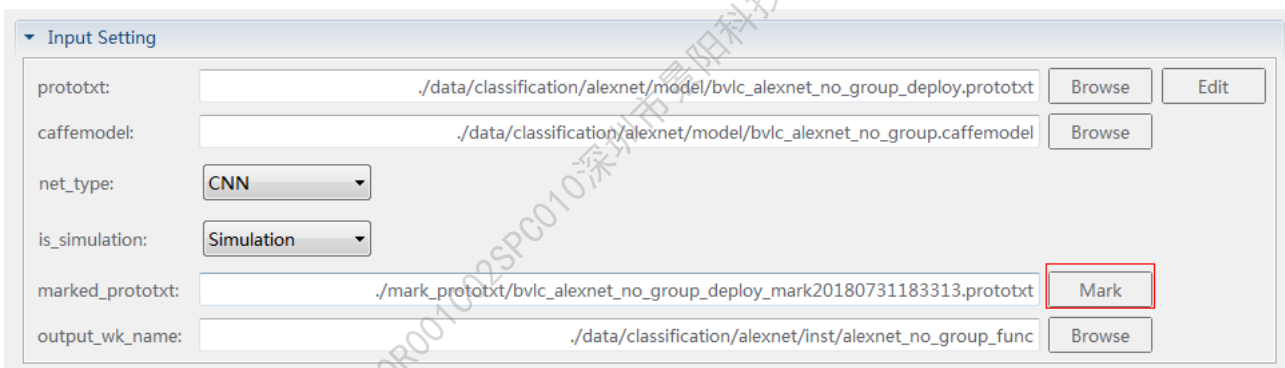
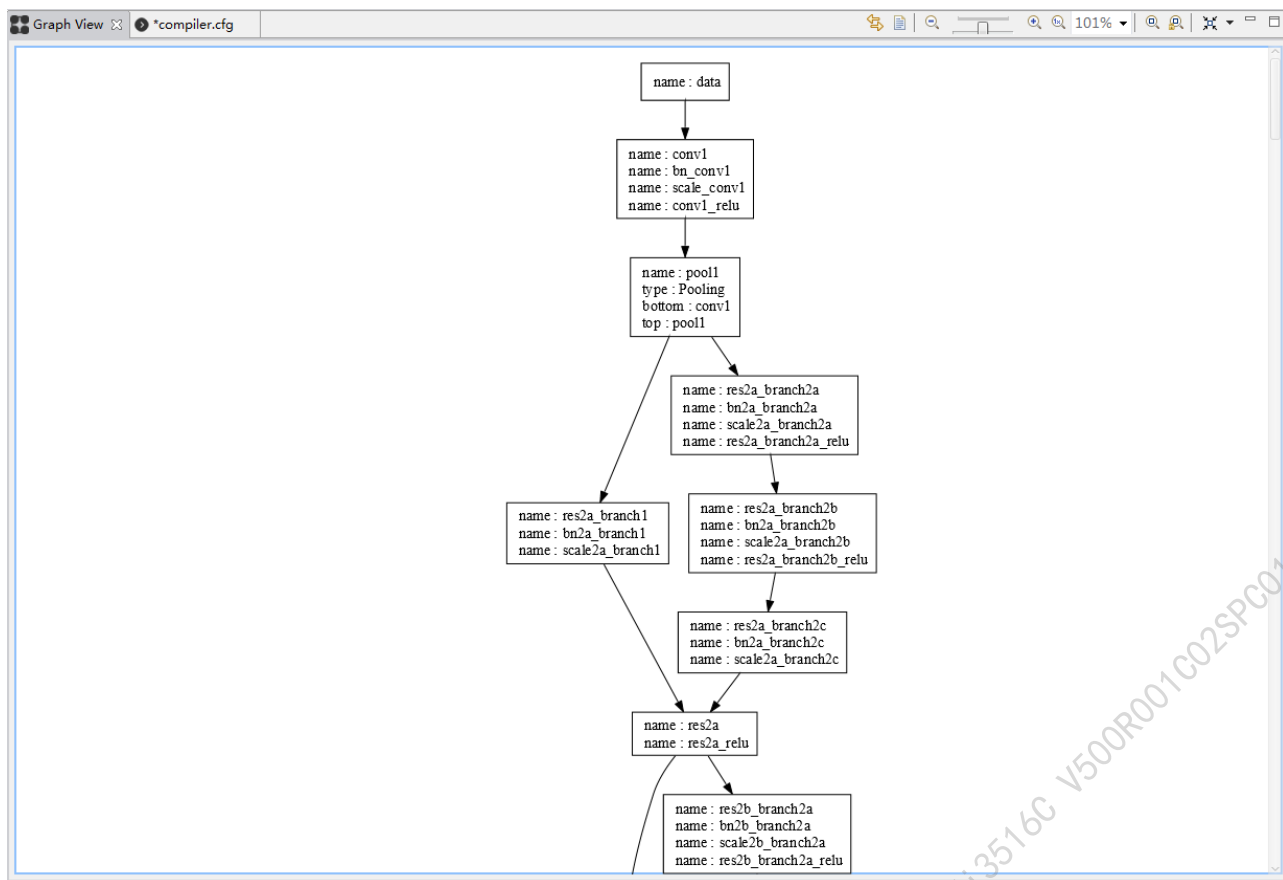


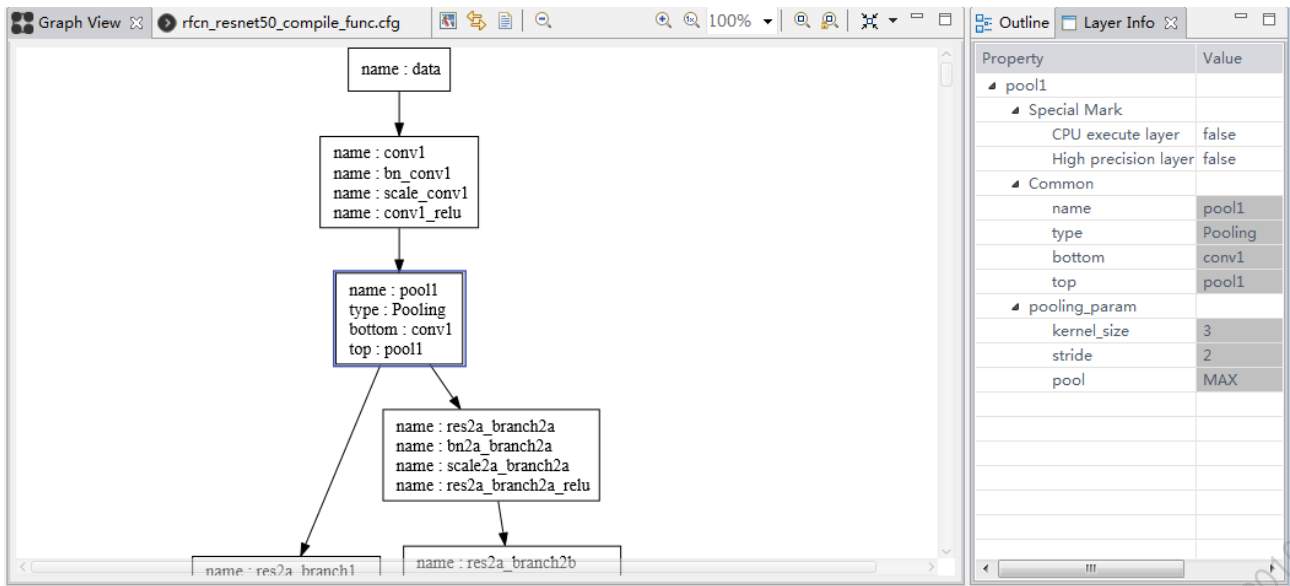
图5-61 标记后的 Prototxt 的网络拓扑图



选中网络拓扑图中的 Layer 节点，被选中的层的外框会被标记为蓝色，如果当前层对应的 Type 是 NNIE 不支持的 Type 类型，则当前层的 Type 会被改为 Custom，且会将其他参数删除，双击选中的层可以在下侧 Property 视图中查看和修改当前 Layer 对应的 Common 基本参数，包括 name, type, bottom, top 等信息，如图 5-62。



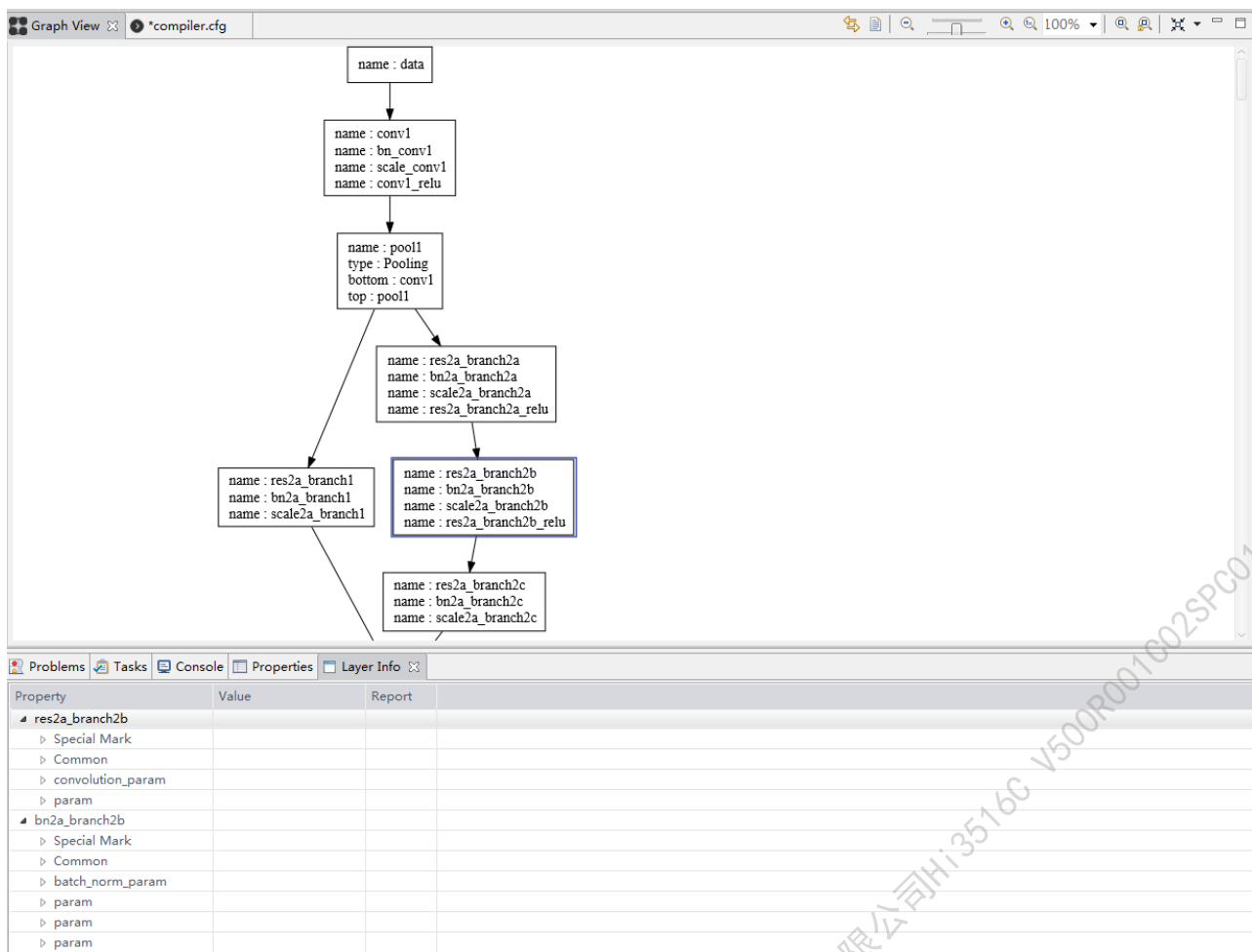
图5-62 标记后的 Prototxt 中 Layer 对应的属性图



Inplace 的层（Inplace 即为相互连接的几个层中 top 和 bottom 完全相同）在网络拓扑图中会被放在一个框节点中，选中一个这样的框，所有层的属性都会显示在 Layer Info 视图中，如图 5-63。



图5-63 Inplace 层节点的 Layer Info 视图



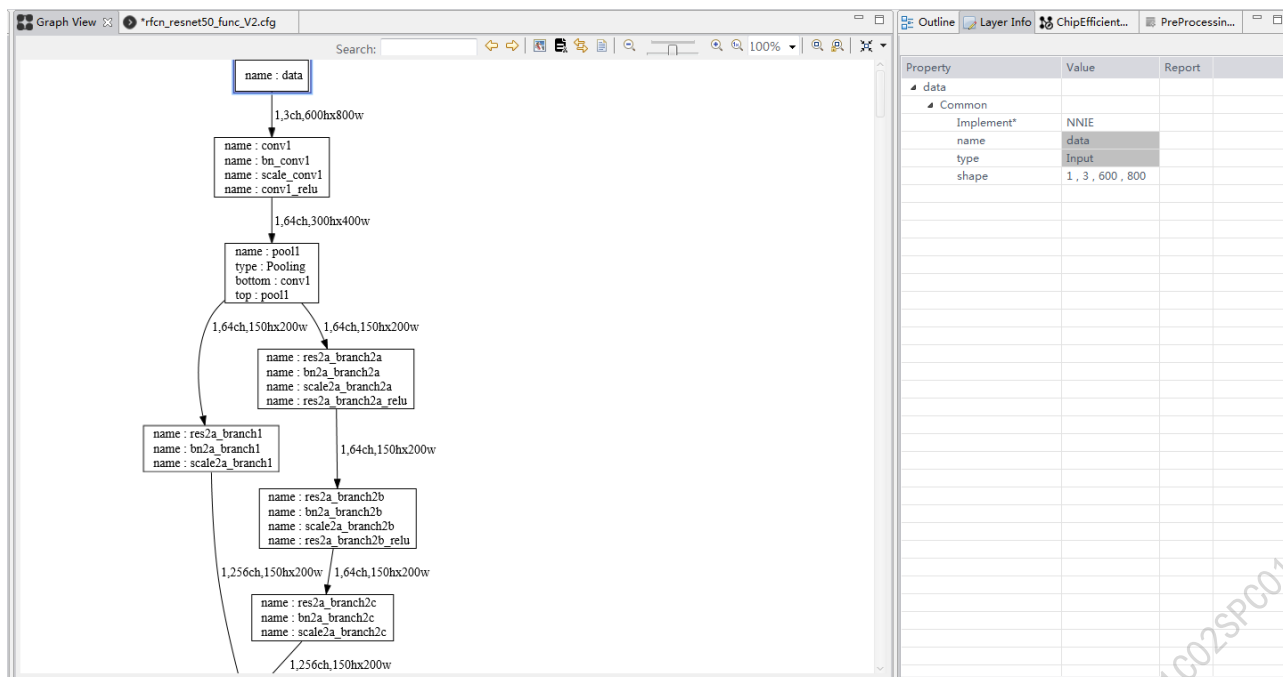
5.3.4 网络拓扑图中显示各原生 caffe 支持层的 Shape 信息

5.3.4.1 Shape 信息显示

支持当前 PC 如果安装了 Python 和 Caffe 环境下（可参考 5.1.2 章节），在按照 5.3.3 操作后，网络拓扑图中可以显示出各层的 shape 信息，如图 5-64。



图5-64 显示 shape 信息的网络拓扑图

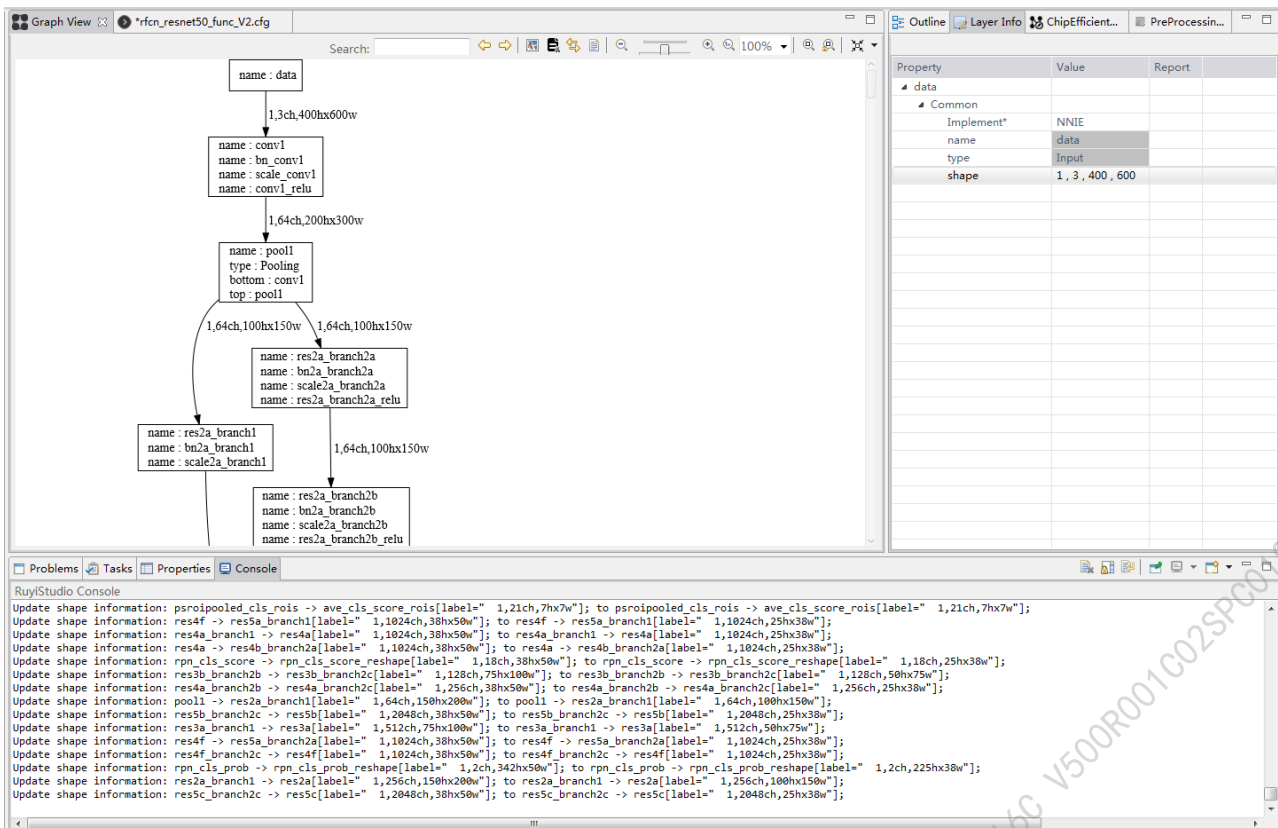


5.3.4.2 Shape 信息更新

在 GraphView 中双击 input 层或 custom 层，然后在层属性 LayerInfo 界面中，修改 shape 信息，然后点击其他空白处确认后，工具会自动更新当前网络拓扑图中各层的 shape 信息并刷新到界面上且修改内容会打印在控制台中，如图 5-65 所示。



图5-65 更新 shape 信息后的网络拓扑图



5.3.5 Prototxt 层属性可视化编辑

NNIE mapper 支持将任意层切换为 CPU 运算，支持自定义高精度层，支持任意层中间上报。

在工具中通过修改层对应的属性，可以将当前层标记为以上三种类型。

- 将当前 Layer 指定为 CPU 运算，在右侧 Property 视图中选择 true 后，工具自动修改 name 添加_cpu 字段，如图 5-66 所示。



图5-66 将当前 Layer 指定为 CPU 运算

The screenshot shows the HiSVP Graph View interface. The main window displays a neural network graph with the following layers:

- name : data
- name : conv1
name : bn_conv1
name : scale_conv1
name : conv1_relu
- name : pool1_cpu
type : Pooling
bottom : conv1
top : pool1
- name : res2a_branch1
name : bn2a_branch1
name : scale2a_branch1
- name : res2a_branch2a
name : bn2a_branch2a
name : scale2a_branch2a
name : res2a_branch2a_relu
- name : res2a_branch2b
name : bn2a_branch2b
name : scale2a_branch2b
name : res2a_branch2b_relu
- name : res2a_branch2c
name : bn2a_branch2c
name : scale2a_branch2c

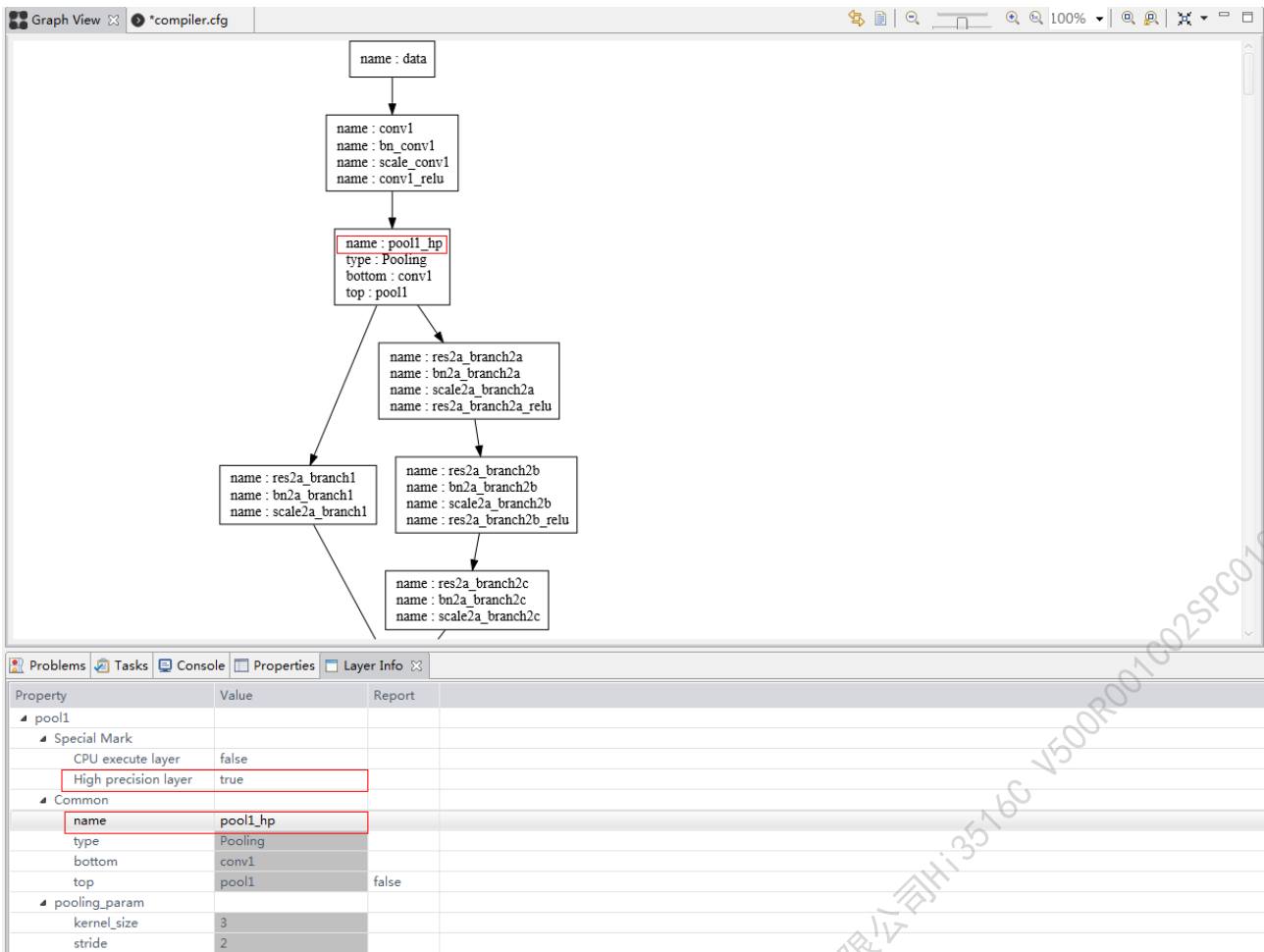
The 'pool1_cpu' layer is highlighted with a red box. Below the graph, the 'Layer Info' panel is visible, showing the properties of the selected layer:

Property	Value	Report
pool1		
Special Mark		
CPU execute layer	true	
High precision layer	false	
Common		
name	pool1_cpu	
type	Pooling	
bottom	conv1	
top	pool1	false
pooling_param		
kernel_size	3	
stride	2	

- 将当前 Layer 指定为高精度层，在右侧 Property 视图中选择 true 后，工具自动修改 name 添加_hp 字段，如图 5-67。



图5-67 将当前 Layer 指定为高精度层



- 将当前 Layer 指定为中间上报层，在右侧 Property 视图中选择 true 后，工具自动修改 top 添加_report 字段，如[图 5-68](#)。



图5-68 将当前 Layer 指定为中间上报层

The screenshot shows the RuyiStudio Graph View interface. The main area displays a neural network graph with the following layers and connections:

- data** (name: data) connects to **conv1**.
- conv1** (name: conv1, name: bn_conv1, name: scale_conv1, name: conv1_relu) connects to **pool1**.
- pool1** (name: pool1, type: Pooling, bottom: conv1, top: pool1_report) connects to **res2a_branch1** and **res2a_branch2a**.
- res2a_branch1** (name: res2a_branch1, name: bn2a_branch1, name: scale2a_branch1) connects to **res2a_branch2c**.
- res2a_branch2a** (name: res2a_branch2a, name: bn2a_branch2a, name: scale2a_branch2a, name: res2a_branch2a_relu) connects to **res2a_branch2b**.
- res2a_branch2b** (name: res2a_branch2b, name: bn2a_branch2b, name: scale2a_branch2b, name: res2a_branch2b_relu) connects to **res2a_branch2c**.
- res2a_branch2c** (name: res2a_branch2c, name: bn2a_branch2c, name: scale2a_branch2c) connects to **res2a_branch2c**.

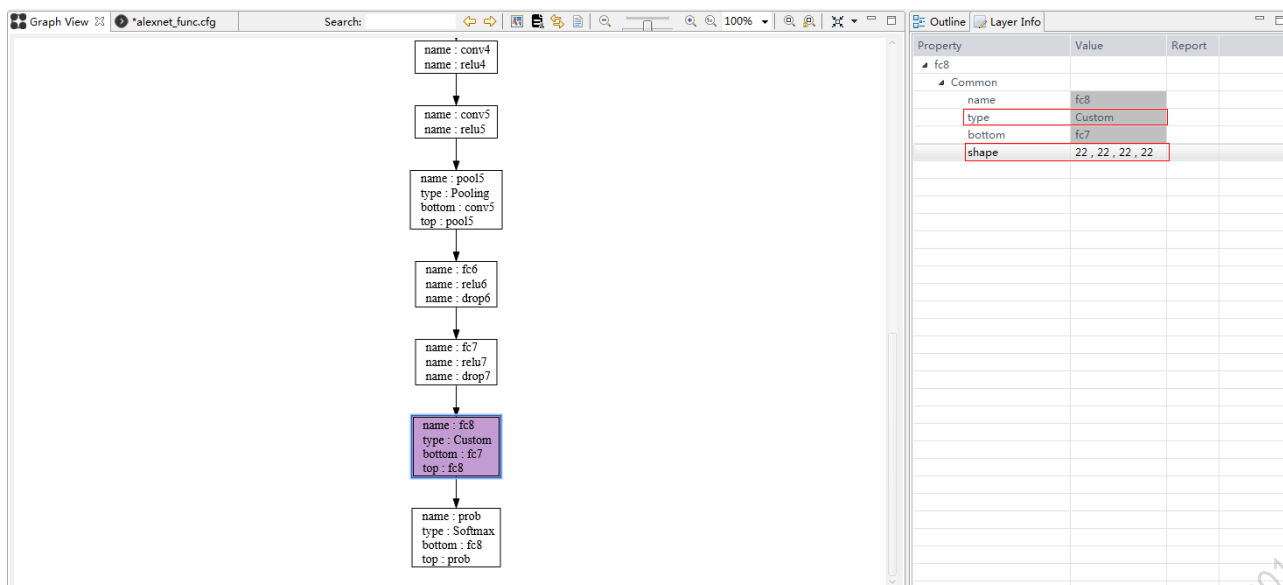
The Layer Info panel at the bottom shows the properties for the selected **pool1** layer:

Property	Value	Report
Special Mark		
CPU execute layer	false	
High precision layer	false	
Common		
name	pool1	
type	Pooling	
bottom	conv1	
top	pool1_report	true
pooling_param		
kernel_size	3	
stride	2	

- 当前选中的 Layer 为 Custom 时，除过基本参数 name、type、bottom、top，其他参数因 nnie_mapper 可能无法识别，工具在自动标记 Prototxt 时会删除这些参数，同时会新增 shape 参数，当前 Custom 层中有多少个 Top 属性就会对应有几个 Shape 参数，shape 默认的参数配置为 0,0,0,0，分别表示 N、C、H、W，这时用户必须修改 shape 参数的值；nnie_mapper 支持的 shape 参数值至少有 2 位，如果填 0，工具默认当做此 dim 值为无效值不传给 nnie_mapper，即 (*, *, 0, 0), (*, *, *, 0), (*, *, *, *) 此三种格式，如图 5-69。



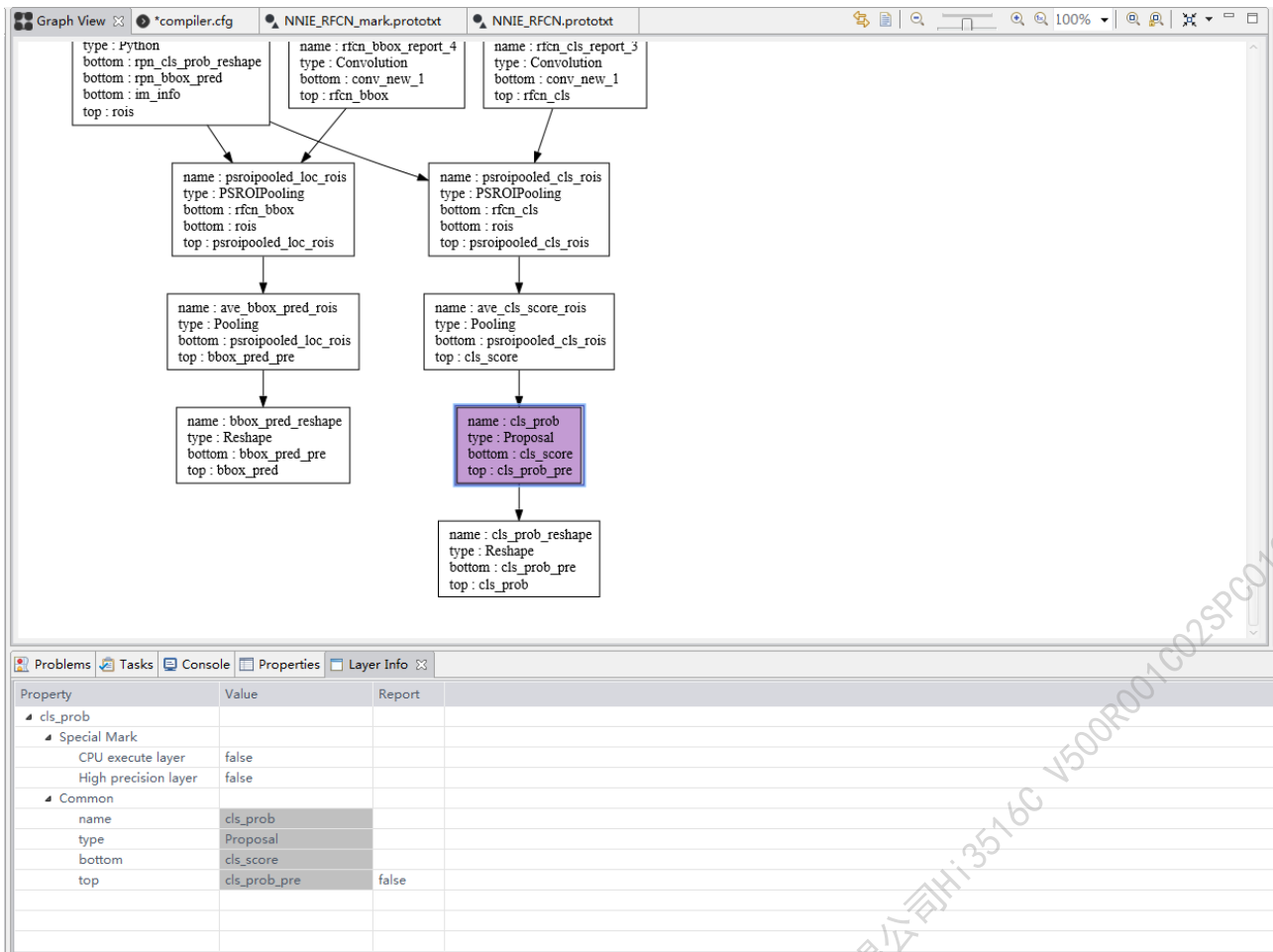
图5-69 当前 Layer 为 Custom 层



- 当前选中的 Layer 为 Proposal 时，除过基本参数 name、type、bottom、top，其他参数因 nnie_mapper 可能无法识别，工具在自动标记会删除这些参数，如图 5-70。



图5-70 当前 Layer 为 Proposal 层



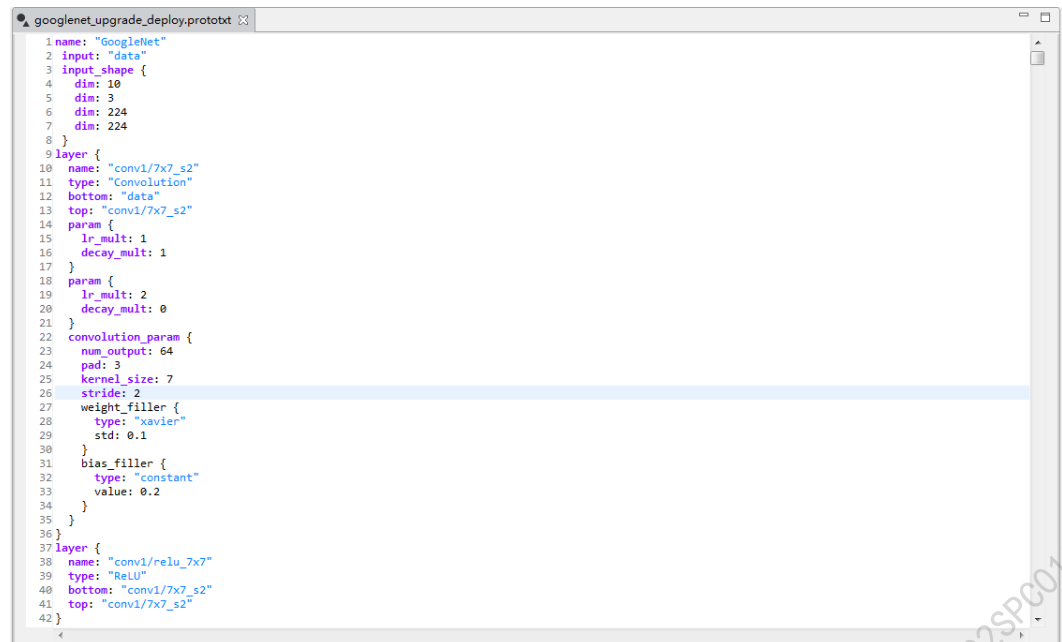
注意

- input 层后的第一个层，不能标记为_cpu
- Type 为 input 的层不能打_cpu, hp_report 标记
- Custom 对应 shape 有 2 个值，3 个值和 4 个值的情况，不允许非 0 值中间出现 0 值
- Custom 层不能打标记
- Proposal 层不能打_hp 标记

5.3.6 原始 prototxt 编辑功能

在 Mapper Configuration Editor 视图中，点击 Edit 按钮，显示如图 5-71，可以对原始的 prototxt 进行编辑；

图5-71 Prototxt 编辑界面



5.3.7 nnie_mapper 配置项自动生成功能

用户在编辑完原始的 prototxt 后，切换回 Mapper Configuration Editor 页面，因 Browse prototxt 操作有标记 custom 和 proposal 的功能，所以如果在 Edit 中 prototxt 文件有内容修改则需要重新点击 Browse 导入 prototxt，工具会根据自动标记后的 Prototxt 生成动态的 nnie_mapper 参数界面，且有些配置按默认值配置，如图 5-72 所示。

自动生成的配置项包含：

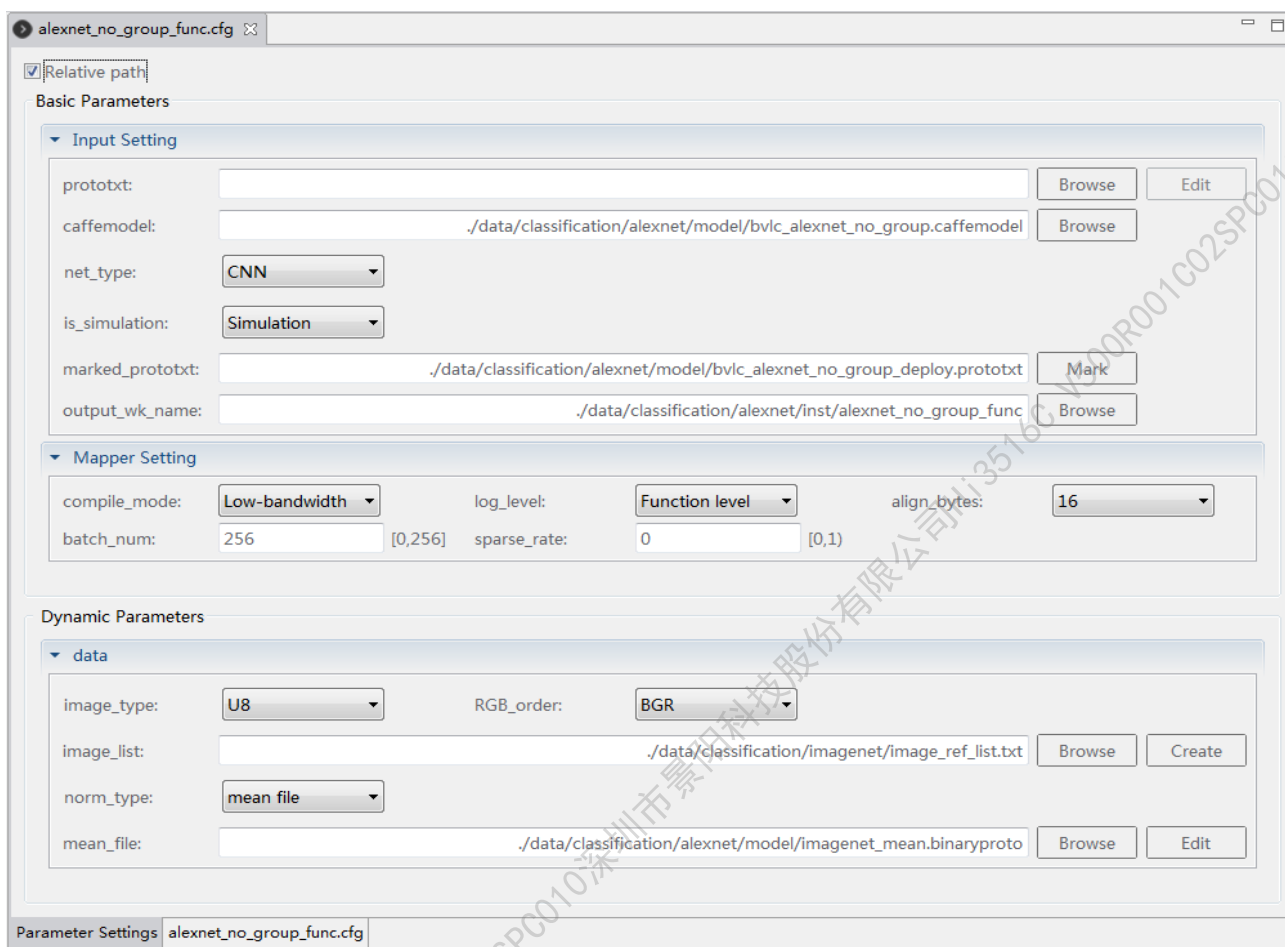
- 1) 静态配置项，即任意 prototxt 都会有的配置；
- 2) 动态配置项，主要有 3 类：
 - 根据网络的输入节点数目，生成 image_type\image_list\norm_type\mean_file 配置项；
 - 有“Proposal”时，会生成 Proposal 层连接的层所需要的 roi_coordinate_file（Bbox）配置，“Proposal”层连接了多少个层就会有多个 roi_coordinate_file 配置；该配置项配置要求见 3.5.2 “配置文件说明”的 roi_coordinate_file 说明；
 - 有“Custom”层时，会生成“Custom”层连接的层所需要的 feature_map_file 配置，“Custom”层连接了多少个层就会有多个 feature_map_file 配置；该配置项配置要求见 3.5.2 “配置文件说明”的【注意】说明。



注意

- roi_coordinate_file 配置的文件，要求用户必须使用 nnie_mapper 中配置的参考图像，在 caffe 中运行该网络直到对应的 Proposal 层，并按 3.5.2 “配置文件说明”的 roi_coordinate_file 格式要求输出。
- feature_map_file 对应 cfg 文件的四个配置项 image_type\image_list\norm_type\mean_file，其中默认 image_type 为 0、norm_type 为 0、mean_file 为 ignore，feature_map_file 配置值对应 image_list 配置值。

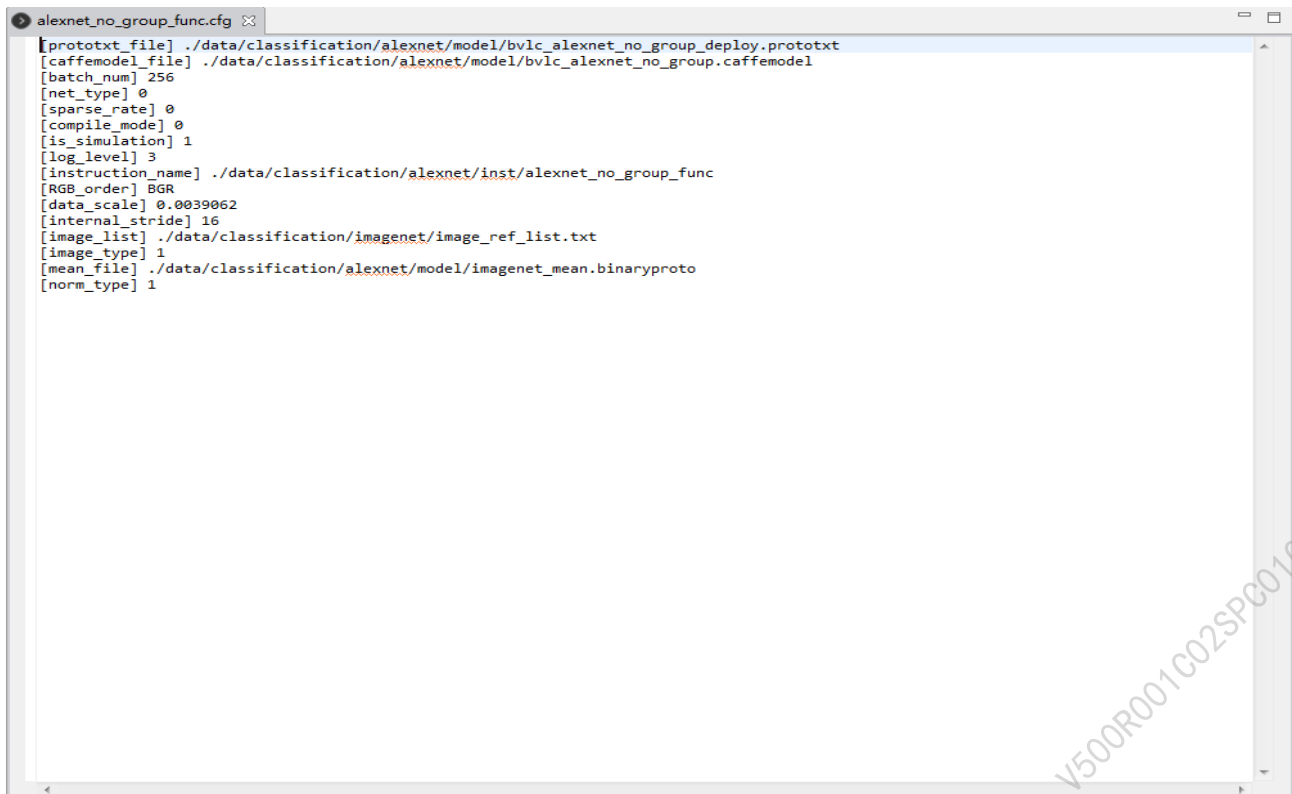
图5-72 Mapper Configuration Editor 参数界面



在 Mapper Configuration Editor 参数界面中设置好后，点击右下角的 alexnet_mapper_func.cfg 按钮，切换到 Mapper Text Editor 视图，可以查看 cfg 文件的文本配置变化，如图 5-73 所示。



图5-73 *.cfg Text Editor 界面

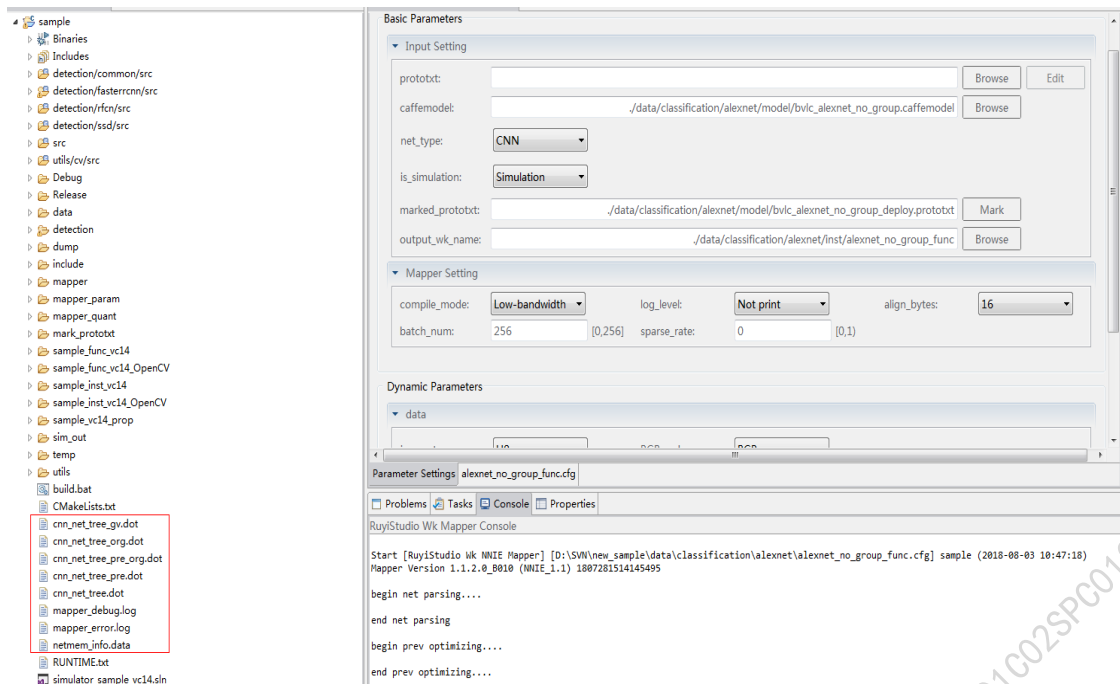


5.3.8 生成 NNIE wk 文件

配置好 cfg 之后，点击左上角工具栏中的 **Make WK** 按钮，工具会将当前的 xxx.cfg 传给 nnie_mapper，转化过程会在控制台显示，转化成功或失败后会在 Project Explorer 中的当前工程的目录下生成多个 dot 文件及编译过程中的 debug 和 error 对应的 log，如图 5-74 所示。



图5-74 Make WK 过程的工具界面



5.3.9 Make wk 时产生的文件描述

当 log_level 选择为 Function Level 时，执行 make wk 时会在工程文件夹下生成 mapper_quant 和 mapper_param 两个文件，在 mapper_quant 中生成的数据是：

- \$(layer_name)_bias.hex 是量化后的 bias，十六进制，24.0 格式。
- \$(layer_name)_weight.hex 是量化后的 weight，十六进制，1.3.4 或 1.5.7 格式。
- \$(layer_name)_weight-anti.float 是反量化后的 weight，浮点。
- \$(layer_name)_input\$(layer_id)_\$C_\$H_\$W_anti_quant.linear.float 是反量化后的 input，浮点。
- \$(layer_name)_input\$(layer_id)_\$C_\$H_\$W_quant.nonlinear.hex 是量化后的 input，十六进制，1.3.4 或 1.5.7 格式。
- \$(layer_name)_output\$(layer_id)_\$C_\$H_\$W_quant.linear.hex 是使用反量化的 input，做 forward 后的 output，十六进制，20.12 格式。
- mapper_param 中的数据是网络各层参数，来源于传入的 caffemodel。

5.4 仿真 NNIE 功能

5.4.1 配置文件可视化编辑

用户双击位于工程 sim_out 目录下的 nnie_sim.ini 文件，如果没有 nnie_sim.ini 文件，用户可以创建一个 nnie_sim.ini 文件，nnie_sim.ini 的文件编辑内容为：

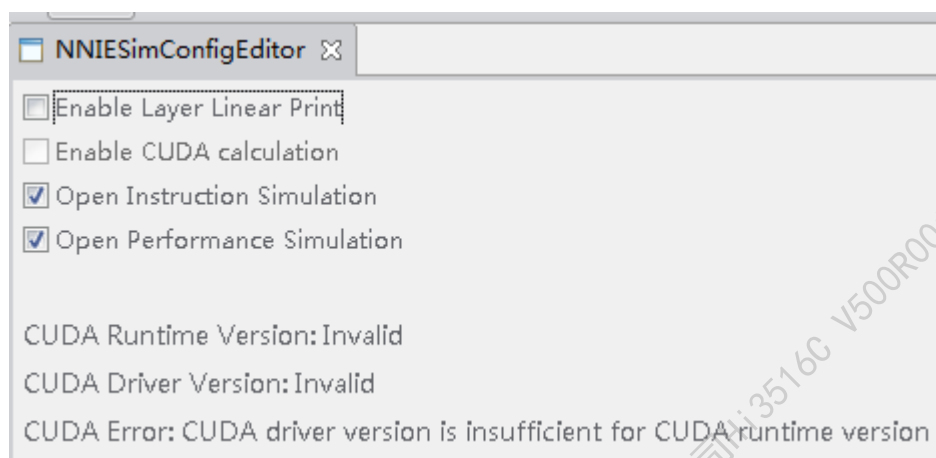
```
#===== CONFIG =====
```



```
[LAYER_LINEAR_PRINT_EN]
0
[CUDA_CALC_EN]
0
[OPEN_INSTSIM]
1
[OPEN_PERFSIM]
0
```

其中 0 表示不可用，1 表示可用，双击可以打开仿真配置文件可视化编辑器，如图 5-75 所示。

图5-75 仿真配置文件可视化编辑



可以设置以下开关：

- **Enable Layer Linear Print:** 激活此选项后，可以在进行指令仿真时输出 caffe 各层的中间线性结果。生成的文件在 func_layer_output_linear 目录下。
- **Enable CUDA calculation:** 激活此选项后，可以在仿真时使用 CUDA 加速特性。需要安装 CUDA Toolkit（版本 8.0 或以上）才可以使用。四个选项的下方会显示 CUDA 运行版本与驱动版本，显示为 Invalid 时，表示当前的 CUDA 安装无效。
- **Open Instruction Simulation:** 激活此选项以支持指令仿真运行。
- **Open Performance Simulation:** 激活此选项以支持性能仿真运行，如果需要激活此选项，必须先要激活指令仿真运行。

编辑完成后，点击 RuyiStudio 工具栏的保存按钮   可将设置保存到所选的 nnie_sim.ini 文件。

5.4.2 Sample 代码中一键切换功能仿和指令仿

导入 HiSVP_PC_Vx.x.x.x\software\sample 工程，默认配置了功能仿的库和使用功能仿 wk 文件的编译宏，如果需要将 sample 目录从指令仿切换到功能仿，仅需要在工程上点击右键->Switch Emulation Library->Function Lib 如图 5-76，即可将当前的工程中



Debug 和 Release 版本对应依赖的库统一切换到功能仿库，切换过程修改的编译参数与依赖的库会在控制台中打印，如图 5-77 所示。

图5-76 切换功能仿/指令仿的操作

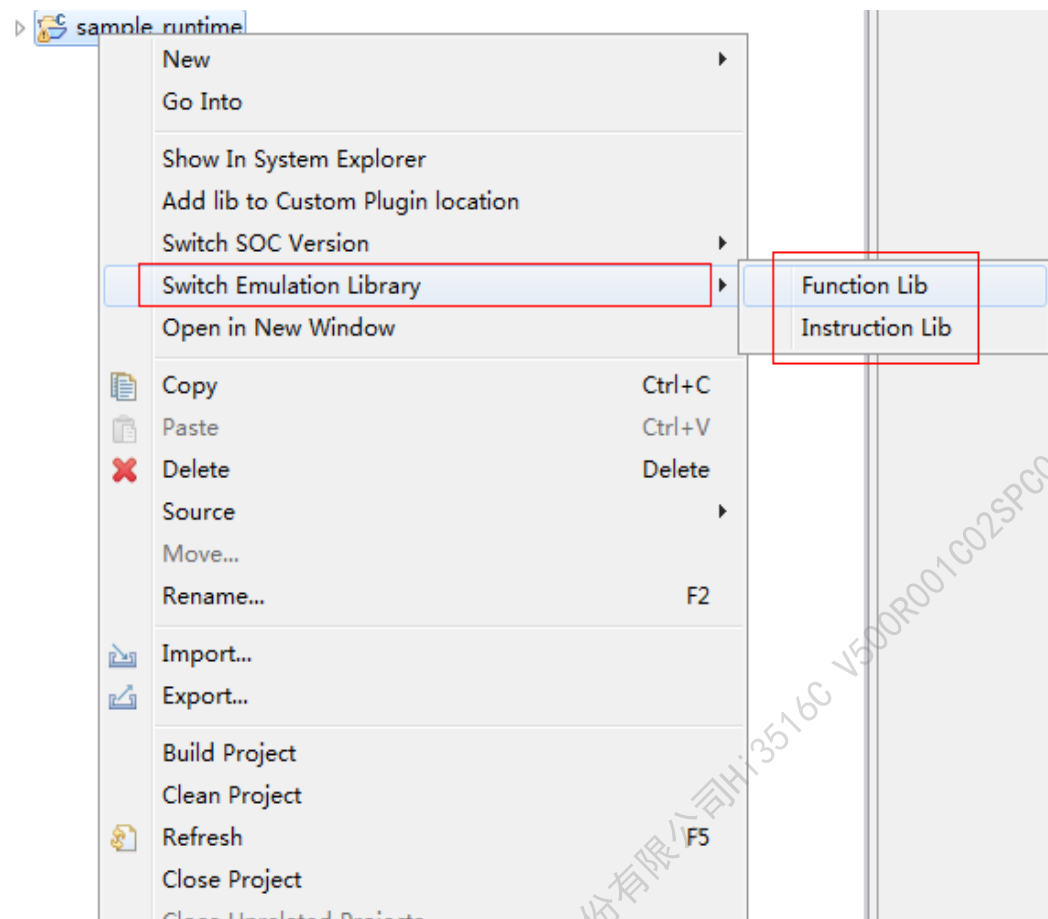
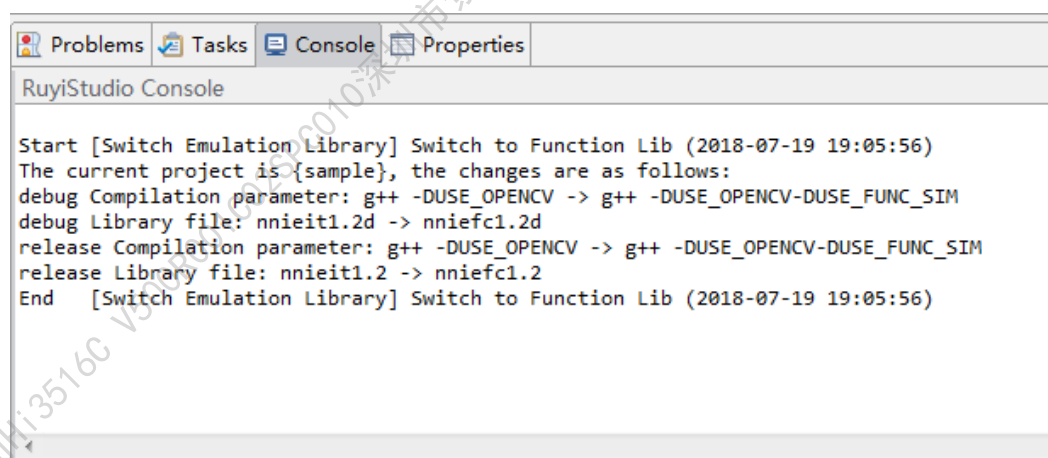


图5-77 控制台打印切换仿真库对应的修改信息



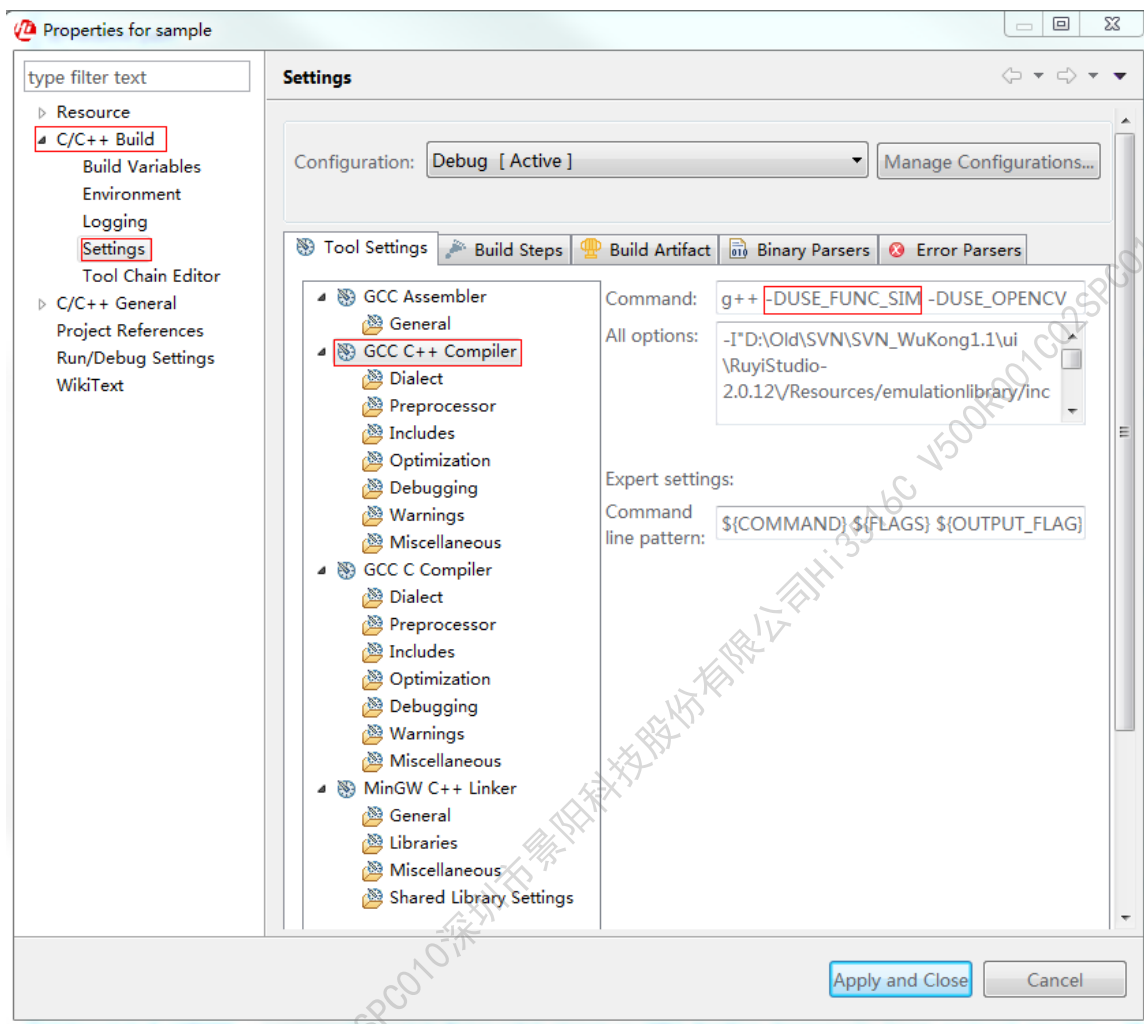


5.4.3 Sample 代码中手动切换功能仿和指令仿切换

导入 HiSVP_PC_Vx.x.x.x\software\sample 工程，默认配置了功能仿的库和使用功能仿 wk 文件的编译宏，如果需要将 sample 目录从功能仿切换到指令仿，需要进行如下步骤：

- 步骤 1. 在项目上点击右键->Properties->C/C++ Build->Settings->GCC C++ Compiler 中 Command:中去掉-DUSE_FUNC_SIM 编译宏，以 Debug 为例，如图 5-78 所示。

图5-78 修改 g++编译宏



编译宏对应的位置在各个不同网络的 sample 代码中，以 SvpSampleClassification.cpp 为例，如图 5-79 所示。



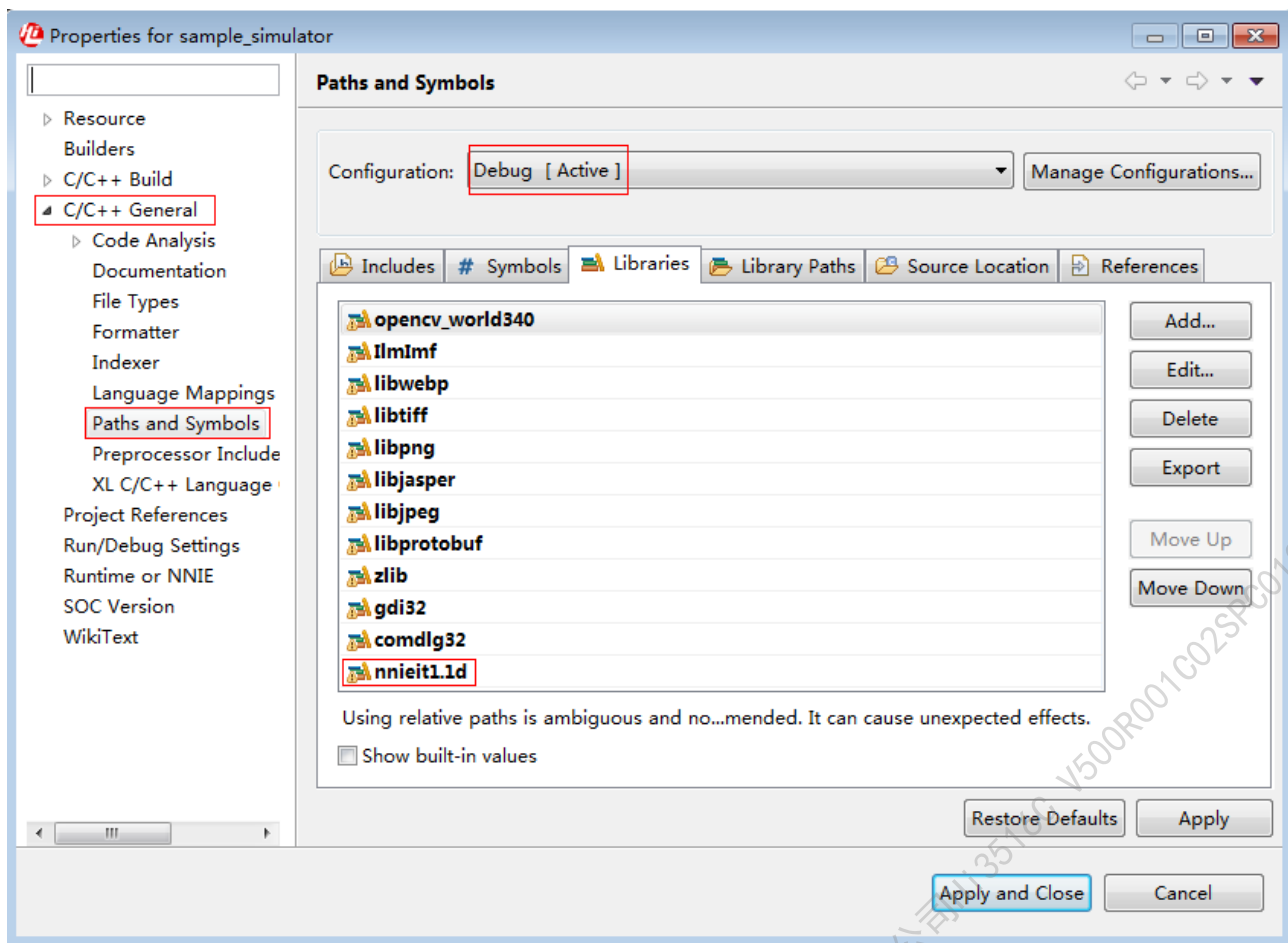
图5-79 编译宏对应的代码位置

```
15
16 #if USE_FUNC_SIM
17 const HI_CHAR *g_paszModelName_c[] = {
18     "../data/classification/lenet/inst/inst_lenet_func_batch.wk",
19     "../data/classification/alexnet/inst/alexnet_no_group_func.wk",
20     "../data/classification/vgg16/inst/vgg16_upgrade_deploy_func.wk",
21     "../data/classification/googlenet/inst/googlenet_upgrade_deploy_func.wk",
22     "../data/classification/resnet50/inst/ResNet50_model_func.wk",
23     "../data/classification/squeezenet/inst/squeezenet_v1.0_func.wk"
24 };
25 #else
26 const HI_CHAR *g_paszModelName_c[] = {
27     "../data/classification/lenet/inst/inst_lenet_inst_batch.wk",
28     "../data/classification/alexnet/inst/alexnet_no_group_inst.wk",
29     "../data/classification/vgg16/inst/vgg16_upgrade_deploy_inst.wk",
30     "../data/classification/googlenet/inst/googlenet_upgrade_deploy_inst.wk",
31     "../data/classification/resnet50/inst/ResNet50_model_inst.wk",
32     "../data/classification/squeezenet/inst/squeezenet_v1.0_inst.wk"
33 };
34 #endif
```

步骤 2. 在项目上点击右键->Properties->C/C++ General->Paths and Symbols->Libraries, 将功能仿的.a 文件删除, 新增指令仿的.a, 如图 5-80 所示。



图5-80 修改工程依赖的 Libraries



----结束

工程依赖的功能仿和指令仿存放在 RuyiStudio 工具目录下的 Resources\emulationlibrary\lib 目录中，Release 版本名称为功能仿 libnniefc1.1.a，指令仿 libnnieit1.1.a，Debug 版本名称为功能仿 libnniefc1.1d.a，指令仿 libnnieit1.1d.a。需要注意在步骤 2 中添加.a 文件时需要去掉原文件名称中的 lib 和后缀.a。

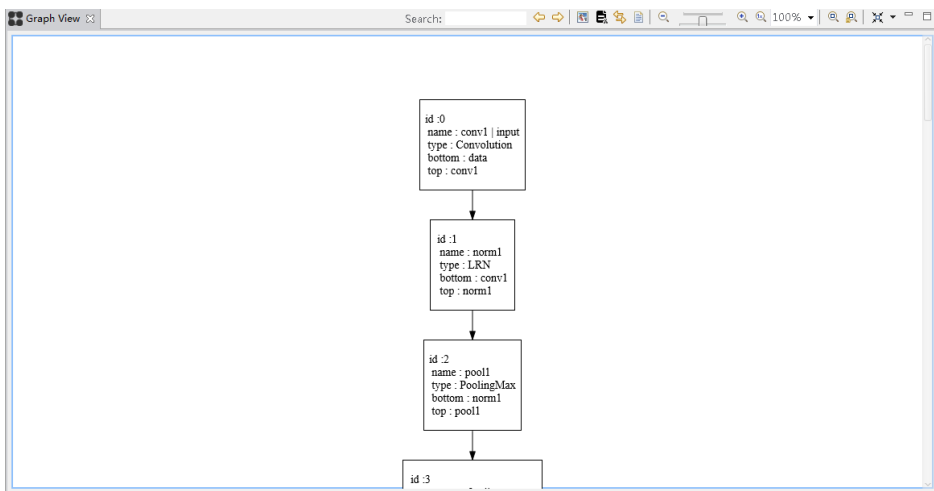
5.5 数据分析工具

5.5.1 网络拓扑图显示功能



在工具栏中点击 Graph View 按钮可以打开网络拓扑图视图，该视图可以显示由 DOT 视图语言描述的网络拓扑图，如图 5-81 所示：

图5-81 网络拓扑图视图



从 GraphView 自带工具栏中可以启用网络拓扑图的各项功能：



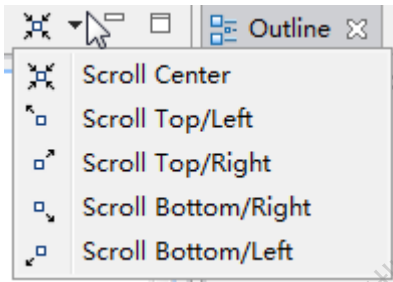
联动按钮：激活联动按钮后，从 Project Explorer 工程视图选择 dot 文件，会将该 dot 文件显示于网络拓扑图视图中；



打开按钮：可选择本地硬盘上的 dot 文件显示；



缩放工具：可以将当前的网络拓扑图缩放显示。



自动滚动工具：可以自动滚动网络拓扑图到指定的方位。



截取按钮：可以截取网络拓扑图中的一层或多层并保存数据到指定文件夹。



保存截图：可以将当前的网络拓扑图保存为 png 文件。



搜索工具：可以按关键字搜索指定的层。在 Search 文本框中输入文字，按回车键即可搜索。网络拓扑图中标签文字包含关键字的层将被高亮显示，并自动定位，在标记 prototxt 后的网络拓扑图上还会在 Layer Info 视图显示层属性。如果有多个层标签匹配，使用切换按钮  可切换层显示。



5.5.2 向量相似性比较功能

提供不同功能配置下的每一层的向量的统计量两两比较的余弦相似度，目的是方便的定位具体是哪个环节引入精度的误差等。

不同配置（建议使用）包括：

- **caffe 推理结果**
使用发布包中的脚本 `CNN_convert_bin_and_print_featuremap.py`，在 `caffe` 环境下运行，会输出各层的中间文件在脚本当前路径的 `output` 目录下，例如：
`conv1_output0_96_55_55_caffe.linear.float` 的文件；
- **nnie_mapper 量化及反量化 bypass 后推理结果**
在 `cfg` 文件中配置 `log_level=3(Function level)`后点击 `make wk` 按钮，生产 `wk` 文件，在 `$(project_root)/mapper_quant` 中也会输出各层的中间文件，例如：
`conv1_output0_96_55_55_quant.linear.hex`
- **指令仿真推理结果**
在 `nnie_sim.ini` 配置选项 `[LAYER_LINEAR_PRINT_EN]`，在 `func_layer_output_linear` 文件夹下也会输出各层的中间文件，例如：
`seg0_layer0_output0_inst.linear.hex`
- **上板推理结果**
上板结果只能打印用户上报输出的层，按
`seg$Id_layer$Id_output$Id_board.linear.hex` 的文件格式打印（打印的 `sample` 已提供）
- **功能仿真推理结果**
同指令仿的配置，输出的中间文件例如：`seg0_layer0_output0_func.linear.hex`

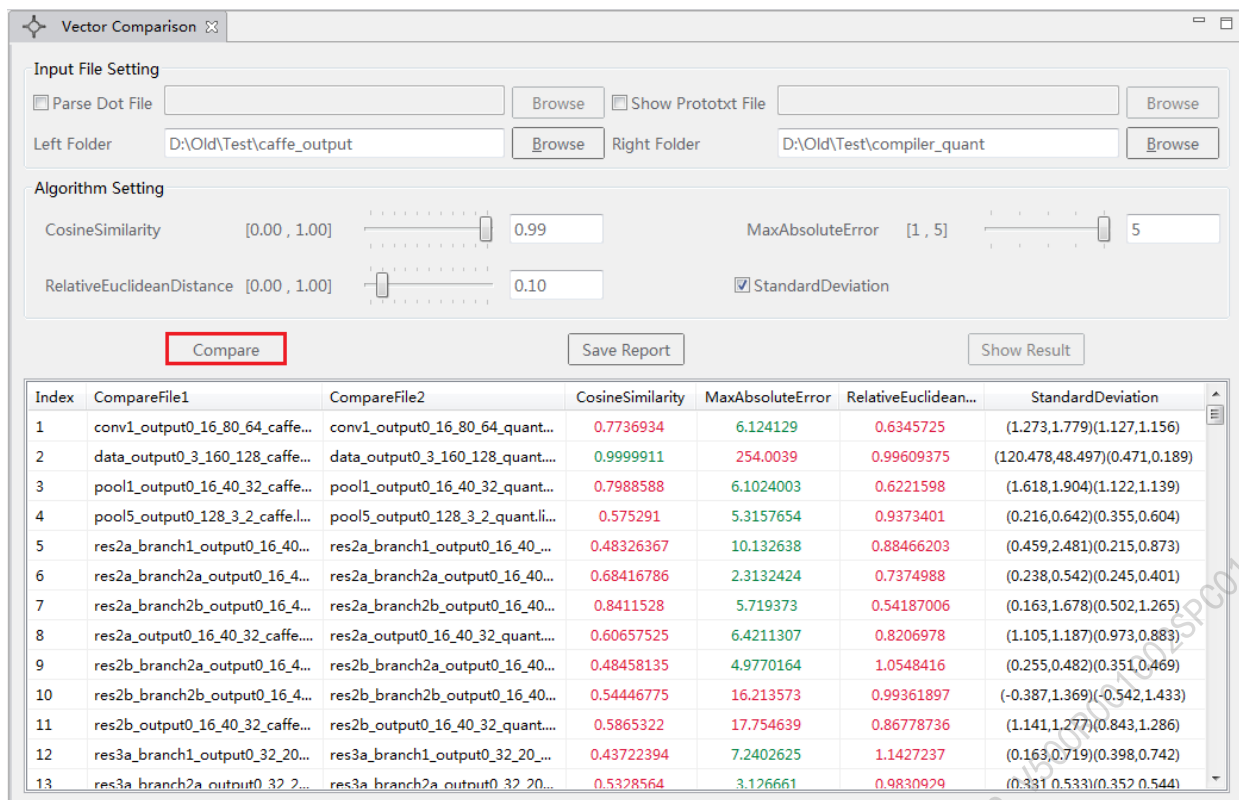
5.5.2.1 展示比较结果到列表功能

操作方式如下：

- 步骤 1. 点击工具栏向量比较按钮 ，弹出向量比较的视图，在 `compare folders` 中选择你需要比较的不同配置的中间输出结果所在的文件夹，如果其中一个是 `caffe` 或者 `nnie_mapper` 的推理结果的数据，而另一个是仿真数据，还需要额外的选择 `nnie_mapper` 生成的 `dot` 文件来进行层的匹配对应，如图 5-82 所示。



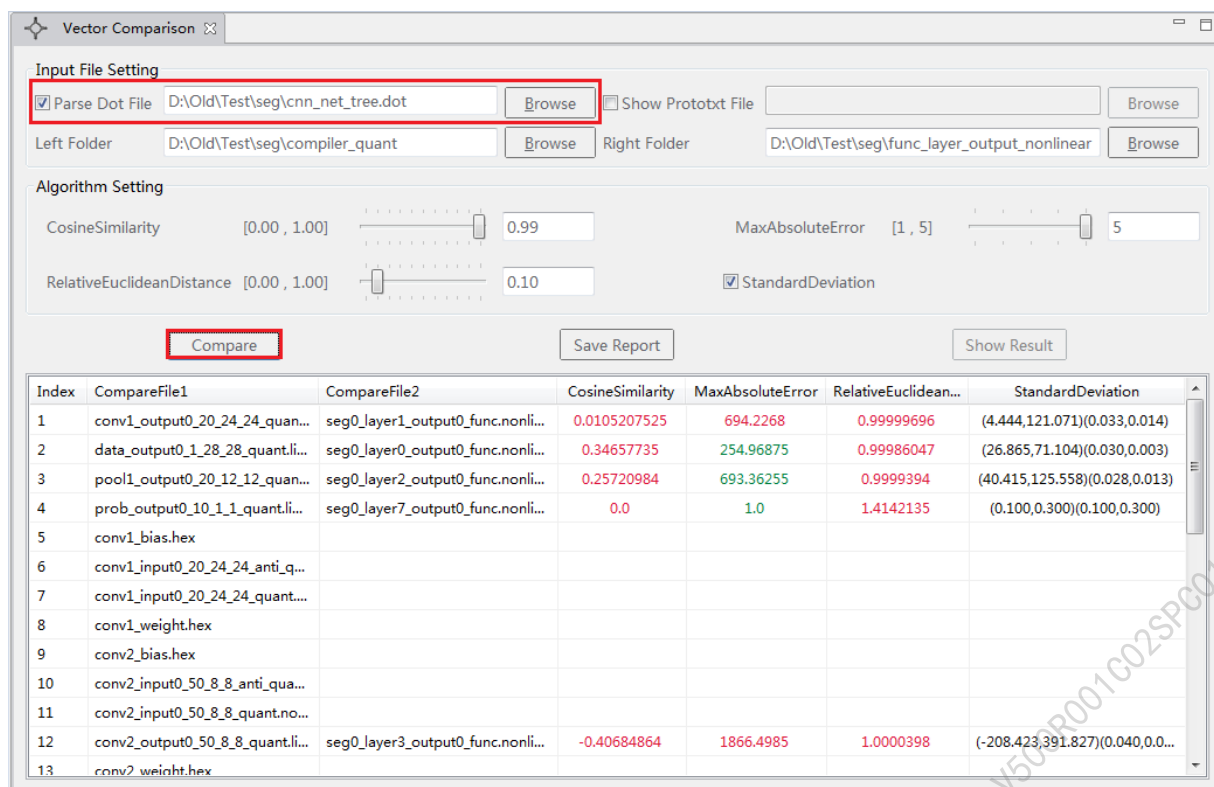
图5-82 向量对比视图



如果当前选择的比较文件中名称为 seg 开头且名称中没有具体的 layer 名称时，则需要先选择 Parse Dot File 再导入比较的文件后点击 Compare 进行比较，这样工具就会根据选择的 dot 文件解析 seg 文件中实际的 layerName，从而匹配 layerName 相同的文件来进行比较，如图 5-83 所示。



图5-83 选择 dot 文件的向量对比视图



步骤 2. 空白区域为未匹配上的文件，用户可以通过选中任意 CompareFile 列中的文件框来可以手动选择文件进来进行比较，点击 Compare 按钮，比较结果会显示到视图的列表中。

算法选择区的主要功能如下：

- CosineSimilarity 滑块选择的阈值用于表示将小于阈值的值在比较后标记为红色。
- RelativeEuclideanDistance 滑块选择的阈值用于表示将大于阈值的值在比较后标记为红色。
- MaxAbsoluteError 滑块选择的阈值用于表示显示列表中最大的绝对误差的个数。
- StandardDeviation 勾选框意为，勾选时比较后显示均值与标准差的结果。

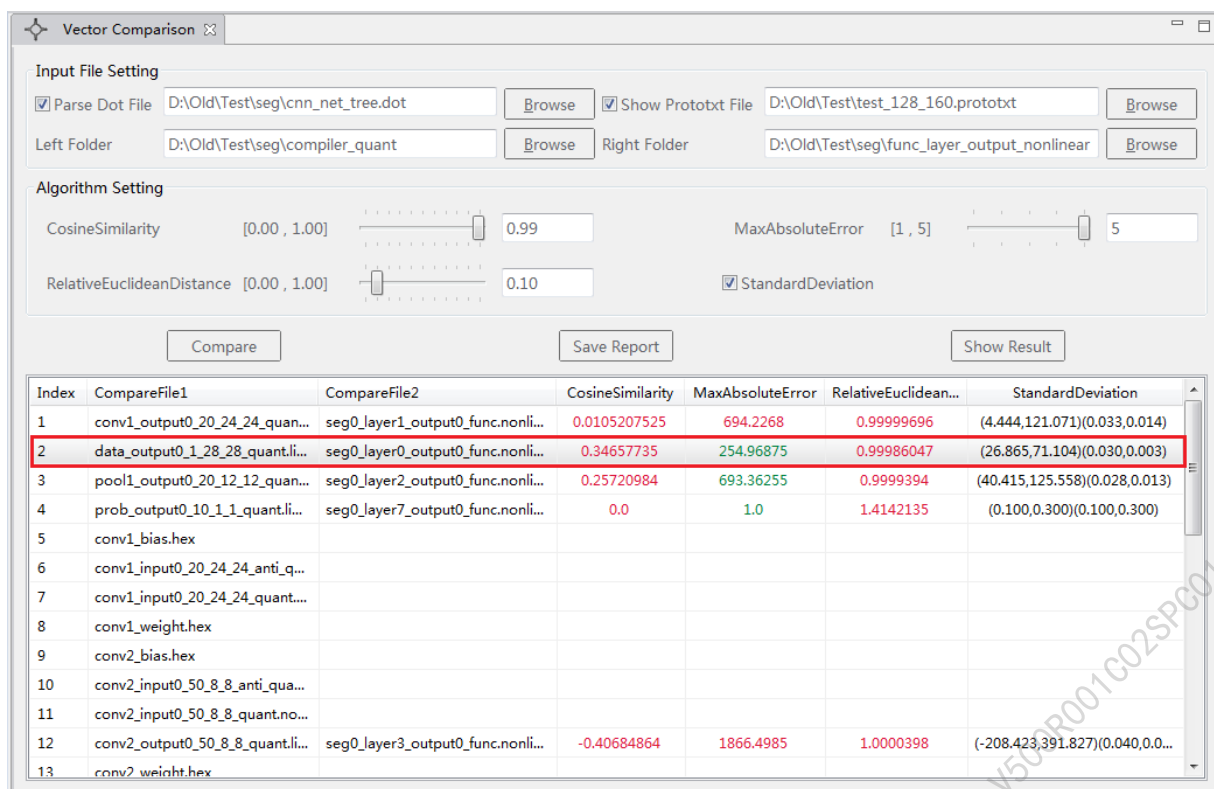
----结束

5.5.2.2 展示详细比较结果功能

在 Compare 完后，双击选择需要查看的行，会弹出详细对比界面，如图 5-84 所示。



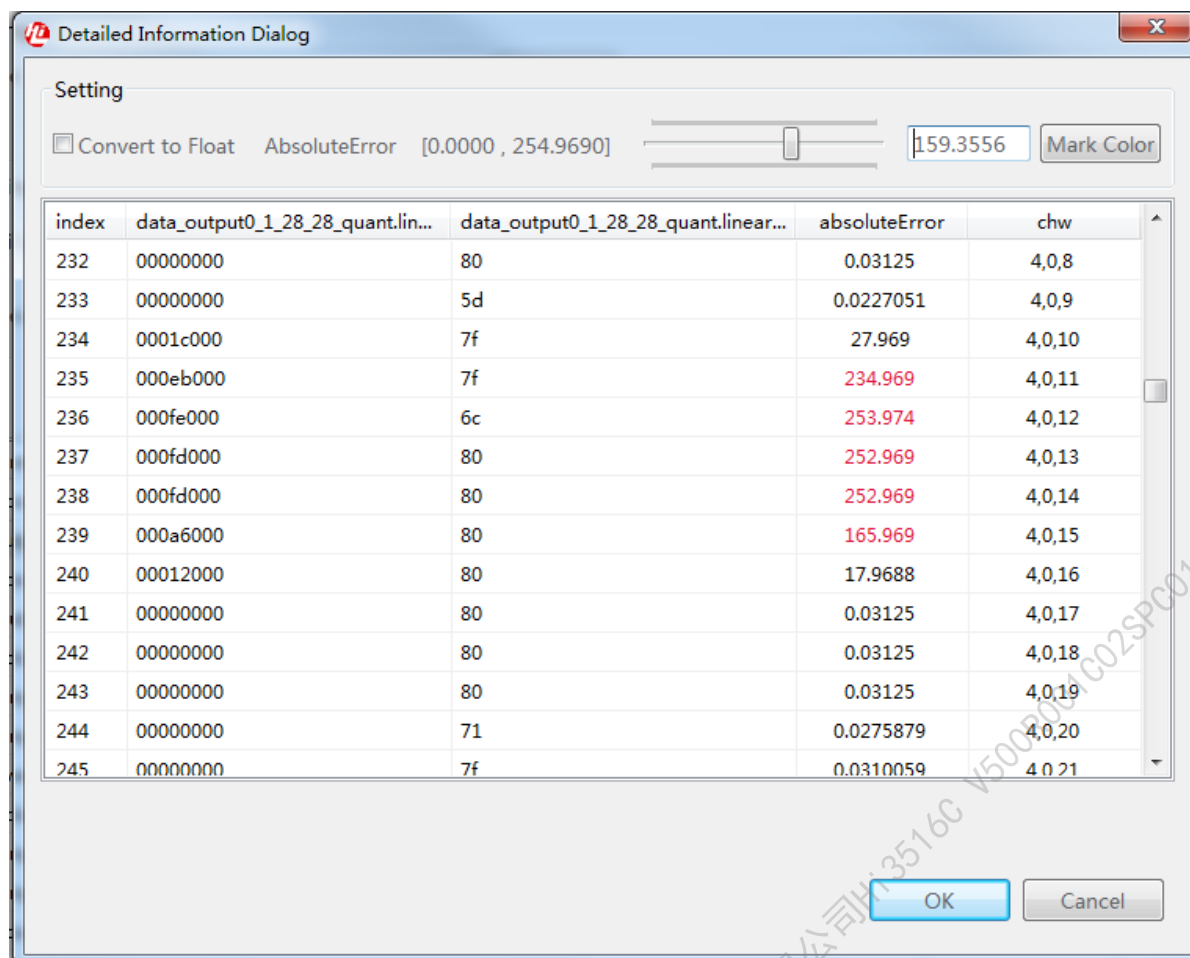
图5-84 双击需要查看的行



如图 5-85，得到详细比较结果，其中 Convert to Float 按钮功能为将列表中每行数据的 hex 值转换为 float 类型显示，方便用户查看。另外，绝对误差对应的滑块和文本框控件用于选择一个阈值，大于此阈值的 absoluteError 值再点击 Mark Color 会被标记为红色。



图5-85 详细比较结果



5.5.2.3 展示比较结果到 GraphView 功能

当用户选择了 Show Prototxt File 时，点击 Compare 比较成功后，再点击 Show Result 按钮，如图 5-86 所示，结果会在工具中的 console 和 Graph View 展示结果，预期结果 ≥ 0.99 不标色； $(0.95 \sim 0.99)$ 给出橙色提示； ≤ 0.95 给出红色预警，如图 5-87 所示。



图5-86 点击 Show Result 按钮

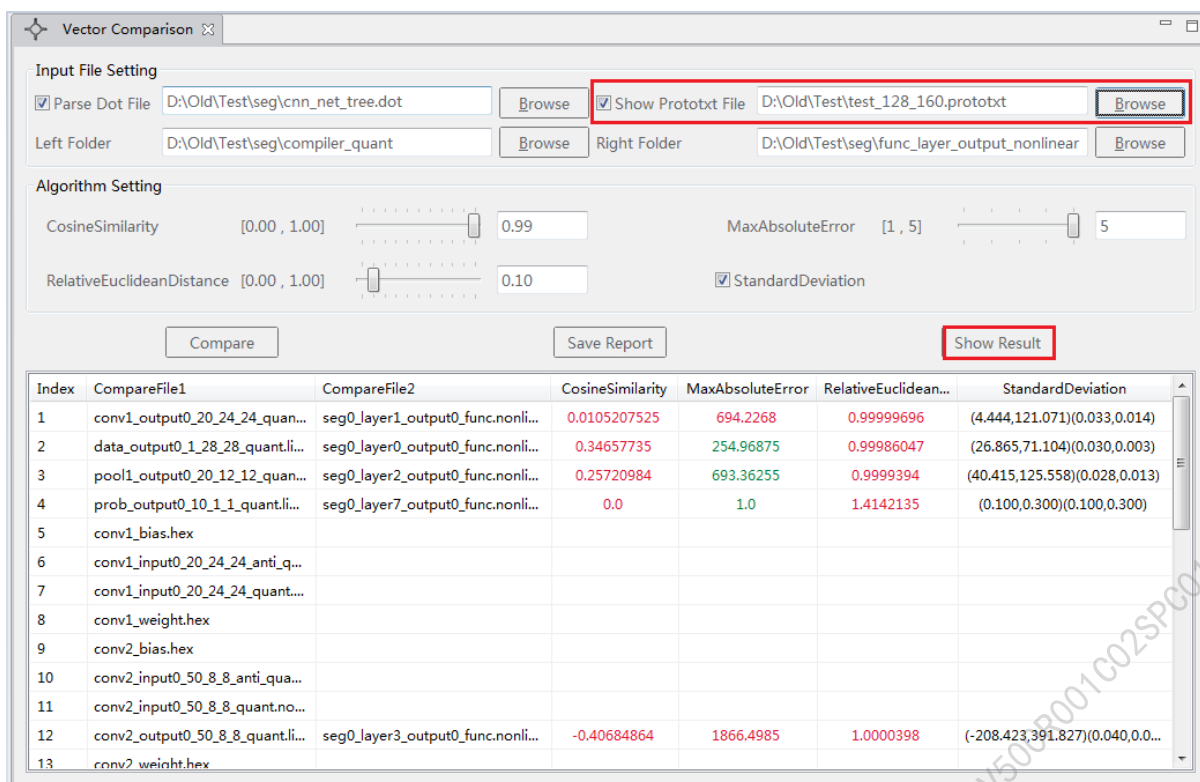
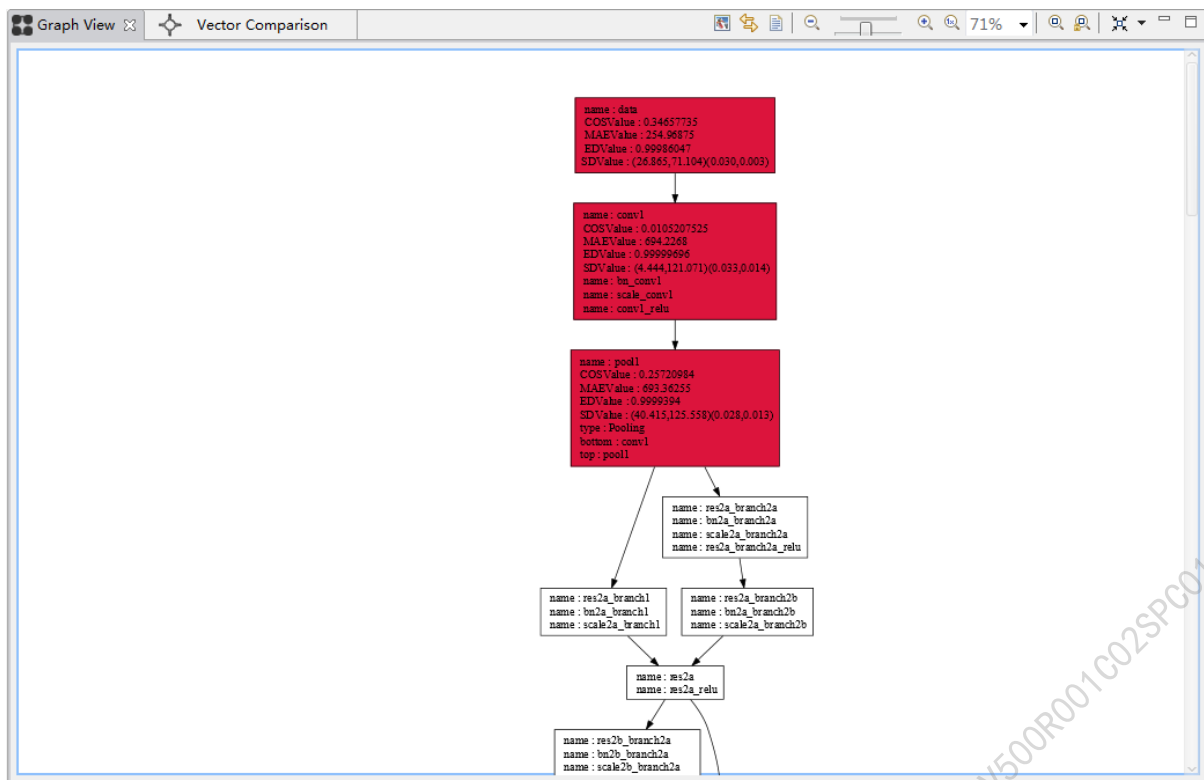




图5-87 Graph View 显示对比结果

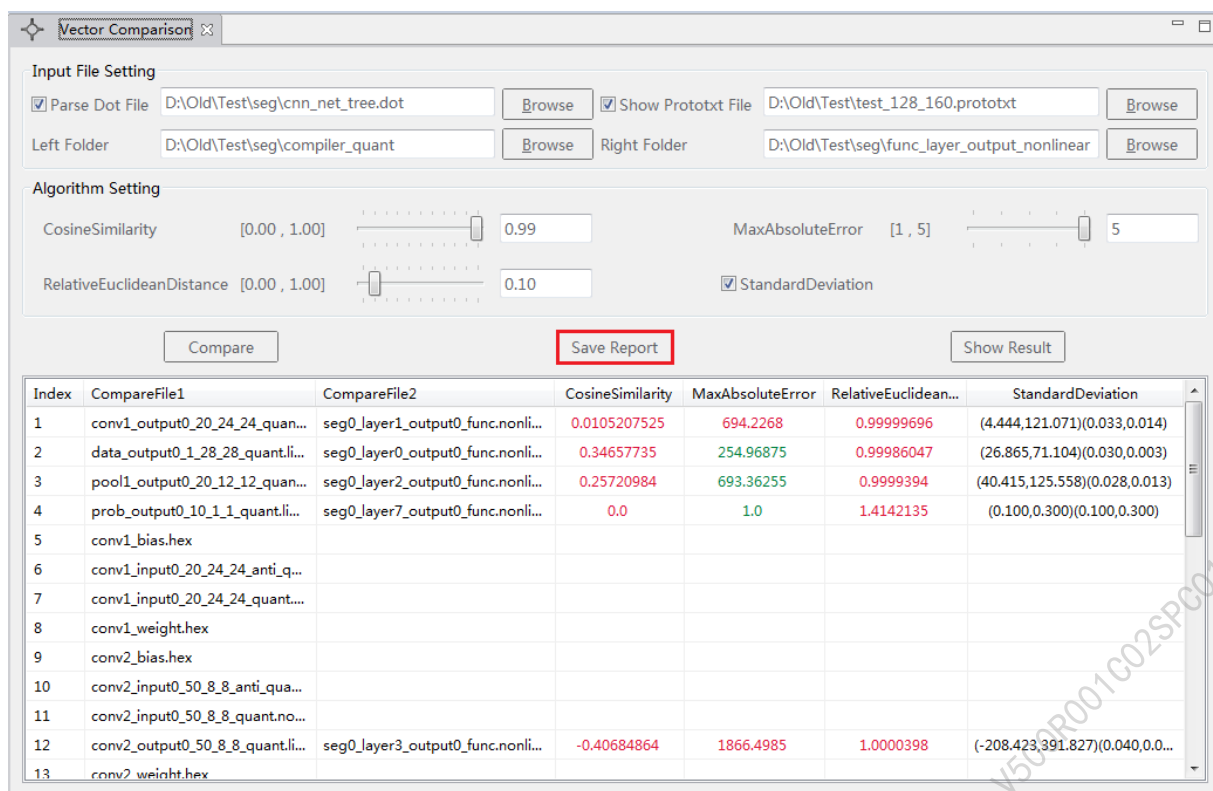


5.5.2.4 保存比较结果生成 csv 报告功能

点击 Save Report 按钮如图 5-88，并选择需要保存报告的路径，点击确定即可将当前列表中的数据保存到 csv 文件中以备查看。



图5-88 保存 csv 格式的报告



5.5.3 调试定位信息获取功能

背景

在用户使用了相似度比较工具确认精度问题不是 nnie_mapper 量化导致的，且定位到某个或者某些非用户自定义层出现精度下降问题（一种情况是断崖式下降，一种情况是相似度不是断崖式的下降，精度逐渐下降或者忽高忽低但是差异不大但整体偏差精度较大），需要将怀疑有问题的层的网络数据发送给海思定位。

此功能提供两种导出场景：

- 从首层开始到问题层的数据，包括：
 - 从 data 层开始到出问题层的 test.prototxt；
 - 从 data 层开始到出问题层的 weight\bias 数据，或者是截断的 test.caffemodel（前者用工具来转为 test.caffemodel）；
 - 给 NNIE 的首层数据（这里是 data 层的）input.bin 和 input.txt (input.bin 是二进制，input.txt 每行表示一个 8bit 的数据，共有 w*h*c 行，input 就是 test.jpg 中的裸数据，是为了规避 caffe 和 nnie 解析 test.jpg 时的差异)；
 - 跟网络预处理方式相关的文件 test.binaryproto 或者 mean.txt 等；
 - 提供一个 Readme 说明相关数据情况，包括数据格式、当前层的问题情况说明等有用的信息。
- 提供中间层数据，包括：



- 相关层 test.prototxt，示例中为 norm1->conv2->pool2 层的 prototxt；
- 前一层输出的 shape 信息，示例中为 relu1 层输出 shape 信息(c, h, w)；
- 相关层的 weight\bias 参数，或者是截断的 test.caffemodel，前者需要一个工具来生成 caffemodel，示例中 指 norm1->pool1->conv2 的 weight\bias 参数；
- 给 NNIE 的上板输入 board_input.bin 和 board_input.txt，示例中为上板跑 relu1 的输出，borad_input.bin 是二进制打印，board_input.txt 每行表示一个 32bit 的定点数据，共有 w*h*c 行；
- 给 nnie_mapper 量化输入的 featuremap.txt，客户在使用 nnie_mapper 转化网络时，配置 log_level=3；由于 conv1->relu1 是 inplace 写法(一个进一个出，进出的 name 相同)，在 mapper 运行生成的文件中找到 conv1_output0.linear.hex () 作为 featuremap.txt；若是 conv1->relu1 是 non-inplace 写法，则在 mapper 运行生成的文件中找到 relu1_output0.linear.hex 作为 featruemap.txt；
- 提供一个 Readme 说明相关数据情况，包括数据格式、当前层的问题情况说明等有用的信息。



注意

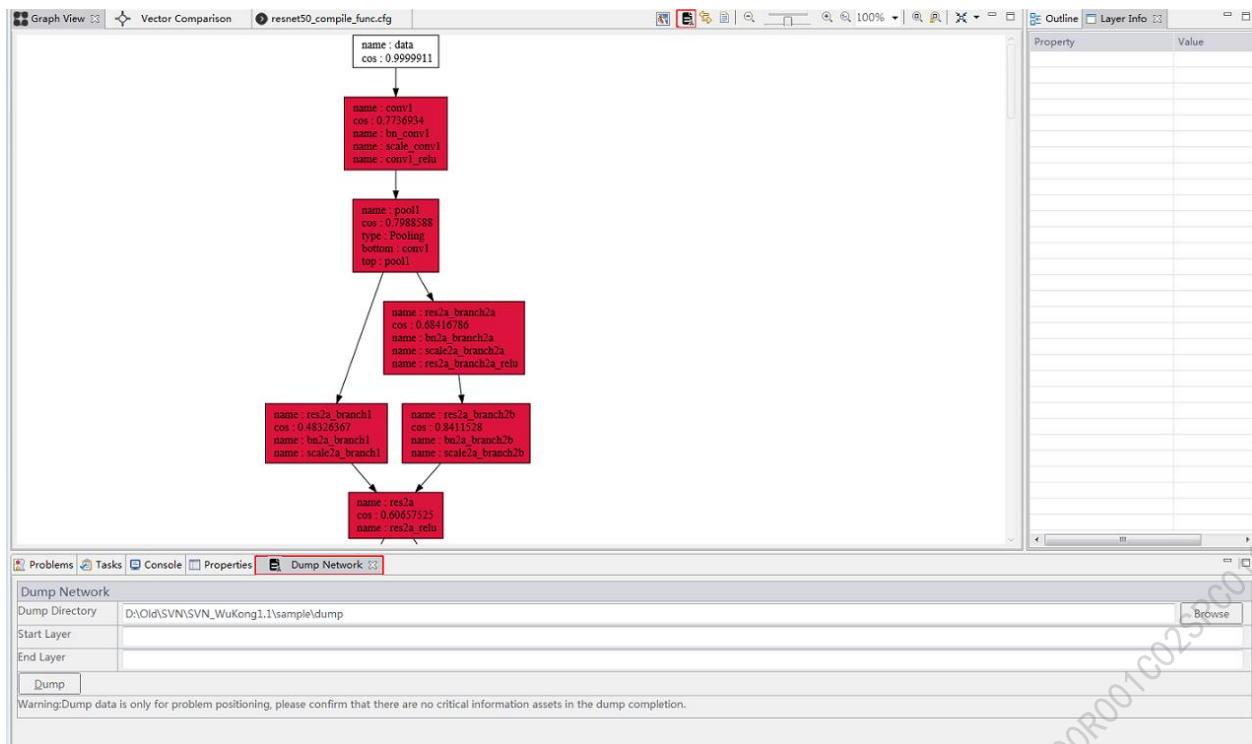
只支持截取一个单输入（起始节点）和单输出（结束节点）的有向无环网络，否则不支持 dump。

操作方式如下：

- 步骤 1. 打开 Dump Network 视图：在向量比较视图中点击“Show Result”按钮后，工具会将算法的值标记在 Graph View 视图对应的 layer 上，点击“Graph View”视图的右上角“Dump Network”按钮，在工具界面视图下侧弹出 Dump Network 视图，如图 5-89 所示。

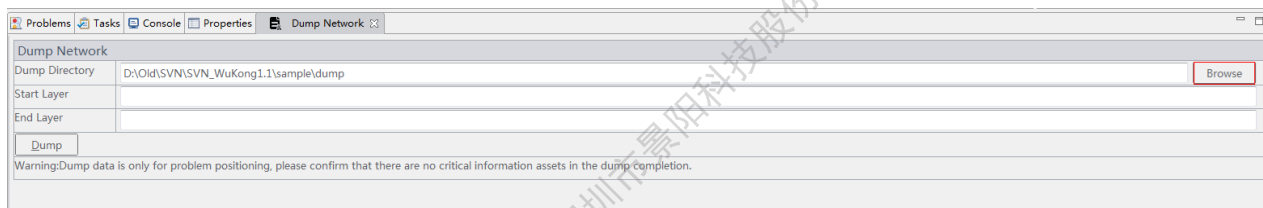


图5-89 Dump Network 视图



步骤 2. 选择保存导出数据的文件夹：当用户默认选择了工程路径时，Dump Directory 会存在默认路径再当前工程目录下的 dump 文件夹，用户可点击“Browse”按钮自行配置导出数据的目录，如图 5-90 所示。

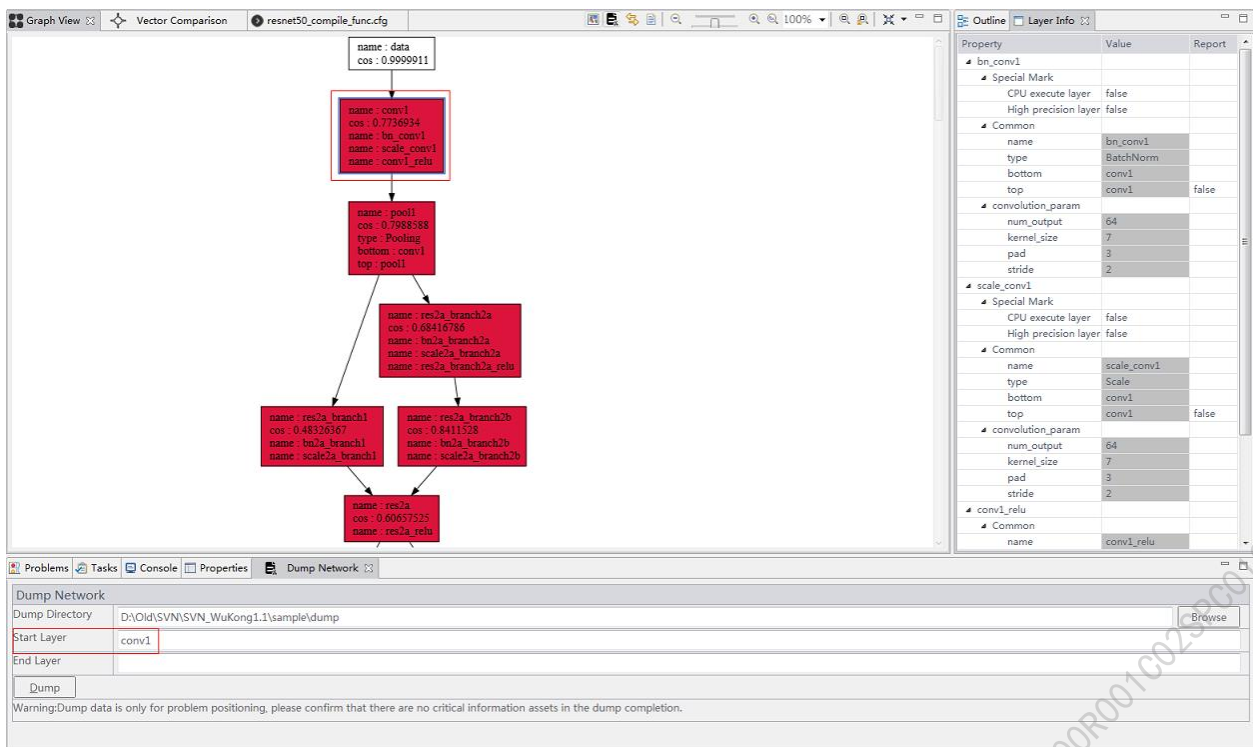
图5-90 选择保存导出数据的文件夹



步骤 3. 选择导出的开始层：选中“Start Layer”文本框，然后在 Graph View 中双击选择需要导出的开始层时，以 conv1 为例，如图 5-91 所示。

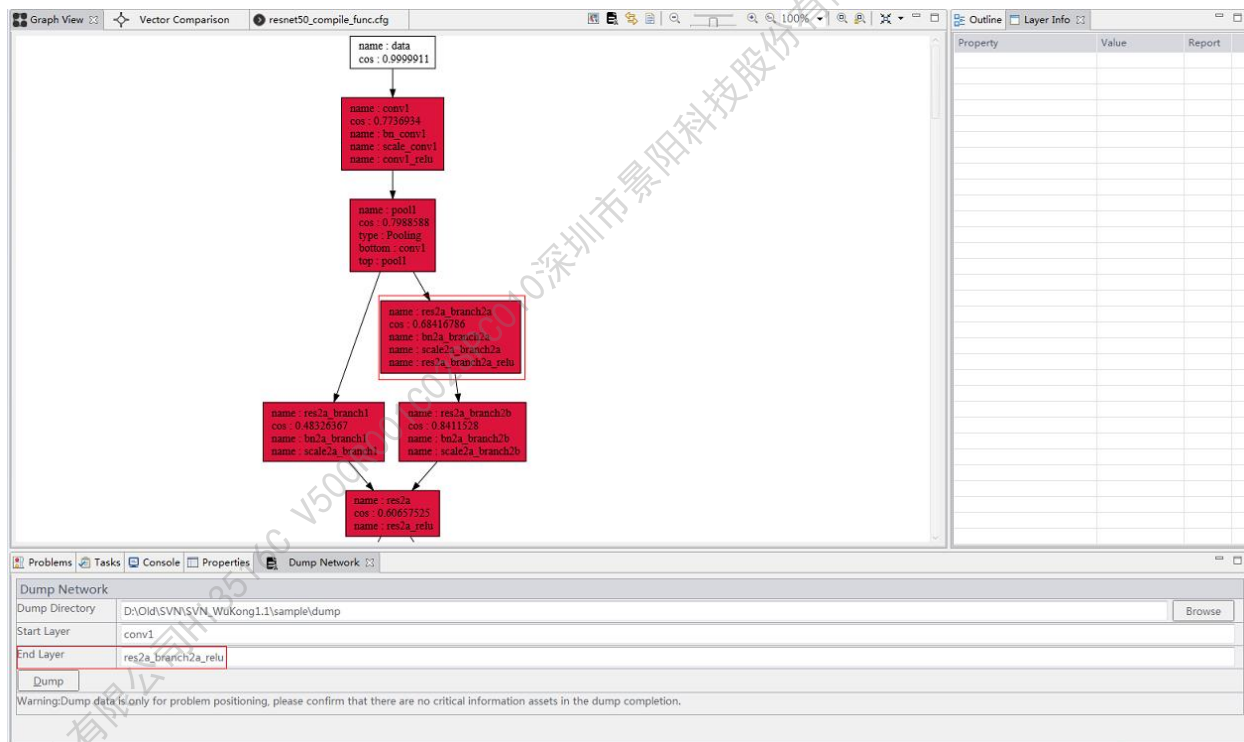


图5-91 选择需要 Dump 的开始层



步骤 4. 选择导出的结束层：选中“End Layer”文本框，然后在 Graph View 中双击选择需要导出的结束层时，以 res2a_branch2a 为例，如图 5-92 所示。

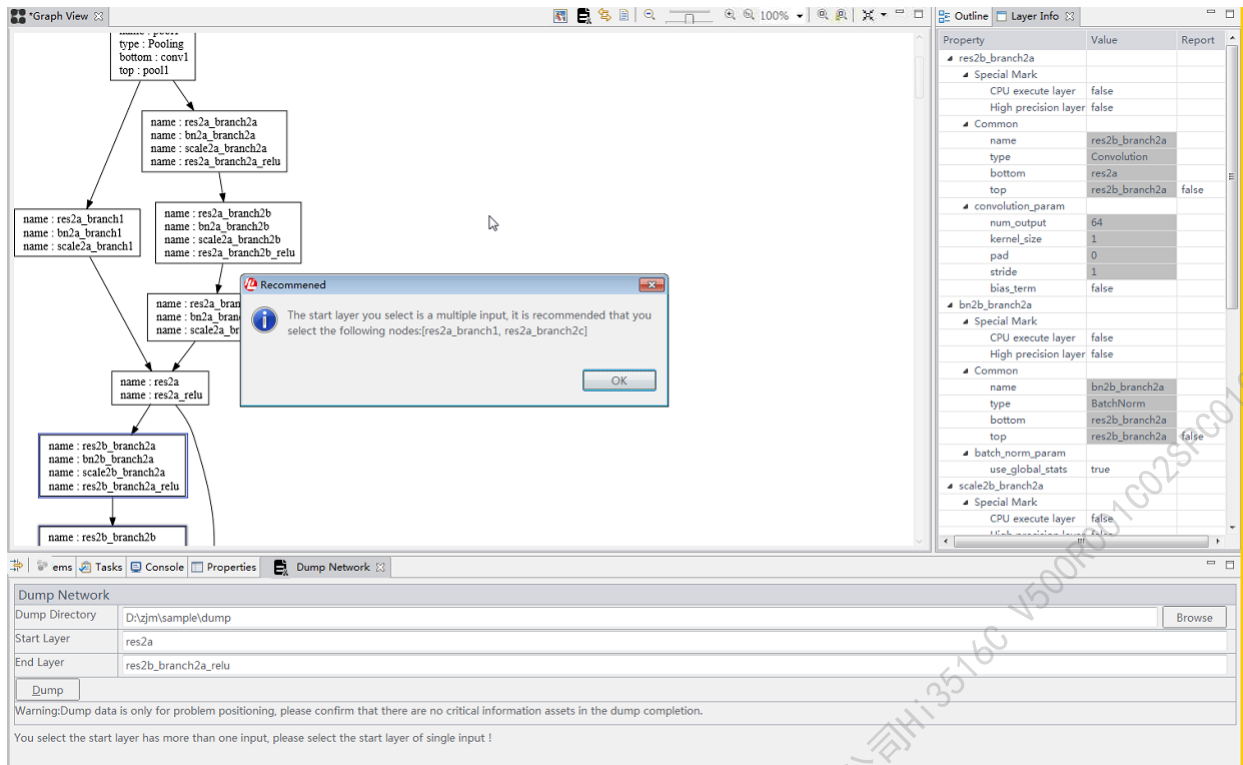
图5-92 选择需要 Dump 的结束层





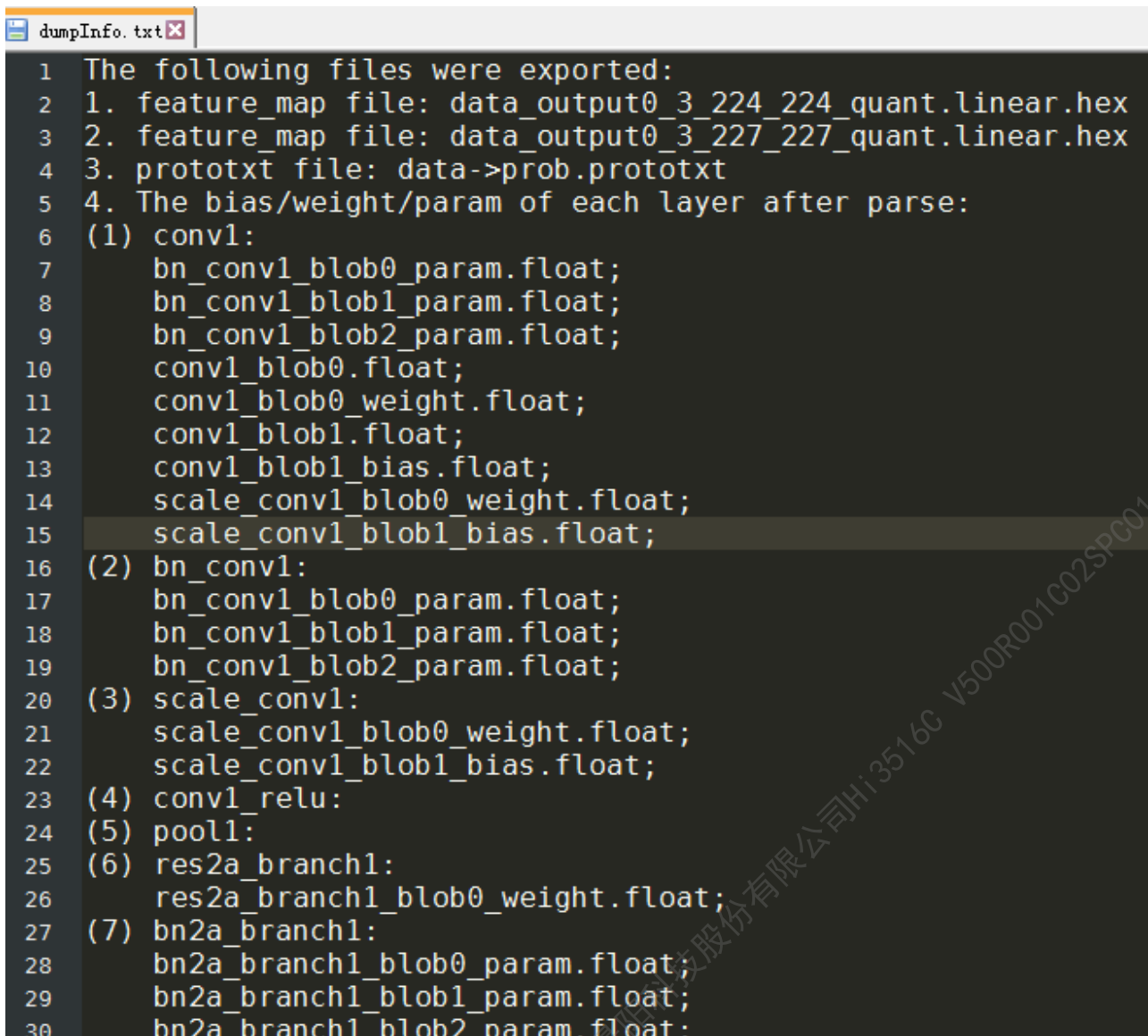
步骤 5. 工具检测当前用户选择的 Dump 层，当选择的起始节点或结束节点不满足单输入单输出的节点时，工具会给出提示，并给出建议的导出层，如图 5-93 所示。

图5-93 Dump 提示推荐层



步骤 6. 查看导出的 dump 信息，以导出的说明信息文件为例，如图 5-94 所示。

图5-94 Dump 导出的信息说明



```
1 The following files were exported:
2 1. feature_map file: data_output0_3_224_224_quant.linear.hex
3 2. feature_map file: data_output0_3_227_227_quant.linear.hex
4 3. prototxt file: data->prob.prototxt
5 4. The bias/weight/param of each layer after parse:
6 (1) conv1:
7     bn_conv1_blob0_param.float;
8     bn_conv1_blob1_param.float;
9     bn_conv1_blob2_param.float;
10    conv1_blob0.float;
11    conv1_blob0_weight.float;
12    conv1_blob1.float;
13    conv1_blob1_bias.float;
14    scale_conv1_blob0_weight.float;
15    scale_conv1_blob1_bias.float;
16 (2) bn_conv1:
17     bn_conv1_blob0_param.float;
18     bn_conv1_blob1_param.float;
19     bn_conv1_blob2_param.float;
20 (3) scale_conv1:
21     scale_conv1_blob0_weight.float;
22     scale_conv1_blob1_bias.float;
23 (4) conv1_relu:
24 (5) pool1:
25 (6) res2a_branch1:
26     res2a_branch1_blob0_weight.float;
27 (7) bn2a_branch1:
28     bn2a_branch1_blob0_param.float;
29     bn2a_branch1_blob1_param.float;
30     bn2a_branch1_blob2_param.float;
```

----结束

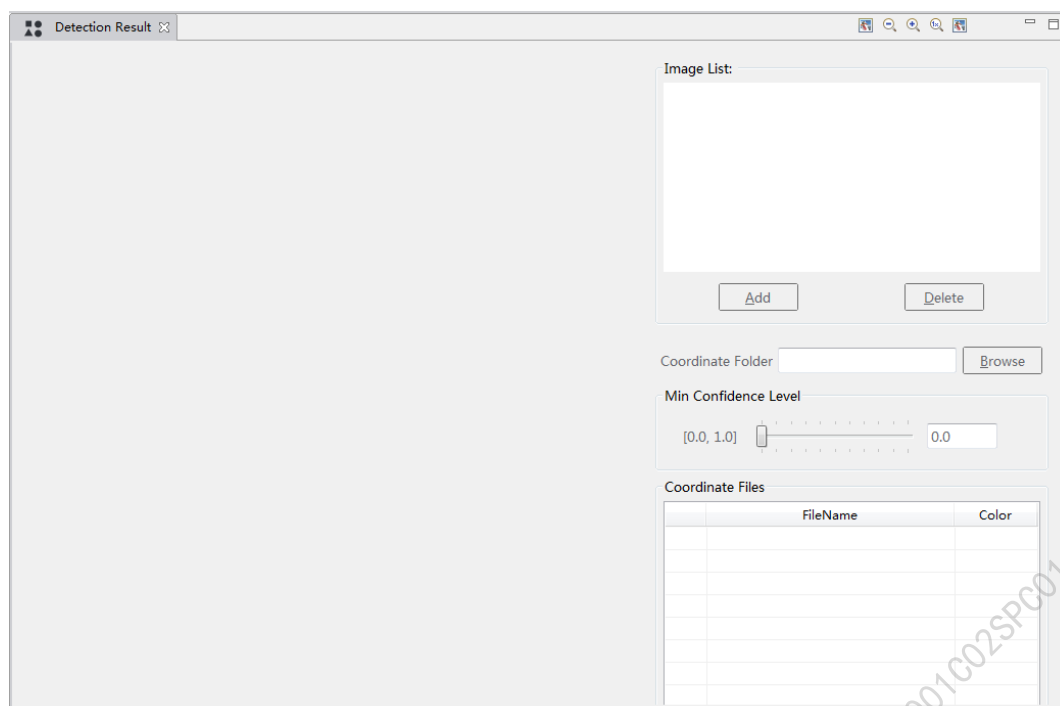
5.5.4 目标检测功能

运行目标检测视图用来查看框选结果，操作如下：

- 步骤 1. 点击左上角工具栏的目标检测按钮，通过 add 或 delete 按钮添加或删除需要检测的图片加入到 Image List 列表中，add 支持单选和多选，delete 支持 ctrl/shift 选中列表中的 ImageList 后删除点击 Detection Result 按钮，打开 Detection Result 视图，如图 5-95。



图5-95 目标检测界面



步骤 2. 导入坐标文件夹 Coordinate Folder，Coordinate 文件的规则如下：

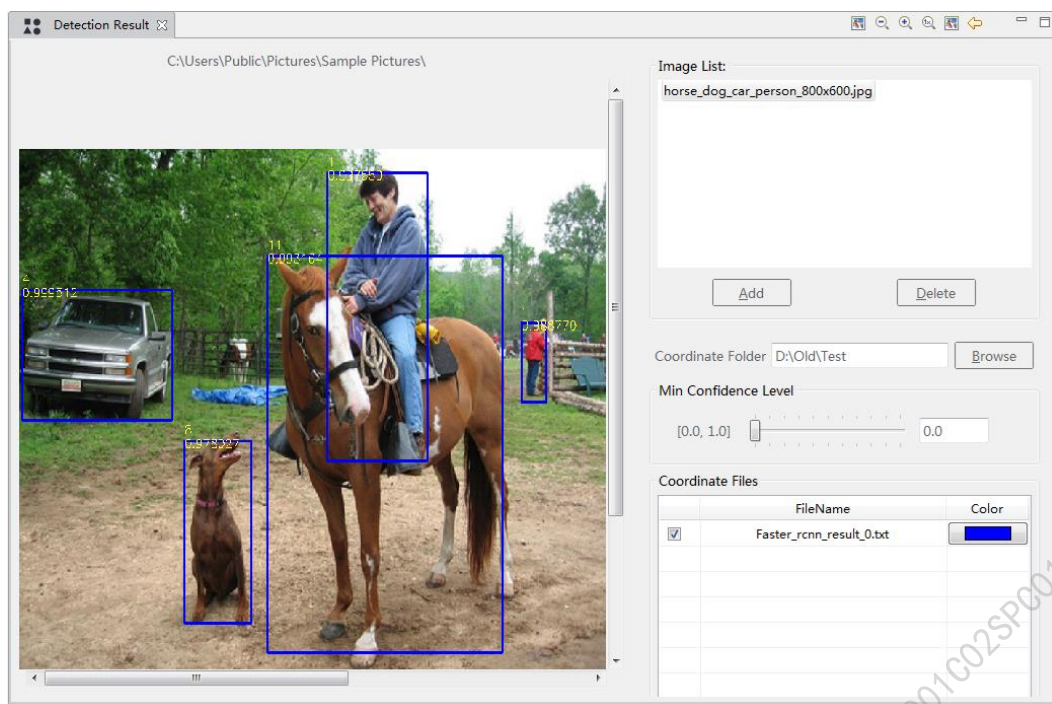
坐标文件首行表示图片缩放到模型输入要求的像素宽和高，从第二行开始，每行由 6 列组成，每列分别对应图片名、类别、置信度、选框左上角 x 坐标、选框左上角 y 坐标、选框右下角 x 坐标、选框右下角 y 坐标。以 horse_dog_car_person_224x224_roi_pos_out_ALL.txt 为例，如图 5-96。

图5-96 坐标检测文件示例

horse_dog_car_person_224x224_roi_pos_out_ALL.txt							
1	224	224					
2	horse_dog_car_person_224x224	7	0.92163086	0	59	62	107
3	horse_dog_car_person_224x224	13	0.81738281	90	30	185	215
4	horse_dog_car_person_224x224	13	0.46826172	150	70	192	192
5	horse_dog_car_person_224x224	15	0.94116211	115	6	159	142
6	horse_dog_car_person_224x224	15	0.62402344	192	67	208	111

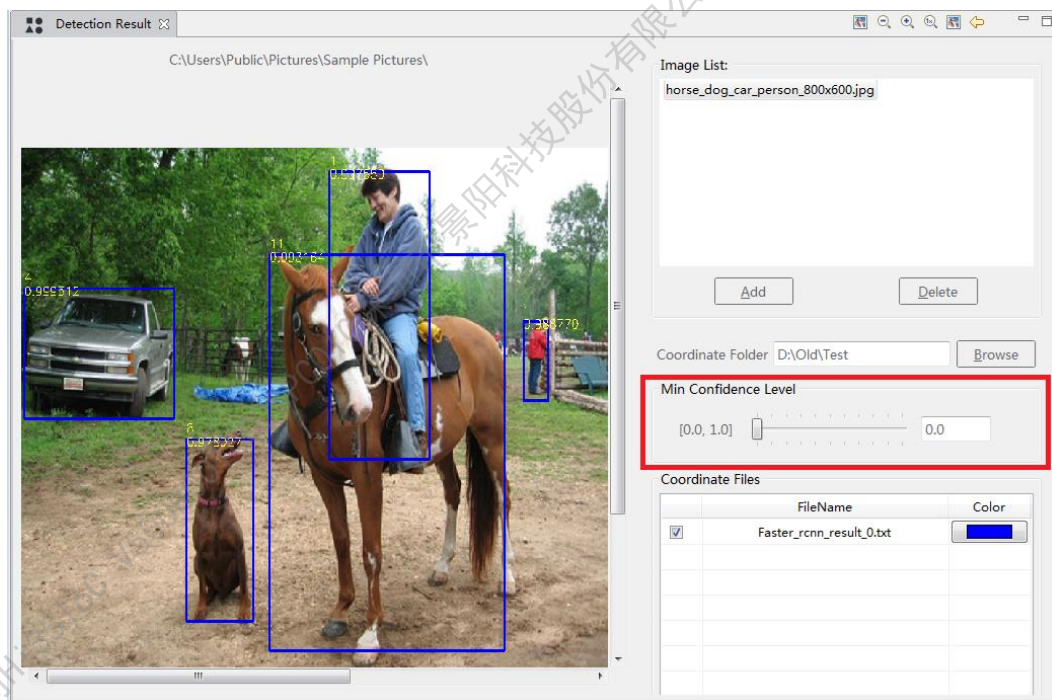
步骤 3. 导入坐标文件后，坐标文件会与当前的 imagelist 中的图片的名称进行匹配，匹配到的坐标文件才会显示到 Coordinate Files 里，如图 5-97。

图5-97 框选结果



步骤 4. 动态调整显示框图的最小置信度，通过修改滑块，可以支持根据选择的最小置信度来过滤框选图，如图 5-98；

图5-98 修改置信度后的框选结果





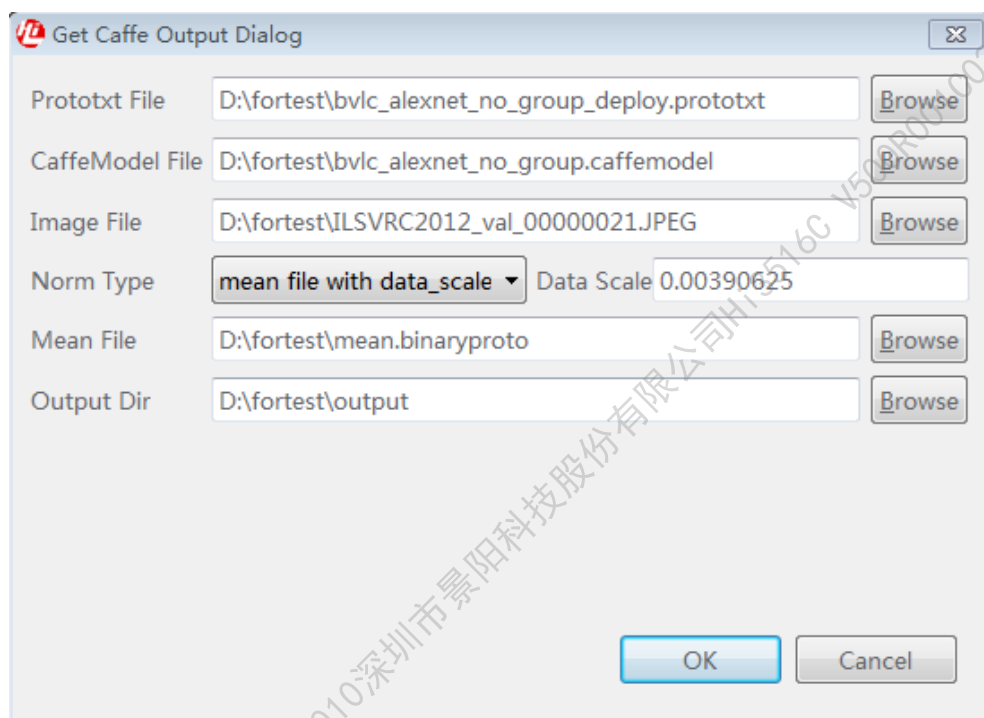
----结束

5.5.5 输出 caffe 的中间结果

本章节和 5.5.6 章节的功能依赖 python3, caffe, 需要客户环境已经具备, 配置方式请参考 5.1.2 “Python3.5+caffe 环境配置” 章节的描述。对于本章节和 5.5.6 “还原网络” 章节, 如果要使用 GPU 加速, 需要保证按照 3.6.4 “功能仿真 CUDA 加速配置” 配置好相关环境, 保证 CUDA 加速使能。如果工具检测到有 CUDA 加速, 会使用 CUDA 加速, 否则使用 CPU 生成 Caffe 结果。

- 步骤 1. 点击工具栏上的  按钮, 弹出如下对话框, 依次设置输出中间结果的网络的 prototxt, caffemodel, ImageFile, 如果要对图片做预处理操作, 需要配置 Norm Type, Data Scale 和 MeanFile, 具体含义可参考 3.5.2 章节中配置文件项的相关介绍, 并且设置 caffe 输出结果所在的路径 Output Dir, 如图 5-99 所示。

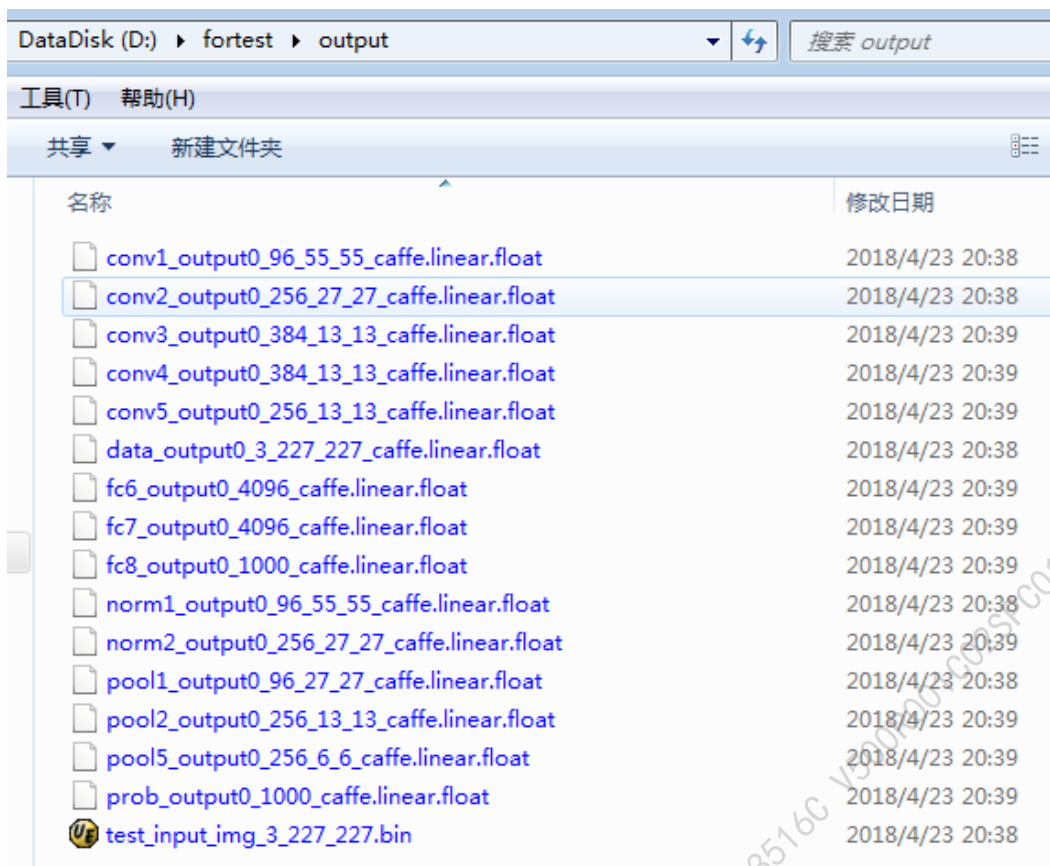
图5-99 输出 caffe 中间结果



- 步骤 2. 点击 OK 键, 在 output 文件夹下得到如下输出, 如图 5-100 所示。



图5-100 在配置的 output Dir 下面得到 caffe 中间结果



----结束

5.5.6 还原网络

5.5.6.1 背景

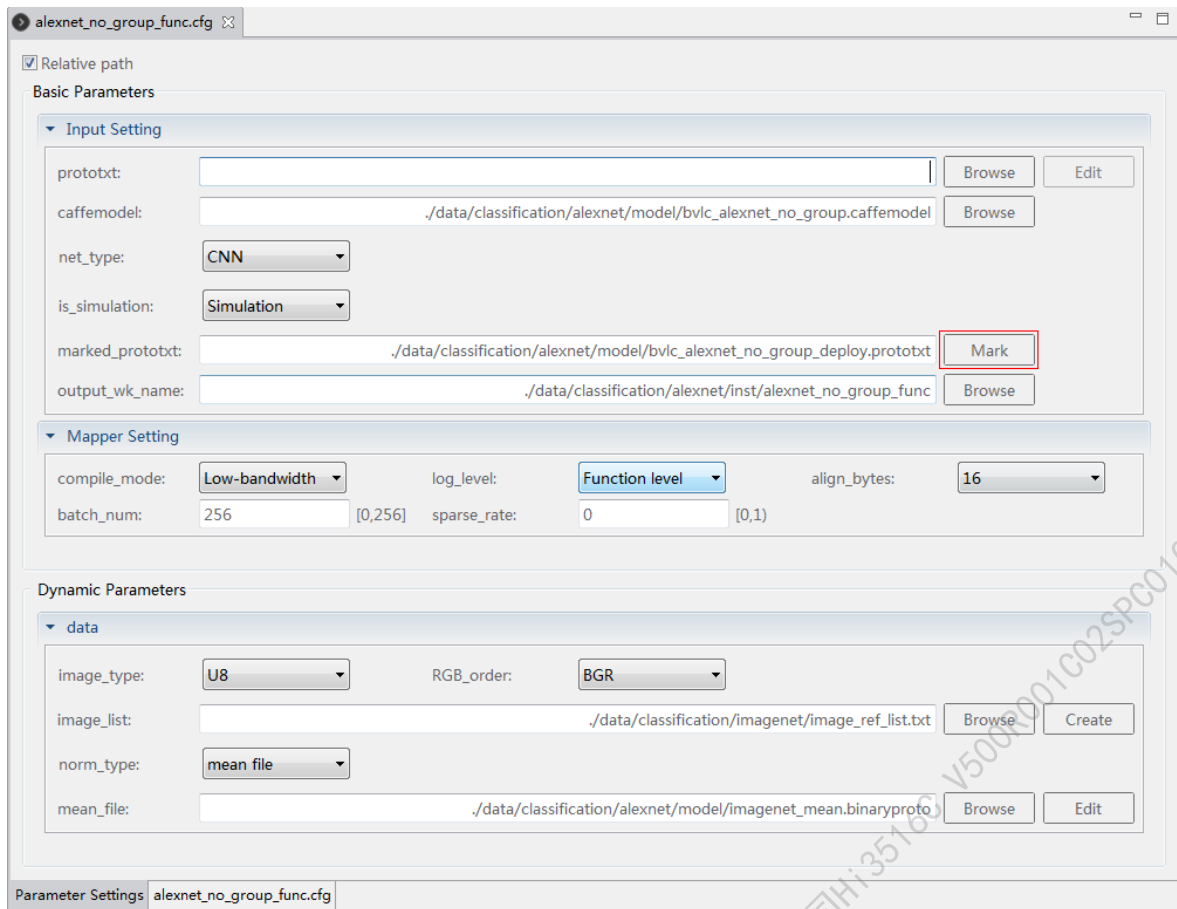
客户已使用相似度比较工具确认精度问题不是 `nnie_mapper` 量化导致的，且定位到某个或者某些非用户自定义层出现精度下降问题（一种情况是断崖式下降，一种情况是相似度不是断崖式的下降，精度逐渐下降或者忽高忽低但是差异不大但整体偏差精度较大），需要客户将怀疑有问题的局部网络的相关数据截取出来并提供给海思定位。工具提供网络模型的导出功能方便还原网络定位问题。

5.5.6.2 操作步骤

- 步骤 1. 利用 5.5.3 章节介绍的调试定位信息获取功能 `dump` 出想要还原的网络数据，如图 5-101 所示。

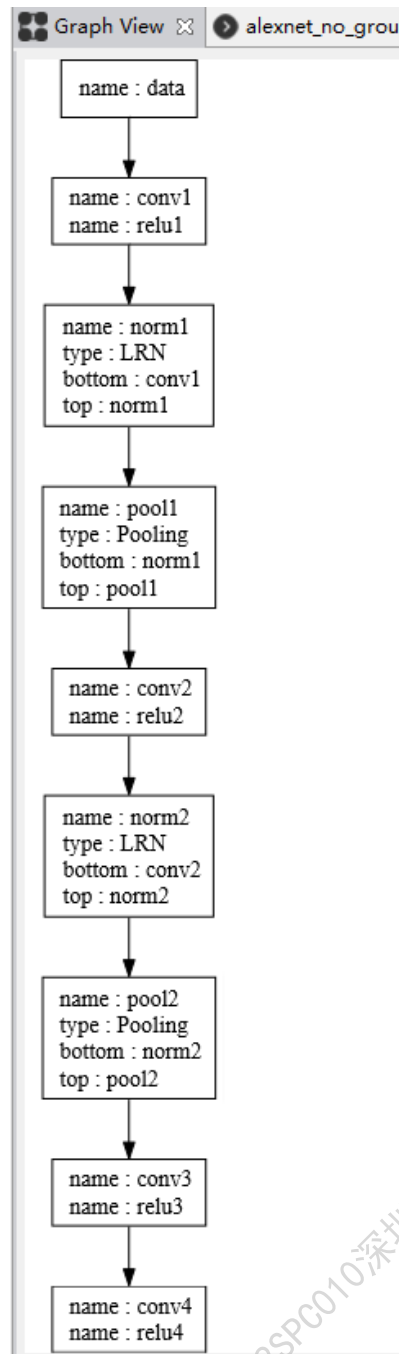


图5-101 标记 prototxt



步骤 2. 得到视图，如图 5-102 所示。

图5-102 在 Graph View 上画出对应视图

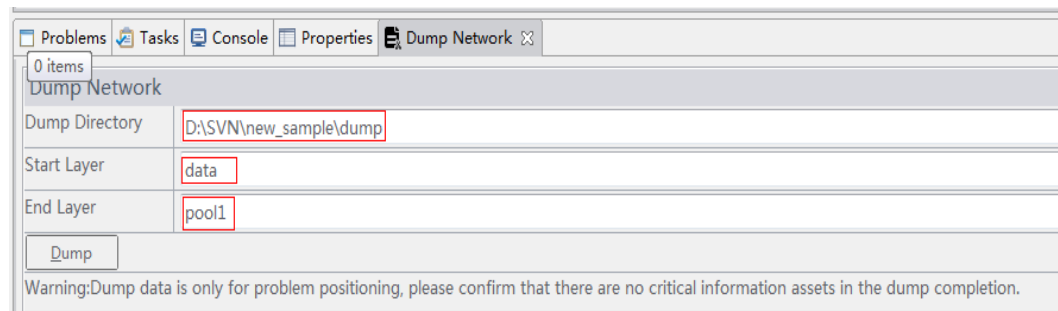


步骤 3. 点击右上角  dump network 按钮。

步骤 4. 选择如下需要 dump 的层，如图 5-103 所示。

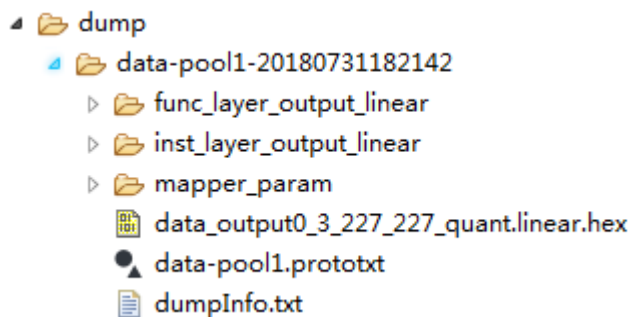


图5-103 Dump Network 视图



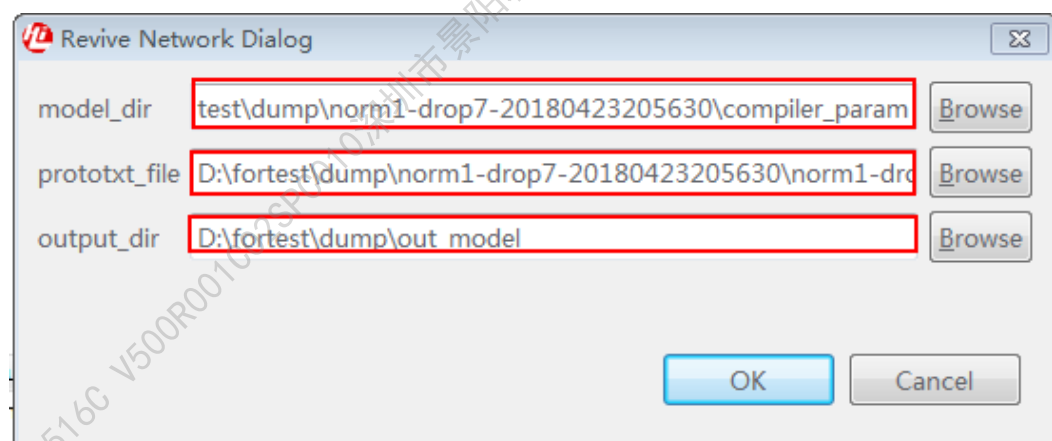
步骤 5. 点击 dump 按钮，生成 dump 出的数据，如图 5-104 所示。

图5-104 Dump 出的数据



步骤 6. 点击工具栏上的还原网络  按钮打开还原网络弹出框，将 mapper_quant, dump 出的 prototxt 作为还原网络的输入，如图 5-105 所示。

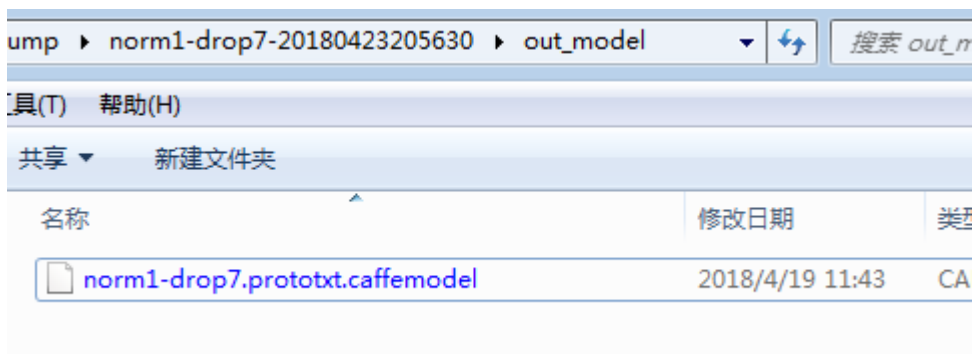
图5-105 还原网络



步骤 7. 得到还原出来的 caffemodel，如图 5-106 所示。



图5-106 还原出 caffemodel



----结束

5.5.7 芯片高效模式和 Un-inplace 功能

5.5.7.1 背景

把激活层合并到前面的 NNE 层，在 cycle 仿或者板端环境下跑网络时候，在计算的时候就把激活执行，不需要一个单独的层来实现激活了，从而减少了内存的分配。

5.5.7.2 芯片高效模式功能介绍

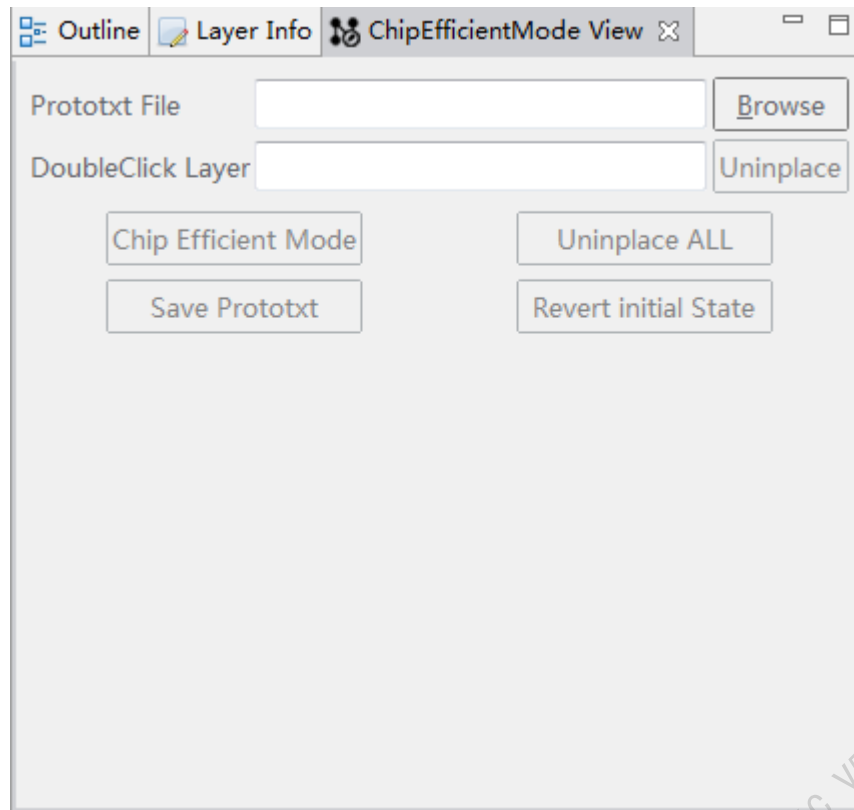
支持将 Prototxt 文件中符合 un-inplace 规范且芯片支持的层集合，改写为 inplace 层。

操作方式如下：

步骤 1. 点击工具栏芯片高效模式按钮 ，打开芯片高效模式视图，如图 5-107 所示。



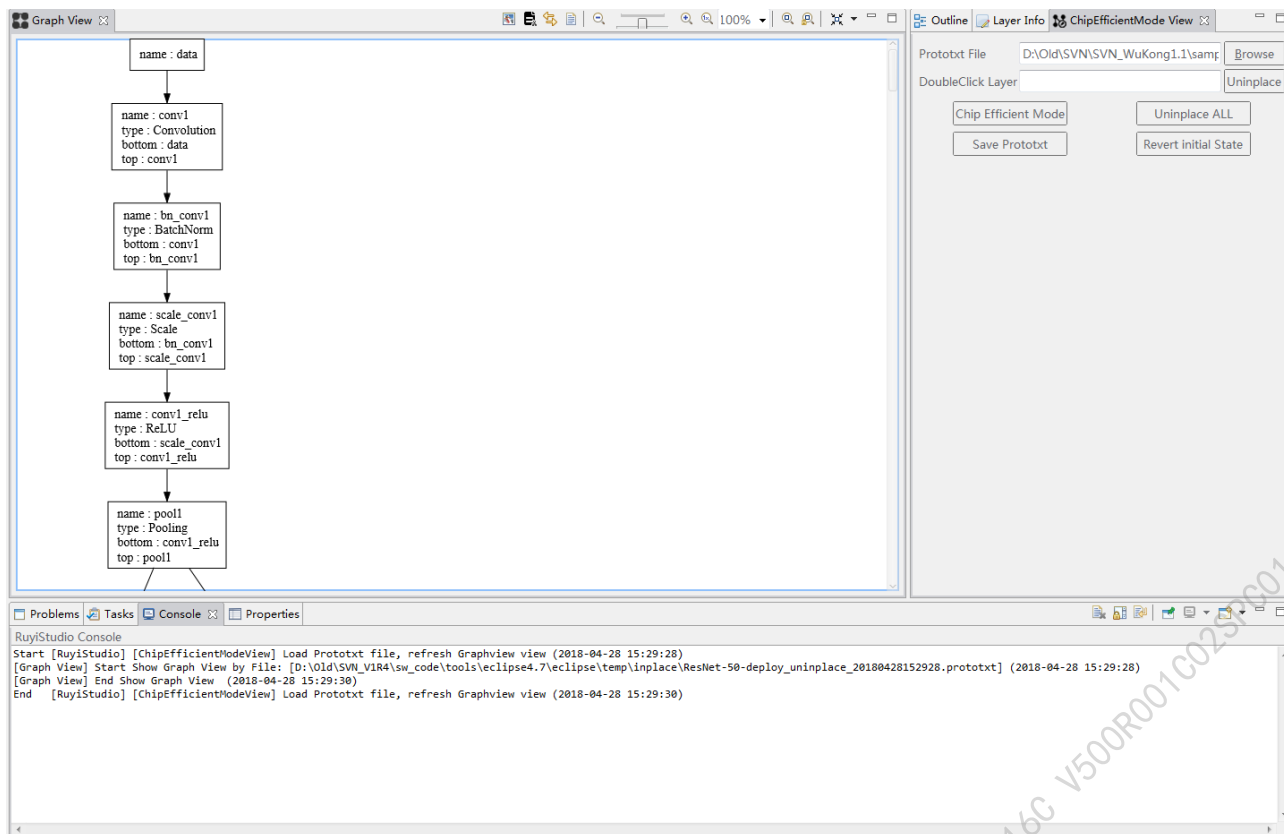
图5-107 芯片高效模式视图



步骤 2. 将需要修改的 Prototxt 导入到 Prototxt File 的文本框，工具会自动加载 Prototxt 显示到 GraphView 中，如[图 5-108](#) 所示。



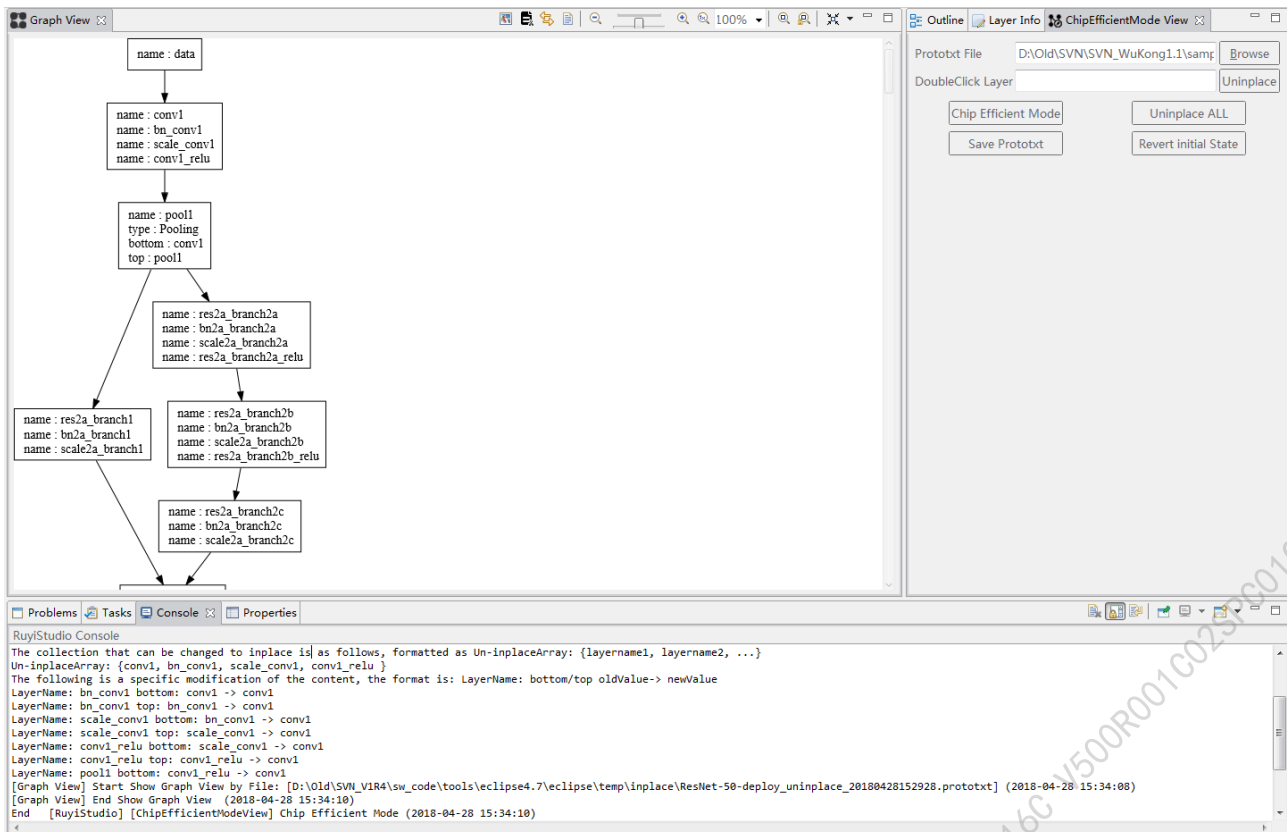
图5-108 打开 Prototxt 显示到 Graph View 中



步骤 3. 点击右侧 ChipEfficientMode View 中的 Chip Efficient Mode 按钮，在确认弹出框中点击确认，工具就会去查找当前 Prototxt 中符合芯片支持的改写为 Inplace 的层的集合，然后将其改写为 Inplace 写法刷新到 Graph View 视图中，下方 Console 控制台会打印 Uninplace 操作的过程信息，如图 5-109，用户可通过此信息了解工具查找到的符合条件的可改写为 inplace 的层集合，以及当前操作修改的对应层中的 top 或 bottom 信息。



图5-109 芯片高效模式执行效果



----结束

5.5.7.3 Un-inplace 所有 inplace 层的功能介绍

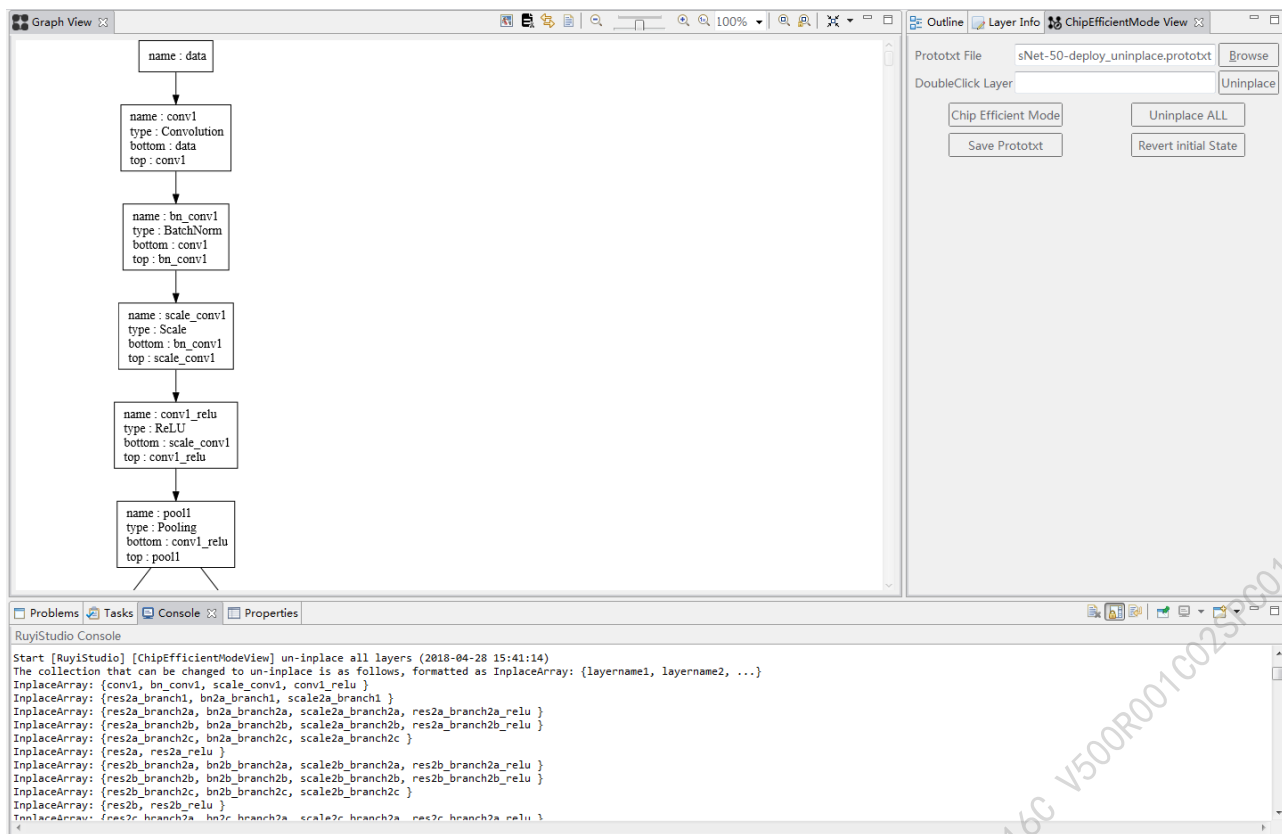
支持将 Prototxt 文件中符合 inplace 规范的层集合，改写为 un-inplace 层。

操作方式如下：

- 步骤 1. 导入需要修改的 Prototxt 文件，操作方式同 5.5.7.2 章节的步骤 2；
- 步骤 2. 点击 Uninplace ALL 按钮，左侧 GraphView 会刷新为 Un-inplace ALL 后的图像，下方 Console 控制台会打印 Un-inplace ALL 操作的过程信息，如图 5-110，用户可通过此信息了解工具查找到的符合条件的可改写为 unInplace 的 inplace 层的集合，以及当前操作修改的对应层中的 top 或 bottom 信息。



图5-110 Un-inplace 所有 inplace 层的功能



----结束

5.5.7.4 Un-inplace 单个 inplace 层功能介绍

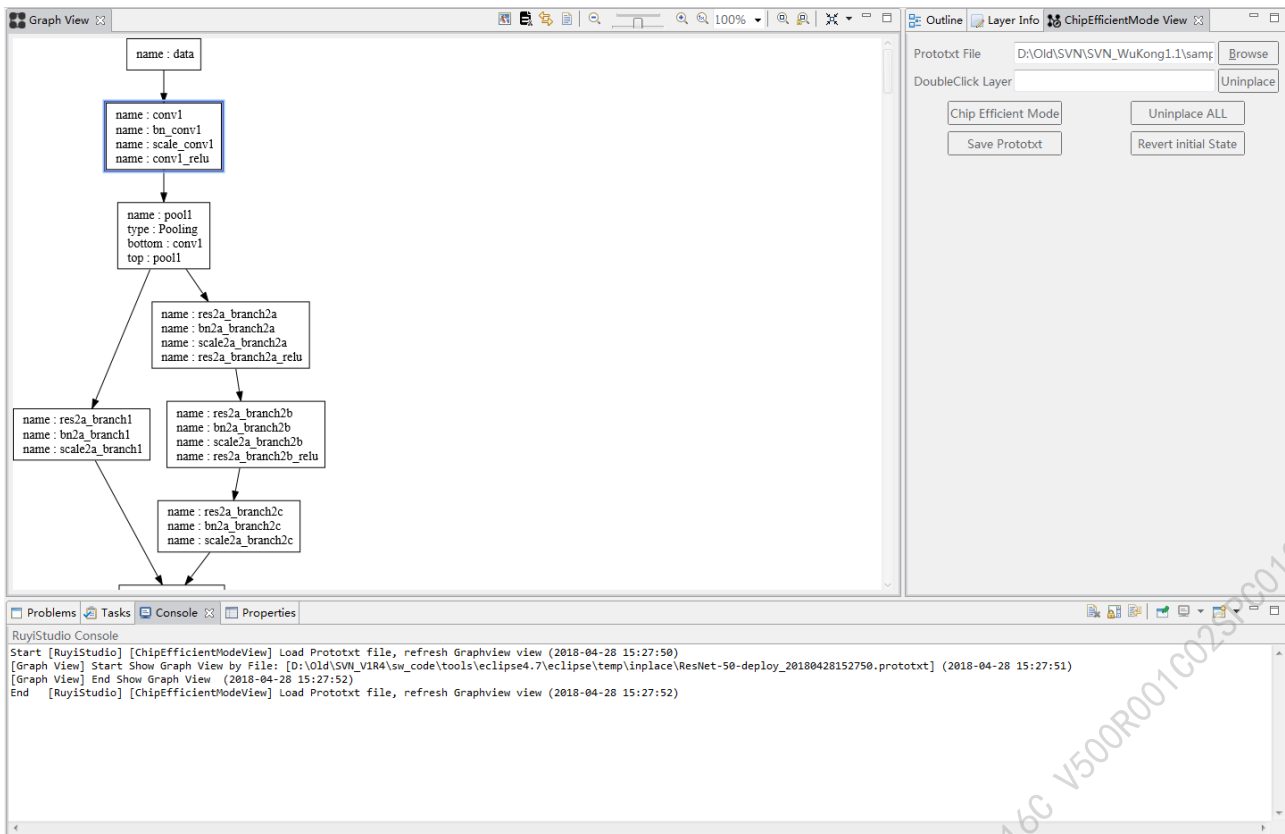
支持将 Prototxt 中芯片支持改写为 Un-inplace 的用户指定的单个 inplace 层自动改写为 Un-inplace 层。

操作方式如下：

- 步骤 1. 在左侧 GraphView 中双击需要执行 Un-inplace 操作的 inplace 层，DoubleClick Layer 文本框中会显示选中的层名，如图 5-111 所示。



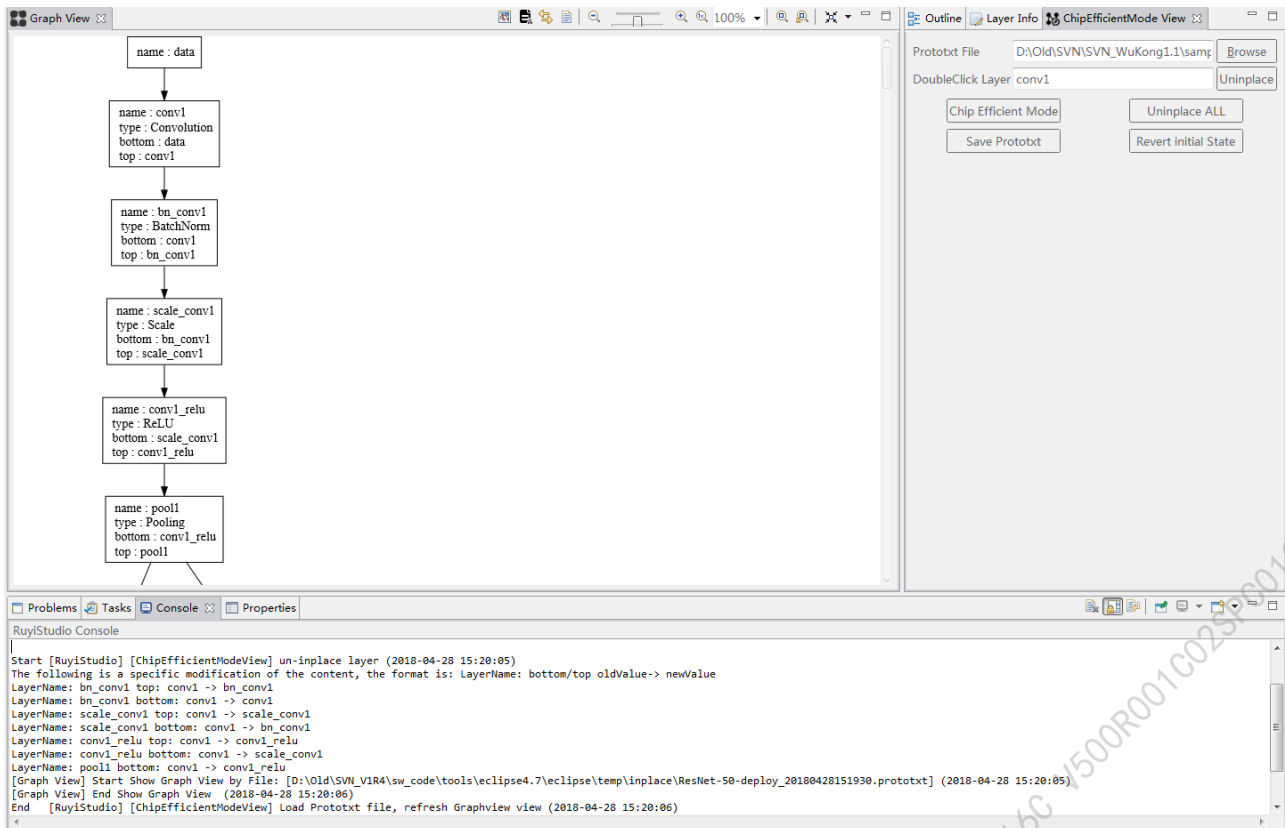
图5-111 双击选中需要 un-inplace 的层



步骤 2. 点击 Uninplace 按钮，左侧 GraphView 会刷新为 Un-inplace 后的图像，下方 Console 控制台会打印 Un-inplace 操作的过程信息，如图 5-112，用户可通过此信息了解当前操作修改的对应层中的 top 或 bottom 信息。



图5-112 Un-inplace 单个 inplace 层的功能



----结束

5.5.7.5 恢复 Prototxt 到原始状态的功能介绍

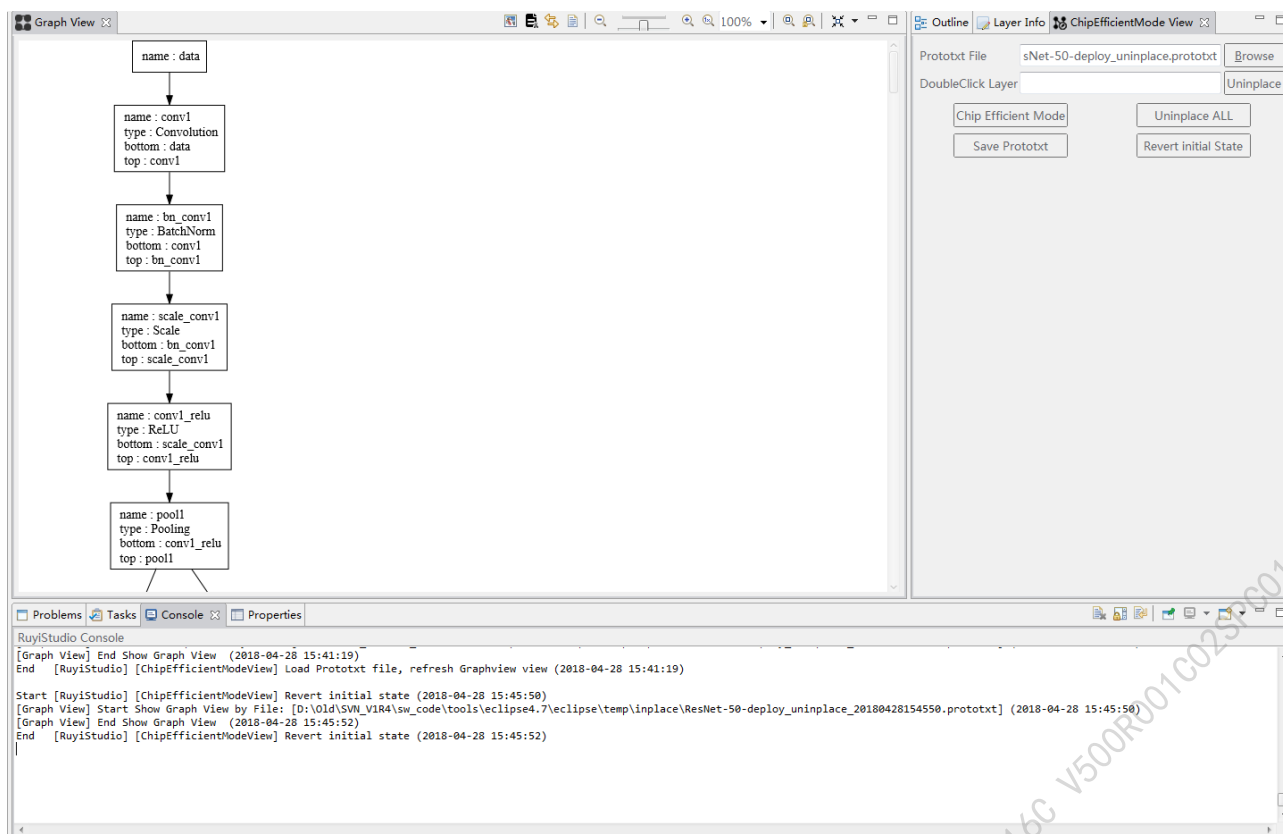
支持将当前 GraphView 中显示的 Prototxt 恢复到原始导入的 Prototxt。

操作方式如下：

- 步骤 1. 点击 Revert initial State 按钮，左侧 GraphView 会刷新为原始的 Prototxt 导入后的图像，如图 5-113。



图5-113 Revert initial State 功能



5.5.7.6 保存当前 GraphView 中修改过的 Prototxt 的功能介绍

支持将当前 GraphView 中显示的修改过的 Prototxt 保存到新的 Prototxt 文件的功能。

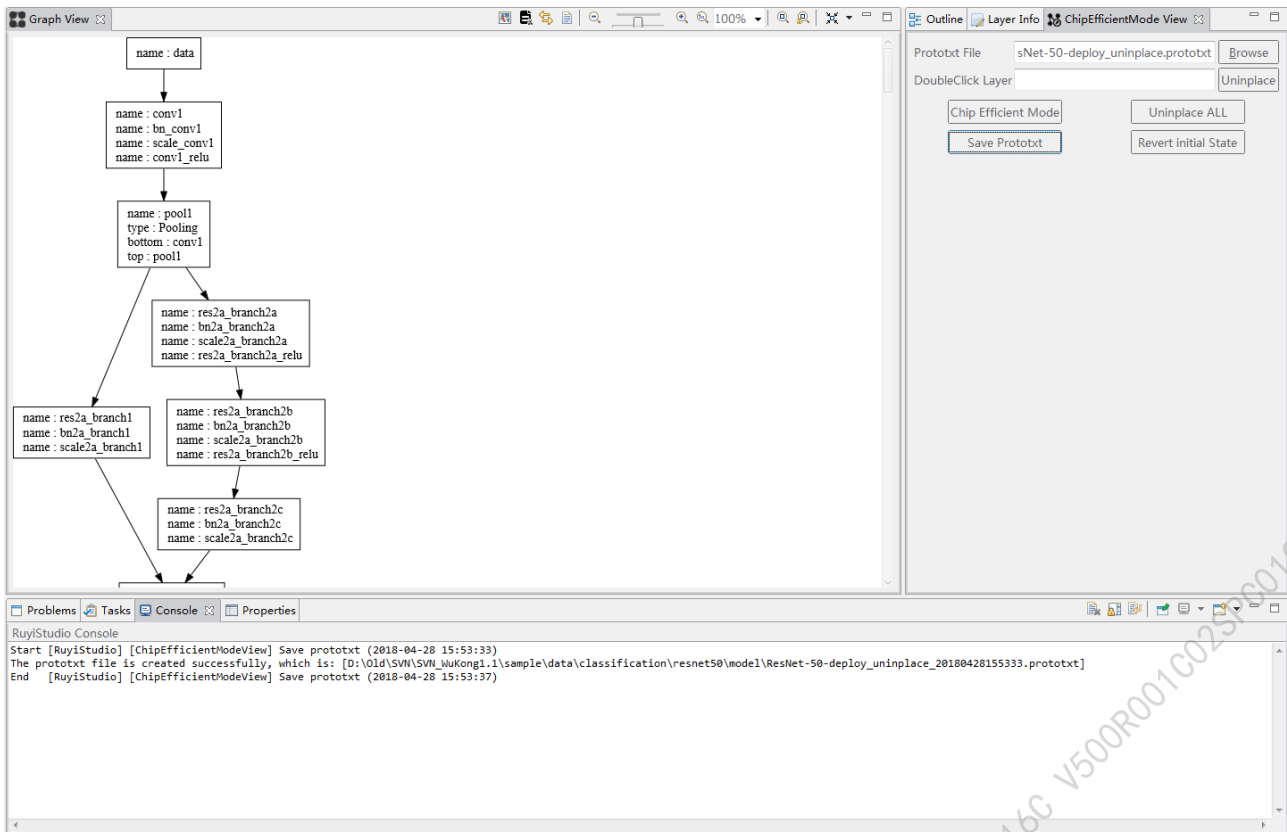
操作方式如下：

步骤 1. 点击 Save Prototxt 按钮，设置保存路径；

步骤 2. 点击确认，保存当前 GraphView 中的 Prototxt 文件成功，如图 5-114。



图5-114 Save Prototxt 功能



----结束

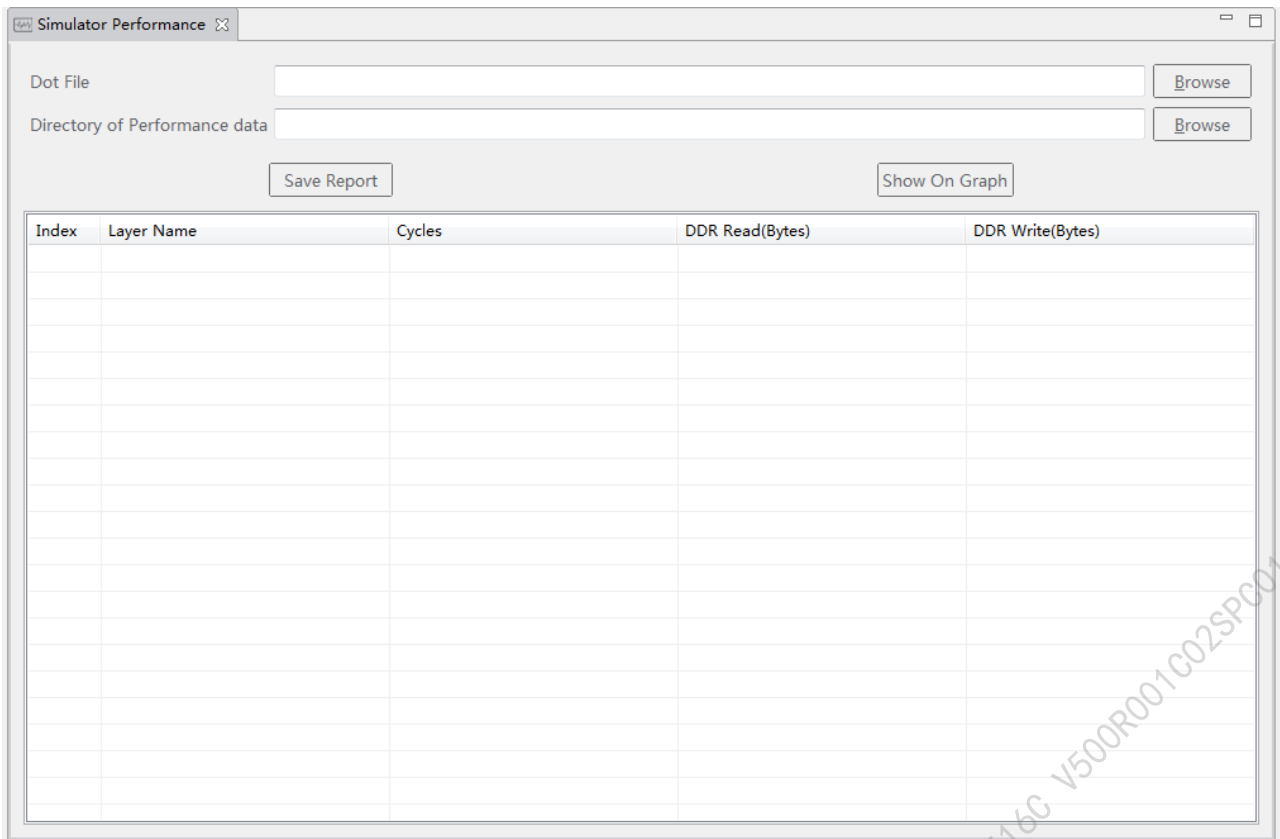
5.5.8 性能仿真结果展示

提供仿真库中 cycle 仿执行过程输入的性能仿真数据的 UI 展示，操作如下：

- 步骤 1. 点击左上角工具栏的性能仿真结果展示按钮 ，在工具编辑区显示性能仿真结果展示视图 Simulator Performance，如图 5-115。



图5-115 性能仿真结果展示界面



步骤 2. 导入 nnie_mapper 生成的 Dot 文件用于解析层名，再导入性能仿真结果数据存在的文件夹，导入操作如[图 5-116](#)，性能仿真数据文件存在于 cycle 仿运行后生成的目录下，以 sample 工程为例，在 sim_out 下的 perf_info 目录中，如 cycs_result_0 文件，示例如[图 5-117](#)。



图5-116 导入 Dot 文件和性能仿真结果文件夹

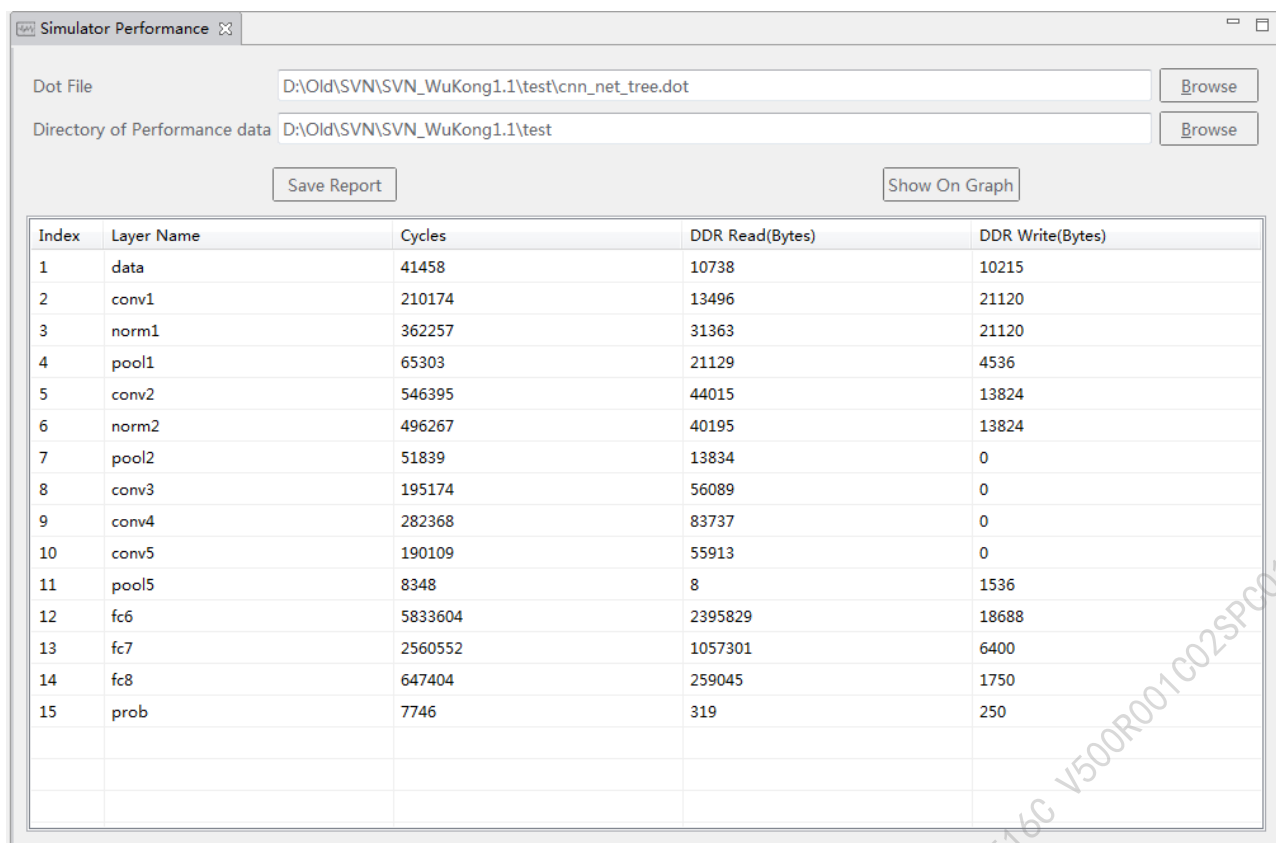
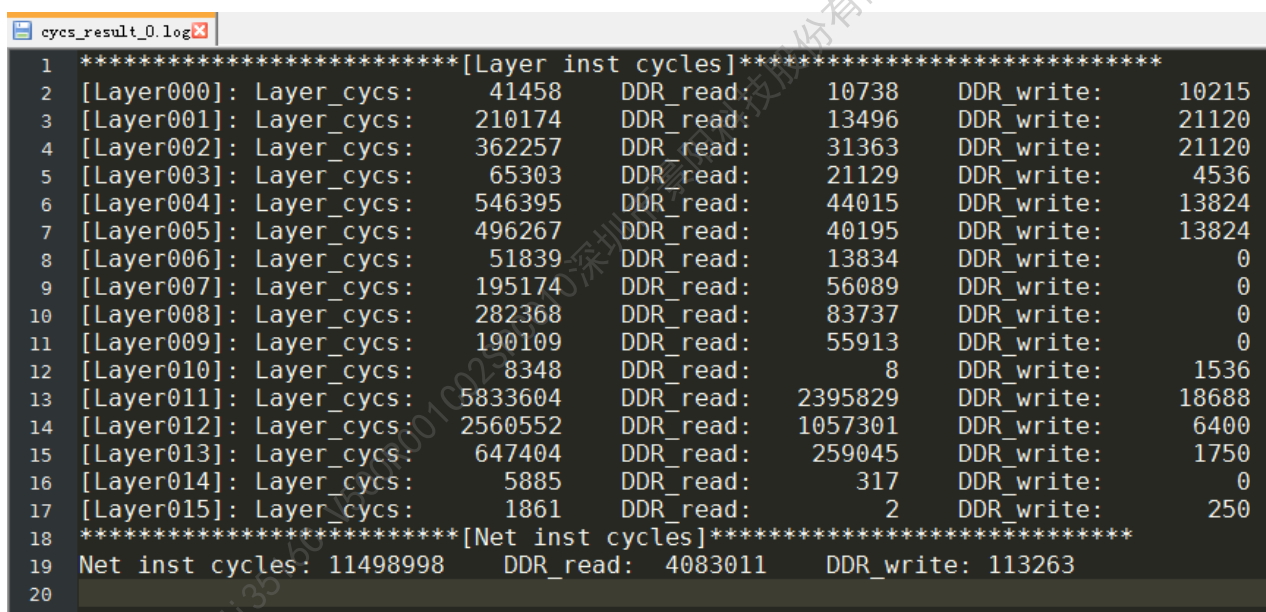


图5-117 性能仿真结果文件示例



----结束



5.5.9 预处理功能

5.5.9.1 背景

将对 Prototx 文件进行格式修改的功能整合到预处理界面下，包含以下功能：

- Convert Data Layerformat
- Convert Efficient Mode
- Add Quant Adaptation Layer
- Remove PriorBox Layer
- Detection Network NNIE1.1 Converted to NNIE1.2

5.5.9.2 功能介绍

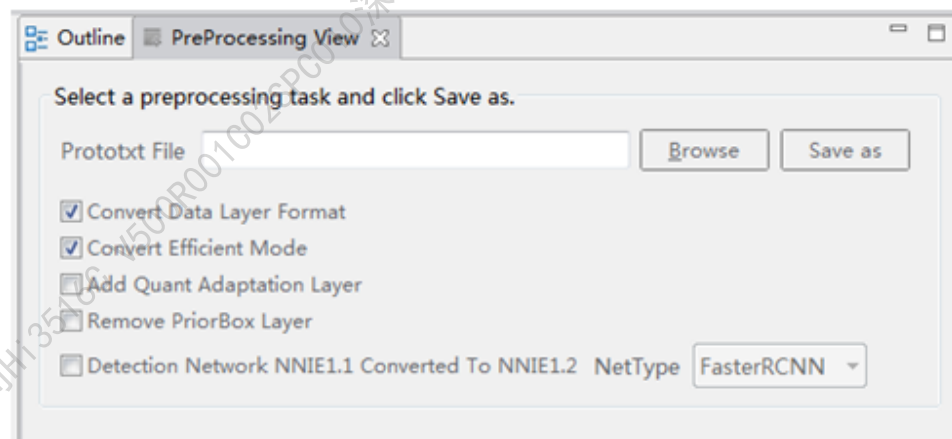
- Convert Data Layerformat，支持将 caffe1.0 标准的 prototxt 文件修改为 NNIE 支持的写法。
- Convert Efficient Mode，支持将 Prototxt 文件中符合 un-inplace 规范且芯片支持的层集合，改写为 inplace 层。
- Add Quant Adaptation Layer，支持在做重训练时，涉及到修改 prototxt 的操作，如果要增加量化层，可以使用工具增加量化层。
- Remove PriorBox Layer，由于 SSD 网络下的 PriorBox 层 NNIE 不支持也不属于 caffe 原生的层，手动改动比较繁琐，效率比较低，故工具支持在在预处理的时候去掉 PriorBox 层。
- Detection Network NNIE1.1 Converted to NNIE1.2，此功能应用于检测网，对 NNIE1.1 不支持的 Proposal 和 DectectionOutput 层添加适配层模板（用户需要自己调整参数）以支持 NNIE1.2 进行了硬件加速。

5.5.9.3 操作步骤

操作方式如下：

步骤 1. 点击工具栏预处理按钮 ，打开预处理视图，如图 5-118 所示。

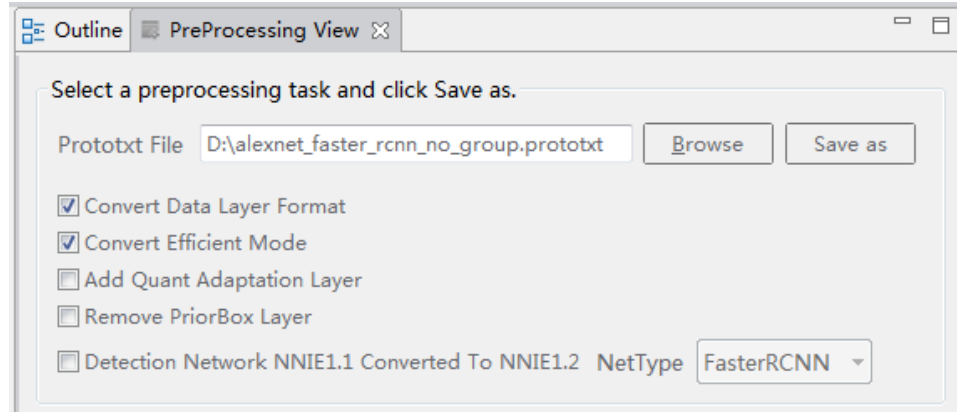
图5-118 预处理视图





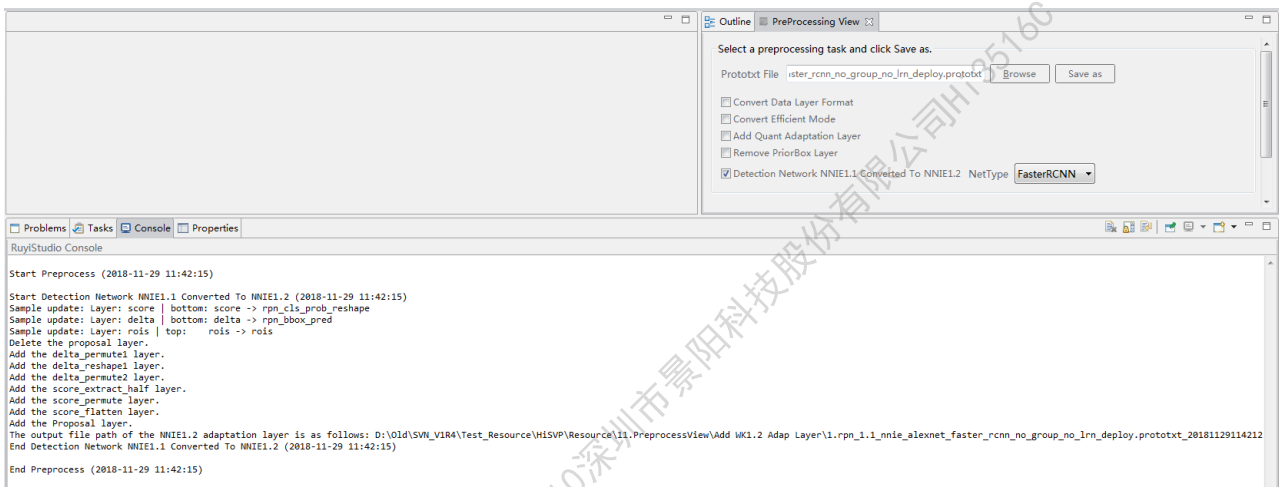
步骤 2. 将需要修改的 Prototxt 导入到 Prototxt File 的文本框，如图 5-119 所示。

图5-119 导入 Prototxt



步骤 3. 选择需要进行预处理修改的选项，点击 Save as 选择需要保存 Prototxt 的路径，点击确认即可将修改的 prototxt 另存，且工具控制台会输出修改详细日志，如图 5-120。

图5-120 进行预处理另存 Prototxt



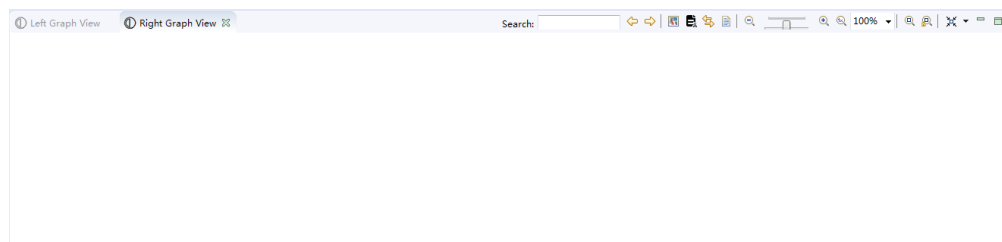
----结束

5.5.10 拓扑图对比功能

步骤 1. 在工具栏点击  按钮，会出现如图 5-121 所示初始化界面。

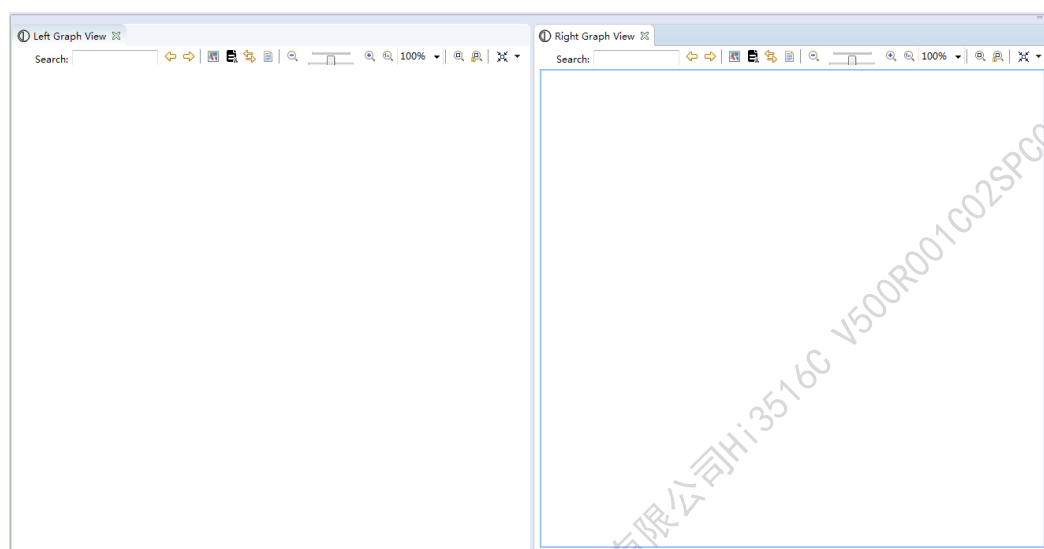


图5-121 拓扑图对比初始化界面



步骤 2. 手动选中 Right GraphView 页签拖动视图布局，切为对比视图，如图 5-122 所示。

图5-122 拓扑图对比视图



步骤 3. 点击上方的导入按钮，导入对应的 prototxt 或者 dot 文件。

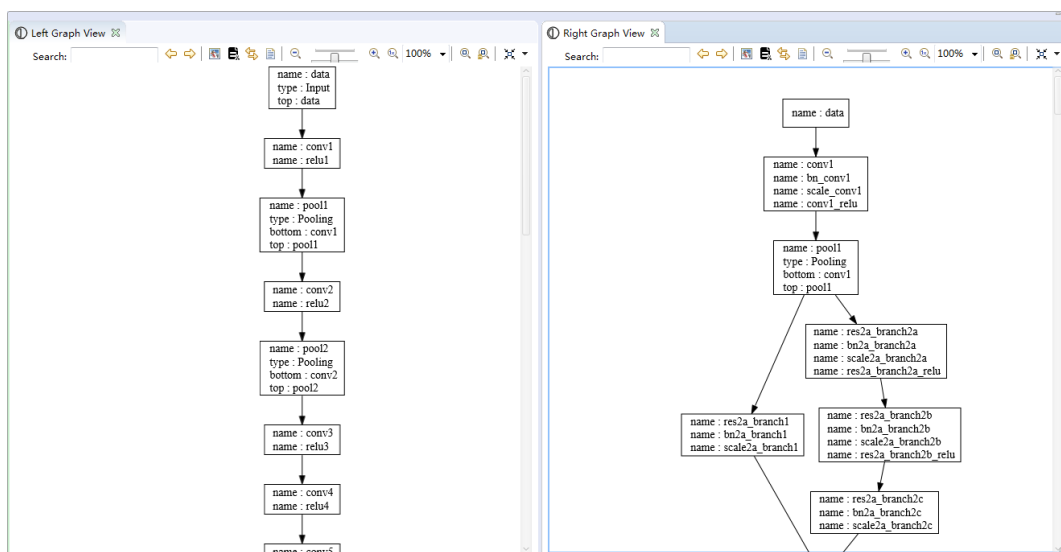
图5-123 导入文件



步骤 4. 对比网络拓扑图，界面显示如图 5-124 所示。



图5-124 网络拓扑图



----结束

5.6 多芯片支持功能

因不同的芯片使用了不同的 nnie_mapper 及仿真库，故 RuyiStudio 工具提供多芯片支持的功能，用户可以指定当前使用的项目工程匹配的是哪一款芯片，具体的支持表格如表 5-1。

表5-1 芯片名称与 mapper 和仿真库的对应关系

芯片名称	与 mapper 和仿真库的对应关系
Hi3559AV100 Hi3559CV100	nnie_mapper_11.exe libnniefc1.1.a, libnnieit1.1.a, libnniefc1.1d.a, libnnieit1.1d.a nniefc1.1.dll, nniefc1.1d.dll, nnieCUDA1.1.dll, nnieCUDA1.1d.dll
Hi3519AV100	nnie_mapper_12.exe libnnieit1.2.a, libnniefc1.2.a, libnnieit1.2d.a, libnniefc1.2d.a nniefc1.2.dll, nniefc1.2d.dll, nnieCUDA1.2.dll, nnieCUDA1.2d.dll
Hi3556AV100	不支持
Hi3516DV300	nnie_mapper_12.exe libnnieit1.2.a, libnniefc1.2.a, libnnieit1.2d.a, libnniefc1.2d.a nniefc1.2.dll, nniefc1.2d.dll, nnieCUDA1.2.dll,

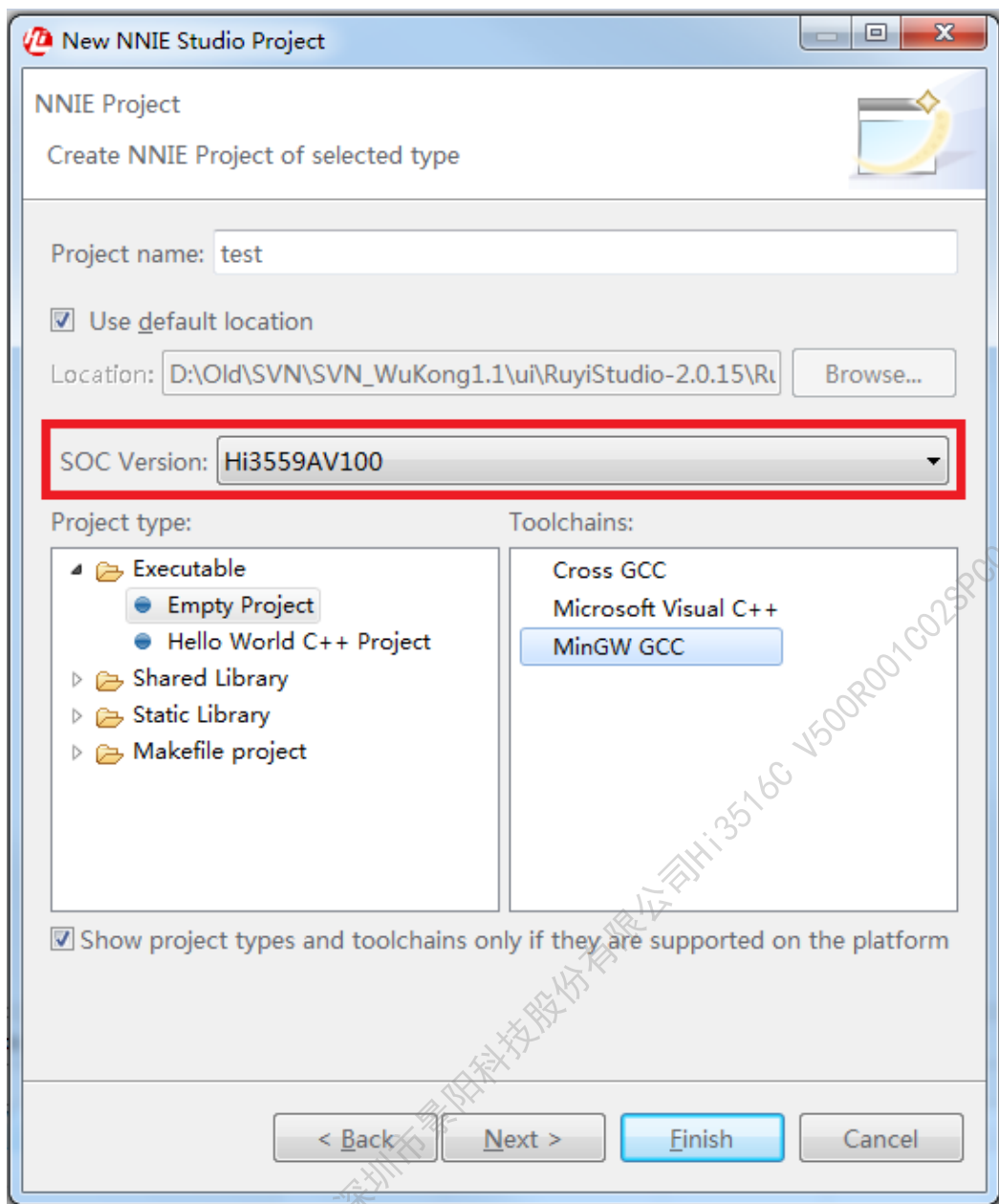


芯片名称	与 mapper 和仿真库的对应关系
	nnieCUDA1.2d.dll
Hi3516CV500	nnie_mapper_12.exe libnnieit1.2.a, libnniefc1.2.a, libnnieit1.2d.a, libnniefc1.2d.a nniefc1.2.dll, nniefc1.2d.dll, nnieCUDA1.2.dll, nnieCUDA1.2d.dll

5.6.2 创建新工程项目时选择对应的芯片

在工具栏上点击 File->New->NNIE Project，打开创建视图，在 SOC Version 处选择项目绑定的芯片名称，如[图 5-125](#)。

图5-125 创建 NNIE Project 时设置 SOC Version

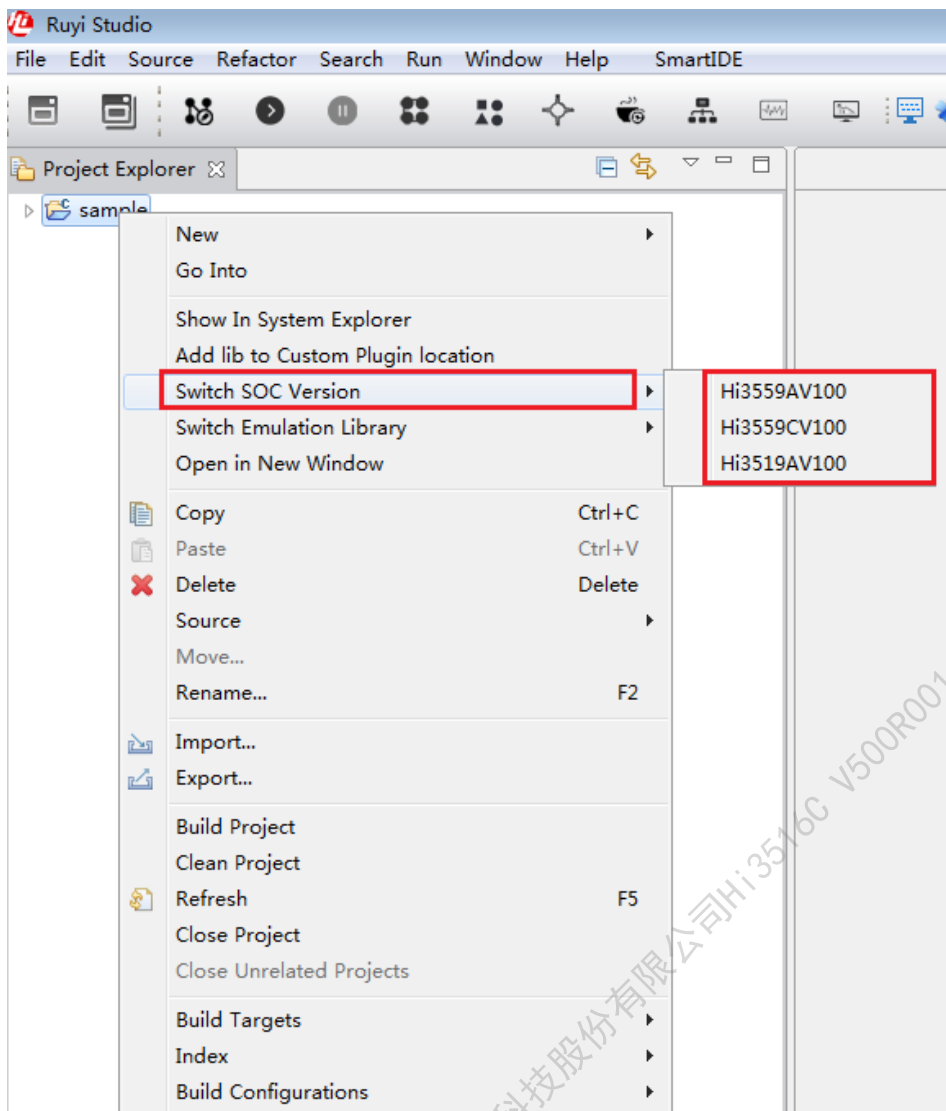


5.6.3 在已有的工程项目中修改对应的芯片

步骤如下：

步骤 1. 在项目上点击右键-> SOC Version 选择对应的 SOC Version，如图 5-126 所示。

图5-126 修改已有工程中对应的芯片



步骤 2. 另外，导入到 RuyiStudio 的工程中若默认没有指定项目对应的芯片，工具会自动添加默认芯片 Hi3559AV100 给当前的工程。

----结束

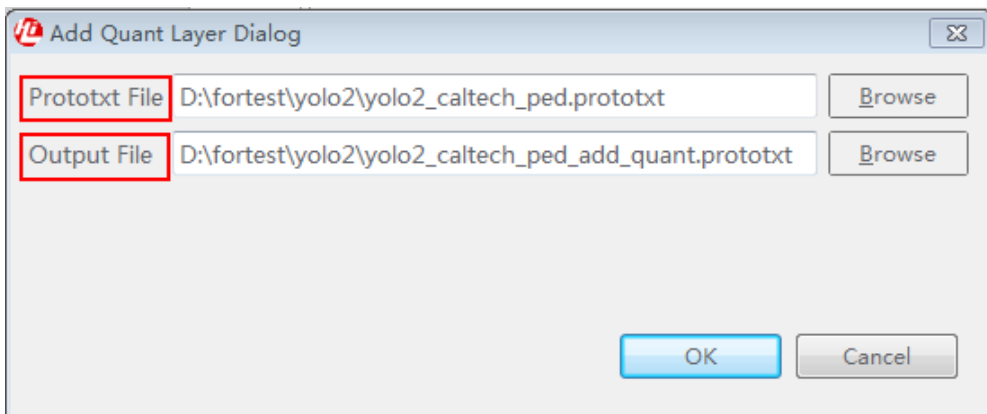
5.7 Fine tune 辅助添加量化层功能

在做重训练时，涉及到修改 prototxt 的操作，如果要增加量化层，可以使用工具增加量化层，操作方式如下：

步骤 1. 点击工具栏添加量化层按钮，弹出添加量化层的视图（如图 5-127 所示），选择需要添加量化层的 prototxt 文件，配置输出 prototxt 文件路径。



图5-127 添加量化层视图



步骤 2. 点击 OK 键，即可在指定路径下获取目标文件。

----结束

5.8 生成 Runtime wk 功能

在 Runtime 工程中，打开一个 cfg。

5.8.1 cfg 文件配置模式

同配置 NNIE 一样。



注意

在打开 cfg 的时候，如果该 cfg 有 prototxt 路径，则会在 prototxt 的路径下生成生成跟 prototxt 同名的 rtcfg 文件。

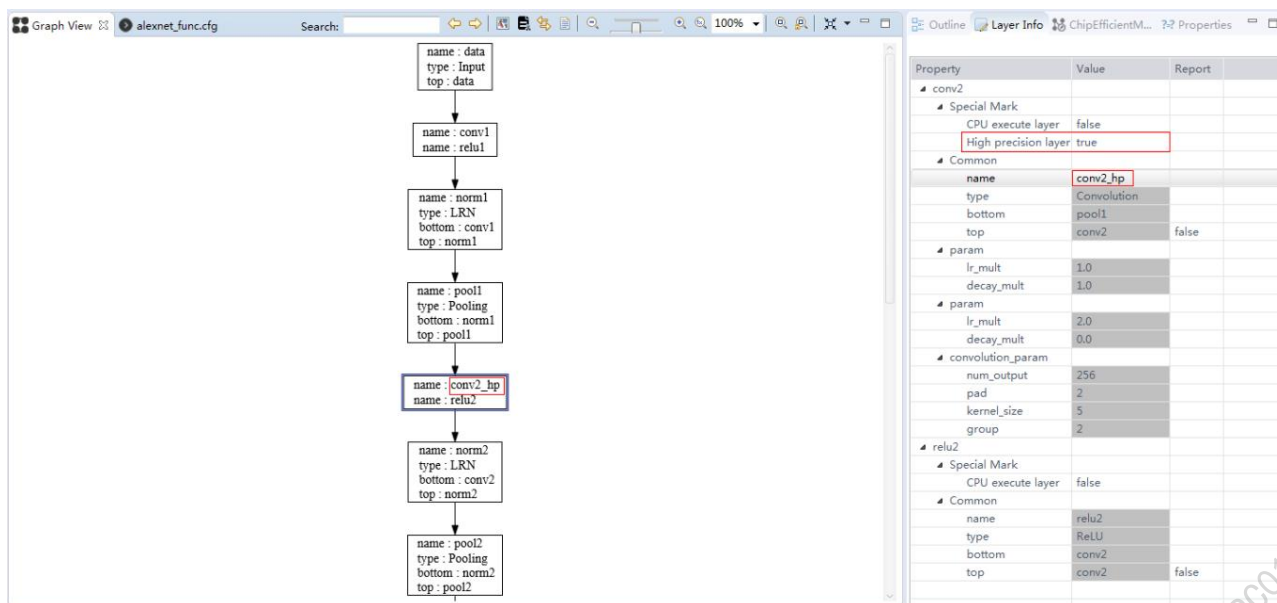
5.8.2 Prototxt 层属性可视化编辑

Runtime mapper 支持将指定的层切换为高精度层，支持任意层中间上报。支持任意层选择客户自定义实现的 Implement。

将当前 Layer 指定为 HP 运算，在右侧 Property 视图选择 true 后，工具自动修改 name 添加 _hp 字段，如图 5-128。



图5-128 将当前 Layer 指定为高精度运算



注意

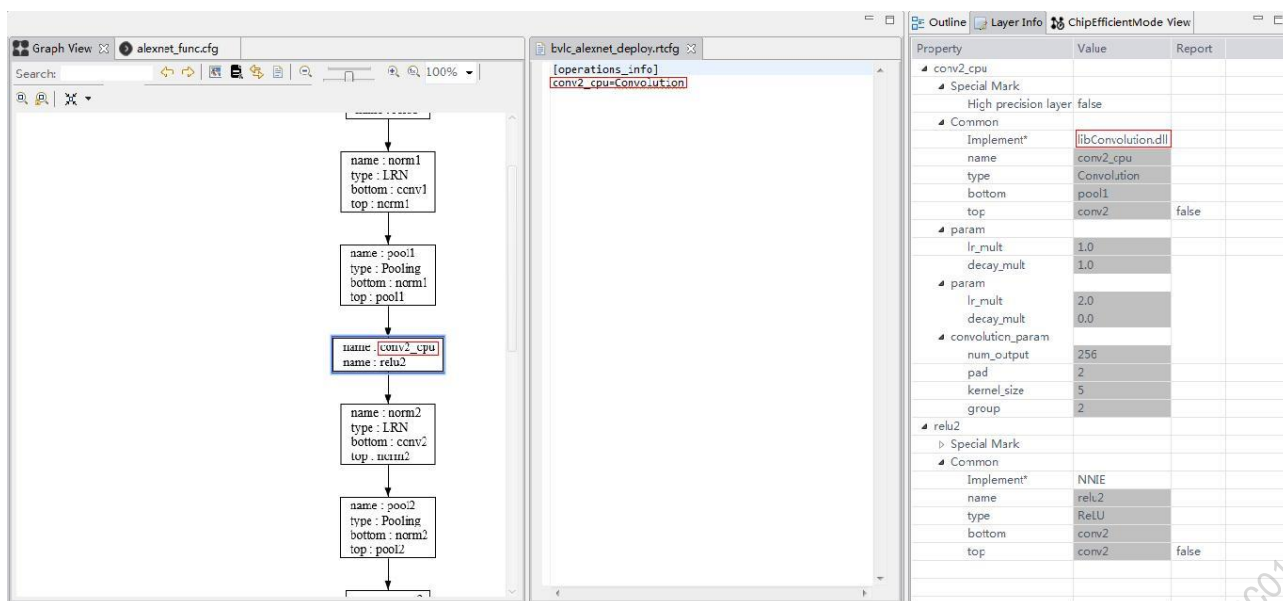
只有当选择层的类型是下列类型中，才可以设置为 HP 运算：

"Convolution", "Deconvolution", "InnerProduct", "LSTM", "Pooling", "RNN", "PSROIPooling", "ROI Pooling", "MatMul", "DepthwiseConv", "PassThrough", 其他的层类型不能设置为 HP 运算。

- 将当前 Layer 指定为 CPU 运算，在右侧 Property 视图中的 Implement，选择了客户自定义插件后，工具会做下面两个操作：
 - 如果当前选择的层不是 Proposal 或者 Custom，工具会自动给层名添加_cpu 字段，
 - 工具会把层名和当前选择的客户自定义插件的名字，写到 rtcfg 文件中，如图 5-129。



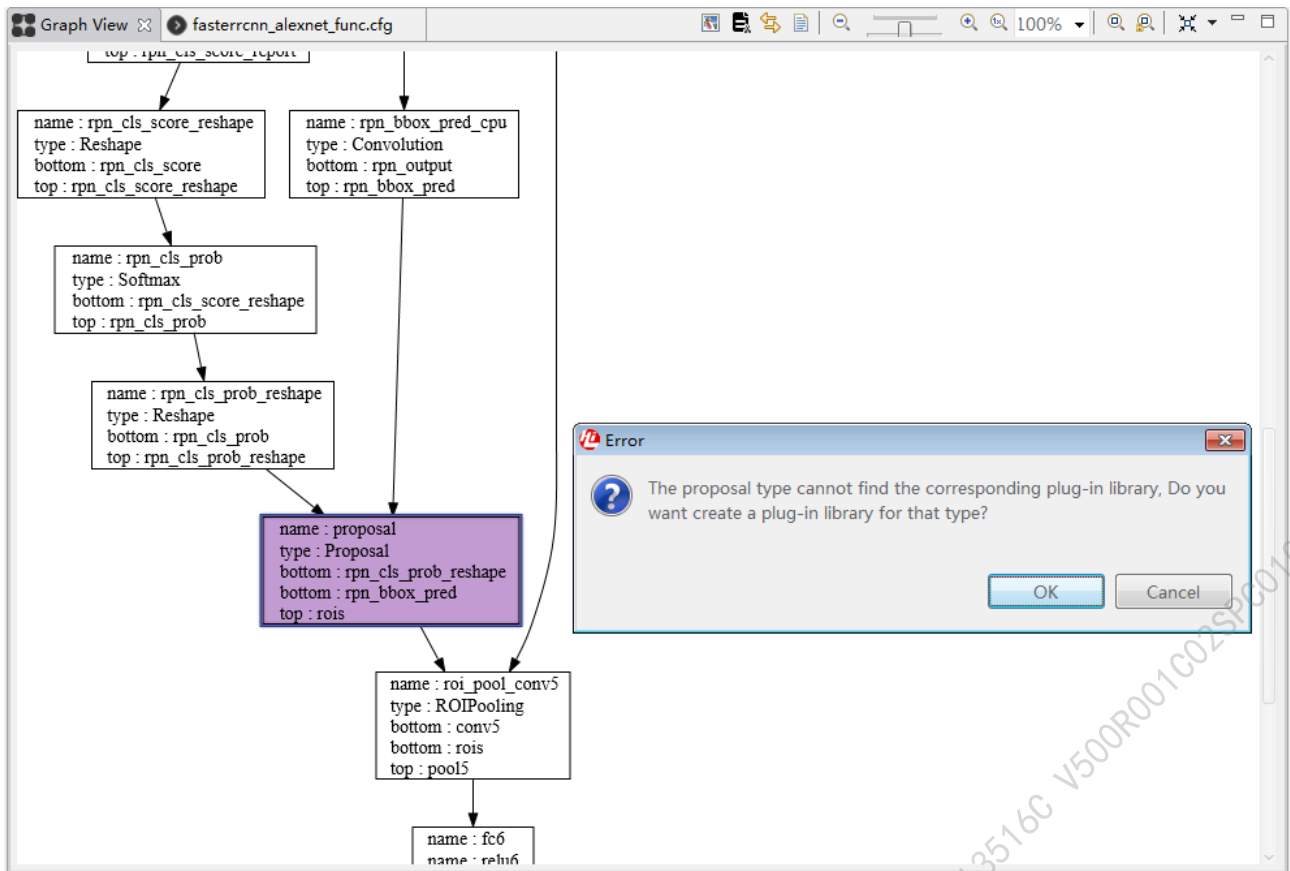
图5-129 将当前 Layer 指定为 cpu 运算



- 让客户自动创建插件，当用户选择 Proposal 层时，此时如果用户自定义的插件目录中没有 Proposal 实现的插件库，会弹出是否需要创建插件库的提示如图 5-130。

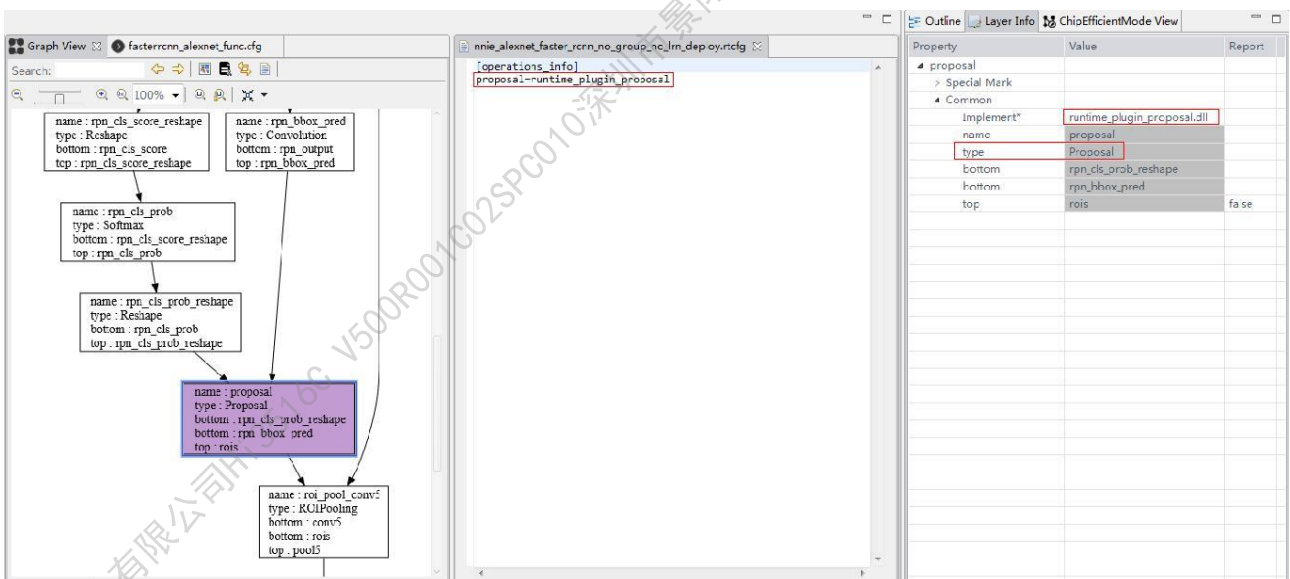


图5-130 提示是否需要创建 Proposal 类型的插件库



- 让客户自动创建插件，当用户选择 Proposal 层时，此时如果用户自定义的插件目录有 Proposal 实现的插件库，会直接选中一个存在的插件库，并把该插件库写到 rtcfg 文件中，并且 proposal 不能设置高精度，如图 5-131。

图5-131 打开 proposal 层的显示

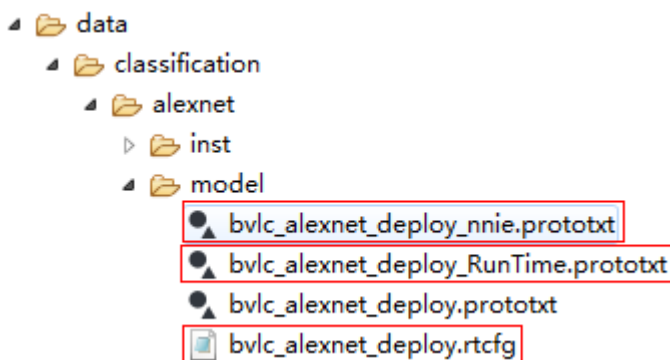




5.8.3 生成 Runtime wk 文件

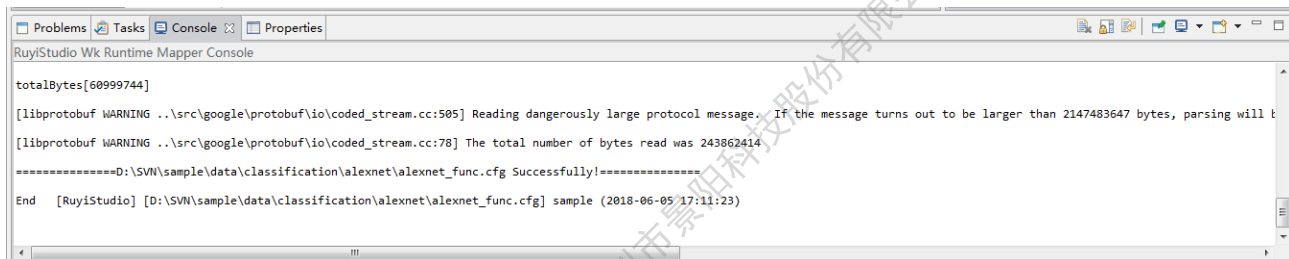
配置好 cfg 之后，点击左上角工具栏中的 Make WK 按钮, 工具会将当前的 xxx.cfg 传给 runtime_mapper，还需传入 NNIE prototxt、runtime prototxt 和 rtcfg 文件，NNIE prototxt 和 runtime prototxt 都是在运行的时候会生成，这些要传入的文件都和 prototxt 在同一个目录。如图 5-132 所示：

图5-132 运行 Runtime 所需要传入的文件



转化过程会在控制台显示，转化成功或失败后会在 Project Explorer 中的当前工程的目录下生成多个 dot 文件及编译过程中的 debug 和 error 对应的 log，如图 5-133 所示。

图5-133 运行 Runtime 的界面



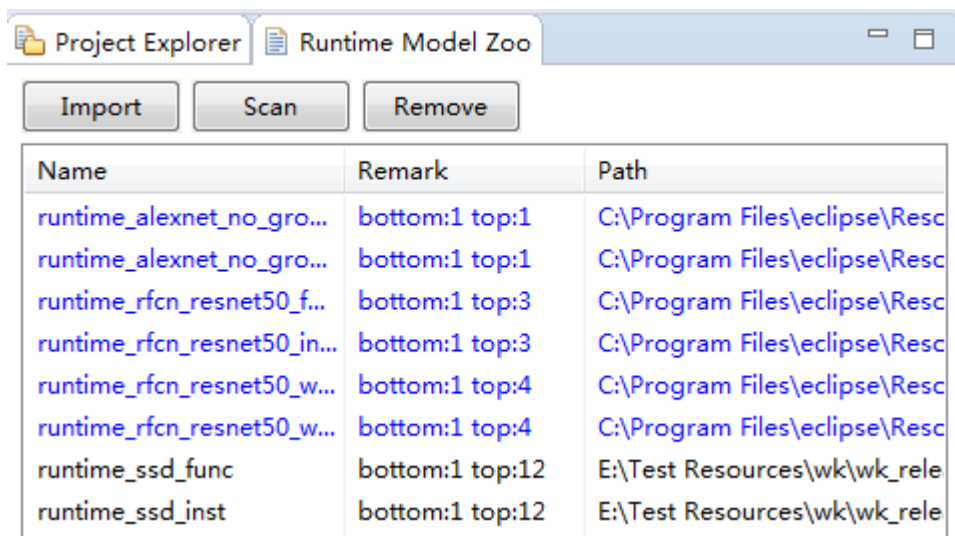
5.9 Runtime 模型管理功能

5.9.1 概述

RuyiStudio 提供了 Runtime 模型管理器功能，可以将 runtime 工程下使用 WK 编译功能生成的 wk 文件统一进行管理，并用于其他的功能，如模型级联编辑功能。

当 RuyiStudio 启动时，Runtime 模型管理器会同时启动，并与工具的工程管理器（Project Explorer）同处在左侧的边栏，以页签形式显示。用户点击 Runtime Model Zoo 页签后，即可从工程管理器切换到 Runtime 模型管理器。如图 5-134 所示：

图5-134 Model Zoo 视图



显示 Runtime 模型管理器时，工具会显示预置在 Ruyi Studio 工具安装目录下的模型（以蓝色文字显示）。模型的名称，其 top/bottom 接口的数目会被显示。

5.9.2 选择文件导入模型

点击 Runtime 模型管理器视图上的 Import 按钮，并在随后弹出的文件选择对话框中选择需要导入的 wk 模型文件（可以多选），可将它们导入到 Runtime 模型管理器。选择完毕后点击 OK 按钮，工具将解析 wk 模型并将其列在列表中（以黑色的文字显示）。

请注意，模型文件名不可重复，即使它们处于不同的目录下。尝试导入一个同名模型文件，工具会告警并跳过。

5.9.3 扫描文件夹导入模型

点击 Runtime 模型管理器视图上的 Scan 按钮，并在随后弹出的文件夹选择对话框中选择一个目录，工具将扫描这个目录及其子目录（全深度），寻找 wk 模型文件，并尝试导入。解析成功的 wk 模型文件会在列表中显示，解析失败，或文件名与列表中已有的模型文件相同，则会跳过该模型文件。

5.9.4 删除导入的模型

从 Runtime 模型管理器中选择文件（支持按住 Ctrl 或 Shift 键进行多选），并点击 Runtime 模型管理器视图上的删除按钮，可以删除由用户导入的模型文件。工具预置的模型文件无法被删除。

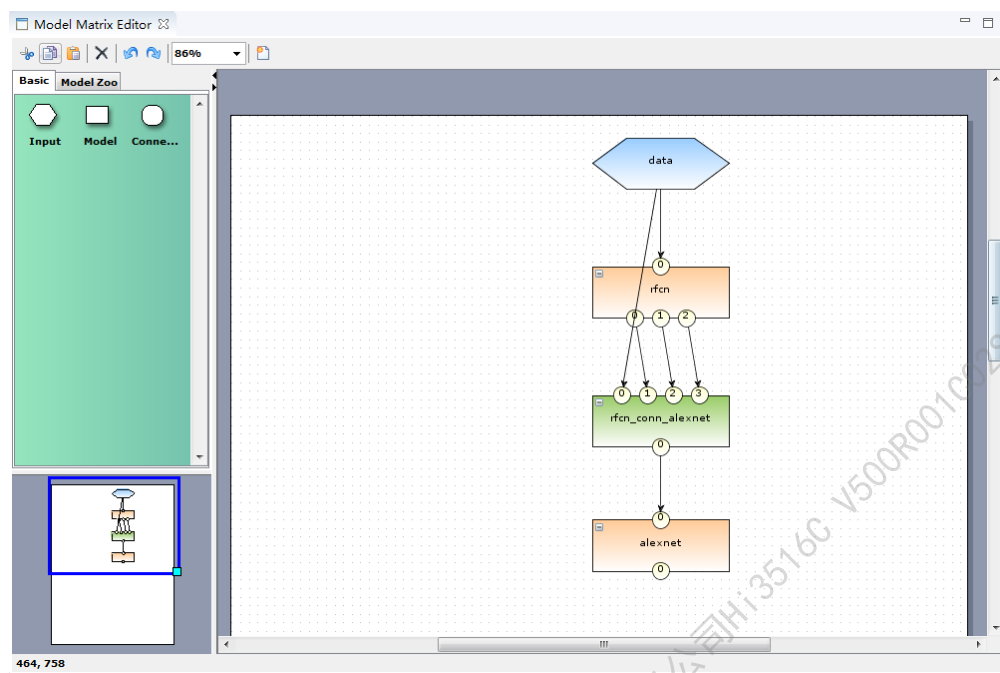


5.10 模型级联编辑功能

5.10.1 概述

工具提供了模型级联编辑功能，允许用户将多个模型进行业务上的组合，形成复杂的业务逻辑。编辑器界面如图 5-135 所示：

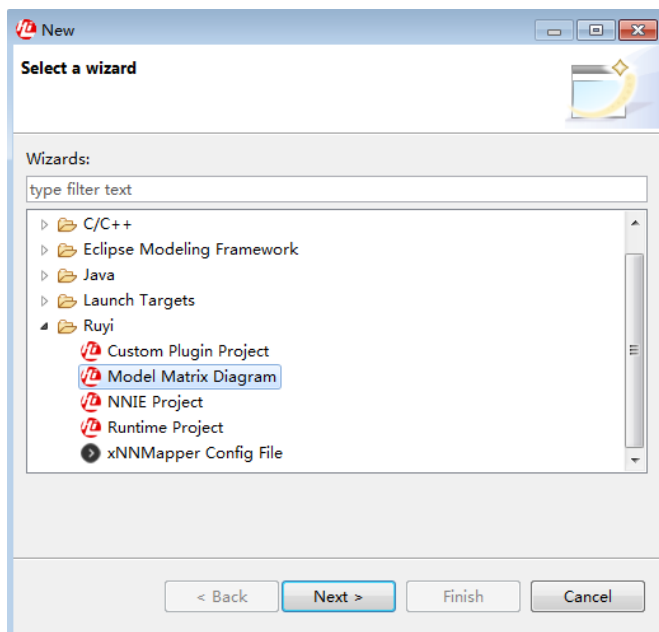
图5-135 模型级联关系图编辑器



5.10.2 新建模型级联关系图

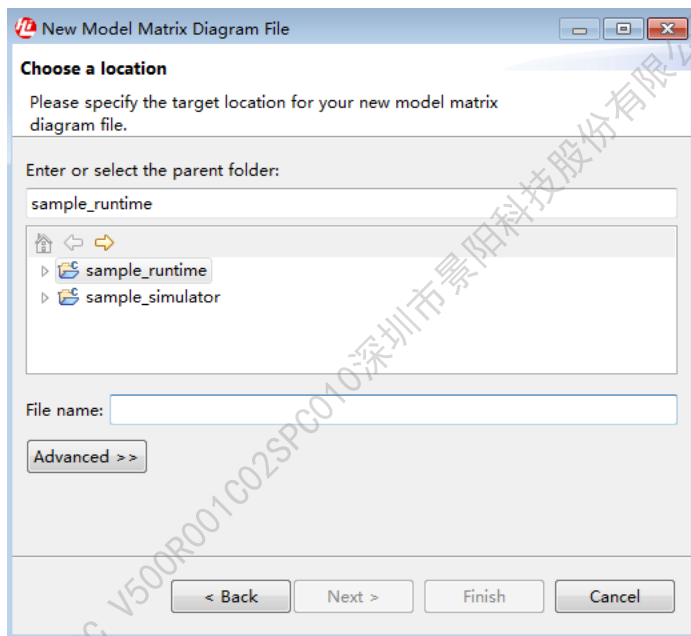
如意工具提供了向导帮助用户创建模型级联关系图。开启文件菜单（File）中的 New/Other 选项，并选中 Ruyi Studio 文件夹下的 Model Matrix Diagram（如图 5-136 所示），即可开启该向导。

图5-136 模型级联关系图向导



在文件基础信息页面，选择模型级联关系图的存放路径，并指定文件名。请注意，模型级联关系编辑功能仅对 runtime 工程有效，在其他的工程中放置该文件没有意义。编辑完成后，点击 Finish 按钮，工具将生成一个空白的模型级联关系图，并打开编辑器（样例如图 5-137 所示）。

图5-137 模型级联关系图基础信息编辑

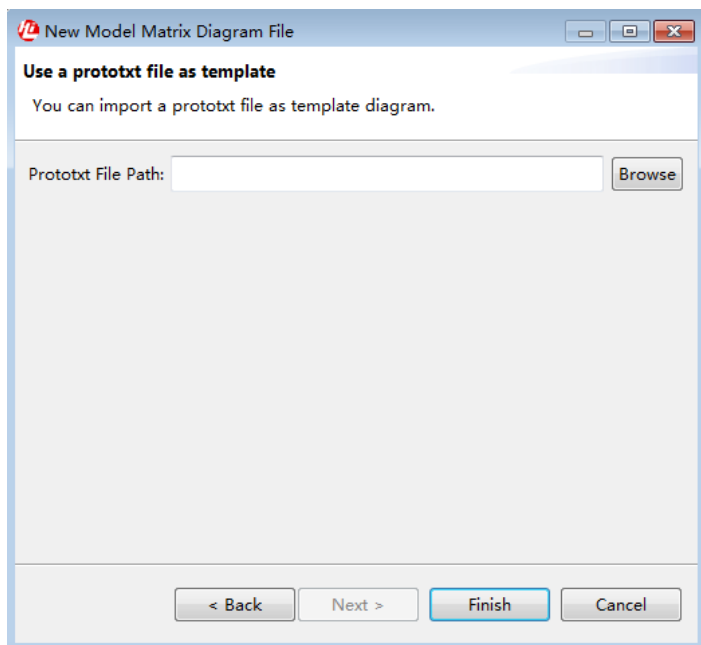


如果用户需要从已有的模型组描述文件（modelgroup）中生成新的模型级联关系图，可以在刚才的向导中编辑基础信息后，点击 Next 按钮开启描述文件导入页面。在该页



面中，指定需要导入的描述文件，即可完成导入。工具打开编辑器，并还原描述文件描述的关系图（样例如图 5-138 所示）。

图5-138 导入模型组描述文件



5.10.3 模型级联关系图编辑

在模型级联关系图中有三个主要元素：输入数据，模型和连接器。

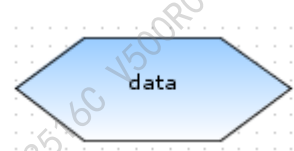
- 其中输入数据用于提供整套业务流程的原始输入数据。
- 模型用于处理数据。
- 连接器用来转换和分配数据。

模型和连接器具备接口，用于与其他模型/连接器对接以传递数据，其中 top 接口传出数据，bottom 接口接受数据。

从模型级联关系图编辑器左侧的工具栏，可以添加这三种主要元素的图元：

- 拖动 Input 图标到关系图中，可以添加一个输入数据图元。添加后，工具以蓝色六边形来显示它。

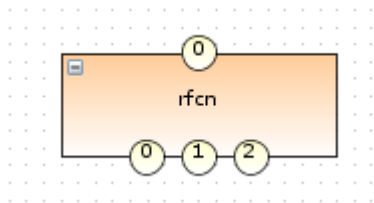
图5-139 输入数据图元



- 拖动 Model 图标到关系图中，可以添加一个模型图元。添加时，用户需要指定其代表哪个模型，可以从弹出的文件弹出对话框中选择。添加后，工具以橙色的方

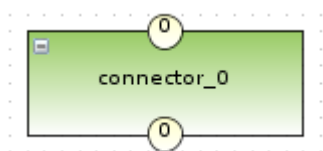
框来显示它，并在它的上边栏显示器所有的 **bottom** 接口，在下边缘上显示所有的 **top** 接口。接口以 0 起始从左到右编号。

图5-140 模型图元



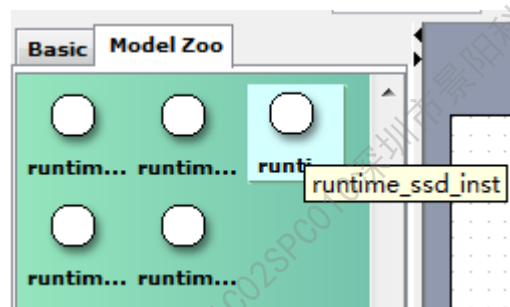
- 拖动 Connector 图标到关系图中，可以添加一个连接器图元。添加后，工具以绿色的方框来显示它，并为其添加 1 个 **bottom** 接口和 1 个 **top** 接口，分别显示在方框的上边缘和下边缘。

图5-141 连接器图元



除此之外，编辑器还提供了从 Model Zoo 快捷导入已有模型的功能。将左侧的工具箱切换到 Model Zoo 页签，选择需要添加的模型拖入即可。由于模型的名称一般较长，若无法看清模型全名，可单击选中模型，之后工具将弹出气泡提示该模型的完整名称。

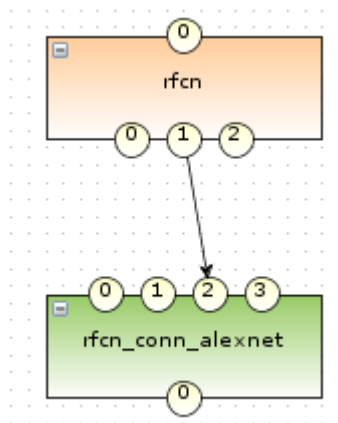
图5-142 从 Model Zoo 添加模型



添加元素后，可以进行接口连接，以表示数据流向。当用户的鼠标停留在输入数据图元或 **bottom** 接口图元中心时，图元将显示绿色边框，此时按住鼠标左键，可以拖出连线，之后将鼠标指针移动到目标的 **bottom** 接口图元中心松开（也有绿色边框提示）即可完成有效的数据传递连接。

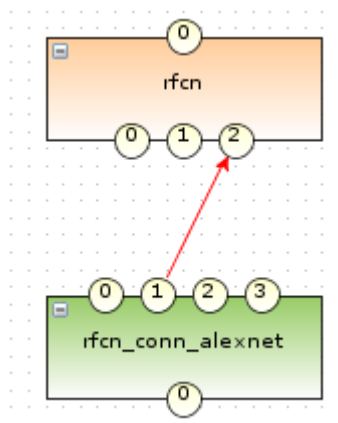


图5-143 有效连线



只有输入数据图元到 bottom 接口图元，以及 top 接口图元到 bottom 接口图元的连接才有效（显示为黑色连线），其余种类的连线均无效（显示为红色连线）。

图5-144 无效连线



5.10.4 编辑元素属性

工具提供了级联关系图上各种属性的设置界面，它位于工具右侧的 Properties 视图中。



图5-145 级联关系图属性视图

[Matrix] rfcn_alexnet	
Group / Key	Value
Common	
Name	rfcn_alexnet
Options	
Priority	1
Max Temp Buffer	3072
Align Size	16

用户使用鼠标单击图的空白部分，可以编辑关系图的属性：

- 名称 (Name)：指定关系图的名称。接受长度 3-64 个字符，且仅包含数字，字符，下划线的字符串；
- 优先级 (Priority)：指定关系图的执行优先级，输入 0-4 的数字，数字越小优先级越高；
- 最大缓存大小 (Max Temp Buffer Size)：指定整个级联关系需要多大的缓存来运行。输入 0~4095 的数值，单位为 MB；
- 对齐大小 (Align Size)：指定关系图的数据在内存中的对齐方式。可选 16/32 字节对齐。

选择模型图元，可以编辑模型的属性：

- HiRT 批次号 (HiRT Batch Number)：指定 HiRT 内的模型组批次号。填写范围 1-256 的数字。

选择连接器图元，可以编辑连接器的属性：

- 名称 (Name)：指定连接器的名称。接受长度 3-64 个字符，且仅包含数字，字符，下划线的字符串；
- Bottom 接口数 (Bottom)：指定连接器有多少个 bottom 接口，范围 1-16。设置后，工具将自动更新连接器图元，添加或删除接口图元到设置的数值。
- Top 接口数 (Top)：指定连接器有多少个 top 接口，范围 1-16。设置后，工具将自动更新连接器图元，添加或删除接口图元到设置的数值。

选择 top 接口图元，可以编辑 top 接口的属性：

- 名称 (Name)：指定 top 接口的名称。接受长度 3-64 个字符，且仅包含数字，字符，下划线的字符串。只有连接器的 top 接口图元才能设置该属性，模型的 top 接口图元只能查看而不能编辑。
- 上报属性 (Report)：指定该 top 接口是否向上报告。只有模型的 top 接口图元才能设置该属性，连接器的 top 接口图元不显示该属性。
- 形态 (Shape)：指定该 top 接口的形态，具备 Num/Channel/Height/Width 四个子值，范围 0-255。只有连接器的 top 接口图元才能设置该属性，模型的 top 接口图元不显示该属性；



- 数据类型 (Type): 指定该 top 接口接受的数据类型。用户可以从 6 中数据类型中选择一个。只有连接器的 top 接口图元才能设置该属性, 模型的 top 接口图元不显示该属性;

选择 bottom 接口图元, 可以编辑 bottom 接口的属性:

- 名称 (Name): 指定 bottom 接口的名称。接受长度 3-64 个字符, 且仅包含数字, 字符, 下划线的字符串。只有连接器的 bottom 接口图元才能设置该属性, 模型的 bottom 接口图元只能查看而不能编辑。
- 帧依赖数目 (FrameDepend): 指定该接口接受图像数据时, 至少需要多少帧数据。

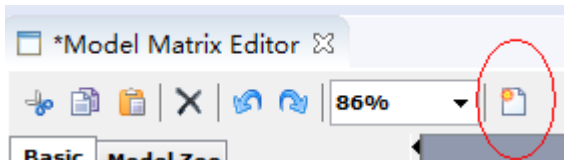
5.10.5 保存级联关系图

点击如意工具工具栏上的保存  或全部保存  按钮, 可以将编辑好的级联关系图进行保存。

5.10.6 导出模型组描述文件

点击级联关系图编辑器上的描述文件导出按钮, 可以将编辑好的级联关系图转换为模型组描述文件。在弹出的文件保存对话框中选择保存路径后点击 OK 按钮, 即可完成保存。

图5-146 模型组描述文件导出按钮的位置



5.11 客户自定义插件

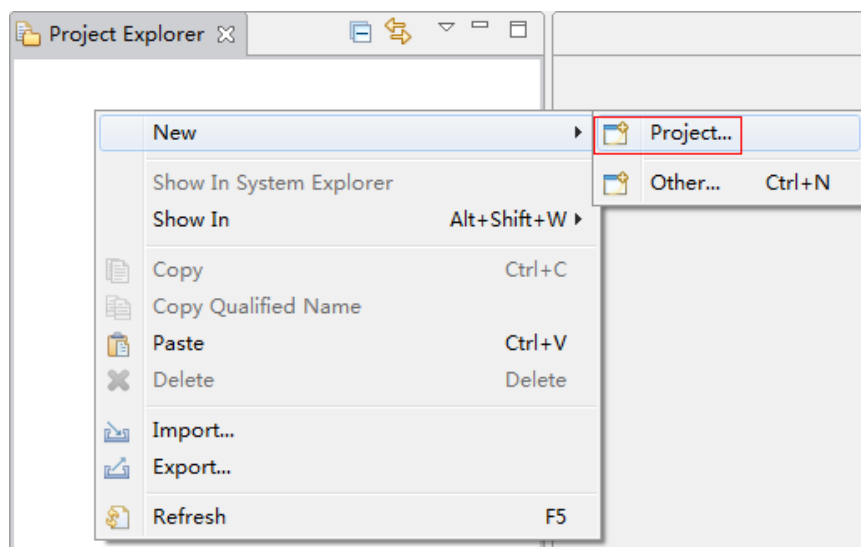
提供用户自己创建自定义插件库, 并且把生成的库添加到用户自定义路径下, 供 runtime 调用。

5.11.1 创建工程

步骤 1. 点击鼠标右键, 弹出图 5-147:



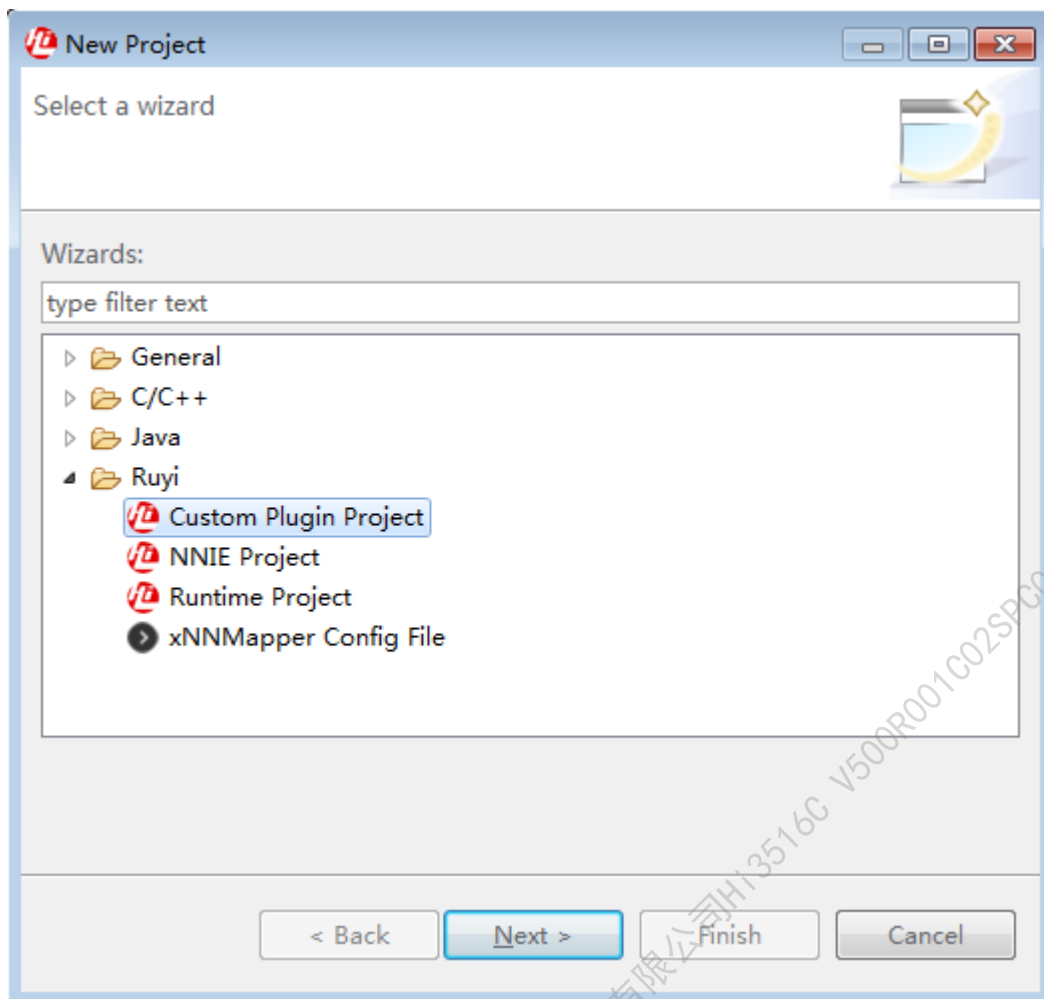
图5-147 通过鼠标右键创建 Project



步骤 2. 选择 NNIE project，点击 Next，如图 5-148 所示。



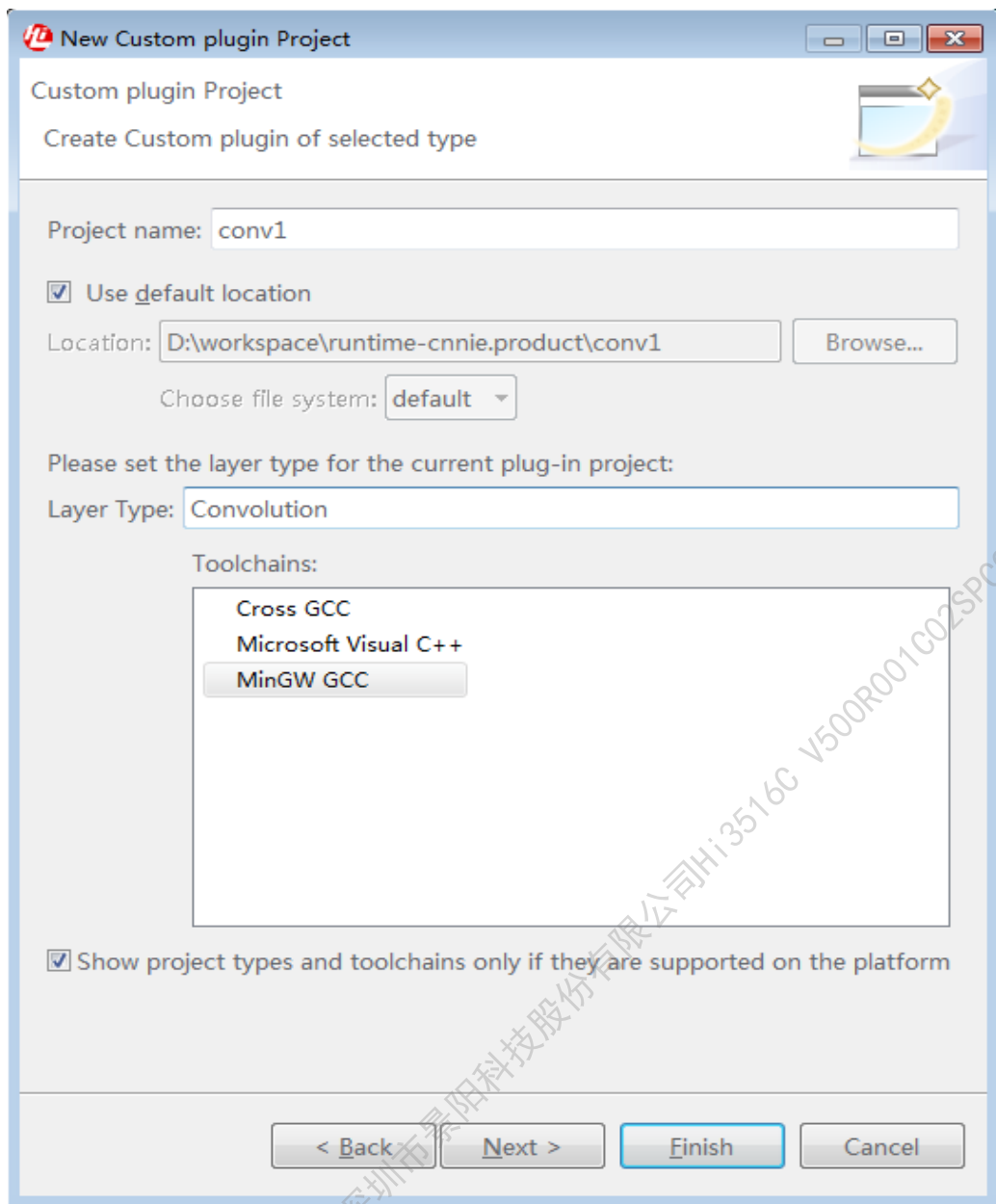
图5-148 选择 Custom plugin Project



步骤 3. 设置 Project 名称，设置 layer type，选择 MinGW GCC，点击 Finish 完成工程创建；注意只有当 MinGW 安装好之后才会在 Toolchains 里显示 MinGW GCC，如图 5-149 所示。



图5-149 设置 Project 和 layer type 的名字并确认创建工程



----结束



注意

创建自定义插件的编译链只支持 minGW GCC 编译链。

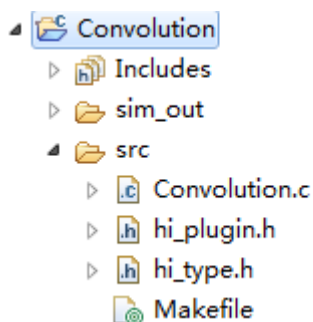


5.11.1.1 工程文件说明

创建后的工程如图如图 5-150 所示。包含两个空文件夹和一个配置文件：

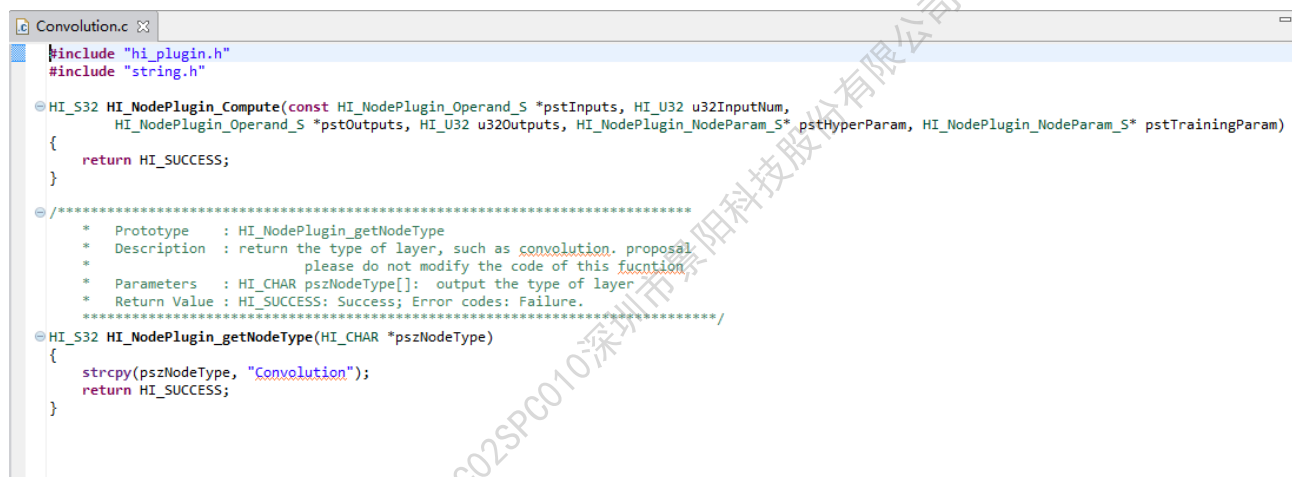
- *.c: 用于实现接口的文件；
- hi_plugin.h: 接口的头文件；
- hi_type.h: 自定义的头文件；
- Makefile: 用于生成该项目的文件。

图5-150 创建工程后的工程视图



其中*.c 文件已经实现了 HI_NodePlugin_getNodeType 的接口，HI_NodePlugin_Compute 需要客户实现，生成的*.c，如图 5-151 所示。

图5-151 生成的*.c 文件

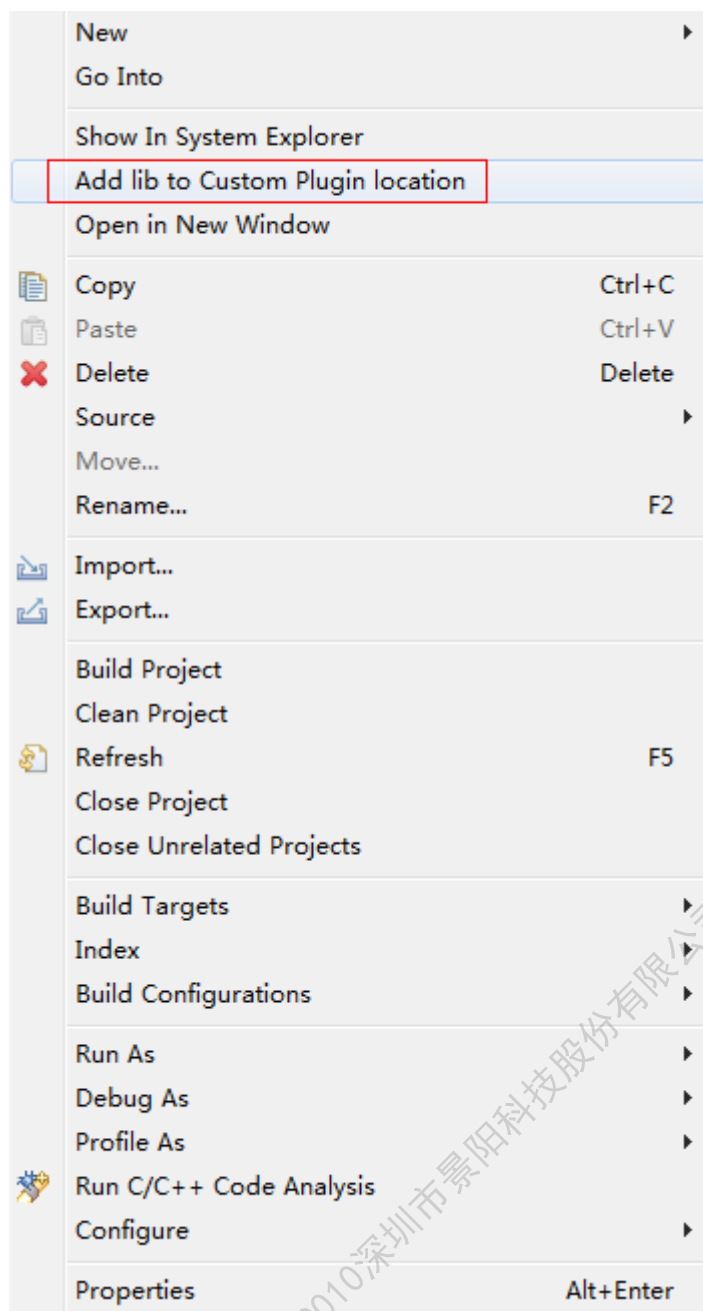


5.11.2 添加客户自定义插件到客户自定义插件路径下

步骤 1. 选择项目，点击鼠标右键，选择 Add lib to Custom plugin location，如图 5-152 所示：



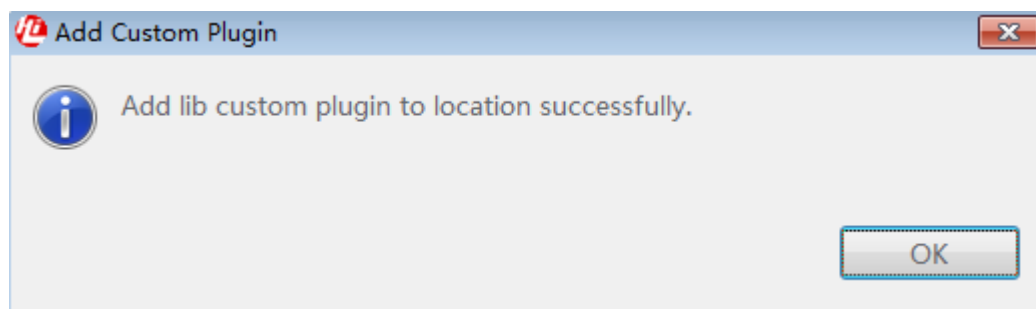
图5-152 通过鼠标右键选择 Add lib to Custom plugin location



弹出选择 dll 的对话框，选择项目生成的 dll 文件。添加成功后会弹出一个提示框。如图 5-153:



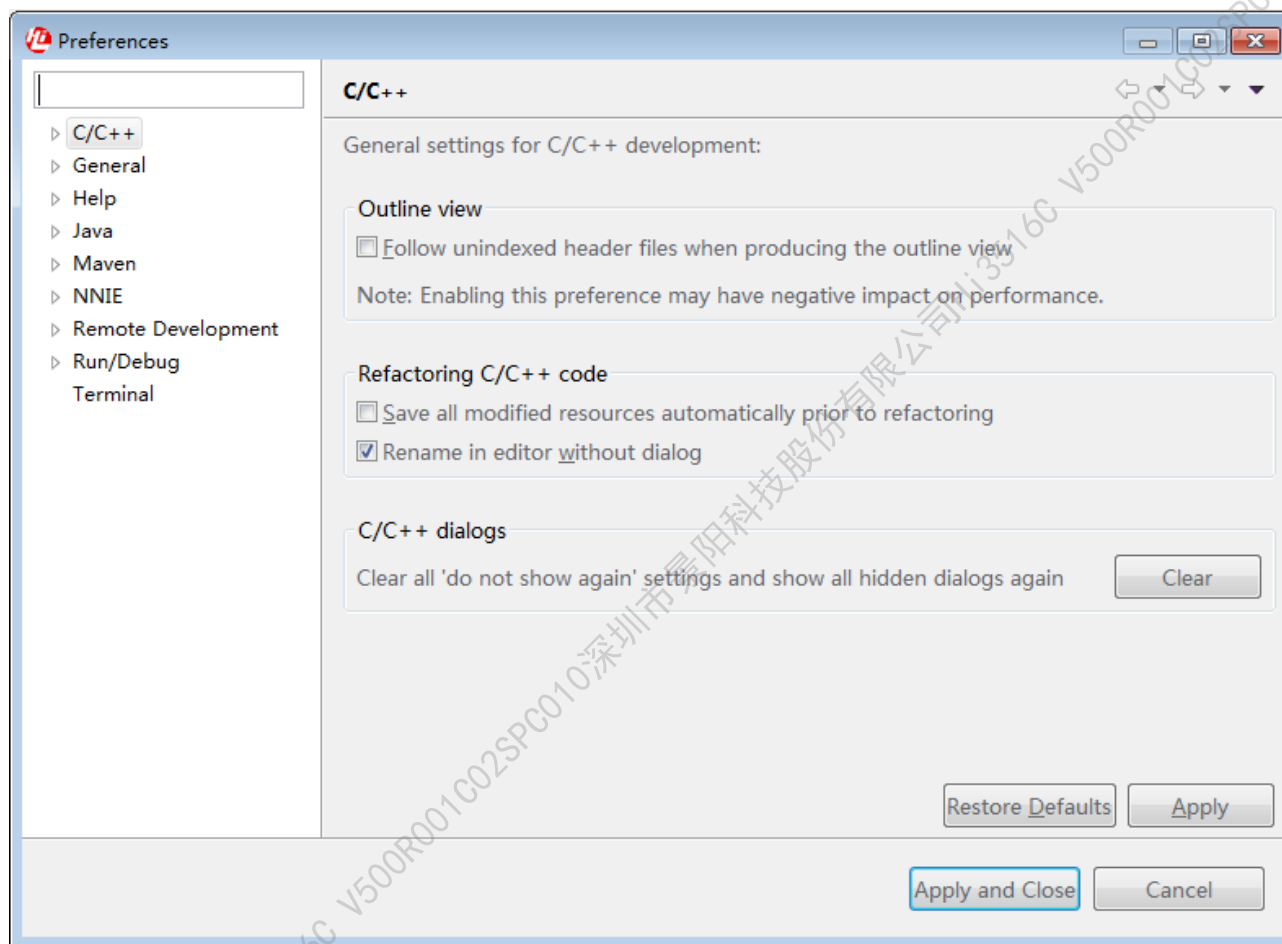
图5-153 添加 dll 成功之后的提示



5.11.3 设置客户自定义插件路径

步骤 1. 依次点击 Window→Preferences，弹出图 5-154：

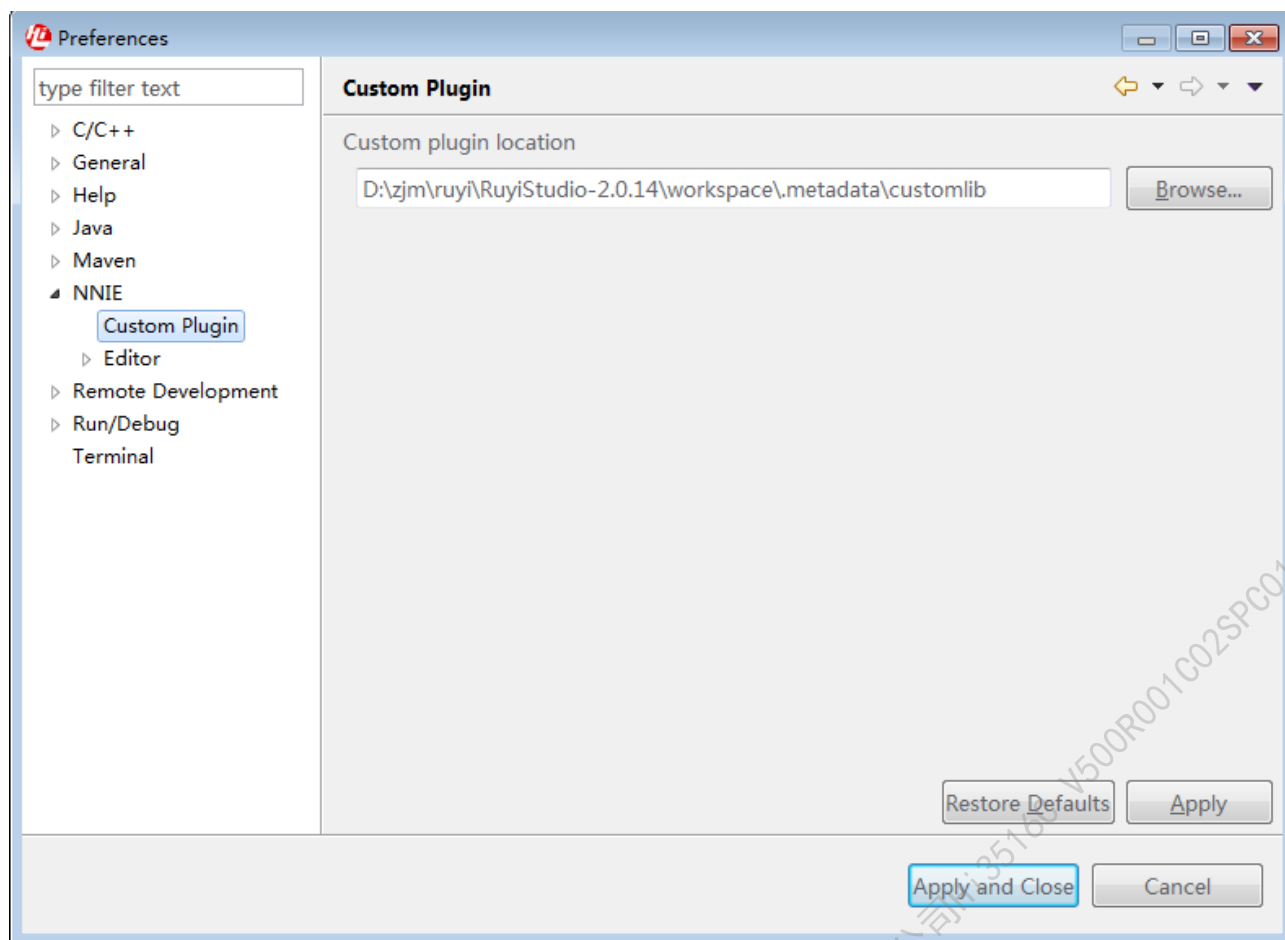
图5-154 打开 preferences 页面



步骤 2. 选择 NNIE->Custom Plugin。如图 5-155：



图5-155 打开设置 Custom Plugin 页面



步骤 3. 点击 Browse 按钮，设置路径，设置完路径后点击 Apply 或者 Apply and Close 就完成了设置。

----结束

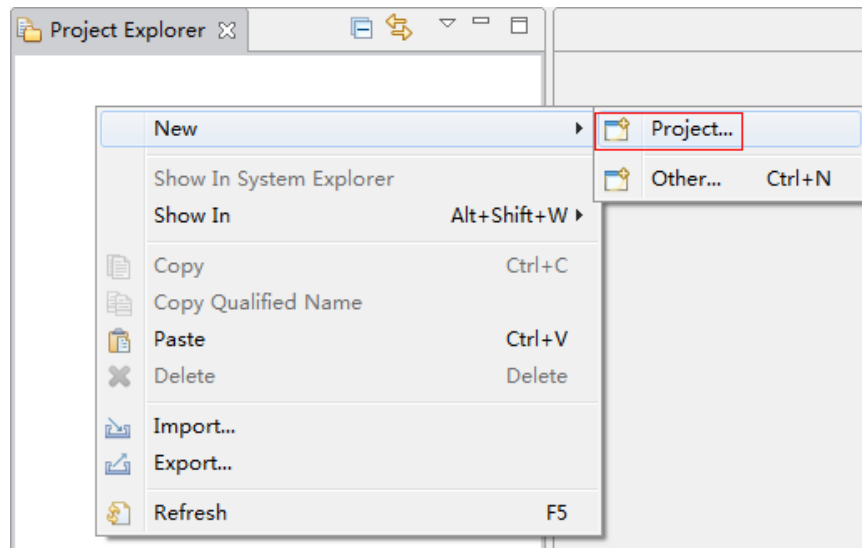
5.12 创建 Runtime 工程和切换芯片

5.12.1 创建 runtime 工程

步骤 1. 工程创建，通过 Project Explorer 视图中鼠标右键创建 Project，如图 5-156 所示。



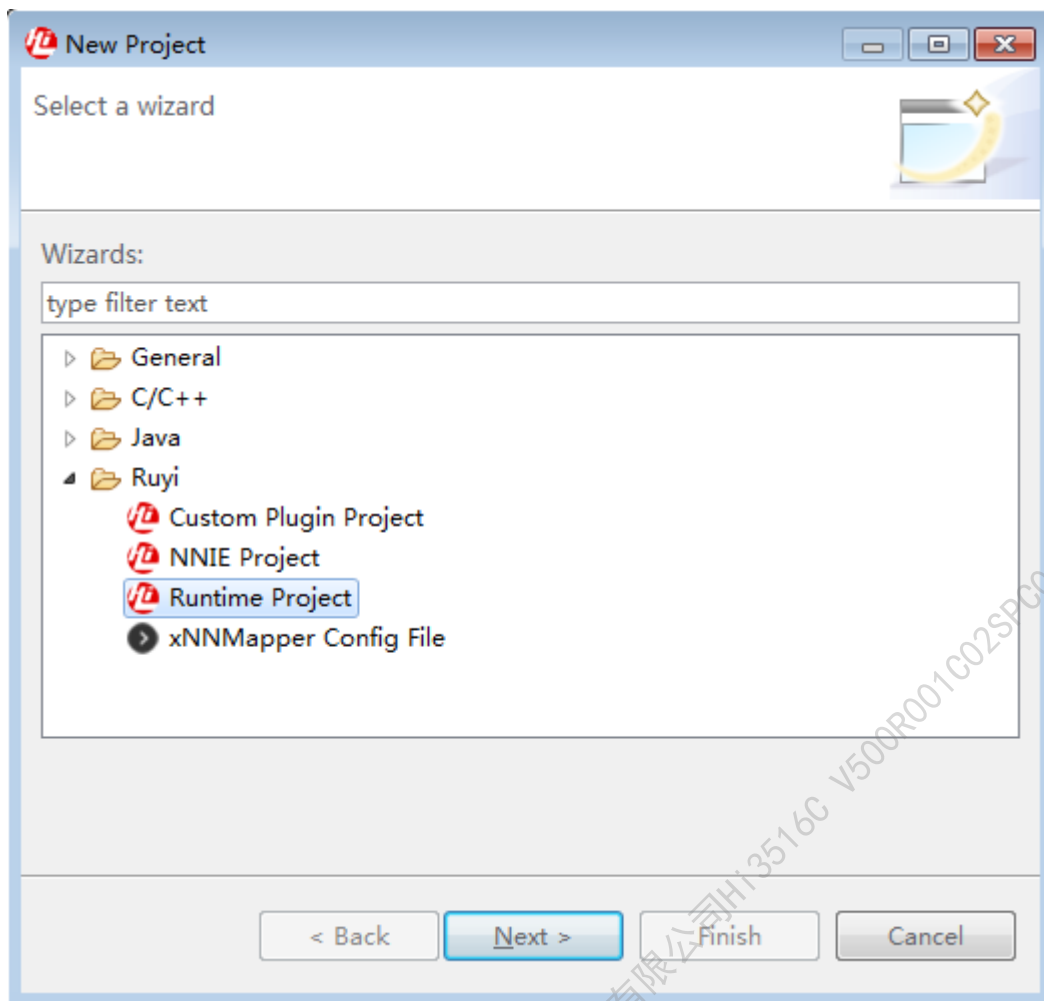
图5-156 通过鼠标右键创建 Project



步骤 2. 选择 Runtime project，点击 Next，如图 5-157 所示。



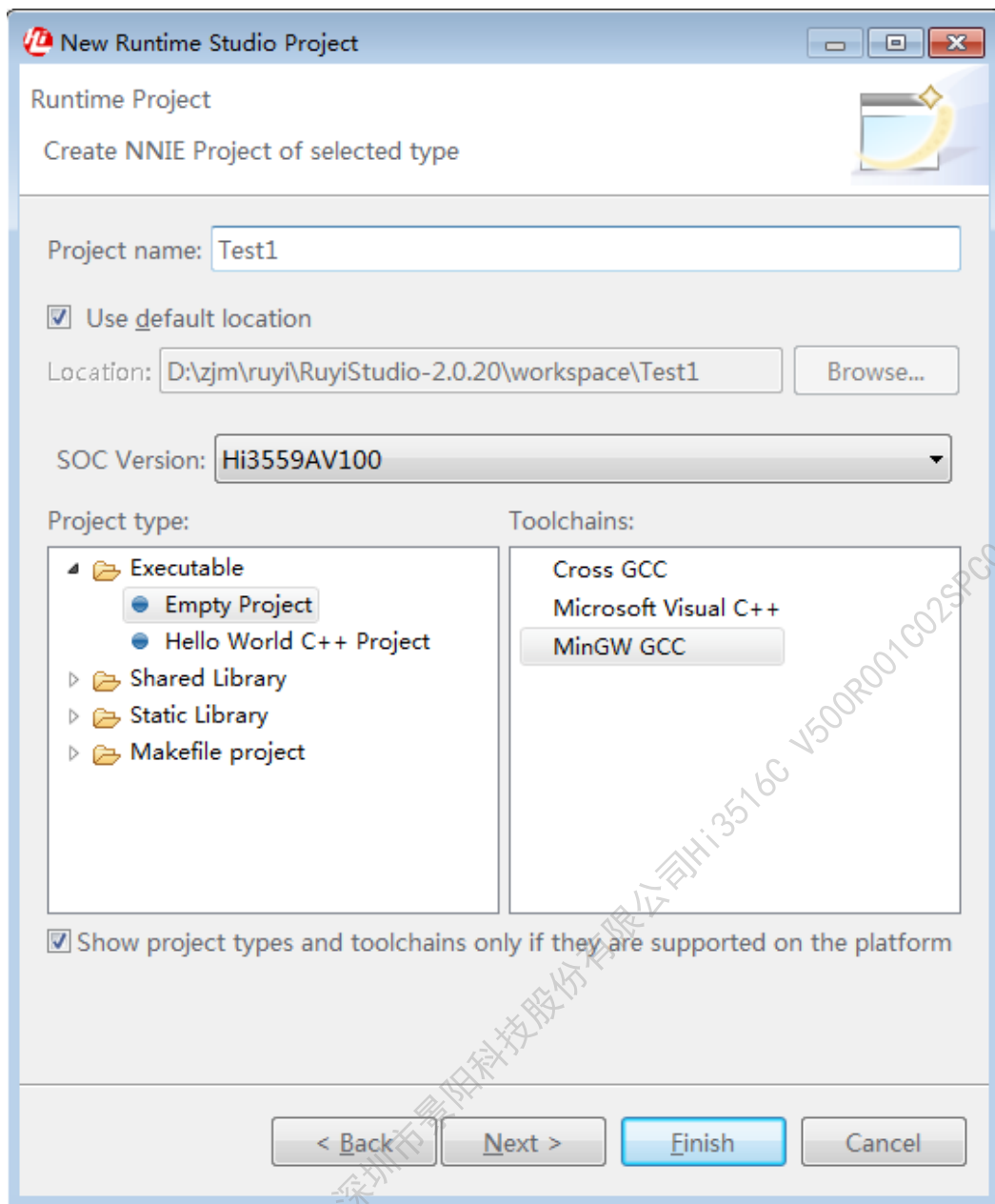
图5-157 选择 Runtime Project



步骤 3. 设置 Project 名称，选择 Hi3559AV100 芯片型号，选择 MinGW GCC，点击 Finish 完成工程创建；注意只有当 MinGW 安装好之后才会在 Toolchains 里显示 MinGW GCC，如图 5-158 所示。



图5-158 设置 Project 的名字并确认创建工程



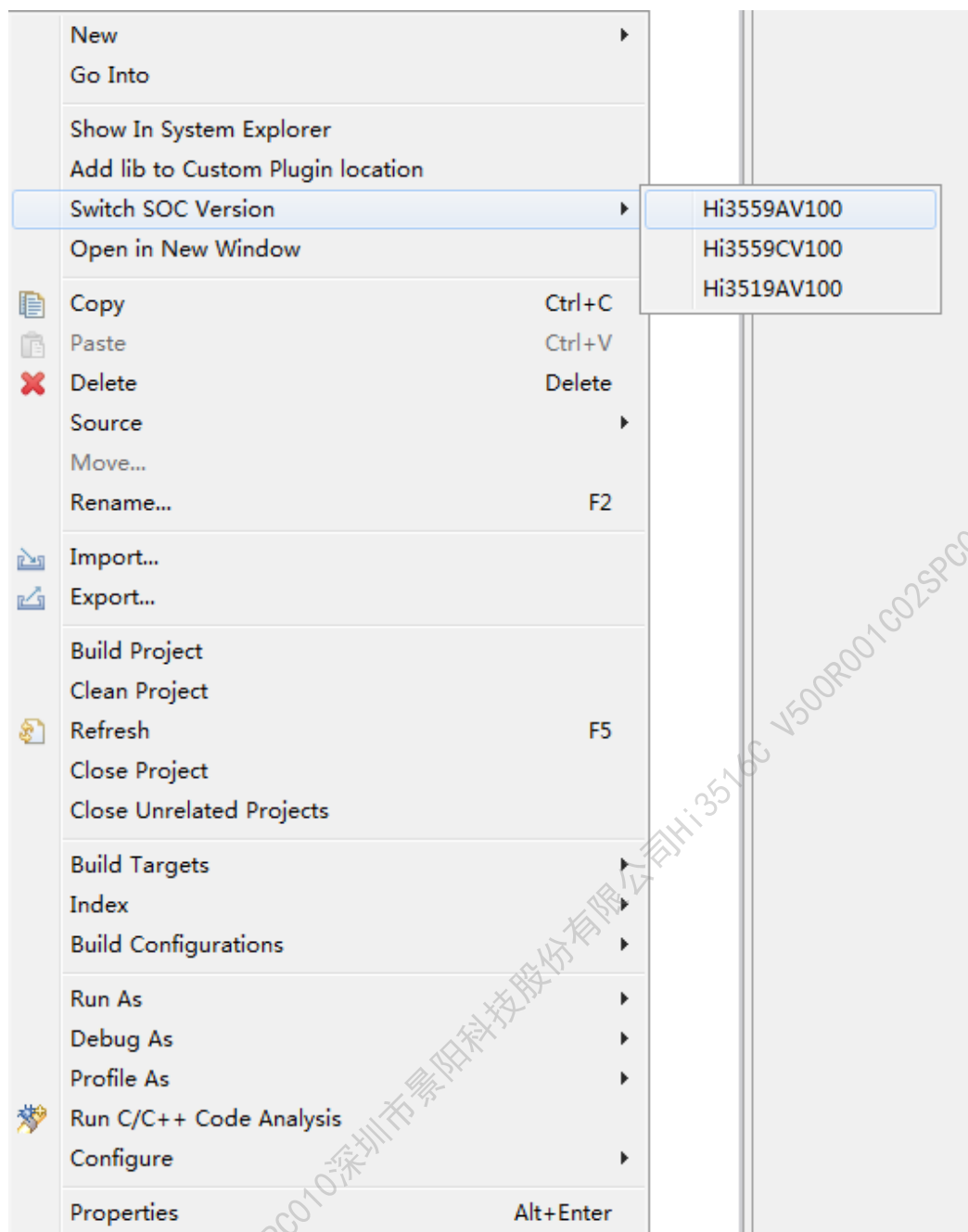
----结束

5.12.2 切换芯片

选择项目，点击右键弹出，菜单栏，选择 Switch SOC Version 菜单，会显示芯片列表，如图 5-159:



图5-159 切换芯片



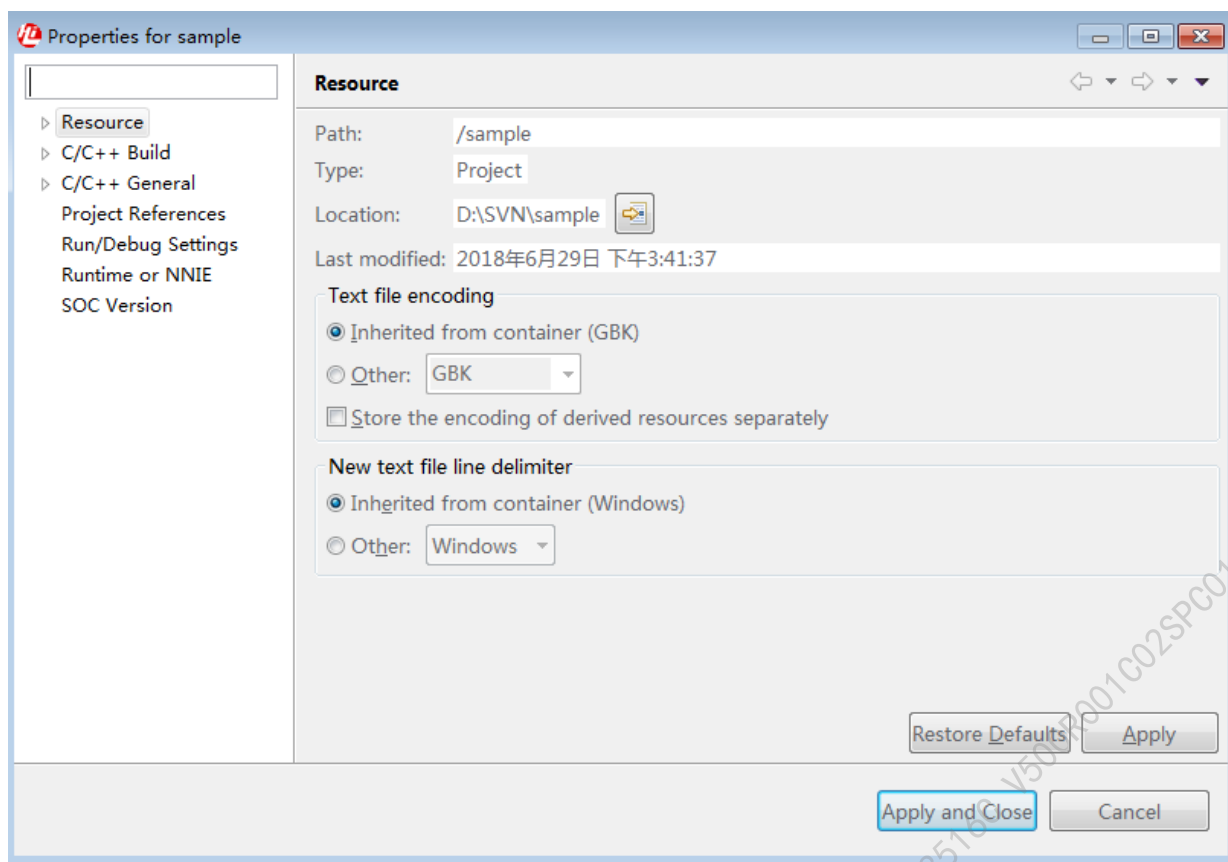
选择对应的芯片，如果该项目是 NNIE 项目，则该项目会切换到当前的芯片。

5.12.3 查看项目 Model 和芯片型号

步骤 1. 选择项目，点击鼠标右键，在弹出的菜单栏选择 Properties，弹出对话框如图 5-160:



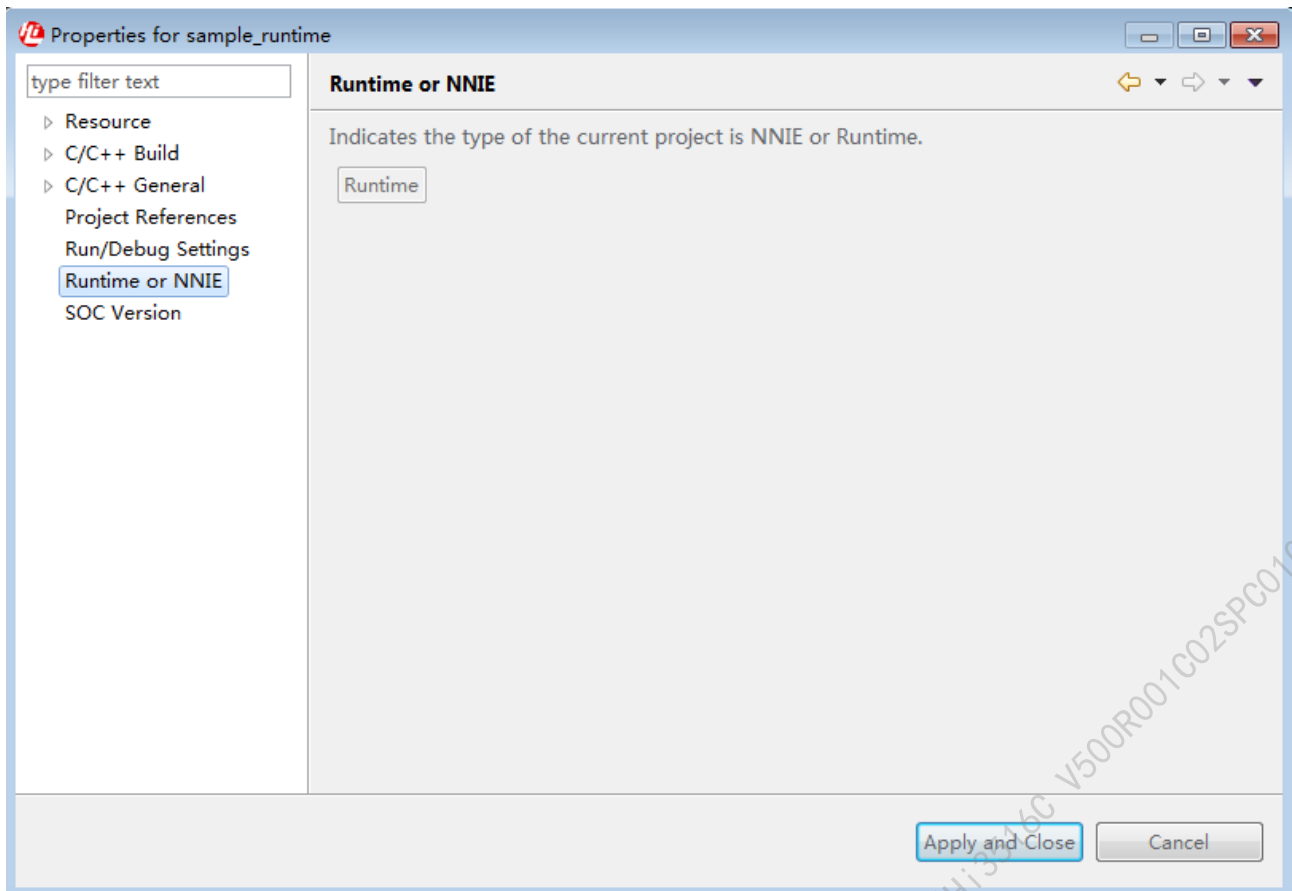
图5-160 属性页



步骤 2. 选择 Runtime or NNIE 查看当前项目所属 Model，如图 5-161：



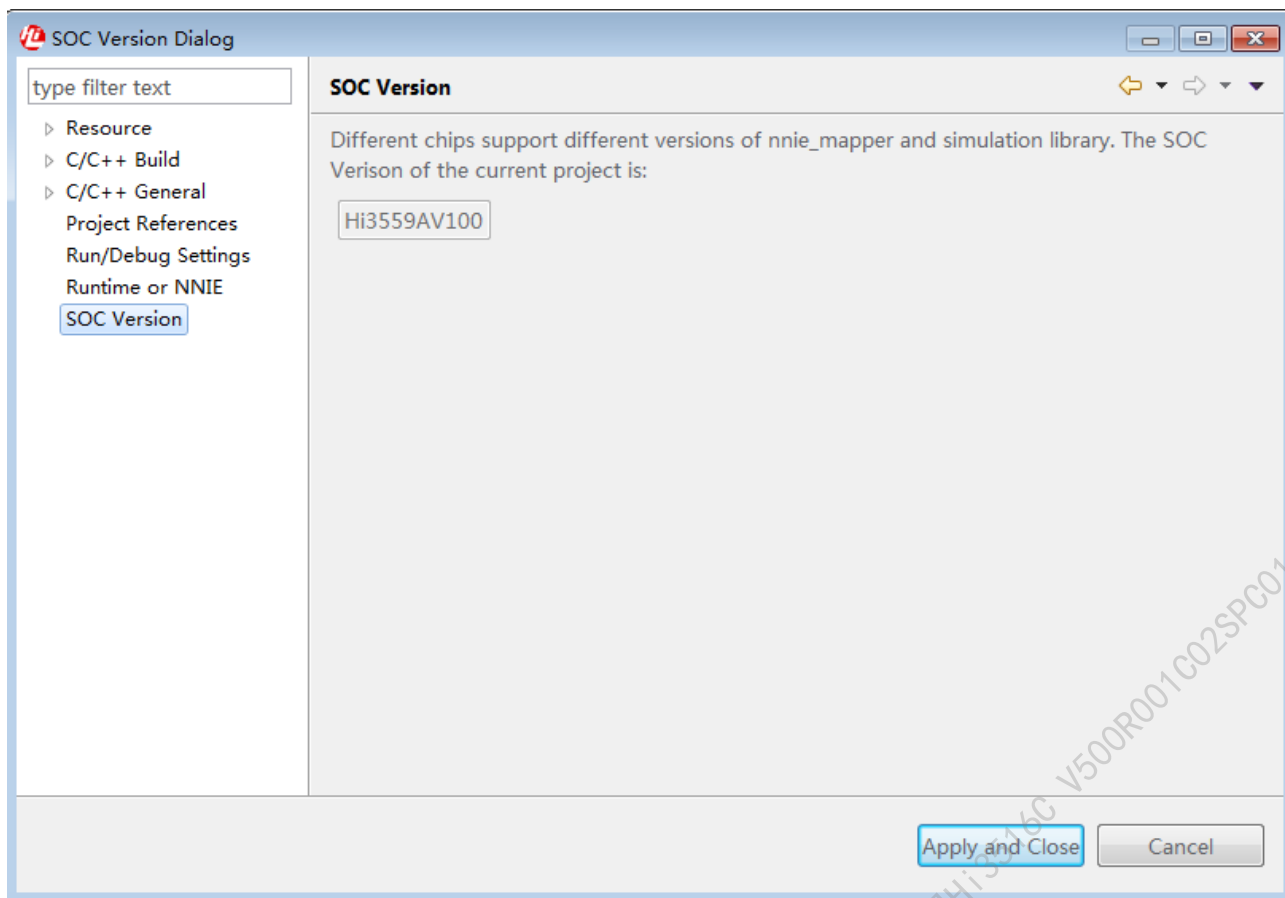
图5-161 项目 Model 页



步骤 3. 选择 SOC Version 查看当前项目的芯片型号如图 5-162。



图5-162 芯片型号页



----结束

5.13 FAQ

5.13.1 工具启动时一直卡在启动画面“Loading org.eclipse...”无法打开工具的解决办法

问题描述

工具启动时一直卡在启动画面，如[图 5-163](#)所示，我该怎么办？

图5-163 工具启动卡主无法打开的现象



解决办法

- 步骤 1. 首先，关闭工具；
- 步骤 2. 找到当前工具加载的 workspace 路径，找到 workspace 下的隐藏文件.matadata 文件，删除如下路径的.metadata\plugins\org.eclipse.e4.workbench\workbench.xml 文件；
- 步骤 3. 最后重启工具，如果在以上路径下找不到 workbench.xml，则删除整个.matadata 文件夹后再重启工具。

----结束

5.13.2 打印 caffe 中间输出或还原网络时提示 Layer xxx with type xxx is not supported, please refer to chapter 3.1.4 and FAQ of "HiSVP Development Guide.pdf" to extend caffe!

当前打印 caffe 中间输出和还原网络的功能提供的一键配置 python+caffe 的环境中的 caffe 并不支持 3.1.4.3 提到的 DepthwiseConv, RReLU, MatMul 等扩展层，而本文档提供的 bat 脚本的配置，只支持 caffe 的标准层和 PassThrough, Permute, ROI Pooling, PSROI Pooling, UpSample, Normalize 这些扩展层，如果涉及到扩展层，需要自行配置。参考 3.1.4 章节的描述，具体操作如下：（默认已经系统安装了 cmake，并且将 cmake 可执行文件所在路径添加到了系统环境变量中，默认已经安装了 VS，推荐使用 VS2015）

- 步骤 1. 从 <https://github.com/BVLC/caffe/tree/windows> 下载 caffe_windows.zip 包，解压到自定义目录；
- 步骤 2. 修改 scripts\build_win.cmd 文件，配置 VS 的版本号；

步骤 3. 修改 scripts\build_win.cmd 文件，依照自己的情况设置 CPU_ONLY，若不用到 GPU，则设置为 1，否则设置为 0，在本例子中，默认为 CPU_ONLY 为 1 的场景；

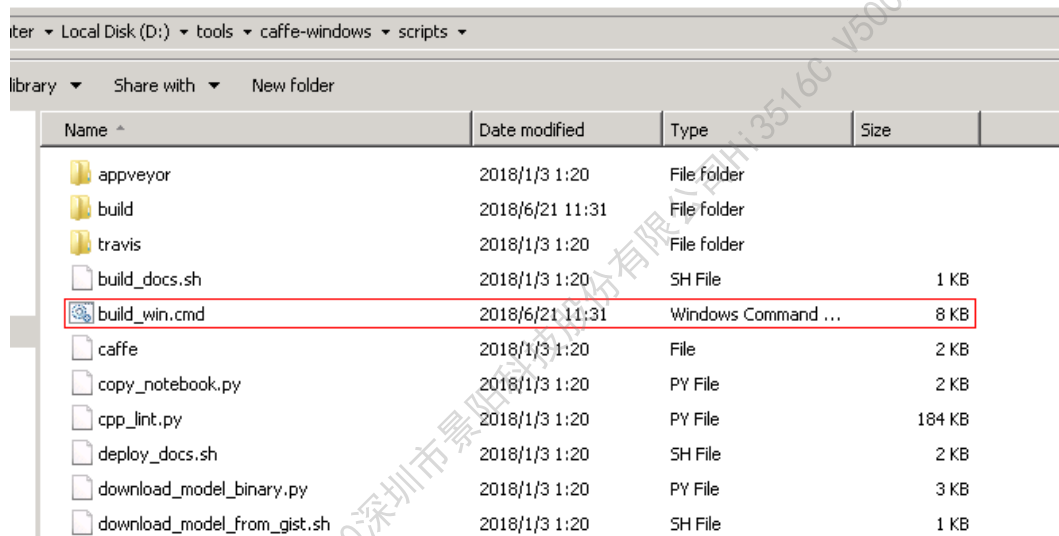
步骤 4. 配置 python 的版本号为 3，也是与我们安装的 python 版本号相对应；

图5-164 修改 build_win.cmd 文件中的部分配置

```
74 if NOT DEFINED WITH_NINJA set WITH_NINJA=0
75 :: Change to 1 to build caffe without CUDA support
76 if NOT DEFINED CPU_ONLY set CPU_ONLY=1
77 :: Change to generate CUDA code for one of the following GPU architectures
78 :: [Fermi Kepler Maxwell Pascal All]
79 if NOT DEFINED CUDA_ARCH_NAME set CUDA_ARCH_NAME=Auto
80 :: Change to Debug to build Debug. This is only relevant for the Ninja generator the Visual Studio
81 if NOT DEFINED CMAKE_CONFIG set CMAKE_CONFIG=Release
82 :: Set to 1 to use NCCL
83 if NOT DEFINED USE_NCCL set USE_NCCL=0
84 :: Change to 1 to build a caffe.dll
85 if NOT DEFINED CMAKE_BUILD_SHARED_LIBS set CMAKE_BUILD_SHARED_LIBS=0
86 :: Change to 3 if using python 3.5 (only 2.7 and 3.5 are supported)
87 if NOT DEFINED PYTHON_VERSION set PYTHON_VERSION=3
```

步骤 5. 点击运行 build_win.cmd 文件；

图5-165 运行 build_win.cmd 文件



步骤 6. 首次运行，会有如下打印，并且会出现卡顿，此时会发现，在 C:\Users\Username(用户名，视用户本身用户名的情况而定)\caffe\dependencies\download 文件夹下会出现一个大小为 0 的 libraries_v140_x64_py35_1.1.0.tar.bz2 文件，需要手动从 <https://github.com/willyd/caffe-builder/releases> 下载 libraries_v140_x64_py35_1.1.0.tar.bz2 并替换到该位置；

图5-166 运行打印，下载卡顿

```

Detecting C compiler ABI info
-- Detecting C compiler ABI info - done
-- Check for working CXX compiler using: Visual Studio 14 2015 Win64
-- Check for working CXX compiler using: Visual Studio 14 2015 Win64 -- works
-- Detecting CXX compiler ABI info
-- Detecting CXX compiler ABI info - done
-- Detecting CXX compile features
-- Detecting CXX compile features - done
-- Found PythonInterp: D:/python/python.exe (found suitable version "3.5.4", minimum required is "2.7")
-- Downloading prebuilt dependencies to C:/Users/Administrator/.caffe/dependencies/download/libraries_v140_x64_py35_1.1.0.tar.bz2

```

步骤 7. 修改 caffe-windows\scripts 下的 download_prebuilt_dependencies.py 文件，注释掉如下内容以及\caffe-windows\cmake\WindowsDownloadPrebuiltDependencies.cmake;

图5-167 download_prebuilt_dependencies.py

```

45     dep_filename = os.path.split(url)[1]
46     # Download binaries
47     #print("Downloading dependencies ({}). Please wait...".format(dep_filename))
48     #urllib.request.urlretrieve(url, dep_filename, reporthook)
49     #if not model_checks_out(dep_filename, sha1):
50     #     print('ERROR: dependencies did not download correctly! Run this again.')
51     #     sys.exit(1)
52     #print("\nDone.")

```

图5-168 WindowsDownloadPrebuiltDependencies.cmake

```

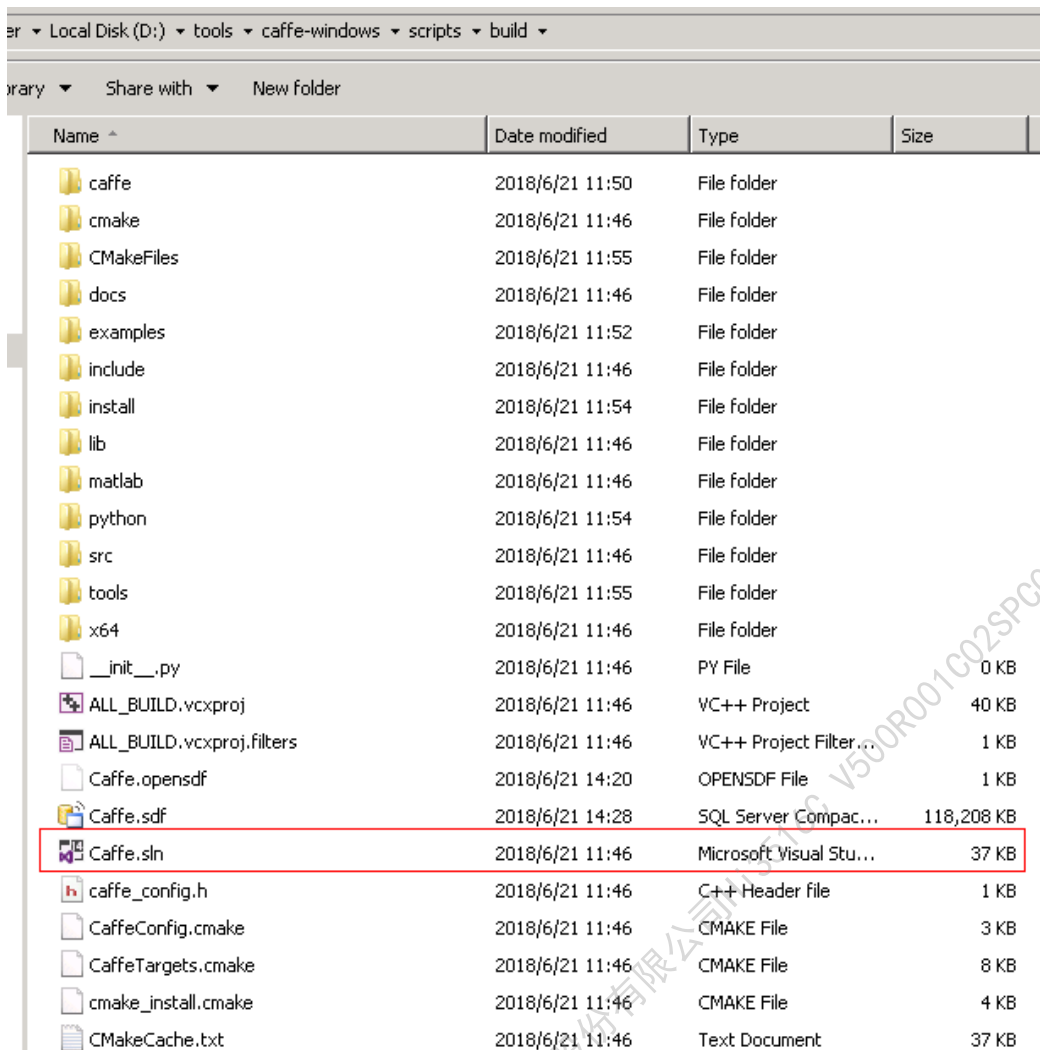
55     # download and extract the file if it does not exist or if does not match the sha1
56     get_filename_component(download_filename ${DEPENDENCIES_URL} NAME)
57     set(download_path ${CAFFE_DEPENDENCIES_DOWNLOAD_DIR}/${download_filename})
58     #set(download_file 1)
59     #if(EXISTS ${download_path})
60     #     file(SHA1 ${download_path} file_sha)
61     #     if("${file_sha}" STREQUAL "${DEPENDENCIES_SHA}")
62     #         set(download_file 0)
63     #     else()
64     #         set(download_file 1)
65     #         message(STATUS "Removing file because sha1 does not match.")
66     #         file(REMOVE ${download_path})
67     #     endif()
68     #endif()
69     #if(download_file)
70     #     message(STATUS "Downloading prebuilt dependencies to ${download_path}")
71     #     file(DOWNLOAD ${DEPENDENCIES_URL}
72     #         "${download_path}"
73     #         EXPECTED_HASH SHA1=${DEPENDENCIES_SHA}
74     #         SHOW_PROGRESS
75     #     )
76     #     if(EXISTS ${CAFFE_DEPENDENCIES_DIR}/libraries)
77     #         file(REMOVE_RECURSE ${CAFFE_DEPENDENCIES_DIR}/libraries)
78     #     endif()
79     #endif()
80     if(EXISTS ${download_path})

```

步骤 8. 运行 build_win.cmd，就会生成 VS 工程文件，整个过程大概要用 7 分钟，此时的工程文件就是编译后未扩展的 caffe1.0 工程；



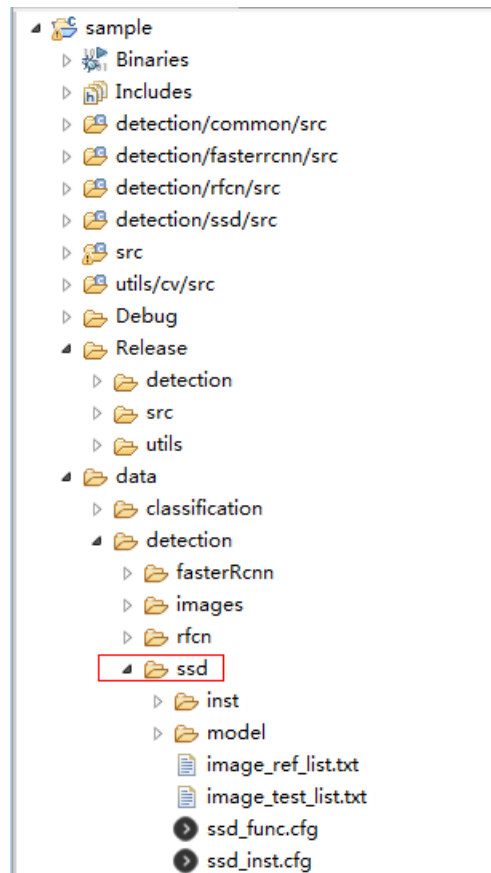
图5-169 Caffe1.0 基础工程



Name	Date modified	Type	Size
caffe	2018/6/21 11:50	File folder	
cmake	2018/6/21 11:46	File folder	
CMakeFiles	2018/6/21 11:55	File folder	
docs	2018/6/21 11:46	File folder	
examples	2018/6/21 11:52	File folder	
include	2018/6/21 11:46	File folder	
install	2018/6/21 11:54	File folder	
lib	2018/6/21 11:46	File folder	
matlab	2018/6/21 11:46	File folder	
python	2018/6/21 11:54	File folder	
src	2018/6/21 11:46	File folder	
tools	2018/6/21 11:55	File folder	
x64	2018/6/21 11:46	File folder	
__init__.py	2018/6/21 11:46	PY File	0 KB
ALL_BUILD.vcxproj	2018/6/21 11:46	VC++ Project	40 KB
ALL_BUILD.vcxproj.filters	2018/6/21 11:46	VC++ Project Filter...	1 KB
Caffe.opensdf	2018/6/21 14:20	OPENSDF File	1 KB
Caffe.sdf	2018/6/21 14:28	SQL Server Compac...	118,208 KB
Caffe.sln	2018/6/21 11:46	Microsoft Visual Stu...	37 KB
caffe_config.h	2018/6/21 11:46	C++ Header file	1 KB
CaffeConfig.cmake	2018/6/21 11:46	CMAKE File	3 KB
CaffeTargets.cmake	2018/6/21 11:46	CMAKE File	8 KB
cmake_install.cmake	2018/6/21 11:46	CMAKE File	4 KB
CMakeCache.txt	2018/6/21 11:46	Text Document	37 KB

步骤 9. 参照 3.1.4 章节的描述，客户需要手动对 caffe.proto 文件进行修改，注册层信息。以检测网的 ssd 网络为例：

图5-170 检测网的 ssd 网络



在 nnie_ssd_deploy.prototxt 文件，会发现有 Normalize 和 Permute 这两个拓展层。

图5-171 nnie_ssd_deploy.prototxt 文件的 Normalize 层

```
815 layer {  
816   name: "conv4 3 norm"  
817   type: "Normalize"  
818   bottom: "conv4_3"  
819   top: "conv4_3_norm"  
820   norm_param {  
821     across_spatial: false  
822     scale_filler {  
823       type: "constant"  
824       value: 20  
825     }  
826     channel_shared: false  
827   }  
828 }
```


图5-172 nnie_ssd_deploy.prototxt 文件的 Permute 层

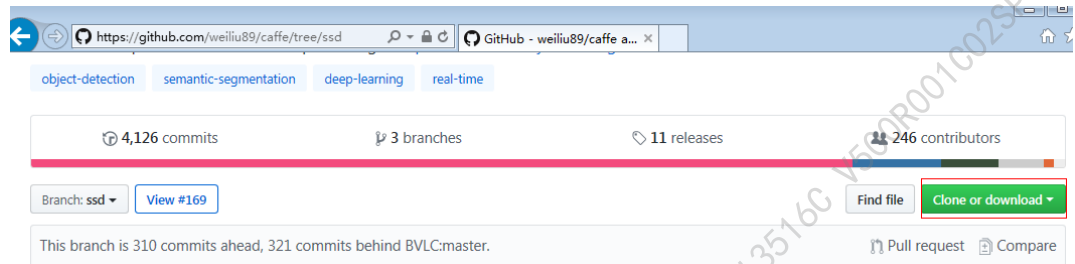
```

856 layer {
857   name: "conv4_3_norm_mbox_loc_perm"
858   type: "Permute"
859   bottom: "conv4_3_norm_mbox_loc"
860   top: "conv4_3_norm_mbox_loc_perm"
861   permute_param {
862     order: 0
863     order: 2
864     order: 3
865     order: 1
866   }
867 }

```

步骤 10. 从 3.1.4.4 提供的 Normalize 层和 Permute 层的开源工程地址
<https://github.com/weiliu89/caffe/tree/ssd> 下载相关代码；

图5-173 下载 caffe-ssd.zip 包



步骤 11. 修改 caffe-windows\src\caffe\proto\caffe.proto 文件，在 LayerParameter 中增加相关拓展层的定义。可以参考 caffe-ssd\src\caffe\proto\caffe.proto 文件的写法；

图5-174 增加 NormalizeParameter 和 PermuteParameter 的定义 (1)

```

caffe.proto  x common.hpp  upsample_layer.cpp  roi_pooling_layers.cpp
403 optional SliceParameter slice_param = 126;
404 optional TanHParameter tanh_param = 127;
405 optional ThresholdParameter threshold_param = 128;
406 optional TileParameter tile_param = 138;
407 optional WindowDataParameter window_data_param = 129;
408
409 optional ROIPoolingParameter roi_pooling_param = 100000;
410 optional NormalizeParameter norm_param = 100001;
411 //optional RSROIPoolingParameter psroi_pooling_param = 100002;
412 optional UpsampleParameter upsample_param = 100003;
413 optional PermuteParameter permute_param = 100004;

```

图5-175 增加 NormalizeParameter 和 PermuteParameter 的定义 (2)

```

caffe.proto x common.hpp upsample_layer.cpp roi_pooling_layers.cpp
1430
1431 // Message that stores parameters used by NormalizeLayer
1432 message NormalizeParameter {
1433     optional bool across_spatial = 1 [default = true];
1434     // Initial value of scale. Default is 1.0 for all
1435     optional FillerParameter scale_filler = 2;
1436     // Whether or not scale parameters are shared across channels.
1437     optional bool channel_shared = 3 [default = true];
1438     // Epsilon for not dividing by zero while normalizing variance
1439     optional float eps = 4 [default = 1e-10];
1440 }
1441
1442 message PermuteParameter {
1443     // The new orders of the axes of data. Notice it should be with
1444     // in the same range as the input data, and it starts from 0.
1445     // Do not provide repeated order.
1446     repeated uint32 order = 1;
1447 }

```

步骤 12. 将 NormalizeParameter 和 PermuteParameter 的实现代码从 caffe-ssd\src\caffe\layers 拷贝到 caffe-windows\src\caffe\layers 文件夹下，包括头文件和源文件。

步骤 13. 在工程文件中，caffe->Source->Layers 右键->Add->Existing Item 加入到工程，注册层信息；

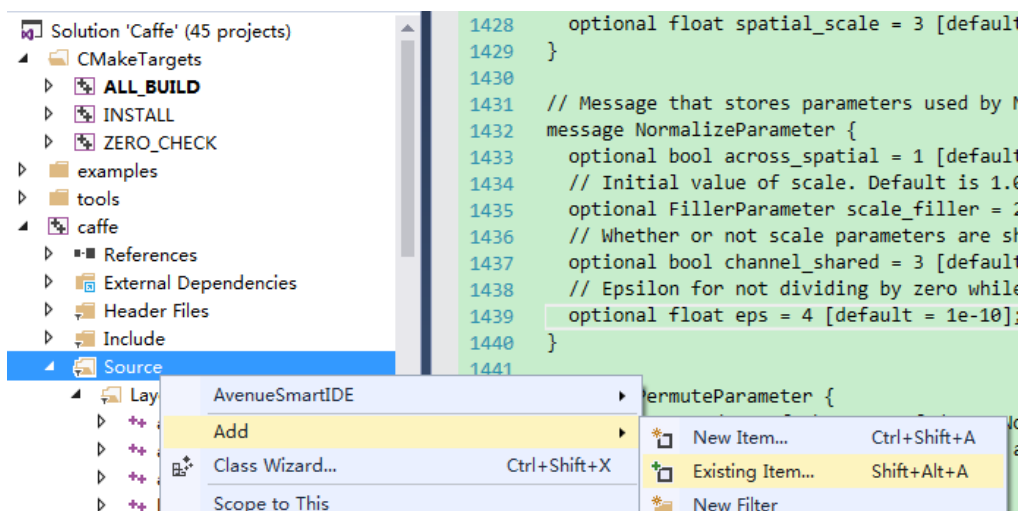
图5-176 NormalizeParameter 和 PermuteParameter 层的代码文件路径

Local Disk (D:) > test > caffe-ssd > src > caffe > layers			
w folder			
Name ^	Date modified	Type	Size
neuron_layer.cpp	2018/1/2 11:07	C++ Source file	1 KB
normalize_layer.cpp	2018/1/2 11:07	C++ Source file	10 KB
normalize_layer.cu	2018/1/2 11:07	CU File	8 KB
parameter_layer.cpp	2018/1/2 11:07	C++ Source file	1 KB
permute_layer.cpp	2018/1/2 11:07	C++ Source file	5 KB
permute_layer.cu	2018/1/2 11:07	CU File	3 KB

cu 结尾的是 cuda 实现，如果要支持 cuda 实现，需要选择.cu 后缀的文件。

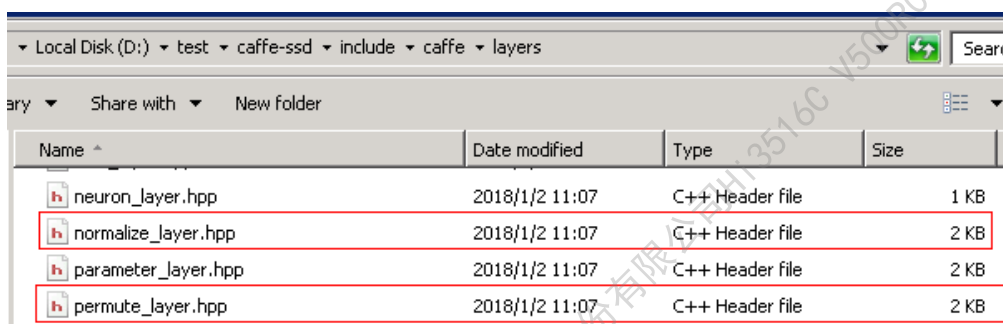


图5-177 添加代码 permute_layer.cpp 和 normalize_layer.cpp 到工程文件



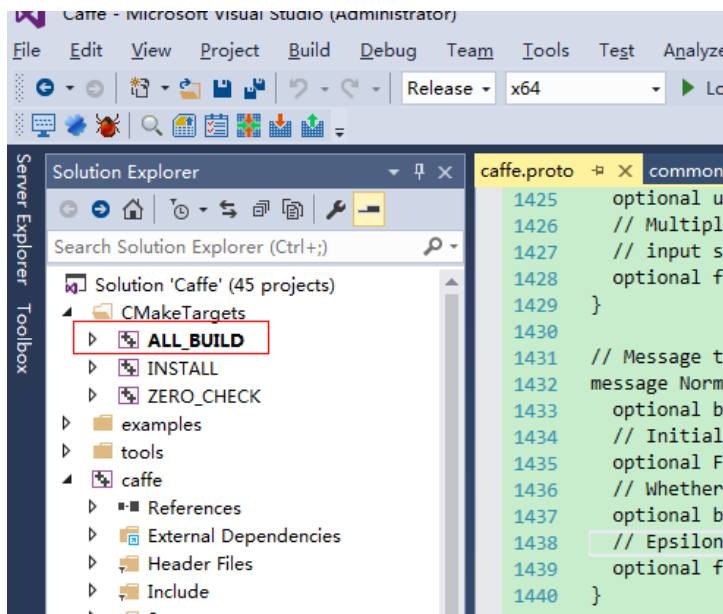
步骤 14. 在工程文件中，caffe->Header Files->Layers 右键->Add->Existing Item 加入到工程。

图5-178 添加代码 permute_layer.hpp 和 normalize_layer.hpp 到工程文件



步骤 15. 重新编译工程；

图5-179 Release 模式编译



- 步骤 16. 将 caffe-windows\scripts\build\lib\Release 下的_caffe.pyd 替换到 python35\Lib\site-packages\caffe\python\caffe 目录下；
- 步骤 17. 将 caffe_windows\caffe-windows\scripts\build\include\caffe\proto 下的_caffe_pb2.py 替换到 python35\Lib\site-packages\caffe\python\caffe\proto 目录下；（这里拷贝的两个文件是必要文件，用户可依据情况，将最新编译出的相关文件选择性的拷贝替换进去）
- 步骤 18. 在 RuyiStudio-xx.xx.xx\Resources\pythonScript 文件夹下，扩展 CNN_convert_to_Caffemodel.py 和 CNN_convert_bin_and_print_featuremap.py 这两个脚本中的 supported_layers，将新增的层增加到 supported_layers 的定义中，如图 5-180 所示：

图5-180 在代码中增加扩展层

```
supported_layers=[  
    "Convolution", "Deconvolution", "Pooling", "InnerProduct", "LRN", "BatchNorm", "Scale", "Bias", "Eltwise",  
    "Parameter", "Reduction", "Proposal", "Custom", "Input", "Dropout", "Normalize", "Permute"]
```

- 步骤 19. 参照 5.5.5 “输出 caffe 的中间结果” 章节，设置 prototxt，caffemodel 以及图片文件，点击 ok 键，成功得到 caffe 的中间输出。如果客户不依赖工具，有自己的一套 python 脚本，可直接参考上一步骤中提到的 CNN_convert_bin_and_print_featuremap.py 中的 print_CNNfeaturemap 函数，运用于自己的脚本中。



图5-181 打印 caffe 中间结果

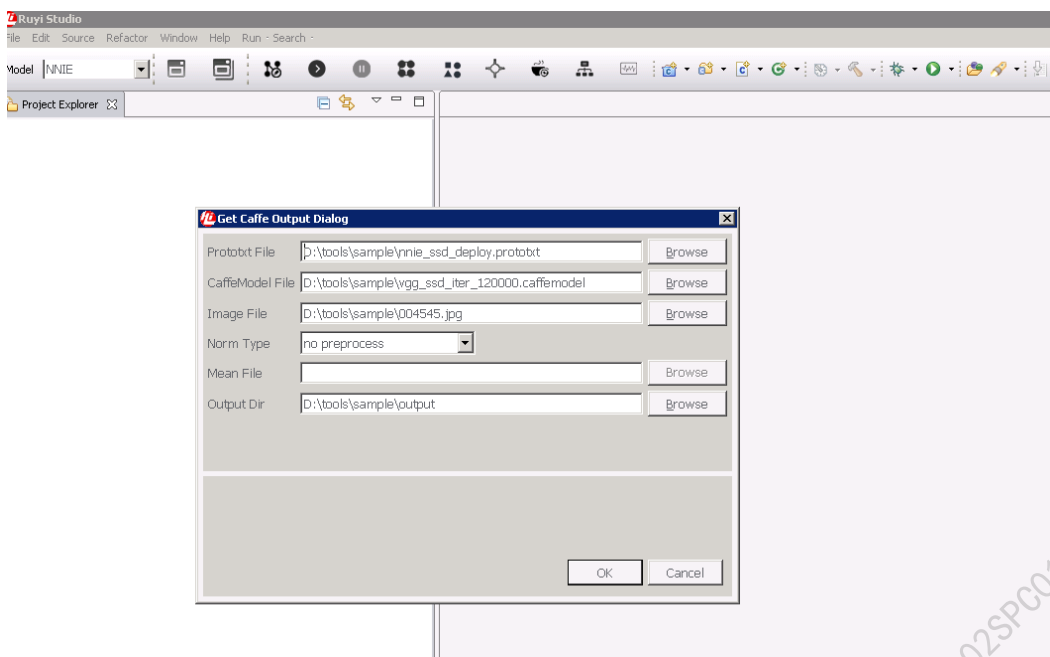
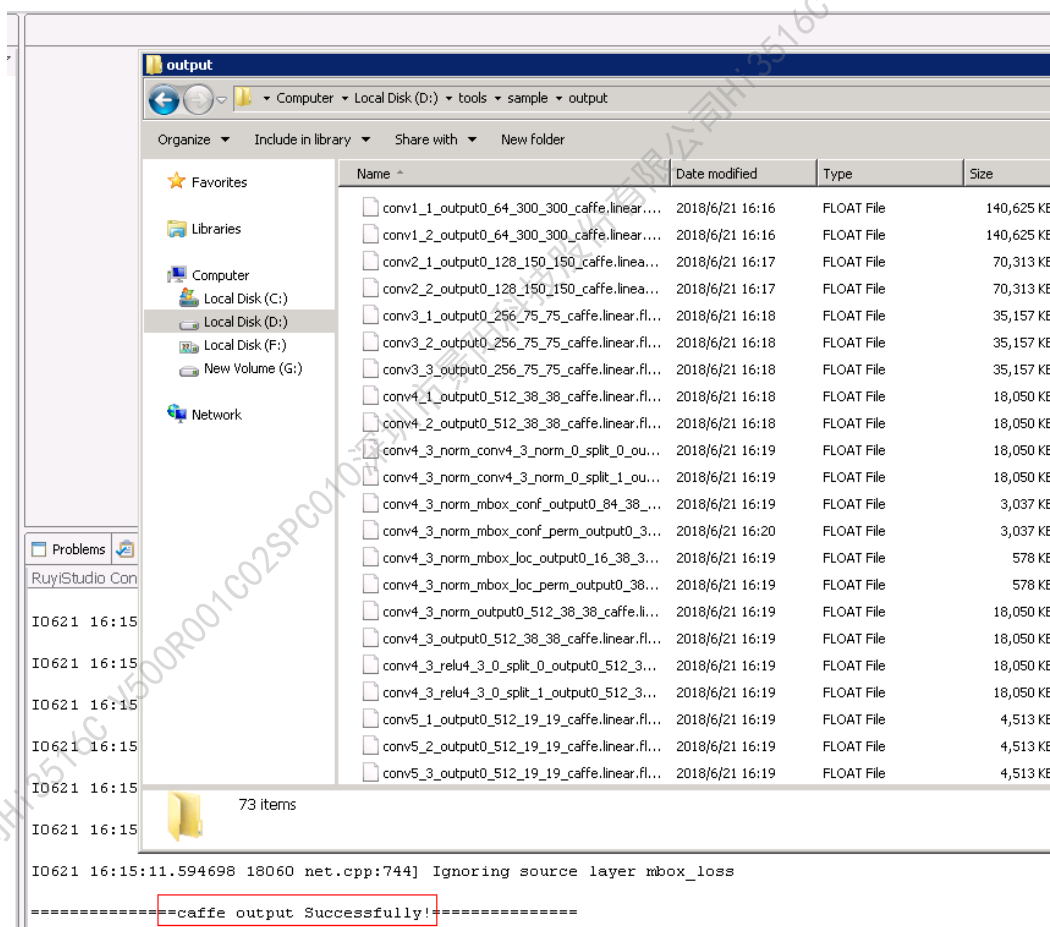


图5-182 成功得到 caffe 的中间输出





----结束

5.13.3 工具在运行仿真库代码时，当仿真库对应的 exe 发生异常崩溃时，所有 printf 信息在控制台上打印不出来的解决办法

问题描述

工具在运行仿真库代码时，当仿真库对应的 exe 发生异常崩溃时，所有 printf 信息在控制台上打印不出来，我该怎么办？

问题原因

因 printf 方法对应的实现原理是将要打印的数据线刷新到缓存流中等待用户只需 fflush 方法才会刷新到控制台中显示，故当程序出现崩溃时，用户未添加 fflush 是无法正常打印之前需要打印的内容的。

解决办法

在代码的 main 方法中最前面添加如下两句 `setbuf(stderr, NULL); setbuf(stdout, NULL);`，如图 5-183，目的是设置输出缓存 BUFF 为 NULL 即可使得 printf 方法做到实时输出。

图5-183 main 方法中设置缓存 BUFF 为 NULL

```
1 #include <stdio.h>
2 #include "SvpSample.h"
3
4 int main(int argc, char* argv[])
5 {
6     /*set stderr & stdout buffer to NULL to flush print info immediately*/
7     setbuf(stderr, NULL);
8     setbuf(stdout, NULL);
9
10    /*Classification*/
11    SvpSampleCnnClfLenet();
12    SvpSampleCnnClfAlexnet();
13    SvpSampleCnnClfVgg16();
14    SvpSampleCnnClfGooglenet();
15    SvpSampleCnnClfResnet50();
16    SvpSampleCnnClfSqueezenet();
17
18    /*Detection*/
19    SvpSampleRoiDetFasterRCNNAlexnet();
20    SvpSampleRoiDetFasterRCNNVGG16();
21    SvpSampleRoiDetFasterRCNNResnet18();
22    SvpSampleRoiDetFasterRCNNResnet34();
23    SvpSampleRoiDetFasterRCNNPvanet();
24    SvpSampleRoiDetFasterRCNNDoubleRoi();
25    SvpSampleRoiDetRCNNResnet50();
26    SvpSampleCnnDetVoloV1();
27    SvpSampleCnnDetVoloV2();
28    SvpSampleCnnDetS50();
29
30    /*Segmentation*/
31    SvpSampleCnnFcnSegnet();
32
33    /*LSTM*/
34    SvpSampleRecurrentLSTMFC();
35    SvpSampleRecurrentLSTMRelu();
36
37    //printf("press any key to exit ... \n");
38    //getchar();
39
40    return 0;
41 }
```



5.13.4 在创建项目时，出现中文时，导致项目修改文件失败，编译和运行失败的问题

问题描述

在创建项目的时候，出现中文时，导致项目修改文件失败，编译和运行失败我该怎么办？

问题原因

因为原生的 CDT 是不支持中文的，所以当出现中文时，就可能会出现上面描述的问题。

解决办法

为了保证创建的工程能够正确编译和运行，以及文件的修改，在创建工程的时候，不要出现中文，以及中文路径。